CSP-S KNOWLEDGE BASE

CSP-S 知识库全集

指南主页全文 • 7 大板块 • 60 篇知识页面完整正文

整理自 csp.wiki CSP-S 文档板块

2026 年 9 月

目录

CSP-S 指南

数学基础 01 • 集合基础知识

数学基础 02 • 容斥原理

数学基础 03 • 欧拉函数

数学基础 04 • 筛法进阶

数学基础 05 • 欧拉定理&费马小定理

数学基础 06 • 同余方程

数学基础 07 • 乘法逆元

数学基础 08 • 裴蜀定理

数学基础 09 • 中国剩余定理

数学基础 10 • 矩阵与矩阵快速幂

搜索 01 • 双向搜索

搜索 02 • A*搜索算法

搜索 03 • 迭代加深搜索

搜索 04 • IDA*算法

搜索 05 • 剪枝

动态规划 01 • 背包 DP 进阶

动态规划 02 • 树形 DP

动态规划 03 • 状压 DP

动态规划 04 • 数位 DP

动态规划 05 • 计数 DP

动态规划 06 • 单调队列/栈优化 DP

动态规划 07 • 四边形不等式优化 DP

动态规划 08 • 状态设计技巧

动态规划 99 • 模板

数据结构 01 • 二叉堆

数据结构 02 • 并查集

数据结构 03 • 单调栈

数据结构 04 • 单调队列

数据结构 05 • ST 表

数据结构 06 • 树状数组

数据结构 07 • 线段树

数据结构 08 • 二叉搜索树与平衡树

数据结构 09 • Splay 树

数据结构 10 • Treap

数据结构 11 • 分块思想
数据结构 12 • 莫队算法
图论 01 • 最小生成树
图论 02 • Floyd 最短路算法
图论 03 • Bellman-Ford 与 SPFA 算法
图论 04 • Dijkstra 算法
图论 05 • 拓扑排序与 Kahn 算法
图论 06 • 欧拉路
图论 07 • 强连通分量与 Tarjan 算法
图论 08 • 割点与割边
图论 09 • 双连通分量
图论 10 • 树的直径与重心
图论 11 • 最近公共祖先 LCA
图论 12 • 树上前缀和与差分
图论 13 • 最长路
图论 14 • 单源次短路径
图论 15 • Prufer 序列与 Caley 公式
字符串 01 • 字符串基础知识及常用术语
字符串 02 • 前缀函数与字符串匹配
字符串 03 • 字符串哈希
字符串 04 • Trie 树
字符串 05 • Manacher 算法
杂项 01 • 离散化
杂项 02 • 双指针
杂项 03 • 悬线法
杂项 04 • 扫描线

CSP-S 指南

CSP-S 介绍

CSP-S 现在俗称提高组，是 CSP-J（普及组）的下一个阶段的考试。

CSP-S 形式与 CSP-J 类似，分为两轮，第一轮为初赛形式为闭卷纸质试卷笔试，第二轮为上机测试。具体详见CSP 介绍。

学习内容

目标奖项的层次决定了学习内容的深度与广度。CSP-S 提高组的竞争更为激烈，“强省”（如浙江、江苏、上海、北京）与“弱省”（如部分西部、边远地区）的获奖分数线差异巨大，竞争格局也完全不同。

在**顶尖强省**，CSP-S 奖项对应的分数线大致如下：

- CSP-S 一等奖（省一）：**220 分左右**（甚至更高）
- CSP-S 二等奖（省二）：**120 分左右**
- CSP-S 三等奖（省三）：**60 分左右**

在**竞争较弱的省份**，奖项对应的分数线大致如下：

- CSP-S 一等奖（省一）：**150 分左右**
- CSP-S 二等奖（省二）：**60 分左右**
- CSP-S 三等奖（省三）：**不设置三等奖**

目标 100 分

这是进入提高组的门槛分数，意味着需要在至少一道题上拿到可观的部分分，或相对稳定地解决第一题。

- **核心要求：**
- **精通 C++ 语法与 STL 基础库：** 熟练使用 `vector`，`map`，`set`，`string`，`algorithm` 等，理解迭代器、函数对象。
- **普及组有较好的基础：** 动态规划（基础背包、线性 DP）、基础图论（DFS/BFS）、基础数论（gcd/lcm、筛法、快速幂）必须达到熟练应用水平。
- **代码调试能力：** 能快速定位并修复语法错误、数组越界、死循环等常见问题。
- **学习重点：** 巩固普及组核心算法，提升代码实现速度和准确性，学习更高效的调试方法。

目标 200 分

这个分数段要求选手具备解决中等难度问题的能力，通常需要稳定做出第一题并拿到第二题的部分分，或者在第三题上拿到基础分。

- **核心要求：**
- **中级数据结构：** 熟练掌握堆（优先队列）、并查集、树状数组、线段树（基础应用）。
- **进阶算法思想：**

搜索优化： 剪枝策略（可行性/最优性）、迭代加深、双向 BFS。 **动态规划深化：** 状态压缩 DP、树形 DP、数位 DP等。 **图论进阶：** 各种常用最短路算法、拓扑排序、强连通分量、LCA等。

- **基础数学能力：** 筛法进阶、同余方程、乘法逆元等。
- **学习重点：** 系统学习上述进阶数据结构和算法，理解其原理和应用场景，进行大量针对性练习，提升分析问题、设计算法的能力。

目标 300 分 +（稳定省一 / 冲击省队）

这是顶尖选手的领域，意味着具备解决难题或在多道题上拿到高分的能力，省队选拔赛（NOIP）的敲门砖。

- **核心要求：**

- **高级数据结构：** 线段树高级应用（扫描线等）、平衡树基础（如 Treap, Splay）、可持久化数据结构（主席树等）、分块。
- **高阶算法与优化：**

动态规划： 单调队列优化、四边形不等式优化、插头 DP 等。 **图论：** 除了本版块的全图论内容，还需要学习网络流算法。 **字符串：** KMP、字符串哈希、trie 树、Manacher 算法。 **数学：** 莫比乌斯反演、生成函数基础、FFT/NTT 等。

- **复杂问题建模能力：** 能将实际问题抽象为合适的数学模型（图、状态机、方程等）。
- **高效实现与常数优化：** 对算法复杂度有精确把握，能进行必要的常数优化。
- **策略与心态：** 具备赛场策略（时间分配、取舍），良好的心理素质应对压力。
- **学习重点：** 深入钻研高阶算法，进行大量综合题、难题训练（尤其是历年 NOIP/省选题），参与模拟赛积累实战经验，学习复杂问题的建模技巧。

目标 AK（满分）

这是极高的目标，需要对知识体系有极其深刻的理解，具备极强的思维能力和近乎完美的代码实现能力，且需要一定的临场发挥和运气。

- **核心要求：**
- **知识体系无短板：** 上述所有内容需达到精通，并能灵活组合应用。
- **极强的思维能力：** 快速洞察问题本质，设计出最优或接近最优的算法。
- **卓越的代码能力：** 代码实现准确、高效、健壮，一次通过率高。
- **海量高质量练习：** 完成大量高难度题目（500+ 题），涵盖各类知识点和题型，尤其注重思维难度大的题目。
- **丰富的比赛经验：** 参加大量线上/线下模拟赛和正式比赛，适应高强度比赛环境。
- **学习重点：** 查漏补缺，追求极致。研究最前沿的算法思想（省选/NOI 级别），进行超高强度的综合训练和模拟赛。此时学习重心应转向省选和 NOI。

NOIP 介绍

NOIP 又称联赛，同时也在以前同样被称作提高组。

NOIP 的难度与 CSP-S 接近，知识点也相同，区别是 NOIP 只有高中生才能参加而 CSP-S 则没有这个限制。

同时，NOIP 的参赛资格需要通过 CSP-S 中取得一定的成绩（超过此年划定的分数线）来获取，CSP-S 的复赛可以当作是 NOIP 的初赛，尽管这二者形式类似难度相同。

具体详见NOIP 介绍。

NOIP 的知识点、难度与分数线与 CSP-S 类似，学习方法相同，不再多作赘述。

学习技巧

选题策略

- **紧跟考纲与趋势：** 重点突破 CSP-S/NOIP 近年高频考点：动态规划（各种模型和优化）、图论、数据结构（线段树、平衡树）、字符串、数学。
- **分层训练：** 根据目标分数选择题目难度，避免长期停留在舒适区或长期被难题打击。
- **专题突破：** 针对薄弱知识点进行集中、高强度的专项训练（如一周专攻状压 DP 或网络流建模等）。

错题分析

- **深度归因：** 不仅记录“超时”、“WA”，更要分析根本原因：算法选择错误？复杂度分析失误？边界条件遗漏？特殊数据未考虑？实现细节出错（初始化、下标）？
- **对比优化：** 研究优秀题解（尤其是不同思路、更优解法），对比自己的代码，学习更精妙的算法设计、更简洁的代码实现、更严谨的边界处理。

- **建立“错题本”：** 系统记录经典错题、难题的核心思路、关键技巧、易错点和优化点，定期重做。

避免“伪勤奋”

- **专注深度而非数量：** 精做一道涵盖多个知识点、需要深入思考的题目，远胜于刷十道简单重复题。追求彻底理解而非“刷过”。
- **独立攻坚：** 遇到难题，给自己设定合理的思考时间（如 30-60 分钟），尝试多种思路，**实在无解再看题解**。看懂后务必独立复现代码，并思考“为什么我没想到？”。

模拟实战与反思

- **限时模拟赛：** 定期（如每周）参加线上模拟赛或进行个人限时训练（4 小时 3 题），严格模拟考场环境（无资料、无提示）。
- **赛后深度复盘：**
- 时间分配是否合理？策略有无失误（死磕/放弃过早）？
- 每道题的解题思路是如何形成的？卡点在哪里？如何突破的（或为何未能突破）？
- 代码实现是否有 BUG？如何避免？
- 哪些知识点/技巧掌握不牢？
- **“讲题”训练：** 尝试清晰地向他人（或虚拟听众）讲解一道难题的**完整解题思路**，包括问题转化、算法选择、关键证明、复杂度分析、实现细节。讲不通顺的地方即是薄弱点。

集合基础知识

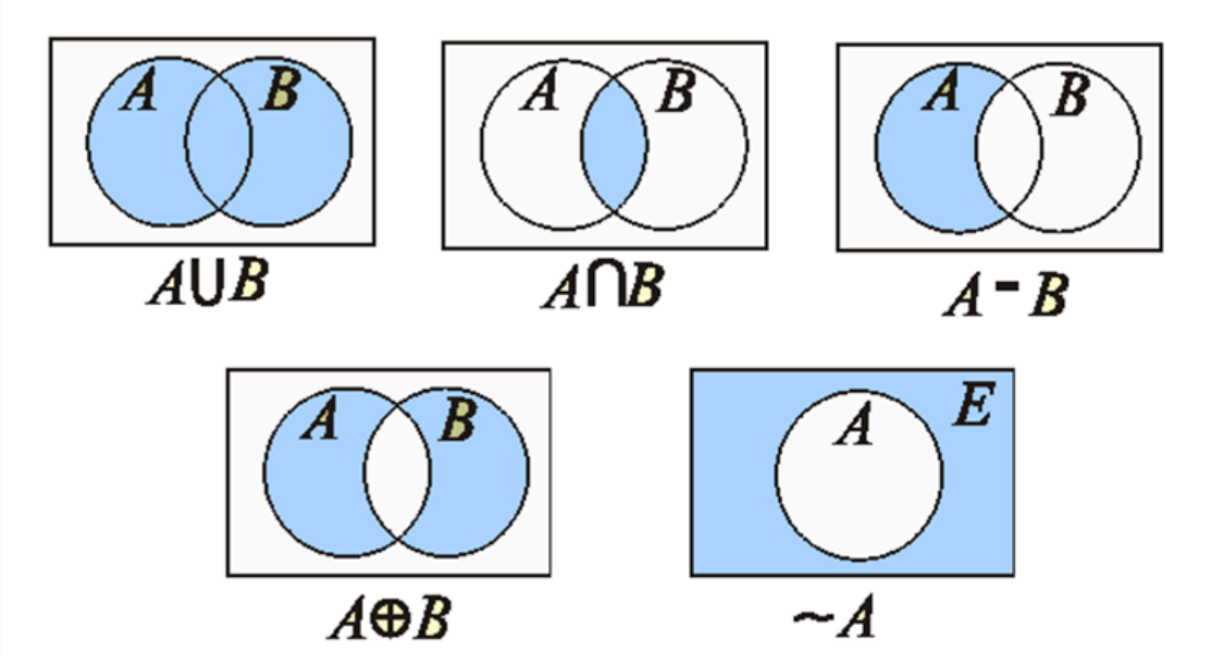
数学基础 01 • 集合基础知识

一、集合的基本概念与性质

1. 集合的定义

集合是由**确定的、互不相同的元素**组成的无序整体。

- **确定性**：元素是否属于集合有明确判定（如“所有正数”是集合，“所有大数”不是）。
- **互异性**：集合中元素唯一（如 `{1, 2, 2}` 非法，应写作 `{1, 2}`）。
- **无序性**：集合中元素顺序无关（如 `{1, 2}` 与 `{2, 1}` 是同一集合）。



集合及其运算

2. 常见表示方法

- **列举法**：直接列出元素，如 `A = {1, 2, 3}`。
- **描述法**：用条件描述元素，如 `B = {x | x 属于 N, x <= 10}`。
- **符号表示**：
- x 属于 A ： x 属于集合 A ；
- x 不属于 A ： x 不属于集合 A ；
- |A| ：集合 A 的元素个数（基数）。

3. 特殊集合

- **空集**：不含任何元素，记作空集 ，是所有集合的子集。
- **全集**：包含讨论范围内所有元素的集合，记作 U 。

二、集合的关系与运算

1. 集合间的关系

- **子集**：若任意 x 属于 A => x 属于 B ，则 A 包含于 B （ A 是 B 的子集）。
- **真子集**：若 A 包含于 B 且 A != B ，则 A 真包含于 B 。
- **相等**：若 A 包含于 B 且 B 包含于 A ，则 A = B 。

2. 基本运算

运算	定义	符号/公式	位运算等价（有限集）
并集	包含所有属于 A 或 B 的元素	A 并 B = x	x 属于 A or x 属于 B
交集	包含所有同时属于 A 和 B 的元素	A 交 B = x	x 属于 A and x 属于 B
差集	包含属于 A 但不属于 B 的元素	A - B = x	x 属于 A and x 不属于 B
对称差集	包含属于 A 或 B ，但不同时属于两者的元素	A xor B = (A - B) 并 (B - A)	a ^ b
补集	包含全集 U 中不属于 A 的元素	A^c = U - A	~ a mask

3. 运算性质

- **交换律**： A 并 B = B 并 A ， A 交 B = B 交 A 。
- **结合律**： (A 并 B) 并 C = A 并 (B 并 C) ，类似适用于交集。
- **分配律**： A 并 (B 交 C) = (A 并 B) 交 (A 并 C) ， A 交 (B 并 C) = (A 交 B) 并 (A 交 C) 。
- **德摩根定律**： A 并 B^c = A^c 交 B^c ， A 交 B^c = A^c 并 B^c 。

CSP-S 指南

容斥原理

数学基础 02 · 容斥原理

引入

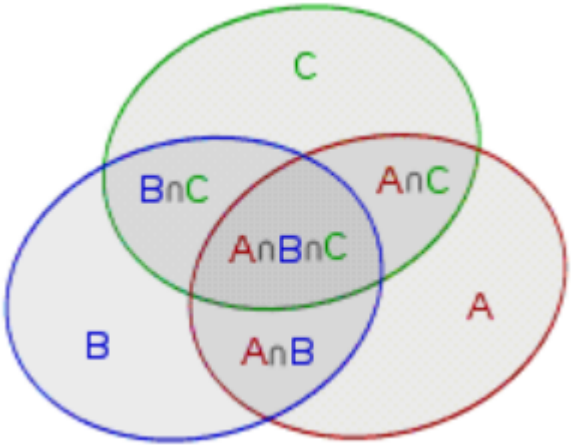
入门例题

假设班里有 10 个学生喜欢数学， 15 个学生喜欢语文， 21 个学生喜欢编程，班里至少喜欢一门学科的有多少个学生呢？

是 10+15+21=46 个吗？不是的，因为有些学生可能同时喜欢数学和语文，或者语文和编程，甚至还有可能三者都喜欢。

为了叙述方便，我们把喜欢语文、数学、编程的学生集合分别用 A,B,C 表示，则学生总数等于 |A 并 B 并 C| 。刚才已经讲过，如果把这三个集合的元素个数 |A|，|B|，|C| 直接加起来，会有一些元素重复统计了，因此需要扣掉 |A 交 B|，|B 交 C|，|C 交 A| ，但这样一来，又有一小部分多扣了，需要加回来，即 |A 交 B 交 C| 。即

|A 并 B 并 C|=|A|+|B|+|C|-|A 交 B|-|B 交 C|-|C 交 A|+|A 交 B 交 C|



图示

把上述问题推广到一般情况，就是我们熟知的容斥原理。

定义

设 U 中元素有 n 种不同的属性，而第 i 种属性称为 P[i] ，拥有属性 P[i] 的元素构成集合 S[i] ，那么

| 并 (i=1 ~ n)S[i]|= sum [i]|S[i]|- sum *(i<j)|S[i] 交 S[j]|+ sum* (i<j<k)|S[i] 交 S[j] 交 S[k]|-... +(-1)^(m-1) sum *a[i]<a[i+1] | 交 (i=1 ~ m)S*(a[i])|+...+(-1)^(n-1)|S[1] 交 ... 交 S[n]|

即

| 并 (i=1 ~ n)S[i]|= sum *(m=1)^n(-1)^(m-1) sum _a[i]<a[i+1] |bigcap*(i=1)^(m)S_(a[i])|

证明

对于每个元素使用二项式定理计算其出现的次数。对于元素 x，假设它出现在 T[1],T[2],...,T_(m) 的集合中，那么它的出现次数为

Cnt= |T[i]|-|T[i] 交 T[j]|i<j|+...+(-1)^(k-1)| 交 (i=1 ~ k)T_(a[i])|a[i]<a[i+1]| +...+(-1)^(m-1)|T[1] 交 ... 交 T_(m)| = C(m,1)-C(m,2)+...+(-1)^(m-1)C(m,m) = C(m,0)- sum _(i=0)^(m)(-1)^iC(m,i) = 1-(1-1)^(m)=1

于是每个元素出现的次数为 1，那么合并起来就是并集。证毕。

补集

对于全集 U 下的 **集合的并** 可以使用容斥原理计算，而集合的交则用全集减去 **补集的并集** 求得：

| 交 (i=1 ~ n)S[i]|=|U|-|bigcup_(i=1)^nS[i]^c|

右边使用容斥即可。

可能接触过容斥的读者都清楚上述内容，而更关心的是容斥的应用。

那么接下来我们给出 3 个层次不同的例题来为大家展示容斥原理的应用。

不定方程非负整数解计数

不定方程非负整数解计数

给出不定方程 sum _(i=1)^nx[i]=m 和 n 个限制条件 x[i] <= b[i] ，其中 m,b[i] 属于 N . 求方程的非负整数解的个数。

没有限制时

如果没有 x[i]<b[i] 的限制，那么不定方程 sum _(i=1)^nx[i]=m 的非负整数解的数目为 C(m+n-1,n-1) .
略证：插板法。
相当于你有 m 个球要分给 n 个盒子，允许某个盒子是空的。这个问题不能直接用组合数解决。
于是我们再加入 n-1 个球，于是问题就变成了在一个长度为 m+n-1 的球序列中选择 n-1 个球，然后这个 n-1 个球把这个序列隔成了 n 份，恰好可以一一对应放到 n 个盒子中。那么在 m+n-1 个球中选择 n-1 个球的方案数就是 C(m+n-1,n-1) 。

容斥模型

接着我们尝试抽象出容斥原理的模型：

1. 全集 U：不定方程 sum _(i=1)^nx[i]=m 的非负整数解
2. 元素：变量 x[i] .
3. 属性： x[i] 的属性即 x[i] 满足的条件，即 x[i] <= b[i] 的条件

目标：所有变量满足对应属性时集合的大小，即 |bigcap_(i=1)^nS[i]| .
这个东西可以用 | 交 (i=1 ~ n)S[i]|=|U|-|bigcup_(i=1)^nS[i]^c| 求解。 |U| 可以用组合数计算，后半部分自然使用容斥原理展开。
那么问题变成，对于一些 S_(a[i])^c 的交集求大小。考虑 S_(a[i]) 的含义，表示 x_(a[i]) >= b_(a[i])+1 的解的数目。而交集表示同时满足这些条件。因此这个交集对应的不定方程中，有些变量有 **下界限制**，而有些则没有限制。
能否消除这些下界限制呢？既然要求的是非负整数解，而有些变量的下界又大于 0 ，那么我们直接**把这个下界减掉**，就可以使得这些变量的下界变成 0 ，即没有下界啦。因此对于
|bigcap_a[i]<a[i+1] ^(1 <= i <= k)S_(a[i])|
的不定方程形式为
sum *(i=1)^nx[i]=m- sum* (i=1)^(k)(b_(a[i])+1)
于是这个也可以组合数计算啦。这个长度为 k 的 a 数组相当于在枚举子集。

HAOI2008 硬币购物

硬币购物
4 种面值的硬币，第 i 种的面值是 C[i] 。 n 次询问，每次询问给出每种硬币的数量 D[i] 和一个价格 S ，问付款方式。
n <= 10^3,S <= 10^5 .
如果用背包做的话复杂度是 O(4nS) ，无法承受。这道题最明显的特点就是硬币一共只有四种。抽象模型，其实就是让我们求方程 sum _(i=1)^4C[i]x[i]=S,x[i] <= D[i] 的非负整数解的个数。
采用同样的容斥方式， x[i] 的属性为 x[i] <= D[i] . 套用容斥原理的公式，最后我们要求解
sum *(i=1)^4C[i]x[i]=S- sum* (i=1)^(k)C_(a[i])(D_(a[i])+1)
也就是无限背包问题。这个问题可以预处理，算上询问，总复杂度 O(4S+2^4n) 。

数论中的容斥

使用容斥原理能够巧妙地求解一些数论问题。

容斥原理求最大公约数为 k 的数对个数

考虑下面的问题：
求最大公约数为 k 的数对个数
设 1 <= x, y <= N ， f(k) 表示最大公约数为 k 的有序数对 (x, y) 的个数，求 f(1) 到 f(N) 的值。
这道题固然可以用欧拉函数或莫比乌斯反演的方法来做，但是都不如用容斥原理来的简单。
由容斥原理可以得知，先找到所有以 k 为 **公约数** 的数对，再从中剔除所有以 k 的倍数为 **公约数** 的数对，余下的数对就是以 k 为 **最大公约数** 的数对。即 f(k)= 以 k 为 **公约数** 的数对个数 - 以 k 的倍数为 **公约数** 的数对个数。

进一步可发现，以 k 的倍数为 **公约数** 的数对个数等于所有以 k 的倍数为 **最大公约数** 的数对个数之和。于是，可以写出如下表达式：

f(k)= floor (N/k) ^2 - sum (i=2 ~ i*k <= N) f(i*k)

由于当 k>N/2 时，我们可以直接算出 f(k)= floor (N/k) ^2 ，因此我们可以倒过来，从 f(N) 算到 f(1) 就可以了。于是，我们使用容斥原理完成了本题。

```
for (long long k = N; k >= 1; k--) {
f[k] = (N / k) * (N / k);
for (long long i = k + k; i <= N; i += k) f[k] -= f[i];
}
```

上述方法的时间复杂度为 O(sum (i=1 ~ N) N/i)=O(N sum (i=1 ~ N) 1/i)=O(N log N) 。

容斥原理一般化

容斥原理常用于集合的计数问题，而对于两个集合的函数 f(S),g(S) ，若

f(S)= sum _(T 包含于 S)g(T)

那么就有

g(S)= sum _(T 包含于 S)(-1)^(|S|-|T|)f(T)

证明

接下来我们简单证明一下。我们从等式的右边开始推：

sum *(T 包含于 S)(-1)^(|S|-|T|)f(T) = sum* (T 包含于 S)(-1)^(|S|-|T|) sum *(Q 包含于 T)g(Q) = sum* (Q)g(Q) sum _(Q 包含于 T 包含于 S)(-1)^(|S|-|T|)

我们发现后半部分的求和与 Q 无关，因此把后半部分的 Q 剔除：

= sum *(Q)g(Q) sum* (T 包含于 (S - Q))(-1)^(|S - Q|-|T|)

记关于集合 P 的函数 F(P)= sum _(T 包含于 P)(-1)^(|P|-|T|) ，并化简这个函数：

F(P) = sum _(T 包含于 P)(-1)^(|P|-|T|) = sum (i=0 ~ |P|)C(|P|,i)(-1)^(|P|-i)= sum (i=0 ~ |P|)C(|P|,i)1^i(-1)^(|P|-i) =(1-1)^(|P|)=0^(|P|)

因此原来的式子的值是

sum *(Q)g(Q) sum* (T 包含于 (S - Q))(-1)^(|S - Q|-|T|)= sum *(Q)g(Q)F(S - Q)= sum* (Q)g(Q)·0^(|S - Q|)

分析发现，仅当 |S - Q|=0 时有 0^0=1 ，这时 Q=S ，对答案的贡献就是 g(S) ，其他时侯 0^(|S - Q|)=0 ，则对答案无贡献。于是得到

sum _(Q)g(Q)· 0^(|S - Q|)=g(S)

综上所述，得证。

推论

该形式还有这样一个推论。在全集 U 下，对于函数 f(S),g(S) ，如果

f(S)= sum _(S 包含于 T)g(T)

那么

g(S)= sum _(S 包含于 T)(-1)^(|T|-|S|)f(T)

这个推论其实就是补集形式，证法类似。

例题：

Status

Problem

Tags

P1287

.

盒子与球

容斥

P3197

.

越狱
容斥
P8557
.
炼金术（Alchemy）
容斥
集合基础知识
欧拉函数

数学基础 03 • 欧拉函数

定义

欧拉函数（Euler's totient function），即 phi(n) ，表示的是小于等于 n 和 n 互质的数的个数。

比如说 phi(1) = 1 。

当 n 是质数的时候，显然有 phi(n) = n - 1 。

性质

- 欧拉函数是积性函数。 即对任意满足 gcd(a, b) = 1 的整数 a,b ，有 phi(ab) = phi(a)phi(b) 。 特别地，当 n 是奇数时 phi(2n) = phi(n) 。
- n = sum _(d | n)phi(d) 。
- 若 n = pˆ(k) ，其中 p 是质数，那么 phi(n) = pˆ(k) - pˆ(k - 1) 。 （根据定义可知）
- 由唯一分解定理，设 n = prod (i=1 ~ s)p[i]ˆ(k[i]) ，其中 p[i] 是质数，有 phi(n) = n * prod _(i = 1)ˆ(s)(p[i] - 1)/(p[i]) 。
- 对任意不全为 0 的整数 m,n ， phi(mn)phi(gcd(m,n))=phi(m)phi(n)gcd(m,n) 。 可由上一条直接计算得出。

实现

如果只要求一个数的欧拉函数值，那么直接根据定义质因数分解的同时求就好了。这个过程可以用 Pollard Rho 算法优化。

示例代码

```
#include <cmath>

int euler_phi(int n) {
int ans = n;
for (int i = 2; i * i <= n; i++)
if (n % i == 0) {
ans = ans / i * (i - 1);
while (n % i == 0) n /= i;
}
if (n > 1) ans = ans / n * (n - 1);
return ans;
}
```

如果是多个数的欧拉函数值，可以利用后面会提到的线性筛法来求得。

例题：

Status
Problem
Tags
P12216
.
互质

欧拉函数
容斥
P2303
.
Longge 的问题
欧拉函数
容斥原理
筛法进阶

数学基础 04 · 筛法进阶

素数筛法

引入

如果我们想要知道小于等于 n 有多少个素数呢？
一个自然的想法是对于小于等于 n 的每个数进行一次质数检验。这种暴力的做法显然不能达到最优复杂度。

埃拉托斯特尼筛法

过程

考虑这样一件事情：对于任意一个大于 1 的正整数 n ，那么它的 x 倍就是合数（ x > 1 ）。利用这个结论，我们可以避免很多次不必要的检测。
如果我们从小到大考虑每个数，然后同时把当前这个数的所有（比自己大的）倍数记为合数，那么运行结束的时候没有被标记的数就是素数了。

实现

```
vector<int> prime;
bool is_prime[N];

void Eratosthenes(int n) {
is_prime[0] = is_prime[1] = false;
for (int i = 2; i <= n; ++i) is_prime[i] = true;
for (int i = 2; i <= n; ++i) {
if (is_prime[i]) {
prime.push_back(i);
if ((long long)i * i > n) continue;
for (int j = i * i; j <= n; j += i)
// 因为从 2 到 i - 1 的倍数我们之前筛过了，这里直接从 i
// 的倍数开始，提高了运行速度
is_prime[j] = false;  // 是 i 的倍数的均不是素数
}
}
}
```

以上为 Eratosthenes 筛法（埃拉托斯特尼筛法，简称埃氏筛法），时间复杂度是 O(nloglog n)。
证明
现在我们就来看看推导过程：
如果每一次对数组的操作花费 1 个单位时间，则时间复杂度为：
O(sum (k=1 ~ pi(n))(n)/(p[k]))=O(n sum (k=1 ~ pi(n))(1)/(p[k]))
其中 p[k] 表示第 k 小的素数， pi(n) 表示 <= n 的素数个数。 sum (k=1 ~ pi(n)) 表示第一层 for 循环，其中累加上界 pi(n) 为 `if (prime[i])` 进入 true 分支的次数； (n)/(p[k]) 表示第二层 for 循环的执行次数。
根据 Mertens 第二定理，存在常数 B[1] 使得：
sum (k=1 ~ pi(n))(1)/(p[k])=loglog n+B[1]+O((1)/(log n))

所以 Eratosthenes 筛法 的时间复杂度为 O(nloglog n) 。接下来我们证明 Mertens 第二定理的弱化版本 sum _(k <= pi(n))1/p[k]=O(loglog n) ：
根据 pi(n)=Theta(n/log n) ，可知第 n 个素数的大小为 Theta(nlog n) 。于是就有
sum (k=1 ~ pi(n))(1)/(p[k]) =O(sum (k=2 ~ pi(n))(1)/(klog k)) =O(int [2]^(pi(n))(mathrm dx)/(xlog x)) =O(loglogpi(n))=O(loglog n)
当然，上面的做法效率仍然不够高效，应用下面几种方法可以稍微提高算法的执行效率。

筛至平方根

显然，要找到直到 n 为止的所有素数，仅对不超过 sqrt (n) 的素数进行筛选就足够了。

```
vector<int> prime;
bool is_prime[N];

void Eratosthenes(int n) {
is_prime[0] = is_prime[1] = false;
for (int i = 2; i <= n; ++i) is_prime[i] = true;
// i * i <= n 说明 i <= sqrt(n)
for (int i = 2; i * i <= n; ++i) {
if (is_prime[i])
for (int j = i * i; j <= n; j += i) is_prime[j] = false;
}
for (int i = 2; i <= n; ++i)
if (is_prime[i]) prime.push_back(i);
}
```

这种优化不会影响渐进时间复杂度，实际上重复以上证明，我们将得到 n ln ln sqrt (n) + o(n) ，根据对数的性质，它们的渐进相同，但操作次数会明显减少。

只筛奇数

因为除 2 以外的偶数都是合数，所以我们可以直接跳过它们，只用关心奇数就好。
首先，这样做能让我们内存需求减半；其次，所需的操作大约也减半。

减少内存的占用

我们注意到筛选时只需要 `bool` 类型的数组。`bool` 数组的一个元素一般占用 1 字节（即 8 比特），但是存储一个布尔值只需要 1 个比特就足够了。
我们可以使用位运算的相关知识，将每个布尔值压到一个比特位中，这样我们仅需使用 n 比特（即 (n)/(8) 字节）而非 n 字节，可以显著减少内存占用。这种方式被称为「位级压缩」。
值得一提的是，存在自动执行位级压缩的数据结构，如 C++ 中的 `vector<bool>` 和 `bitset<>` 。
另外，`vector<bool>` 和 `bitset<>` 对程序有常数优化，时间复杂度 O(n log log n) 的埃氏筛在使用 `bitset<>` 或 `vector<bool>` 优化后，性能甚至超过时间复杂度 O(n) 的欧拉筛。

分块筛选

由优化「筛至平方根」可知，不需要一直保留整个 `is_prime[1...n]` 数组。为了进行筛选，只保留到 sqrt (n) 的素数就足够了，即 `prime[1...sqrt(n)]` 。并将整个范围分成块，每个块分别进行筛选。这样，我们就不必同时在内存中保留多个块，而且 CPU 可以更好地处理缓存。
设 s 是一个常数，它决定了块的大小，那么我们就有了 ceil (n)/(s) 个块，而块 k (k = 0 … floor (n)/(s)) 包含了区间 [ks, ks + s - 1] 中的数字。我们可以依次处理块，也就是说，对于每个块 k ，我们将遍历所有质数（从 1 到 sqrt (n) ）并使用它们进行筛选。
值得注意的是，我们在处理第一个数字时需要稍微修改一下策略：首先，应保留 [1, sqrt (n)] 中的所有的质数；第二，数字 0 和 1 应该标记为非素数。在处理最后一个块时，不应该忘记最后一个数字 n 并不一定位于块的末尾。
以下实现使用块筛选来计算小于等于 n 的质数数量。

```
int count_primes(int n) {
constexpr static int S = 10000;
vector<int> primes;
int nsqrt = sqrt(n);
vector<char> is_prime(nsqrt + 1, true);
for (int i = 2; i <= nsqrt; i++) {
if (is_prime[i]) {
primes.push_back(i);
for (int j = i * i; j <= nsqrt; j += i) is_prime[j] = false;
}
}
int result = 0;
vector<char> block(S);
for (int k = 0; k * S <= n; k++) {
fill(block.begin(), block.end(), true);
int start = k * S;
for (int p : primes) {
int start_idx = (start + p - 1) / p;
int j = max(start_idx, p) * p - start;
for (; j < S; j += p) block[j] = false;
}
if (k == 0) block[0] = block[1] = false;
for (int i = 0; i < S && start + i <= n; i++) {
if (block[i]) result++;
}
}
return result;
}
```

分块筛法的渐进时间复杂度与埃氏筛法是一样的（除非块非常小），但是所需的内存将缩小为 O(sqrt (n) + S) ，并且有更好的缓存结果。

另一方面，对于每一对块和区间 [1, sqrt (n)] 中的素数都要进行除法，而对于较小的块来说，这种情况要糟糕得多。

因此，在选择常数 S 时要保持平衡。

块大小 S 取 10^4 到 10^5 之间，可以获得最佳的速度。

线性筛法

埃氏筛法仍有优化空间，它会将一个合数重复多次标记。有没有什么办法省掉无意义的步骤呢？答案是肯定的。

如果能让每个合数都只被标记一次，那么时间复杂度就可以降到 O(n) 了。

	i	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
vis	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
prm	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

欧拉筛

```
vector<int> pri;
bool not_prime[N];

void pre(int n) {
for (int i = 2; i <= n; ++i) {
if (!not_prime[i]) {
pri.push_back(i);
}
for (int pri_i : pri) {
if (i * pri_i > n) break;
not_prime[i * pri_i] = true;
if (i % pri_i == 0) {
// i % pri_i == 0
// 换言之，i 之前被 pri_i 筛过了
// 由于 pri 里面质数是从小到大的，所以 i 乘上其他的质数的结果一定会被
// pri_i 的倍数筛掉，就不需要在这里先筛一次，所以这里直接 break
// 掉就好了
break;
}
}
}
}
```

上面的这种 **线性筛法** 也称为 Euler **筛法**（欧拉筛法）。

???+ note

注意到筛法求素数的同时也得到了每个数的最小质因子。

筛法求欧拉函数

注意到在线性筛中，每一个合数都是被最小的质因子筛掉。比如设 p[1] 是 n 的最小质因子， n' = (n)/(p[1]) ，那么线性筛的过程中 n 通过 n' * p[1] 筛掉。

观察线性筛的过程，我们还需要处理两个部分，下面对 n' mod p[1] 分情况讨论。

如果 n' mod p[1] = 0 ，那么 n' 包含了 n 的所有质因子。

phi(n) = n * prod *(i = 1)^(s)(p[i] - 1)/(p[i]) = p[1] * n' * prod* (i = 1)^(s)(p[i] - 1)/(p[i]) = p[1] * phi(n')

那如果 n' mod p[1] != 0 呢，这时 n' 和 p[1] 是互质的，根据欧拉函数性质，我们有：

phi(n) = phi(p[1]) * phi(n') = (p[1] - 1) * phi(n')

实现

```
vector<int> pri;
bool not_prime[N];
int phi[N];

void pre(int n) {
phi[1] = 1;
for (int i = 2; i <= n; i++) {
if (!not_prime[i]) {
pri.push_back(i);
phi[i] = i - 1;
}
for (int pri_j : pri) {
if (i * pri_j > n) break;
not_prime[i * pri_j] = true;
if (i % pri_j == 0) {
phi[i * pri_j] = phi[i] * pri_j;
break;
}
phi[i * pri_j] = phi[i] * phi[pri_j];
}
}
}
```

筛法求莫比乌斯函数

定义

根据莫比乌斯函数的定义，设 n 是一个合数， p[1] 是 n 的最小质因子， n'=(n)/(p[1]) ，有：

mu(n)= 0 n' mod p[1] = 0 -mu(n') otherwise

若 n 是质数，有 mu(n)=-1 。

实现

```
vector<int> pri;
bool not_prime[N];
int mu[N];

void pre(int n) {
mu[1] = 1;
for (int i = 2; i <= n; ++i) {
if (!not_prime[i]) {
mu[i] = -1;
pri.push_back(i);
}
for (int pri_j : pri) {
if (i * pri_j > n) break;
not_prime[i * pri_j] = true;
if (i % pri_j == 0) {
mu[i * pri_j] = 0;
break;
}
mu[i * pri_j] = -mu[i];
}
}
}
```

筛法求约数个数

用 d[i] 表示 i 的约数个数， num[i] 表示 i 的最小质因子出现次数。

约数个数定理

定理：若 n= prod *(i=1)^(m) p[i]^(c[i])* 则 *d[i]= prod* (i=1)^(m) (c[i]+1) 。

证明：我们知道 p[i]^(c[i]) 的约数有 p[i]^0,p[i]^1,… ,p[i]^(c[i]) 共 c[i]+1 个，根据乘法原理， n 的约数个数就是 prod _(i=1)^(m) (c[i]+1) 。

实现

因为 d[i] 是积性函数，所以可以使用线性筛。

在这里简单介绍一下线性筛实现原理。

1. 当 i 为质数时， num[i] <- 1,d[i] <- 2 ，同时设 q = floor (i)/(p) ，其中 p 为 i 的最小质因子。
2. 当 p 为 q 的质因子时， num[i] <- num_(q) + 1,d[i] <- (d_(q))/(num[i]) * (num[i] + 1) 。
3. 当 p,q 互质时， num[i] <- 1,d[i] <- d_(q) * (num[i]+1) 。

```
vector<int> pri;
bool not_prime[N];
int d[N], num[N];

void pre(int n) {
d[1] = 1;
for (int i = 2; i <= n; ++i) {
if (!not_prime[i]) {
pri.push_back(i);
d[i] = 2;
num[i] = 1;
}
for (int pri_j : pri) {
if (i * pri_j > n) break;
not_prime[i * pri_j] = true;
if (i % pri_j == 0) {
num[i * pri_j] = num[i] + 1;
d[i * pri_j] = d[i] / num[i * pri_j] * (num[i * pri_j] + 1);
break;
}
num[i * pri_j] = 1;
d[i * pri_j] = d[i] * 2;
}
}
}
```

筛法求约数和

f[i] 表示 i 的约数和， g[i] 表示 i 的最小质因子的 p^0+p^1+p^2+… p^(k) .

实现

```
vector<int> pri;
bool not_prime[N];
int g[N], f[N];

void pre(int n) {
g[1] = f[1] = 1;
for (int i = 2; i <= n; ++i) {
if (!not_prime[i]) {
pri.push_back(i);
g[i] = i + 1;
f[i] = i + 1;
}
for (int pri_j : pri) {
if (i * pri_j > n) break;
not_prime[i * pri_j] = true;
if (i % pri_j == 0) {
g[i * pri_j] = g[i] * pri_j + 1;
f[i * pri_j] = f[i] / g[i] * g[i * pri_j];
break;
}
f[i * pri_j] = f[i] * f[pri_j];
g[i * pri_j] = 1 + pri_j;
}
}
}
```

例题：

Status

Problem

Tags

P1621

.

集合
筛法
P1835
.
素数密度
筛法
P2092
.
数字游戏
筛法
P2158
.
[SDOI2008] 仪仗队
筛法
欧拉函数
P2568
.
GCD
筛法
前缀和
欧拉函数
欧拉定理&费马小定理

数学基础 05 · 欧拉定理&费马小定理

费马小定理

定义

若 p 为素数， gcd(a, p) = 1 ，则 a^(p - 1) == 1 (mod p) 。
另一个形式：对于任意整数 a ，有 a^(p) == a (mod p) 。

证明

设一个质数为 p ，我们取一个不为 p 倍数的数 a 。
构造一个序列： A=1,2,3…,p-1 ，这个序列有着这样一个性质：
prod (i=1 ~ p-1)space A[i] == prod (i=1 ~ p-1) (A[i] * a) mod p
证明：
because (A[i],p)=1,(A[i] * a,p)=1
又因为每一个 A[i] * a mod p 都是独一无二的，且 A[i] * a mod p < p
得证（每一个 A[i] * a 都对应了一个 A[i] ）
设 f=(p-1)! , 则 f == a * A[1] * a * A[2] * a * A[3] … * A[p-1] mod p
a^(p-1) * f == f mod p a^(p-1) == 1 mod p
证毕。
也可用归纳法证明：
显然 1^(p) == 1 mod p ，假设 a^(p) == a mod p 成立，那么通过二项式定理有
(a+1)^(p)=a^(p)+C(p,1)a^(p-1)+C(p,2)a^(p-2)+... +C(p,p-1)a+1
因为 C(p,k)=(p(p-1)... (p-k+1))/(k!) 对于 1 <= k <= p-1 成立，在模 p 意义下 C(p,1) == C(p,2) == ... == C(p,p-1) == 0 mod p ，那么 (a+1)^(p) == a^(p) +1 mod p ，将 a^(p) == a mod p 带入得 (a+1)^(p) == a+1 mod p 得证。

欧拉定理

在了解欧拉定理（Euler's theorem）之前，请先了解欧拉函数。定理内容如下：

定义

若 gcd(a, m) = 1 ，则 a^(phi(m)) == 1 (mod m) 。

证明

实际上这个证明过程跟上文费马小定理的证明过程是非常相似的：**构造一个与 m 互质的数列**，再进行操作。

设 r[1], r[2], ..., r_(phi(m)) 为模 m 意义下的一个简化剩余系，则 ar[1], ar[2], ..., ar_(phi(m)) 也为模 m 意义下的一个简化剩余系。所以 r[1]r[2] ... r_(phi(m)) == ar[1] • ar[2] ... ar_(phi(m)) == a^(phi(m))r[1]r[2] ... r_(phi(m)) (mod m) ，可约去 r[1]r[2] ... r_(phi(m)) ，即得 a^(phi(m)) == 1 (mod m) 。

当 m 为素数时，由于 phi(m) = m - 1 ，代入欧拉定理可立即得到费马小定理。

筛法进阶

同余方程

数学基础 06 • 同余方程

同余符号

形如″ == ″ 的符号称作同余符号，对于以下式子：

a == b mod n

其意义为： a 除以 n 的余数等于 b 除以 n 的余数。

同余方程定义

形如

ax == b mod n

的方程称为 **线性同余方程**（Linear Congruence Equation）。其中， a 、 b 和 n 为给定整数， x 为未知数。需要从区间 [0, n-1] 中求解 x ，当解不唯一时需要求出全体解。

用逆元求解

首先考虑简单的情况，当 a 和 n 互素（coprime 或 relatively prime）时，即 gcd(a, n) = 1 。此时可以计算 a 的逆元，并将方程的两边乘以 a 的逆元，可以得到唯一解。

证明

x == ba ^ - 1 mod n

接下来考虑 a 和 n 不互素（not coprime），即 gcd(a, n) != 1 的情况。此时不一定有解。例如，2x == 1 mod 4 没有解。

设 g = gcd(a, n) ，即 a 和 n 的最大公约数，其中 a 和 n 在本例中大于 1。

当 b 不能被 g 整除时无解。此时，对于任意的 x ，方程 ax == b mod n 的左侧始终可被 g 整除，而右侧不可被 g 整除，因此无解。

如果 g 整除 b ，则通过将方程两边 a 、 b 和 n 除以 g ，得到一个新的方程：

a^(')x == b^(') (mod n^('))

其中 a^(') 和 n^(') 已经互素，这种情形已经解决，于是得到 x^(') 作为 x 的解。

很明显， x^(') 也将是原始方程的解。这不是唯一的解。可以看出，原始方程有如下 g 个解：

x[i] == (x^(') + i • n^(')) mod n for i = 0 … g-1

总之，线性同余方程的 **解的数量** 等于 g = gcd(a, n) 或等于 0 。

用扩展欧几里得算法求解

根据以下两个定理，可以求出线性同余方程 ax == b mod n 的解。

定理 1：线性同余方程 ax == b mod n 可以改写为如下线性不定方程：

ax + nk = b

其中 x 和 k 是未知数。这两个方程是等价的，有整数解的充要条件为 gcd(a,n) | b 。

应用扩展欧几里德算法可以求解该线性不定方程。根据定理 1，对于线性不定方程 ax+nk=b ，可以先用扩展欧几里得算法求出一组 x[0],k[0] ，也就是 ax[0]+nk[0]=gcd(a,n) ，然后两边同时除以 gcd(a,n) ，再乘 b 。就得到了方程

a(b)/(gcd(a,n))x[0]+n(b)/(gcd(a,n))k[0]=b
于是找到方程的一个解。
定理 2：若 gcd(a,n)=1 ，且 x[0] 、 k[0] 为方程 ax+nk=b 的一组解，则该方程的任意解可表示为：
x=x[0]+nt
k=k[0]-at
并且对任意整数 t 都成立。
根据定理 2，可以从已求出的一个解，求出方程的所有解。实际问题中，往往要求出一个最小整数解，也就是一个特解
x=(x mod t+t) mod t
其中有
t=(n)/(gcd(a,n))
如果仔细考虑，用扩展欧几里得算法求解与用逆元求解，两种方法是等价的。
实现

```
int ex_gcd(int a, int b, int& x, int& y) {
if (b == 0) {
x = 1;
y = 0;
return a;
}
int d = ex_gcd(b, a % b, x, y);
int temp = x;
x = y;
y = temp - a / b * y;
return d;
}

bool liEu(int a, int b, int c, int& x, int& y) {
int d = ex_gcd(a, b, x, y);
if (c % d != 0) return false;
int k = c / d;
x *= k;
y *= k;
return true;
}
```

欧拉定理&费马小定理
乘法逆元

数学基础 07 • 乘法逆元

定义
如果一个线性同余方程 ax == 1 mod b ，则 x 称为 a mod b 的逆元，记作 a^(-1) 。
如何求逆元
扩展欧几里得法

```
void exgcd(int a, int b, int& x, int& y) {
if (b == 0) {
x = 1; y = 0;
return;
}
exgcd(b, a % b, y, x);
y -= a / b * x;
}
```

扩展欧几里得法和求解线性同余方程是一个原理，在这里不展开解释。
快速幂法

证明

因为 ax == 1 mod b ；

所以 ax == a^(b-1) mod b （根据费马小定理）；

所以 x == a^(b-2) mod b 。

然后我们就可以用快速幂来求了。

```
int qpow(long long a, int b) {
int ans = 1;
a = (a % p + p) % p;
for (; b; b >>= 1) {
if (b & 1) ans = (a * ans) % p;
a = (a * a) % p;
}
return ans;
}
```

注意：快速幂法使用了费马小定理，要求 b 是一个素数；而扩展欧几里得法只要求 gcd(a, b) = 1 。

线性求逆元

求出 1,2,…,n 中每个数关于质数 p 的逆元。

如果对于每个数进行单次求解，以上两种方法就显得慢了，很有可能超时，所以下面来讲一下如何线性（ O(n) ）求逆元。

首先，很显然的 1^(-1) == 1 mod p ；

???+ note "证明"

对于任意 p 属于 Z ，有 1 * 1 == 1 mod p 恒成立，故在 p 下 1 的逆元是 1 ，而这是推算出其他情况的基础。

其次对于递归情况 i^(-1) ，我们令 k = floor (p)/(i) ， j = p mod i ，有 p = ki + j 。再放到 mod p 意义下就会得到： ki+j == 0 mod p ；

当 p 为质数时，可知 j 在模 p 意义下的乘法逆元必定存在。这时在上式 ki+j == 0 mod p 的两边同时乘 i^(-1) * j^(-1) ：

kj^(-1)+i^(-1) == 0 mod p

i^(-1) == -kj^(-1) mod p

再带入 j = p mod i ，有 p = ki + j ，有：

i^(-1) == - floor (p)/(i) (p mod i)^(-1) mod p

我们注意到 p mod i < i ，而在迭代中我们完全可以假设我们已经知道了所有的模 p 下的逆元 j^(-1)，j < i 。

故我们就可以推出逆元，利用递归的形式，而使用迭代实现：

i^(-1) == 1, if i = 1, - floor (p)/(i) (p mod i)^(-1), otherwise. mod p

```
inv[1] = 1;
for (int i = 2; i <= n; ++i) {
inv[i] = (long long)(p - p / i) * inv[p % i] % p;
}
```

使用 p- floor (p)/(i) 来防止出现负数。

当 p 不为质数

线性同余方程中指出，如果 i 与 p 不互素时不存在相应的逆元。如果 p 不为质数，则至少有一个 i 的逆元不存在。此时这个建立在递推式上的方法就不能保证结果的正确性。例如当 p = 8 且 i = 3 时，根据递推式，逆元需要从 `inv[p % i]` 即 `inv[2]` 计算。而 2 在模 8 意义下不存在逆元，`inv[2]` 的值是未定义的（准确来讲，递推时 `inv[2]` 依赖于 `inv[0]` 的初始值）。因此 3 在模 8 意义下的乘法逆元便无法正确求出。

另外，根据线性求逆元方法的式子： i^(-1) == -kj^(-1) mod p

递归求解 j^(-1) ，直到 j=1 返回 1 。
中间优化可以加入一个记忆化来避免多次递归导致的重复，这样求 1,2,…,n 中所有数的逆元的时间复杂度仍是 O(n) 。
注意：如果用以上给出的式子递归进行单个数的逆元求解，目前已知的时间复杂度的上界为 O(n^((1)/(3)))，具体请看知乎讨论。算法竞赛中更好地求单个数的逆元的方法有扩展欧几里得法和快速幂法。
线性求任意 n 个数的逆元
上面的方法只能求 1 到 n 的逆元，如果需要求任意给定 n 个数（ 1 <= a[i] < p ）的逆元，就需要下面的方法：
首先计算 n 个数的前缀积，记为 s[i] ，然后使用快速幂或扩展欧几里得法计算 s[n] 的逆元，记为 sv[n] 。
因为 sv[n] 是 n 个数的积的逆元，所以当我们把它乘上 a[n] 时，就会和 a[n] 的逆元抵消，于是就得到了 a[1] 到 a[n-1] 的积逆元，记为 sv[n-1] 。
同理我们可以依次计算出所有的 sv[i] ，于是 a[i]^(-1) 就可以用 s[i-1] * sv[i] 求得。
所以我们就在 O(n + log p) 的时间内计算出了 n 个数的逆元。

```
s[0] = 1;
for (int i = 1; i <= n; ++i) s[i] = s[i - 1] * a[i] % p;
sv[n] = qpow(s[n], p - 2);
// 当然这里也可以用 exgcd 来求逆元，视个人喜好而定
for (int i = n; i >= 1; --i) sv[i - 1] = sv[i] * a[i] % p;
for (int i = 1; i <= n; ++i) inv[i] = sv[i] * s[i - 1] % p;
```

例题：
Status
Problem
Tags
【模板】模意义下的乘法逆元
.
逆元
组合数问题
.
逆元
[SDOI2016] 排列计数
.
逆元
同余方程
裴蜀定理

数学基础 08 • 裴蜀定理

定义
裴蜀定理，又称贝祖定理（Bézout's lemma）、贝祖等式（Bézout's identity）。是一个关于最大公约数的定理。
其内容是：
设 a,b 是不全为零的整数，对任意整数 x,y ，满足 gcd(a,b) | ax+by ，且存在整数 x,y ，使得 ax+by=gcd(a,b) .
证明
对于第一点
由于 gcd(a,b) | a,gcd(a,b) | b

所以 gcd(a,b) | ax,gcd(a,b) | by ，其中 x,y 均为整数
因此 gcd(a,b) | ax+by
对于第二点
1. 若任何一个等于 0 ，则 gcd(a,b)=a . 这时定理显然成立。
2. 若 a,b 不等于 0 . 由于 gcd(a,b)=gcd(a,-b) ，

不妨设 a,b 都大于 0 ， a >= b,gcd(a,b)=d .
对 ax+by=d ，两边同时除以 d ，可得 a[1]x+b[1]y=1 ，其中 (a[1],b[1])=1 .
转证 a[1]x+b[1]y=1 .
我们先回顾一下辗转相除法是怎么做的，由 gcd(a, b) -> gcd(b,a mod b) -> … 我们把模出来的数据叫做 r 于是，有
gcd(a[1],b[1])=gcd(b[1],r[1])=gcd(r[1],r[2])=...=(r[n-1],r[n])=1
把辗转相除法中的运算展开，做成带余数的除法，得
a[1] = q[1]b[1]+r[1] (0 <= r[1]<b[1])
b[1] = q[2]r[1]+r[2] (0 <= r[2]<r[1])
r[1] = q[3]r[2]+r[3] (0 <= r[3]<r[2])
...
r[n-3] = q[n-1]r[n-2]+r[n-1]
r[n-2] = q[n]r[n-1]+r[n]
r[n-1] = q[n+1]r[n]
不妨令辗转相除法在除到互质的时候退出则 r[n]=1 所以有（ q 被换成了 x ，为了符合最终形式）
r[n-2]=x[n]r[n-1]+1
即
1=r[n-2]-x[n]r[n-1]
由倒数第三个式子 r[n-1]=r[n-3]-x[n-1]r[n-2] 代入上式，得
1=(1+x[n]x[n-1])r[n-2]-x[n]r[n-3]
然后用同样的办法用它上面的等式逐个地消去 r[n-2],...,r[1] ，
可证得 1=a[1]x+b[1]y .
这样等于是一般式中 d=1 的情况。
推广
逆定理
设 a, b 是不全为零的整数，若 d > 0 是 a, b 的公因数，且存在整数 x, y ，使得 ax+by=d ，则 d = gcd(a, b) 。
特殊地，设 a, b 是不全为零的整数，若存在整数 x, y ，使得 ax+by=1 ，则 a, b 互质。
多个整数
裴蜀定理可以推广到 n 个整数的情形：设 a[1], a[2], …, a[n] 是不全为零的整数，则存在整数 x[1], x[2], …, x[n] ，使得 a[1] x[1] + a[2] x[2] + ... + a[n] x[n]=gcd(a[1], a[2], …, a[n]) 。其逆定理也成立：设 a[1], a[2], …, a[n] 是不全为零的整数， d > 0 是 a[1], a[2], …, a[n] 的公因数，若存在整数 x[1], x[2], …, x[n] ，使得 a[1] x[1] + a[2] x[2] + ... + a[n] x[n]=d ，则 d = gcd(a[1], a[2], …, a[n]) 。
乘法逆元
中国剩余定理

数学基础 09 • 中国剩余定理

引入

> 「物不知数」问题：有物不知其数，三三数之剩二，五五数之剩三，七七数之剩二。问物几何？

即求满足以下条件的整数：除以 3 余 2 ，除以 5 余 3 ，除以 7 余 2 。
该问题最早见于《孙子算经》中，并有该问题的具体解法。宋朝数学家秦九韶于 1247 年《数书九章》卷一、二《大衍类》对「物不知数」问题做出了完整系统的解答。上面具体问题的解答口诀由明朝数学家程大位在《算法统宗》中给出：

> 三人同行七十希，五树梅花廿一支，七子团圆正半月，除百零五便得知。

2 * 70+3 * 21+2 * 15=233=2 * 105+23 ，故答案为 23 。

定义

中国剩余定理 (Chinese Remainder Theorem, CRT) 可求解如下形式的一元线性同余方程组（其中 n[1], n[2], ..., n[k] 两两互质）：
x == a[1] (mod n[1]) x == a[2] (mod n[2]) ... x == a[k] (mod n[k])
上面的「物不知数」问题就是一元线性同余方程组的一个实例。

过程

1. 计算所有模数的积 n ；
2. 对于第 i 个方程：
3. 计算 m[i]=(n)/(n[i]) ；
4. 计算 m[i] 在模 n[i] 意义下的逆元 m[i]^(-1) ；
5. 计算 c[i]=m[i]m[i]^(-1) （**不要对 n[i] 取模**）。
6. 方程组在模 n 意义下的唯一解为： x= sum _(i=1)^(k) a[i]c[i] mod n 。

实现

```
LL CRT(int k, LL* a, LL* r) {
LL n = 1, ans = 0;
for (int i = 1; i <= k; i++) n = n * r[i];
for (int i = 1; i <= k; i++) {
LL m = n / r[i], b, y;
exgcd(m, r[i], b, y);  // b * m mod r[i] = 1
ans = (ans + a[i] * m * b % n) % n;
}
return (ans % n + n) % n;
}
```

证明

我们需要证明上面算法计算所得的 x 对于任意 i=1,2,...,k 满足 x == a[i] (mod n[i]) 。
当 i != j 时，有 m[j] == 0 (mod n[i]) ，故 c[j] == m[j] == 0 (mod n[i]) 。又有 c[i] == m[i] • (m[i]^(-1) mod n[i]) == 1 (mod n[i]) ，所以我们有：
x == sum _(j=1)^(k) a[j]c[j] (mod n[i]) == a[i]c[i] (mod n[i]) == a[i] • m[i] • (m^(-1)[i] mod n[i]) (mod n[i]) == a[i] (mod n[i])
即对于任意 i=1,2,...,k ，上面算法得到的 x 总是满足 x == a[i] (mod n[i]) ，即证明了解同余方程组的算法的正确性。
因为我们没有对输入的 a[i] 作特殊限制，所以任何一组输入 a[i] 都对应一个解 x 。
另外，若 x != y ，则总存在 i 使得 x 和 y 在模 n[i] 下不同余。
故系数列表 a[i] 与解 x 之间是一一映射关系，方程组总是有唯一解。

解释

下面演示 CRT 如何解「物不知数」问题。

1. n=3 * 5 * 7=105 ；
2. 三人同行 七十 希： n[1]=3, m[1]=n/n[1]=35, m[1]^(-1) == 2 mod 3 ，故 c[1]=35 * 2=70 ；

3. 五树梅花 廿一 支： n[2]=5, m[2]=n/n[2]=21, m[2]^(-1) == 1 mod 5 ，故 c[2]=21 * 1=21 ；
4. 七子团圆正 半月： n[3]=7, m[3]=n/n[3]=15, m[3]^(-1) == 1 mod 7 ，故 c[3]=15 * 1=15 ；
5. 所以方程组的唯一解为 x == 2 * 70+3 * 21+2 * 15 == 233 == 23 (mod 105) 。（除 百零五便得知）

例题：

Status
Problem
Tags
P1495
.
【模板】中国剩余定理（CRT）/ 曹冲养猪
中国剩余定理
P1516
.
青蛙的约会
中国剩余定理
P3868
.
[TJOI2009] 猜数字
中国剩余定理
P3951
.
[NOIP 2017 提高组] 小凯的疑惑
中国剩余定理
P4409
.
[ZJOI2006] 皇帝的烦恼
中国剩余定理
裴蜀定理
矩阵与矩阵快速幂

数学基础 10・矩阵与矩阵快速幂

向量与矩阵

在线性代数中，向量分为列向量和行向量。
线性代数的主要研究对象是列向量，约定使用粗体小写字母表示列向量。在用到大量向量与矩阵的线性代数中，不引起混淆的情况下，在手写时，字母上方的向量记号可以省略不写。
向量也是特殊的矩阵。如果想要表示行向量，需要在粗体小写字母右上方写转置记号。行向量在线性代数中一般表示方程。

引入

矩阵的引入来自于线性方程组。与向量类似，矩阵体现了一种对数据「打包处理」的思想。
例如，将线性方程组：
7x[1]+8x[2]+9x[3]=13 4x[1]+5x[2]+6x[3]=12 x[1]+2x[2]+3x[3]=11
一般用圆括号或方括号表示矩阵。将上述系数抽出来，写成矩阵乘法的形式：
7 8 9 4 5 6 1 2 3x[1] x[2] x[3]=13 12 11

简记为：
Ax=b
即未知数列向量 x，左乘一个矩阵 A，得到列向量 b。这个式子可以认为是线性代数的基本形式。
线性代数主要研究的运算模型是内积。内积是先相乘再相加，是行向量左乘列向量，得到一个数的过程。
矩阵乘法是内积的拓展。矩阵乘法等价于左边矩阵抽出一行，与右边矩阵抽出一列进行内积，得到结果矩阵的对应元素，口诀「左行右列」。
当研究对象是右边的列向量时，矩阵乘法相当于对列向量进行左乘。在左乘的观点下，矩阵就是对列向量的变换，将矩阵乘法中右边矩阵的每一个列向量进行变换，对应地得到结果矩阵中每一个列向量。
矩阵可以对一个列向量进行变换，也可以对一组列向量进行「打包」变换，甚至可以对整个空间——即全体列向量进行变换。当矩阵被视为对整个空间变换的时候，也就脱离了空间，成为了纯粹变换的存在。
定义
对于矩阵 A ，主对角线是指 A_(i,i) 的元素。
一般用 I 来表示单位矩阵，就是主对角线上为 1，其余位置为 0。
同型矩阵
两个矩阵，行数与列数对应相同，称为同型矩阵。
方阵
行数等于列数的矩阵称为方阵。方阵是一种特殊的矩阵。对于「 n 阶矩阵」的习惯表述，实际上讲的是 n 阶方阵。阶数相同的方阵为同型矩阵。
研究方程组、向量组、矩阵的秩的时候，使用一般的矩阵。研究特征值和特征向量、二次型的时候，使用方阵。
主对角线
方阵中行数等于列数的元素构成主对角线。
对称矩阵
如果方阵的元素关于主对角线对称，即对于任意的 i 和 j ， i 行 j 列的元素与 j 行 i 列的元素相等，则将方阵称为对称矩阵。
对角矩阵
主对角线之外的元素均为 0 的方阵称为对角矩阵，一般记作：
diaglambda[1],...,lambda[n]
式中的 lambda[1],...,lambda[n] 是主对角线上的元素。
对角矩阵是对称矩阵。
如果对角矩阵的元素均为 1 ，称为单位矩阵，记为 I 。只要乘法可以进行，无论形状，任何矩阵乘单位矩阵仍然保持不变。
三角矩阵
如果方阵主对角线左下方的元素均为 0 ，称为上三角矩阵。如果方阵主对角线右上方的元素均为 0 ，称为下三角矩阵。
两个上（下）三角矩阵的乘积仍然是上（下）三角矩阵。如果对角线元素均非 0 ，则上（下）三角矩阵可逆，逆也是上（下）三角矩阵。
单位三角矩阵
如果上三角矩阵 A 的对角线全为 1 ，则称 A 是单位上三角矩阵。如果下三角矩阵 A 的对角线全为 1 ，则称 A 是单位下三角矩阵。
两个单位上（下）三角矩阵的乘积仍然是单位上（下）三角矩阵，单位上（下）三角矩阵的逆也是单位上（下）三角矩阵。
运算
矩阵的线性运算

矩阵的线性运算分为加减法与数乘，它们均为逐个元素进行。只有同型矩阵之间可以对应相加减。

矩阵的转置

矩阵的转置，就是在矩阵的右上角写上转置「T」记号，表示将矩阵的行与列互换。

对称矩阵转置前后保持不变。

矩阵乘法

矩阵的乘法是向量内积的推广。

矩阵相乘只有在第一个矩阵的列数和第二个矩阵的行数相同时才有意义。

设 A 为 P * M 的矩阵， B 为 M * Q 的矩阵，设矩阵 C 为矩阵 A 与 B 的乘积，

其中矩阵 C 中的第 i 行第 j 列元素可以表示为：

C_(i,j) = sum *(k=1)^(M)A*(i,k)B_(k,j)

在矩阵乘法中，结果 C 矩阵的第 i 行第 j 列的数，就是由矩阵 A 第 i 行 M 个数与矩阵 B 第 j 列 M 个数分别 **相乘再相加** 得到的。这里的 **相乘再相加**，就是向量的内积。乘积矩阵中第 i 行第 j 列的数恰好是乘数矩阵 A 第 i 个行向量与乘数矩阵 B 第 j 个列向量的内积，口诀为 **左行右列**。

线性代数研究的向量多为列向量，根据这样的对矩阵乘法的定义方法，经常研究对列向量左乘一个矩阵的左乘运算，同时也可以在这里看出「打包处理」的思想，同时处理很多个向量内积。

矩阵乘法满足结合律，不满足一般的交换律。

利用结合律，矩阵乘法可以利用快速幂的思想来优化。

在比赛中，由于线性递推式可以表示成矩阵乘法的形式，也通常用矩阵快速幂来求线性递推数列的某一项。

优化

首先对于比较小的矩阵，可以考虑直接手动展开循环以减小常数。

可以重新排列循环以提高空间局部性，这样的优化不会改变矩阵乘法的时间复杂度，但是会得到常数级别的提升。

```
// 以下文的参考代码为例
mat operator*(const mat& T) const {
mat res;
for (int i = 0; i < sz; ++i)
for (int j = 0; j < sz; ++j)
for (int k = 0; k < sz; ++k) {
res.a[i][j] += mul(a[i][k], T.a[k][j]);
res.a[i][j] %= MOD;
}
return res;
}

// 不如
mat operator*(const mat& T) const {
mat res;
int r;
for (int i = 0; i < sz; ++i)
for (int k = 0; k < sz; ++k) {
r = a[i][k];
for (int j = 0; j < sz; ++j)
res.a[i][j] += T.a[k][j] * r, res.a[i][j] %= MOD;
}
return res;
}
```

方阵的逆

方阵 A 的逆矩阵 P 是使得 A * P = I 的矩阵。

逆矩阵不一定存在。如果存在，可以使用高斯消元进行求解。

方阵的行列式

行列式是方阵的一种运算。

参考代码

一般来说，可以用一个二维数组来模拟矩阵。

```
struct mat {
LL a[sz][sz];

mat() { memset(a, 0, sizeof a); }

mat operator-(const mat& T) const {
mat res;
for (int i = 0; i < sz; ++i)
for (int j = 0; j < sz; ++j) {
res.a[i][j] = (a[i][j] - T.a[i][j]) % MOD;
}
return res;
}

mat operator+(const mat& T) const {
mat res;
for (int i = 0; i < sz; ++i)
for (int j = 0; j < sz; ++j) {
res.a[i][j] = (a[i][j] + T.a[i][j]) % MOD;
}
return res;
}

mat operator*(const mat& T) const {
mat res;
int r;
for (int i = 0; i < sz; ++i)
for (int k = 0; k < sz; ++k) {
r = a[i][k];
for (int j = 0; j < sz; ++j)
res.a[i][j] += T.a[k][j] * r, res.a[i][j] %= MOD;
}
return res;
}

mat operator^(LL x) const {
mat res, bas;
for (int i = 0; i < sz; ++i) res.a[i][i] = 1;
for (int i = 0; i < sz; ++i)
for (int j = 0; j < sz; ++j) bas.a[i][j] = a[i][j] % MOD;
while (x) {
if (x & 1) res = res * bas;
bas = bas * bas;
x >>= 1;
}
return res;
}
};
```

看待线性方程组的两种视角

看待矩阵 A，或者变换 A，有两种视角。

第一种观点：按行看，观察 A 的每一行。这样一来把 A 看作方程组。于是就有了消元法解方程的过程。

第二种观点：按列看，观察 A 的每一列。A 本身也是由列向量构成的。此时相当于把变换 A 本身看成了列向量组，而 x 是未知数系数，思考 A 当中的这组列向量能不能配上未知数，凑出列向量 b。

例如，文章开头的例子变为：

7 4 1x[1]+8 5 2x[2]+9 6 3x[3]=13 12 11

解方程变为研究，是否可以通过调整三个系数 x，使得给定的三个基向量能够凑出结果的向量。

按列看比按行看更新颖。在按列看的视角下，可以研究线性无关与线性相关。

矩阵乘法的应用

矩阵加速递推

以斐波那契数列（Fibonacci Sequence）为例。在斐波那契数列当中， F[1] = F[2] = 1 ， F[i] = F_(i - 1) + F_(i - 2)(i >= 3) 。

如果有一道题目让你求斐波那契数列第 n 项的值，最简单的方法莫过于直接递推了。但是如果 n 的范围达到了 10^(18) 级别，递推就不行了，此时我们可以考虑矩阵加速递推。

F[n-1] F[n-2] 1 1 1 0 = F[n] F[n-1]

定义初始矩阵 ans = F[2] F[1] = 1 1, base = 1 1

1 0 。那么， F[n] 就等于 ans base^(n-2) 这个矩阵的第一行第一列元素，也就是 1 1 1 1

1 0^(n-2) 的第一行第一列元素。

矩阵乘法不满足交换律，所以一定不能写成 1 1

1 0^(n-2) 1 1 的第一行第一列元素。另外，对于 n <= 2 的情况，直接输出 1 即可，不需要执行矩阵快速幂。

为什么要乘上 base 矩阵的 n-2 次方而不是 n 次方呢？因为 F[1], F[2] 是不需要进行矩阵乘法就能求的。也就是说，如果只进行一次乘法，就已经求出 F[3] 了。如果还不是很理解为什么幂是 n-2 ，建议手算一下。

下面是求斐波那契数列第 n 项对 10^9+7 取模的示例代码（核心部分）。

```
constexpr int mod = 1000000007;

struct Matrix {
int a[3][3];

Matrix() { memset(a, 0, sizeof a); }

Matrix operator*(const Matrix &b) const {
Matrix res;
for (int i = 1; i <= 2; ++i)
for (int j = 1; j <= 2; ++j)
for (int k = 1; k <= 2; ++k)
res.a[i][j] = (res.a[i][j] + a[i][k] * b.a[k][j]) % mod;
return res;
}
} ans, base;

void init() {
base.a[1][1] = base.a[1][2] = base.a[2][1] = 1;
ans.a[1][1] = ans.a[1][2] = 1;
}

void qpow(int b) {
while (b) {
if (b & 1) ans = ans * base;
base = base * base;
b >>= 1;
}
}

int main() {
int n = read();
if (n <= 2) return puts("1"), 0;
init();
qpow(n - 2);
println(ans.a[1][1] % mod);
}
```

这是一个稍微复杂一些的例子。
f[1] = f[2] = 0 f[n] = 7f[n-1]+6f[n-2]+5n+4 * 3^n
我们发现， f[n] 和 f[n-1], f[n-2], n 有关，于是考虑构造一个矩阵描述状态。
但是发现如果矩阵仅有这三个元素 f[n] f[n-1] n 是难以构造出转移方程的，因为乘方运算和 +1 无法用矩阵描述。
于是考虑构造一个更大的矩阵。
f[n] f[n-1] n 3^n 1
我们希望构造一个递推矩阵可以转移到
f[n+1] f[n] n+1 3^(n+1) 1
转移矩阵即为
7 1 0 0 0 6 0 0 0 0 5 0 1 0 0 12 0 0 3 0 5 0 1 0 1
定长路径统计
问题描述
给一个 n 阶有向图，每条边的边权均为 1 ，然后给一个整数 k ，你的任务是对于所有点对 (u,v) 求出从 u 到 v 长度为 k 的路径的数量（不一定是简单路径，即路径上的点或者边可能走多次）。
我们将这个图用邻接矩阵 G （对于图中的边 (u -> v) ，令 G[u,v]=1 ，其余为 0 的矩阵；如果有重边，则设 G[u,v] 为重边的数量）表示这个有向图。下述算法同样适用于图有自环的情况。
显然，该邻接矩阵对应 k=1 时的答案。
假设我们知道长度为 k 的路径条数构成的矩阵，记为矩阵 C[k] ，我们想求 C[k+1] 。显然有 DP 转移方程
C[k+1][i,j] = sum (p = 1 ~ n) C[k][i,p] • G[p,j]
我们可以把它看作矩阵乘法的运算，于是上述转移可以描述为
C[k+1] = C[k] • G
那么把这个递推式展开可以得到
C[k] = underbraceG • G ... G_(k 次) = G^(k)
要计算这个矩阵幂，我们可以使用快速幂（二进制取幂）的思想，在 O(n^3 log k) 的复杂度内计算结果。
例题：
Status
Problem
Tags
Pyundou/luogu/P3390
.
【模板】矩阵快速幂
矩阵快速幂
Pyundou/luogu/P2233
.
[HNOI2002] 公交车路线
矩阵快速幂
Pyundou/luogu/P1962
.
斐波那契数列
矩阵快速幂
Pyundou/luogu/P1707
.
刷题比赛
矩阵快速幂

中国剩余定理
双向搜索

搜索 01 • 双向搜索

本页面将简要介绍两种双向搜索算法：「双向同时搜索」和「Meet in the middle」。

双向同时搜索

定义

双向同时搜索的基本思路是从状态图上的起点和终点同时开始进行广搜或深搜。
如果发现搜索的两端相遇了，那么可以认为是获得了可行解。

过程

双向广搜的步骤：

```
将开始结点和目标结点加入队列 q
标记开始结点为 1
标记目标结点为 2
while (队列 q 不为空)
{
从 q.front() 扩展出新的 s 个结点

如果 新扩展出的结点已经被其他数字标记过
那么 表示搜索的两端碰撞
那么 循环结束

如果 新的 s 个结点是从开始结点扩展来的
那么 将这个 s 个结点标记为 1 并且入队 q

如果 新的 s 个结点是从目标结点扩展来的
那么 将这个 s 个结点标记为 2 并且入队 q
}
```

Meet in the middle

本节要介绍的不是 **二分搜索**（二分搜索的另外一个译名为「折半搜索」）。

引入

Meet in the middle 算法没有正式译名，常见的翻译为「折半搜索」、「双向搜索」或「中途相遇」。
它适用于输入数据较小，但还没小到能直接使用暴力搜索的情况。

过程

Meet in the middle 算法的主要思想是将整个搜索过程分成两半，分别搜索，最后将两半的结果合并。

性质

暴力搜索的复杂度往往是指数级的，而改用 meet in the middle 算法后复杂度的指数可以减半，即让复杂度从 O(a^(b)) 降到 O(a^(b/2)) 。

例题

有 n 盏灯，每盏灯与若干盏灯相连，每盏灯上都有一个开关，如果按下一盏灯上的开关，这盏灯以及与之相连的所有灯的开关状态都会改变。一开始所有灯都是关着的，你需要将所有灯打开，求最小的按开关次数。
1 <= n <= 35 。
解题思路
如果这道题暴力 DFS 找开关灯的状态，时间复杂度就是 O(2^n) ，显然超时。不过，如果我们用 meet in middle 的话，时间复杂度可以优化至 O(n2^(n/2)) 。meet in middle 就是让我们先找一半的状态，也就是找出只使用编号为 1 到 mid 的开关能够到达的状态，再找出只使用另一半开关能到达的状态。如果前半段和后半段开启的灯互补，将这两段合并起来就得到了一种将所有灯打开的方

案。具体实现时，可以把前半段的状态以及达到每种状态的最少按开关次数存储在 map 里面，搜索后半段时，每搜出一种方案，就把它与互补的第一段方案合并来更新答案。
示例代码

```
#include <algorithm>
#include <iostream>
#include <map>
using namespace std;

int n, m, ans = 0x7fffffff;
map<long long, int> f;
long long a[40];

int main() {
cin >> n >> m;
a[0] = 1;
for (int i = 1; i < n; ++i) a[i] = a[i - 1] * 2;  // 进行预处理

for (int i = 1; i <= m; ++i) {  // 对输入的边的情况进行处理
int u, v;
cin >> u >> v;
--u;
--v;
a[u] |= ((long long)1 << v);
a[v] |= ((long long)1 << u);
}

for (int i = 0; i < (1 << (n / 2)); ++i) {  // 对前一半进行搜索
long long t = 0;
int cnt = 0;
for (int j = 0; j < n / 2; ++j) {
if ((i >> j) & 1) {
t ^= a[j];
++cnt;
}
}
if (!f.count(t))
f[t] = cnt;
else
f[t] = min(f[t], cnt);
}

for (int i = 0; i < (1 << (n - n / 2)); ++i) {  // 对后一半进行搜索
long long t = 0;
int cnt = 0;
for (int j = 0; j < (n - n / 2); ++j) {
if ((i >> j) & 1) {
t ^= a[n / 2 + j];
++cnt;
}
}
if (f.count((((long long)1 << n) - 1) ^ t))
ans = min(ans, cnt + f[(((long long)1 << n) - 1) ^ t]);
}

cout << ans;

return 0;
}
```

例题：

Status

Problem

Tags
P1032
.
[NOIP 2002 提高组] 字串变换
搜索
P4799
.
[CEOI 2015] 世界冰球锦标赛 (Day2)
搜索
P9234
.
[蓝桥杯 2023 省 A] 买瓜
搜索
P10848
.
送礼物
搜索
矩阵与矩阵快速幂
A*搜索算法

搜索 02 • A*搜索算法

定义

A * 搜索算法（英文：A*search algorithm，A * 读作 A-star），简称 A * 算法，是一种在图形平面上，对于有多个节点的路径求出最低通过成本的算法。它属于图遍历（英文：Graph traversal）和最佳优先搜索算法（英文：Best-first search），亦是 BFS 的改进。

过程

定义起点 s ，终点 t ，从起点（初始状态）开始的距离函数 g(x) ，到终点（最终状态）的距离函数 h(x) ， h^(*)(x) [^note1]，以及每个点的估价函数 f(x)=g(x)+h(x) 。

A * 算法每次从优先队列中取出一个 f 最小的元素，然后更新相邻的状态。

如果 h <= h* ，则 A * 算法能找到最优解。

上述条件下，如果 h 满足三角形不等式，则 A * 算法不会将重复结点加入队列。

当 h=0 时，A * 算法变为 Dijkstra；当 h=0 并且边权为 1 时变为 BFS。

例题

八数码

题目大意：在 3 * 3 的棋盘上，摆有八个棋子，每个棋子上标有 1 至 8 的某一数字。棋盘中留有一个空格，空格用 0 来表示。空格周围的棋子可以移到空格中，这样原来的位置就会变成空格。给出一种初始布局和目标布局（为了使题目简单，设目标状态如下），找到一种从初始布局到目标布局最少步骤的移动方法。

```
123
804
765
```

思路

h 函数可以定义为，不在应该在的位置的棋子个数。

容易发现 h 满足以上两个性质，此题可以使用 A * 算法求解。

示例代码

```
#include <algorithm>
#include <cstring>
#include <iostream>
#include <queue>
#include <set>
using namespace std;
constexpr int dx[4] = {1, -1, 0, 0}, dy[4] = {0, 0, 1, -1};
int fx, fy;
char ch;

struct matrix {
int a[5][5];

bool operator<(matrix x) const {
for (int i = 1; i <= 3; i++)
for (int j = 1; j <= 3; j++)
if (a[i][j] != x.a[i][j]) return a[i][j] < x.a[i][j];
return false;
}
} f, st;

int h(matrix a) {
int ret = 0;
for (int i = 1; i <= 3; i++)
for (int j = 1; j <= 3; j++)
if (a.a[i][j] != st.a[i][j] && a.a[i][j] != 0) ret++;
return ret;
}

struct node {
matrix a;
int t;

bool operator<(node x) const { return t + h(a) > x.t + h(x.a); }
} x;

priority_queue<node> q;  // 搜索队列
set<matrix> s;           // 防止搜索队列重复

int main() {
cin.tie(nullptr)->sync_with_stdio(false);
st.a[1][1] = 1;  // 定义标准表
st.a[1][2] = 2;
st.a[1][3] = 3;
st.a[2][1] = 8;
st.a[2][2] = 0;
st.a[2][3] = 4;
st.a[3][1] = 7;
st.a[3][2] = 6;
st.a[3][3] = 5;
for (int i = 1; i <= 3; i++)  // 输入
for (int j = 1; j <= 3; j++) {
cin >> ch;
f.a[i][j] = ch - '0';
}
s.insert(f);
q.push({f, 0});
while (!q.empty()) {
x = q.top();
q.pop();
if (!h(x.a)) {  // 判断是否与标准矩阵一致
cout << x.t << '\n';
return 0;
}
for (int i = 1; i <= 3; i++)
```

```
for (int i = 1; i <= 3; i++)
if (!x.a.a[i][i]) fx = i, fy = i;  // 查找空格子（0 号点）的位置
for (int i = 0; i < 4; i++) {  // 对四种移动方式分别进行搜索
int xx = fx + dx[i], yy = fy + dy[i];
if (1 <= xx && xx <= 3 && 1 <= yy && yy <= 3) {
swap(x.a.a[fx][fy], x.a.a[xx][yy]);
if (!s.count(x.a))
s.insert(x.a)
q.push({x.a, x.t + 1});  // 这样移动后，将新的情况放入搜索队列中
swap(x.a.a[fx][fy], x.a.a[xx][yy]);  // 如果不这样移动的情况
}
}
}
return 0;
}
```

例题：

Status

Problem

Tags

P1379

.

八数码难题

搜索

P5507

.

机关

搜索

双向搜索

迭代加深搜索

搜索 03 • 迭代加深搜索

定义

迭代加深是一种 **每次限制搜索深度的** 深度优先搜索。

解释

迭代加深搜索的本质还是深度优先搜索，只不过在搜索的同时带上了一个深度 d ，当 d 达到设定的深度时就返回，一般用于找最优解。如果一次搜索没有找到合法的解，就让设定的深度加一，重新从根开始。

既然是为了找最优解，为什么不用 BFS 呢？我们知道 BFS 的基础是一个队列，队列的空间复杂度很大，当状态比较多或者单个状态比较大时，使用队列的 BFS 就显出了劣势。事实上，迭代加深就类似于用 DFS 方式实现的 BFS，它的空间复杂度相对较小。

当搜索树的分支比较多时，每增加一层的搜索复杂度会出现指数级爆炸式增长，这时前面重复进行的部分所带来的复杂度几乎可以忽略，这也就是为什么迭代加深是可以近似看成 BFS 的。

过程

首先设定一个较小的深度作为全局变量，进行 DFS。每进入一次 DFS，将当前深度加一，当发现 d 大于设定的深度 limit 就返回。如果在搜索的途中发现了答案就可以回溯，同时在回溯的过程中可以记录路径。如果没有发现答案，就返回到函数入口，增加设定深度，继续搜索。

伪代码

```
IDDFS(u,d)
if d>limit
return
else
for each edge (u,v)
IDDFS(v,d+1)
return
```

注意事项

在大多数的题目中，广度优先搜索还是比较方便的，而且容易判重。当发现广度优先搜索在空间上不够优秀，而且要找最优解的问题时，就应该考虑迭代加深。

实例

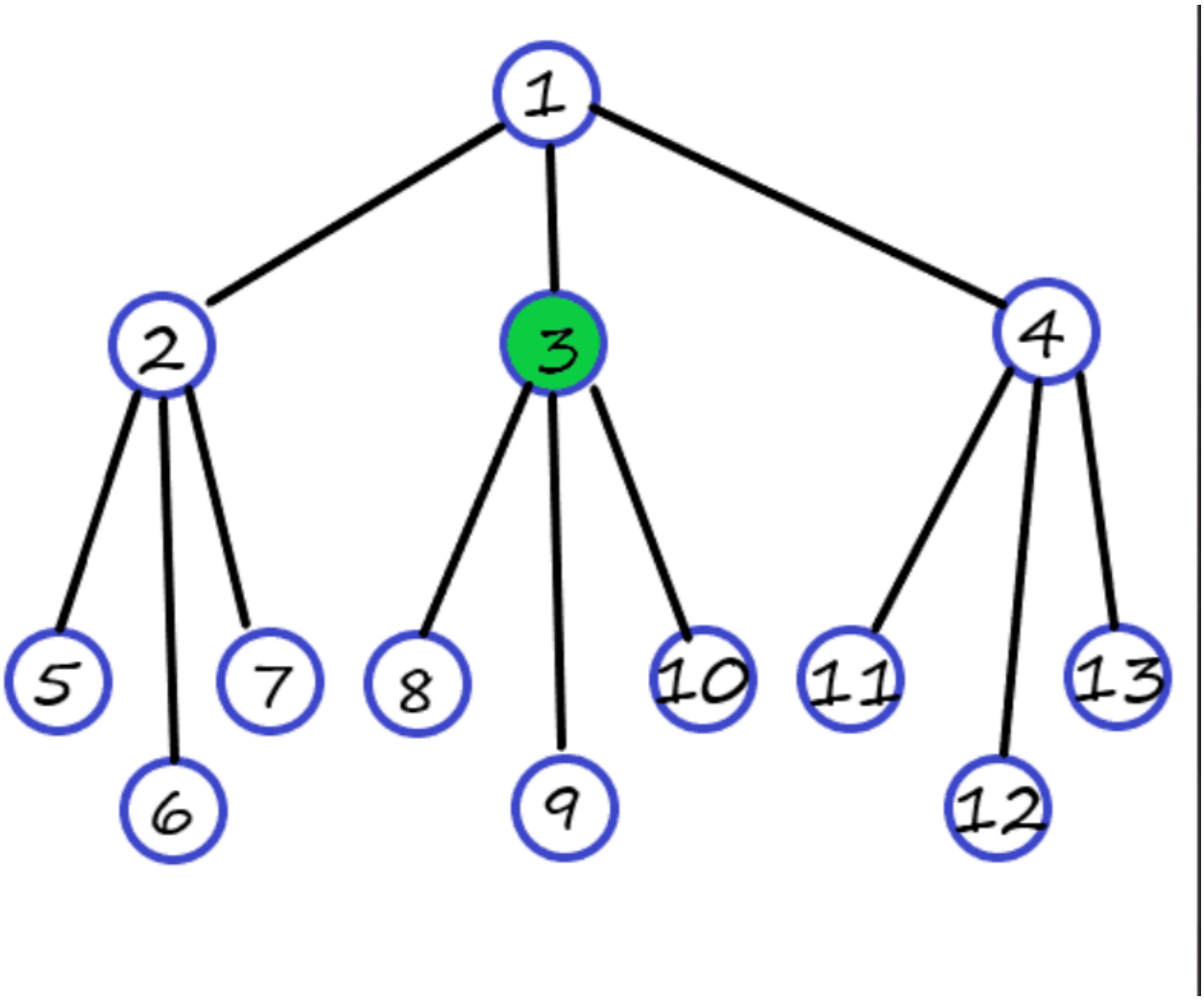
假设一张图，ans1 在很深的地方，ans2 离搜索树的根节点最近，但是需要找到的答案为 ans3。

首先考虑 DFS，一般是一搜搜到底，很有可能找到 ans1。若继续查找，很有可能花费太多时间。时间效率低。

再来考虑 BFS，它可以找到最近的答案 ans2。若继续查找，很有可能存储状态的队列会浪费巨大空间。空间效率低

现在引出 IDDFS，它通常适用于有两个条件的问题：一是它是个最优解问题，二是最优的答案深度最小。且能够快速地找到答案。

假设在搜索树中，每层树都有 3 个方案，即是搜索树为一颗 3 叉树，共 2 层，ans 在 3。



图示

先来对比 DFS，搜索路径为 1-2-5-2-6-2-7-2-1-3，找到答案。有最坏情况，即每一个分支都是一个无底洞，若永远搜索不到答案，就会卡在里面。

再来对比 BFS，搜索路径为 1-2-3，看起来比较短，但是队列中有 1,2,5,6,7,3 的信息，若答案更深一些，那么就会炸空间。
通过上述两个例子，可以知道 DFS 和 BFS 的局限性，但也各有千秋。结合两种算法，就有了迭代加深。首先限定一个层数，对于搜索树进行深度优先搜索。假设这个层数为 1，那么深搜只会搜索到 2，不会继续加深。首先试探性地来找答案，直到找到答案位置。很明显，上面几层的点会搜到很多遍，但时间复杂度对于 DFS 来说比较优，而在空间复杂度上比 BFS 上略胜一筹。
很容易就写出模板：

```
 int max_depth = min_depth;
Id_Dfs( int current_depth  ,  int max_depth ) {
if( current_depth > max_depth ) return ;
if( 找到答案 ){ 输出答案 ; (exit(0) ;  ||  return ;) }
for_each ( 当前节点的儿子节点 )
Id_Dfs(current_depth + 1, max_depth) ;
}
for(; ; max_depth++ ) {
Id_Dfs( 0 , i ) ;
}
```

示例题目
一个与 n 有关的整数加成序列 a[0],a[1],a[2],...,a_(m) 满足以下四个条件：

1. a[0] = 1
2. a_(m) = n
3. a[0] <= a[1] <= a[2] <= ... <= a_(m-1) <= a_(m)
4. 对于每一个 k(1 <= k <= m) 都存在有两个整数 i 和 j(0 <= i,j <= k-1 ，i 和 j 可以相等))，使得 a[k] = a[i]+a[j]

你的任务是：给定一个整数 n，找出符合上述四个条件的长度最小的整数加成序列。如果有多个满足要求的答案，只需要输出任意一个解即可。
思路
按照 1,2,4,8... 这样来排列，找出最少需要的次数那么最少的层数就找到了，就减少了之前做的无用功。
树上的子节点也较为好找，只需要将之前搜索到的数字，按照题意两两搭配找到下一项。
只需要按照 IDDFS 的规则搜索就行了。但重点在于剪枝，写在注释里。
示例代码

```
#include <cstdio>
#include <cstring>
bool Quick_Read(int &N) {
N = 0;
int op = 1;
char c = getchar();
while(c < '0' || c > '9') {
if(c == '-')
op = -1;
c = getchar();
}
while(c >= '0' && c <= '9') {
N = (N << 1) + (N << 3) + (c ^ 48);
c = getchar();
}
N *= op;
return N != 0;
}
void Quick_Write(int N) {
if(N < 0) {
putchar('-');
N = -N;
}
if(N >= 10)
Quick_Write(N / 10);
putchar(N % 10 + 48);
}
const int MAXN = 1e5 + 5;
int ans[MAXN];
int limit;
bool flag;
int n;
void Id_Dfs(int nowdepth) {
if(nowdepth > limit || flag)//达到层数不在恋战或找到答案，直接跳出
return;
if(ans[nowdepth] == n) {//满足题意
flag = true;
return;
}
for(int i = nowdepth; i >= 1; i--) {
for(int j = nowdepth; j >= i; j--) {//两两搭配，且答案越大越容易找到解，故而到着找
if(ans[i] + ans[j] <= n && ans[i] + ans[j] > ans[nowdepth]) {//满足题意 1，2 两点的搜索
int now;//找到下一项
ans[nowdepth + 1] = now = ans[i] + ans[j];
for(int k = nowdepth + 2; k <= limit; k++)
//从 nowdepth + 1 这一项开始，后面最大时也就是 now 不停扩大 2 倍，若最大都达不到 n，舍去不求
now <<= 1;
if(now < n)
continue;
Id_Dfs(nowdepth + 1);//搜索下一层
if(flag)//找到答案
return;
}
}
}
}
void Work() {
for(; !flag; limit++)//直到找到答案时停止搜索
Id_Dfs(1);
for(int i = 1; i < limit; i++) {//输出
Quick_Write(ans[i]);
putchar(' ');
}
putchar('\n');
}
```

```
void Init() {
limit = 1;
int test = 1;
while(test < n) {//找到最小层数
test <<= 1;
limit++;
}
ans[1] = 1;
flag = false;
}
int main() {
while(Quick_Read(n)) {//多组输入输出，到 0 为止
Init();
Work();
}
return 0;
}
```

例题：

Status

Problem

Tags

Pyundou/luogu/P2346

.

四子连棋

搜索

Pyundou/luogu/P10488

.

Booksort

搜索

A*搜索算法

IDA*算法

搜索 04 • IDA*算法

引子

一位探险家在探险时不慎身陷迷宫。这个迷宫非常复杂，假设你是一名探险家，有很多死路和岔路。这位探险家需要在迷宫中不断探索，寻找最短的路径。不过很幸运的是，你在下迷宫之前在入口留下了一个信号接收装置，这个装置能让你知道你到入口的直线距离。由于有限的物资，这位探险家想要以最短的路径找到出口。最后，这位聪明的探险家想到了一个绝妙的点子…

正文

探险家的做法

他用到了一根很长很长的绳子，在出发前，他记住了自己到入口的直线距离，然后他将绳子一大部分绑在了出发点的柱子上，另一端绑在了自己身上，让他能自由活动的距离刚好是他一开始期望到出口的距离，即直线距离。出发后，每走几十步，他就确认一次自己到入口的直线距离，如果他发现自己走的步数 + 现在的直线距离反而比出发时的直线距离大了，那么他判定这条路线是不符合他期望的，因此原路返回到出发点，选择另一条符合他期望的道路继续走到发现不符合期望，直到所有道路走完为止，返回出发点；接下来，他将绑在柱子上的绳子解开一些，使自己能自由活动的距离大了一些，然后重新出发。他重复这个过程，直到他找到了入口为止。另外，当他走到一条死路时，他会直接原地返回，并在下次重新出发时不再选择这条道路。

IDA*算法

解析

毫无疑问，这实在是个聪明的探险家！同时，他可能自己都没意识到，他还是一个极有天赋的程序员。他找到入口的这个过程正是非常经典的寻路算法——IDA* 算法。

IDA *算法，顾名思义，就是用 IDDFS 的方式去实现 A* 算法的过程。这个算法的具体思想，在上面的例子中已经展示的较为明确了，不过我们还是较为形式化的说明一下。

IDA *算法和 A* 算法一样，也需要一个估价函数：F=G+H，但它多了一个深度限制，也就是探险家对找到出口的路径长度的期望值。

1. 我们一开始将起点的 F 值设为这个深度限制，也就是探险家一开始记住的自己到入口的直线距离。
2. 我们从起点开始进行 DFS。在上面的例子中，探险家选择一条路一直走到他身上的绳子不允许他再向前走，他没有考虑其他的道路。至少在走这条道路的时候没有。
3. 重复这个过程，一直到探索了所有可探索的道路，此时我们增加找到出口所经过路径长度的深度限制，对于探险家来说，就是解开一部分绳子，增加了他找到出口的期望值。
4. 发现死路时，我们原路返回，深度限制不变，在下次搜索中不会再走这条路。
5. 重复这个过程直到找到出口为止。

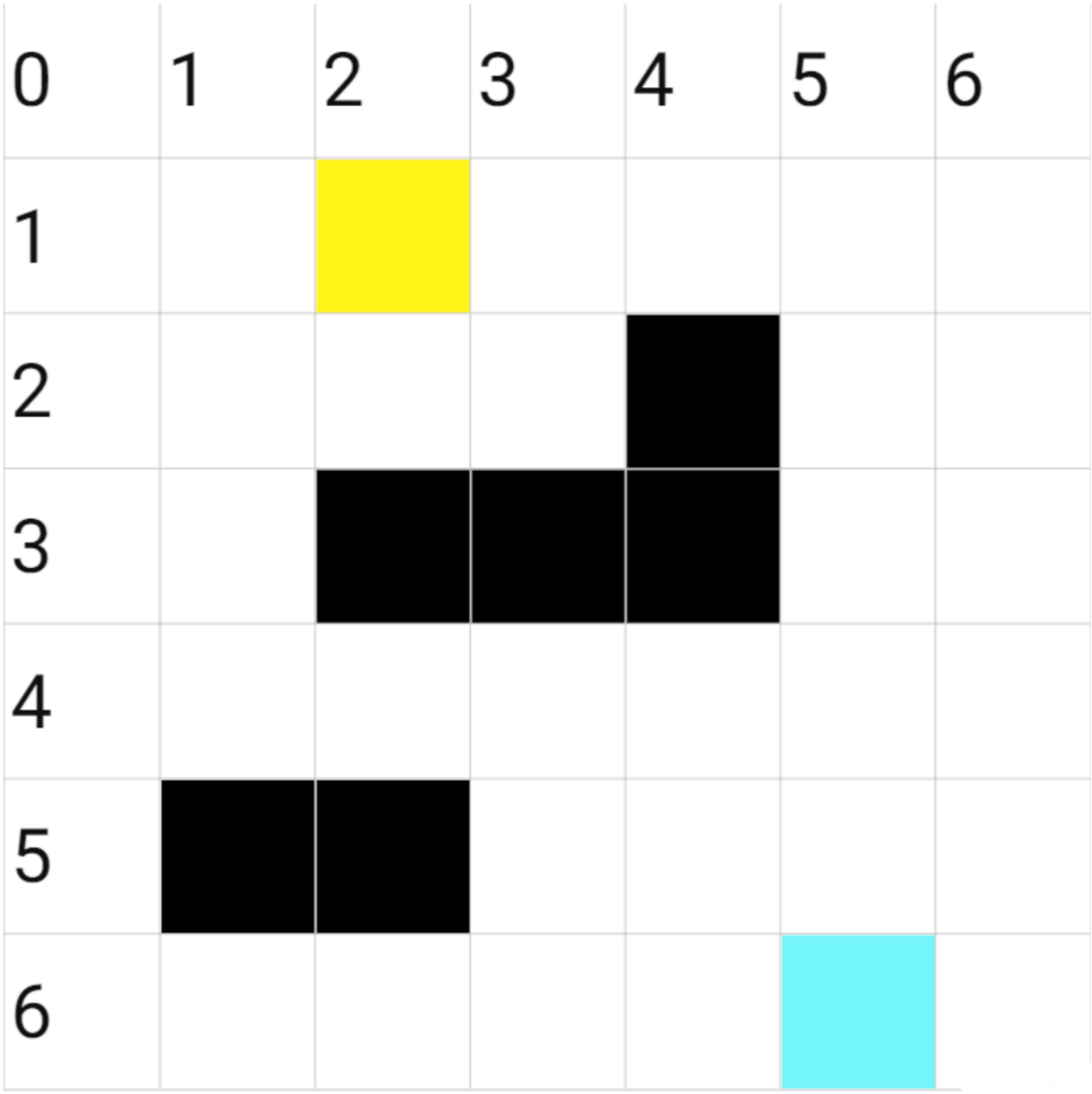
这就是大名鼎鼎的 IDA*算法。

图示

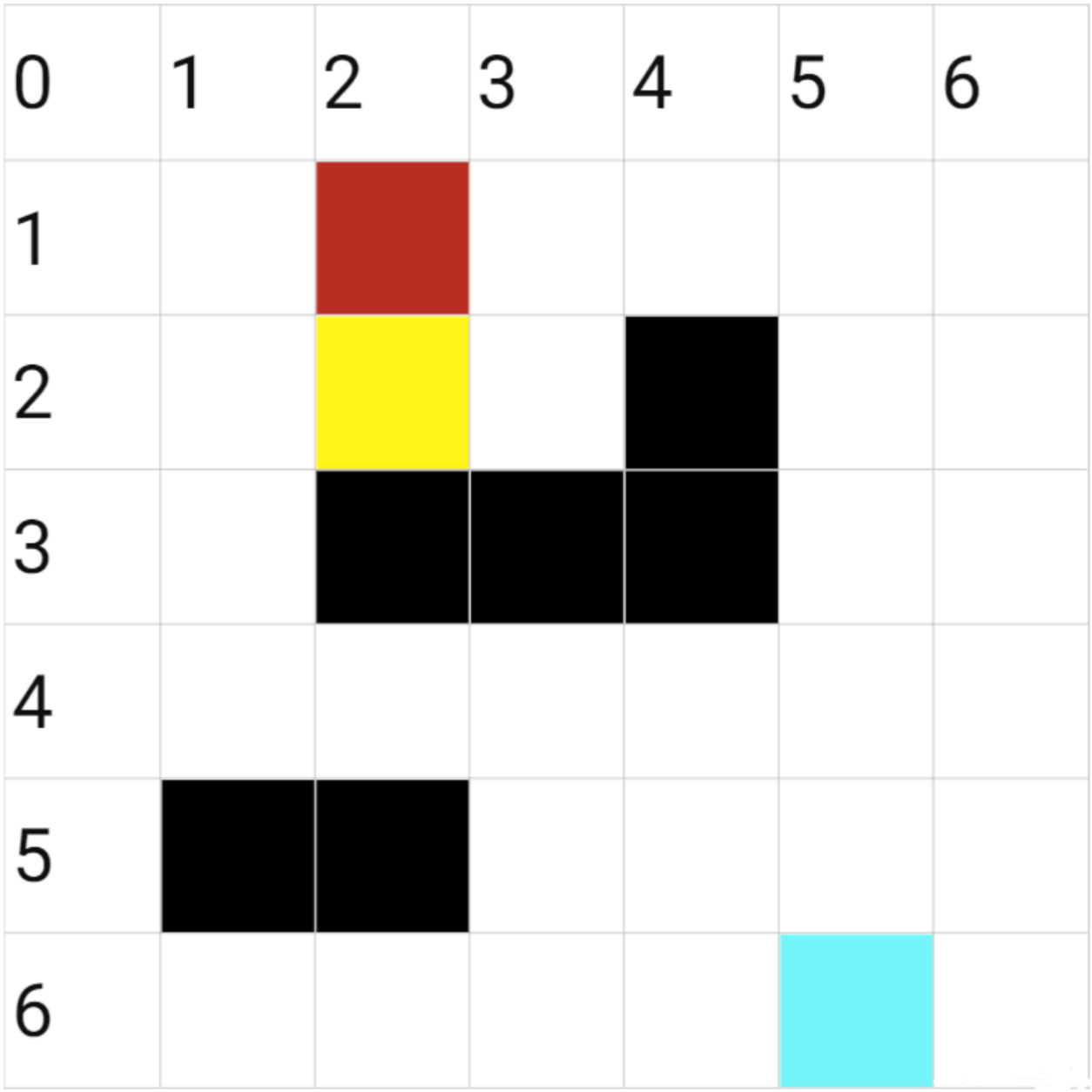
我们依然用 A*算法中用过的例子。黄色块代表当前搜索的点，红色块代表在当前深搜中正在搜索的路径，浅蓝色块代表终点，黑色块代表障碍物。这里多加一个绿色块，用来标记我们在当前深搜中搜索过的路径。

我们依然利用曼哈顿式启发函数来估价

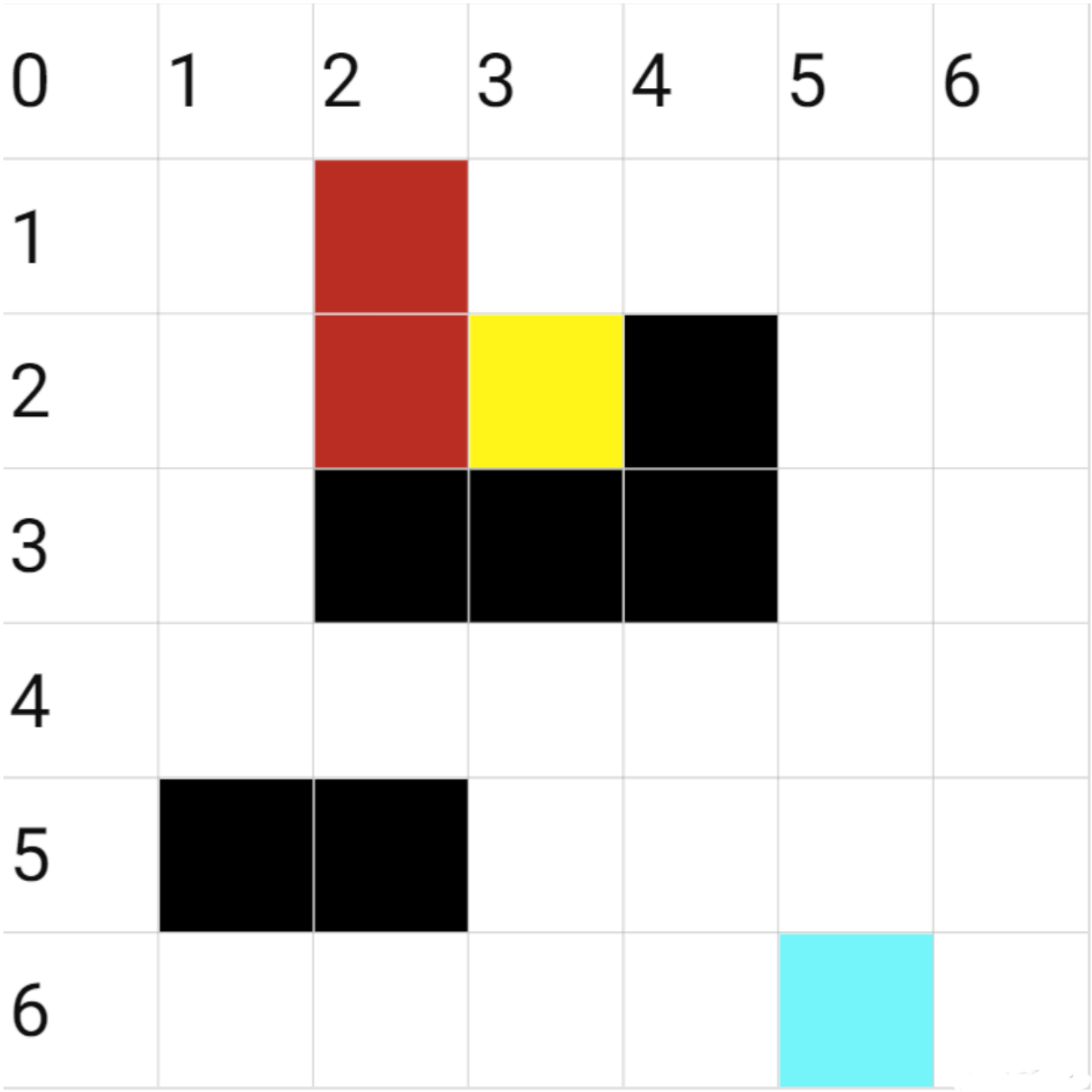
1. 原图，我们将一开始的深度限制设为起点的曼哈顿值 8。



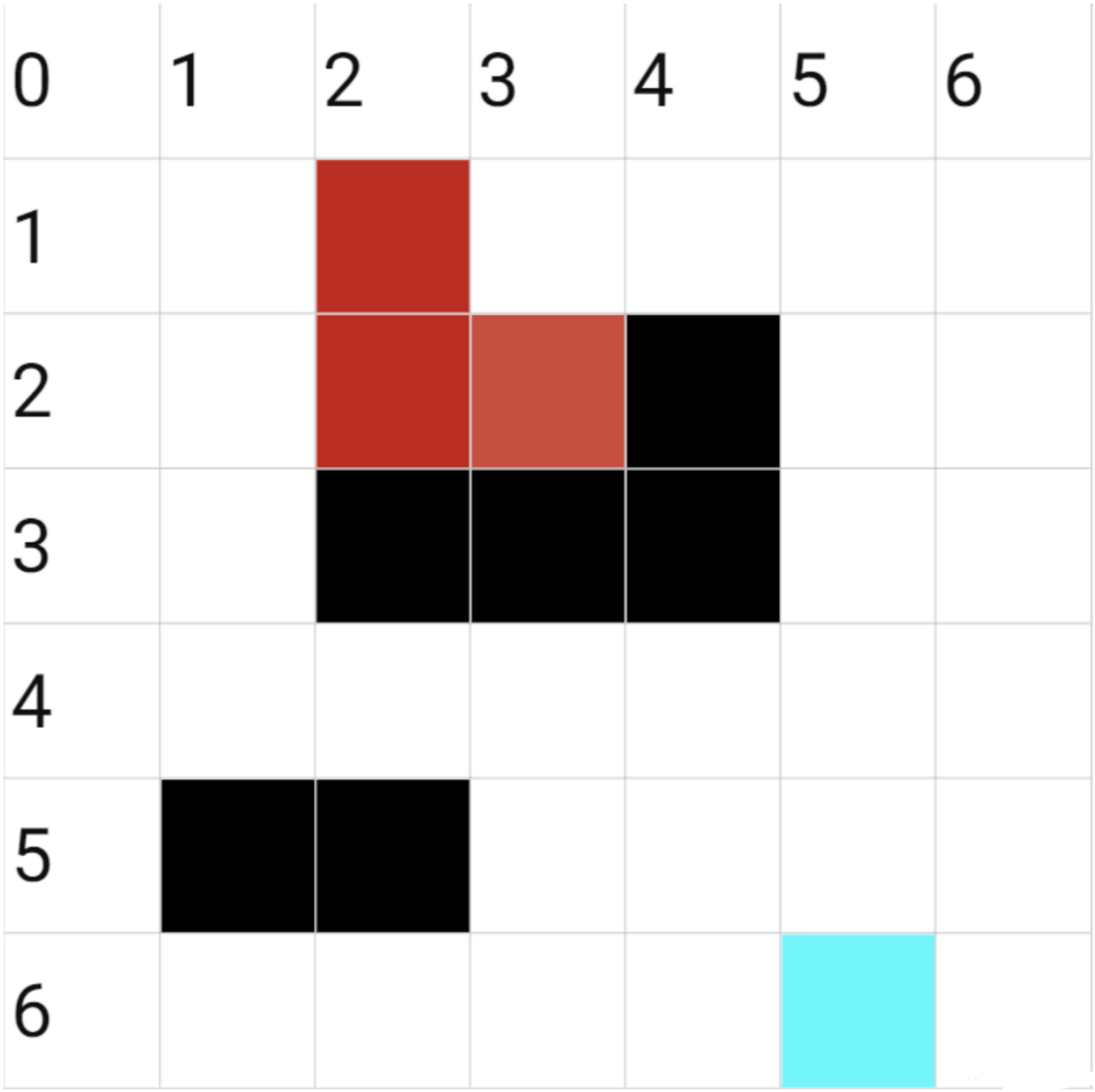
1. 我们按照下右左上的方式进行深搜，先搜索下面的结点。

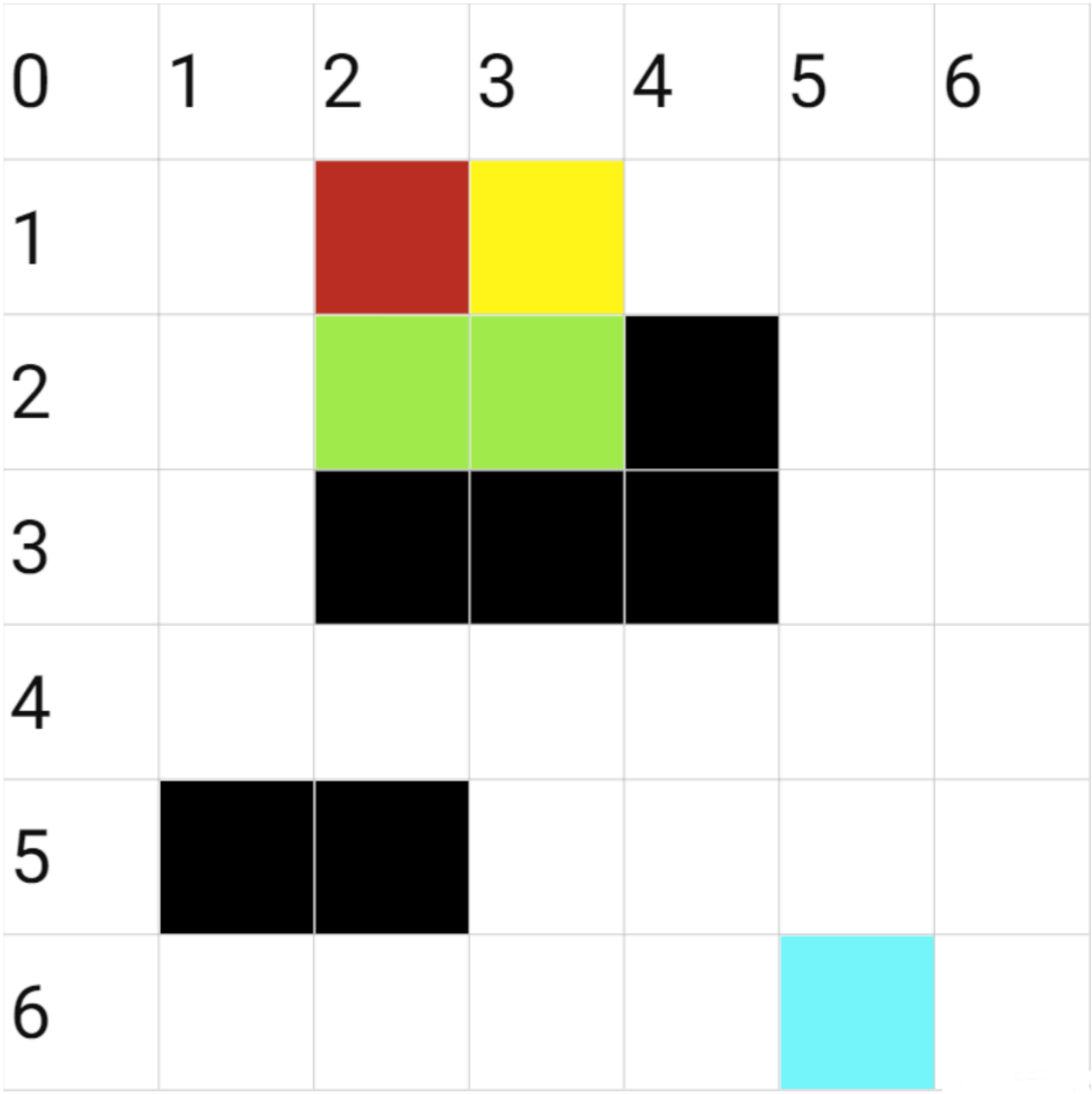


1. 进行深搜。



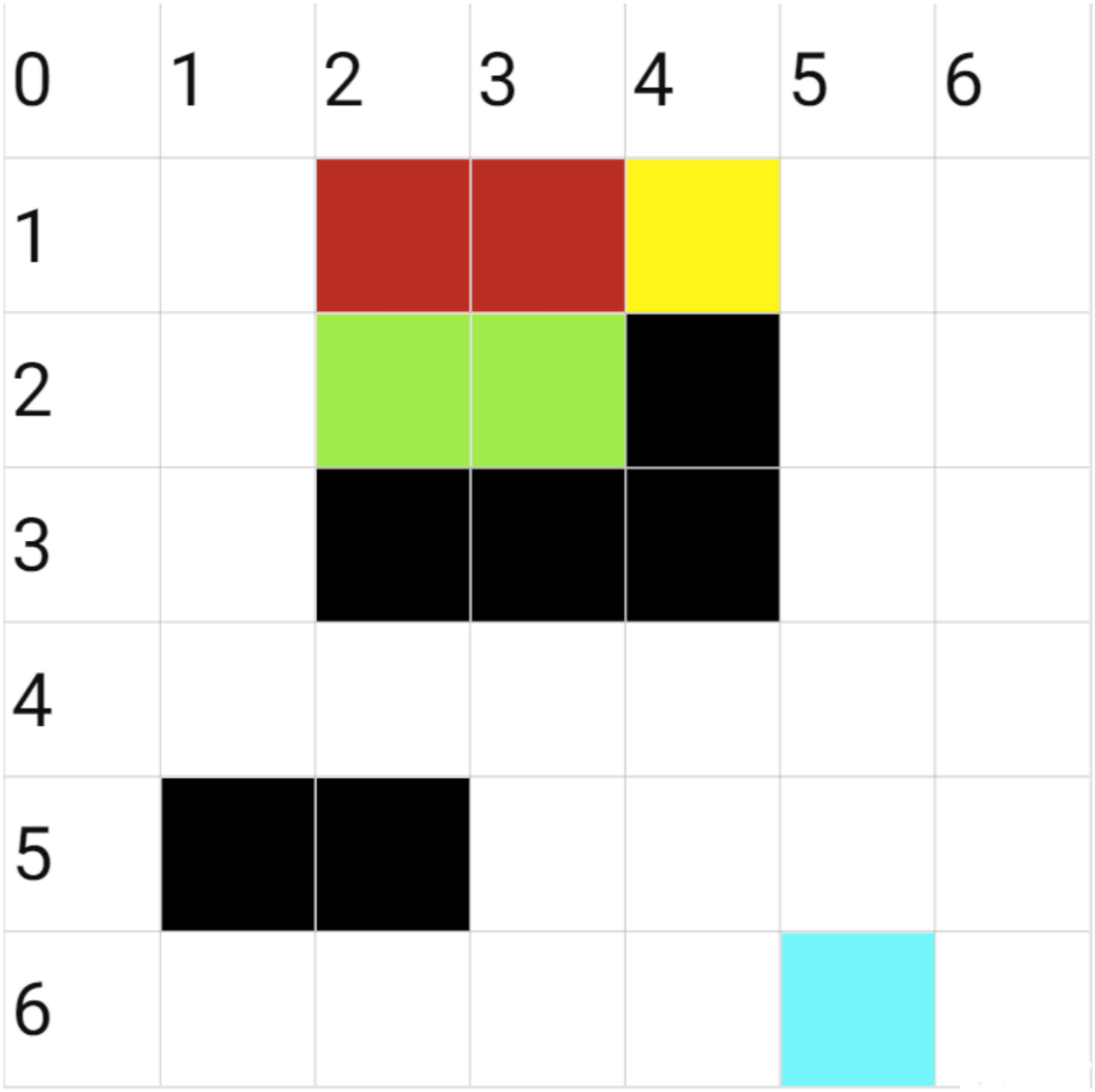
1. 发现到这一步再往上走的话，估价函数值会超过深度限制，于是回溯到上一格，依然没有找到其他路径，再次回溯到起点，这一次我们发现，右边的那一格没有超过深度限制，于是向右深搜。


0
1
2
3
4
5
6
1
2
3
4
5
6

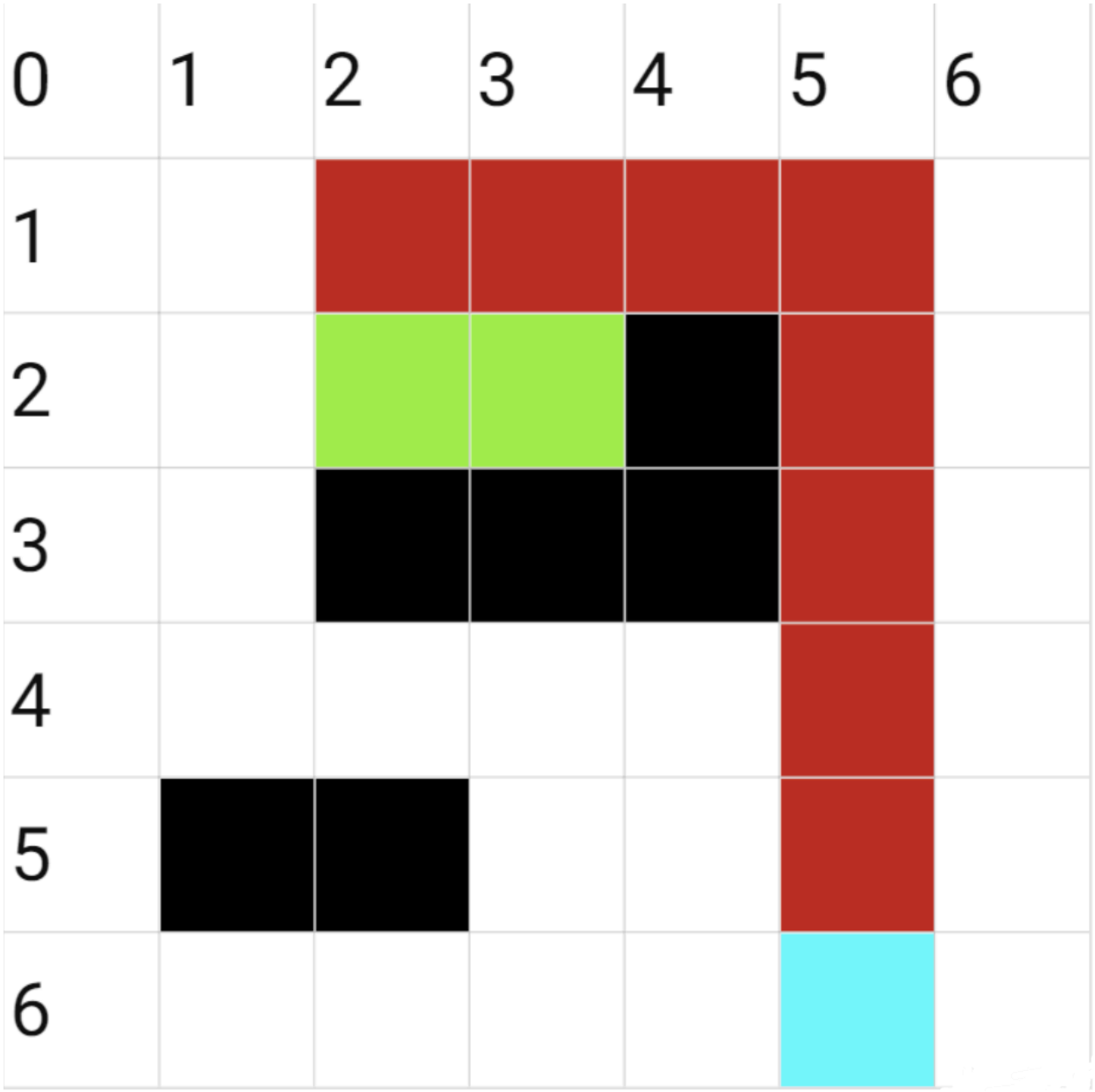


（再次强调，绿色格子是这一轮次深搜中搜索过的路径）

1. 下面的方块在上一条路径中被搜索过，因此搜索右边的方块。



1. 持续深搜，最后能发现这样的一条路径。



IDA*算法的运行过程很简单，下面贴上代码：

代码

```
#include<algorithm>
#include<iostream>
using namespace std;
typedef struct node {
int x;
int y;
int v;
}node;
node* G;
bool* visited;
int width;//图的边长
int max_depth;
int over;//标记当前路径是否搜索到超过深度的地步，如果有则一路返回到起点
int h(node A,node B)//定义点
{
int i, j;
if (B.x > A.x)i = B.x - A.x;
else i = A.x - B.x;
if (B.y > A.y)j = B.y - A.y;
else j = A.y - B.y;
return i+j;
}
void ini()//初始化函数
{
scanf("%d", &width);
G = (node*)malloc(sizeof(node) * ((width) * (width)));
visited = (bool*)malloc(sizeof(bool) * (width) * (width));
for (int i = 0; i < (width) * (width); i++) {
G[i].x = i / width;
G[i].y = i % width;
G[i].v = 0;
visited[i] = 0;
}
//初始化特定区域，例如起点块设置为 1，终点块设置为 2，障碍物设置为 -1
int x;
int y;
int v;
while (scanf("%d %d %d", &x, &y, &v)) {
G[x * width + y].v = v;
}
}
bool blind_alley(node a){}//这个函数用于判断是否是死路，大致思路就是看周围的方块是否都是之前走过的路径或者障碍物，如果是则返回 true
bool IDA_star(int g,node a,node b)//g 是估价函数中的 G，a 是当前搜索的点，b 是终点
{
if (a.v == 1)over = 0;
if (a.x == b.x && a.y == b.y)return true;//找到终点
if (over == 1)return false;
if (g + h(a, b) > max_depth) {
max_depth++;
over = 1;
return false;//超过了估价函数
}
if (blind_alley(a)) {//发现是死路
over = 1;
return false;
}
int self = a.x * width + a.y;
visited[self] = 1;
//通过下右左上的顺序来计算代码
int below = (a.x + 1) * width + a.y;
int right = a.x * width + (a.y + 1);
int left = a.x * width + (a.y - 1);
int above = (a.x - 1) * width + (a.y);
if (visited[below]==0&&G[below].v != -1)IDA_star(g + 1, G[below], b);
```

```
else if (visited[right] == 0&&G[right].v != -1)IDA_star(g + 1, G[right], b);
else if (visited[left] == 0 && G[left].v != -1)IDA_star(g + 1, G[left], b);
else if (visited[above] == 0 && G[above].v != -1)IDA_star(g + 1, G[above], b);
visited[self] = 0;
}
```

例题：

Status

Problem

Tags

Pyundou/luogu/P10494

.

[USACO02FEB] Power Hungry Cows

搜索

迭代加深搜索

剪枝

搜索 05 • 剪枝

前言

DFS（深度优先搜索）是一种常见的算法，大部分的题目都可以用 DFS 解决，但是大部分情况下，这都是骗分算法，很少会有爆搜为正解的题目。因为 DFS 的时间复杂度特别高。（没学过 DFS 的请自行补上这一课）

既然不能成为正解，那就多骗一点分吧。那么这一篇文章将介绍一些实用的优化算法（俗称「剪枝」）。

先来一段深搜模板，之后的模板将在此基础上进行修改。

```
int ans = 最坏情况, now;  // now 为当前答案

void dfs(传入数值) {
if (到达目的地) ans = 从当前解与已有解中选最优;
for (遍历所有可能性)
if (可行) {
进行操作;
dfs(缩小规模);
撤回操作;
}
}
```

其中的 ans 可以是解的记录，那么从当前解与已有解中选最优就变成了输出解。

剪枝方法

最常用的剪枝有三种，记忆化搜索、最优性剪枝、可行性剪枝。

记忆化搜索

因为在搜索中，相同的传入值往往会带来相同的解，那我们就可以用数组来记忆。

模板：

```
int g[MAXN];  // 定义记忆化数组
int ans = 最坏情况, now;

void dfs_f(传入数值) {
if (g[规模] != 无效数值) return;  // 或记录解，视情况而定
if (到达目的地) ans = 从当前解与已有解中选最优;  // 输出解，视情况而定
for (遍历所有可能性)
if (可行) {
进行操作;
dfs(缩小规模);
撤回操作;
}
}

int main() {
// ...
memset(g, 无效数值, sizeof(g));  // 初始化记忆化数组
// ...
}
```

最优性剪枝

在搜索中导致运行慢的原因还有一种，就是在当前解已经比已有解差时仍然在搜索，那么我们只需要判断一下当前解是否已经差于已有解。

模板：

```
int ans = 最坏情况, now;

void dfs(传入数值) {
if (now比ans的答案还要差) return;
if (到达目的地) ans = 从当前解与已有解中选最优;
for (遍历所有可能性)
if (可行) {
进行操作;
dfs(缩小规模);
撤回操作;
}
}
```

可行性剪枝

在搜索过程中当前解已经不可用了还继续搜索下去也是运行慢的原因。

模板：

```
int ans = 最坏情况, now;

void dfs(传入数值) {
if (当前解已不可用) return;
if (到达目的地) ans = 从当前解与已有解中选最优;
for (遍历所有可能性)
if (可行) {
进行操作;
dfs(缩小规模);
撤回操作;
}
}
```

剪枝思路

剪枝思路有很多种，大多需要对于具体问题来分析，在此简要介绍几种常见的剪枝思路。

- 极端法：考虑极端情况，如果最极端（最理想）的情况都无法满足，那么肯定实际情况搜出来的结果不会更优了。
- 调整法：通过对子树的比较剪掉重复子树和明显不是最有「前途」的子树。

- 数学方法：比如在图论中借助连通分量，数论中借助模方程的分析，借助不等式的放缩来估计下界等等。

例题

题目描述

有 n 份工作要分配给 n 个人来完成，每个人完成一份。第 i 个人完成第 k 份工作所用的时间为一个正整数 t_(i,k) ，其中 1 <= i, k <= n 。试确定一个分配方案，使得完成这 n 份工作的时间总和最小。

输入包含 n + 1 行。

第 1 行为一个正整数 n 。

第 2 行到第 n + 1 行中每行都包含 n 个正整数，形成了一个 n * n 的矩阵。在该矩阵中，第 i 行第 k 列元素 t_(i,k) 表示第 i 个人完成第 k 件工作所要用的时间。

输出包含一个正整数，表示所有分配方案中最小的时间总和。

数据范围

1 <= n <= 15

1 <= t_(i,k) <= 10^4

输入样例

```
5
9 2 9 1 9
1 9 8 9 6
9 9 9 9 1
8 8 1 8 4
9 1 7 8 9
```

输出样例

```
5
```

由于每个人都必须分配到工作，在这里可以建一个二维数组 `time[i][j]` ，用以表示 i 个人完成 j 号工作所花费的时间。给定一个循环，从第 1 个人开始循环分配工作，直到所有人都分配到。为第 i 个人分配工作时，再循环检查每个工作是否已被分配，没有则分配给 i 个人，否则检查下一个工作。可以用一个一维数组 `is_working[j]` 来表示第 j 号工作是否已被分配，未分配则 `is_working[j]=0` ，否则 `is_working[j]=1` 。利用回溯思想，在工人循环结束后回到上一工人，取消此次分配的工作，而去分配下一工作直到可以分配为止。这样，一直回溯到第 1 个工人后，就能得到所有的可行解。

检查工作分配，其实就是判断取得可行解时的二维数组的第一维下标各不相同并且第二维下标各不相同。而我们是要得到完成这 n 份工作的最小时间总和，即可行解中时间总和最小的一个，故需要再定义一个全局变量 `cost_time_total_min` 表示目前找到的解中最小的时间总和，初始 `cost_time_total_min` 为 `time[i][i]` 之和，即对角线工作时间相加之和。在所有人分配完工作时，比较 `count` 与 `cost_time_total_min` 的大小，如果 `count` 小于 `cost_time_total_min` ，说明找到了一个最优解，此时就把 `count` 赋给 `cost_time_total_min` 。

但考虑到算法的效率，这里还有一个剪枝优化的工作可以做。就是在每次计算局部费用变量 `count` 的值时，如果判断 `count` 已经大于 `cost_time_total_min` ，就没必要再往下分配了，因为这时得到的解必然不是最优解。

示例代码

```
#include <iostream>
constexpr int N = 16;
int is_working[N] = {0};  // 某项工作是否被分配
int tm[N][N];             // 完成某项工作所需的时间
int cost_time_total_min;  // 完成 n 份工作的最小时间总和

// i 表示第几个人，count 表示工作费用总和
void work(int i, int count, int n) {
// 如果 i 超出了所能分配的最大工作件数，表示分配完成，并且 count 比原来
// cost_time_total_min 花费少，则更新 cost_time_total_min 的值
if (i > n && count < cost_time_total_min) {
cost_time_total_min = count;
return;
}
// 回溯思想
if (count < cost_time_total_min) {
// j 表示第几件工作
for (int j = 1; j <= n; j++) {
// 如果工作未被分配 is_working = 0
if (is_working[j] == 0) {
// 分配工作 is_working = 1
is_working[j] = 1;
// 工作交给第 i + 1 个人
work(i + 1, count + tm[j][i], n);
// 在一轮迭代完成之后，返回到上一个人，要对此次的工作进行重新分配
// 将 is_working[j] 重设为 0
is_working[j] = 0;
}
}
}
}

using std::cin;
using std::cout;

int main() {
cin.tie(nullptr)->sync_with_stdio(false);
int n;
cin >> n;
for (int i = 1; i <= n; i++) {
for (int j = 1; j <= n; j++) {
cin >> tm[i][j];
}
cost_time_total_min += tm[i][i];
}
work(1, 0, n);
cout << cost_time_total_min << '\n';
return 0;
}
```

例题：

Status

Problem

Tags

P1025

.

数的划分

搜索

P8854

.

超级马

搜索
P2329
.
栅栏
搜索
P1120
.
小木棍
搜索
P1312
.
Mayan 游戏
搜索
IDA*算法
背包 DP 进阶

动态规划 01 · 背包 DP 进阶

引入

背包 DP 除了最基础的 01 背包以外，还有多种更加复杂的模型，本篇将对这些模型进行进阶讲解。

完全背包

解释

完全背包模型与 0-1 背包类似，与 0-1 背包的区别仅在于一个物品可以选取无限次，而非仅能选取一次。

我们可以借鉴 0-1 背包的思路，进行状态定义：设 f_(i,j) 为只能选前 i 个物品时，容量为 j 的背包可以达到的最大价值。

需要注意的是，虽然定义与 0-1 背包类似，但是其状态转移方程与 0-1 背包并不相同。

过程

可以考虑一个朴素的做法：对于第 i 件物品，枚举其选了多少个来转移。这样做的时间复杂度是 O(n^3) 的。

状态转移方程如下：

f_(i,j)=max(k=0 ~ + INF)(f_(i-1,j-k * w[i])+v[i] * k)

考虑做一个简单的优化。可以发现，对于 f_(i,j) ，只要通过 f_(i,j-w[i]) 转移就可以了。因此状态转移方程为：

f_(i,j)=max(f_(i-1,j),f_(i,j-w[i])+v[i])

理由是当我们这样转移时， f_(i,j-w[i]) 已经由 f_(i,j-2 * w[i]) 更新过，那么 f_(i,j-w[i]) 就是充分考虑了第 i 件物品所选次数后得到的最优结果。换言之，我们通过局部最优子结构的性质重复使用了之前的枚举过程，优化了枚举的复杂度。

与 0-1 背包相同，我们可以将第一维去掉来优化空间复杂度。如果理解了 0-1 背包的优化方式，就不难明白压缩后的循环是正向的（也就是上文中提到的错误优化）。

疯狂的采药

题意概要：有 n 种物品和一个容量为 W 的背包，每种物品有重量 w[i] 和价值 v[i] 两种属性，要求选若干个物品放入背包使背包中物品的总价值最大且背包中物品的总重量不超过背包的容量。

示例代码

```
#include <iostream>
using namespace std;
constexpr int MAXN = 1e4 + 5;
constexpr int MAXW = 1e7 + 5;
int n, W, w[MAXN], v[MAXN];
long long f[MAXW];

int main() {
cin >> W >> n;
for (int i = 1; i <= n; i++) cin >> w[i] >> v[i];
for (int i = 1; i <= n; i++)
for (int l = w[i]; l <= W; l++)
if (f[l - w[i]] + v[i] > f[l]) f[l] = f[l - w[i]] + v[i];  // 核心状态方程
cout << f[W];
return 0;
}
```

多重背包

多重背包也是 0-1 背包的一个变式。与 0-1 背包的区别在于每种物品有 k[i] 个，而非一个。

一个很朴素的想法就是：把「每种物品选 k[i] 次」等价转换为「有 k[i] 个相同的物品，每个物品选一次」。这样就转换成了一个 0-1 背包模型，套用上文所述的方法就可已解决。状态转移方程如下：

f_(i,j)=max(k=0 ~ k[i])(f_(i-1,j-k * w[i])+v[i] * k)

时间复杂度 O(W sum _(i=1)^nk[i]) 。

??? note "核心代码"

```
for (int i = 1; i <= n; i++) {
for (int weight = W; weight >= w[i]; weight--) {
// 多遍历一层物品数量
for (int k = 1; k * w[i] <= weight && k <= cnt[i]; k++) {
dp[weight] = max(dp[weight], dp[weight - k * w[i]] + k * v[i]);
}
}
}
```

二进制分组优化

考虑优化。我们仍考虑把多重背包转化成 0-1 背包模型来求解。

解释

显然，复杂度中的 O(nW) 部分无法再优化了，我们只能从 O(sum k[i]) 处入手。为了表述方便，我们用 A_(i,j) 代表第 i 种物品拆分出的第 j 个物品。

在朴素的做法中， 任意 j <= k[i] ， A_(i,j) 均表示相同物品。那么我们效率低的原因主要在于我们进行了大量重复性的工作。举例来说，我们考虑了「同时选 A_(i,1),A_(i,2) 」与「同时选 A_(i,2),A_(i,3) 」这两个完全等效的情况。这样的重复性工作我们进行了许多次。那么优化拆分方式就成为了解决问题的突破口。

过程

我们可以通过「二进制分组」的方式使拆分方式更加优美。

具体地说就是令 A_(i,j)(j 属于 [0, floor log2 -1]) 分别表示由 2^(j) 个单个物品「捆绑」而成的大物品。特殊地，若 k[i]+1 不是 2 的整数次幂，则需要在最后添加一个由 k[i]-2^(floor log2 -1) 个单个物品「捆绑」而成的大物品用于补足。

举几个例子：

- 6=1+2+3
- 8=1+2+4+1
- 18=1+2+4+8+3
- 31=1+2+4+8+16

显然，通过上述拆分方式，可以表示任意 <= k[i] 个物品的等效选择方式。将每种物品按照上述方式拆分后，使用 0-1 背包的方法解决即可。
时间复杂度 O(W sum _(i=1)^nlog[2]k[i])

实现

示例代码

```
index = 0;
for (int i = 1; i <= m; i++) {
int c = 1, p, h, k;
cin >> p >> h >> k;
while (k > c) {
k -= c;
list[++index].w = c * p;
list[index].v = c * h;
c *= 2;
}
list[++index].w = p * k;
list[index].v = h * k;
}
```

混合背包

混合背包就是将前面三种的背包问题混合起来，有的只能取一次，有的能取无限次，有的只能取 k 次。
这种题目看起来很吓人，可是只要领悟了前面几种背包的中心思想，并将其合并在一起就可以了。下面给出伪代码：

```
for (循环物品种类) {
if (是 0 - 1 背包)
套用 0 - 1 背包代码;
else if (是完全背包)
套用完全背包代码;
else if (是多重背包)
套用多重背包代码;
}
```

例题

樱花

有 n 种樱花树和长度为 T 的时间，有的樱花树只能看一遍，有的樱花树最多看 A[i] 遍，有的樱花树可以看无数遍。每棵樱花树都有一个美学值 C[i] ，求在 T 的时间内看哪些樱花树能使美学值最高。

核心代码

```
for (int i = 1; i <= n; i++) {
if (cnt[i] == 0) {  // 如果数量没有限制使用完全背包的核心代码
for (int weight = w[i]; weight <= W; weight++) {
dp[weight] = max(dp[weight], dp[weight - w[i]] + v[i]);
}
} else {  // 物品有限使用多重背包的核心代码，它也可以处理 0-1 背包问题
for (int weight = W; weight >= w[i]; weight--) {
for (int k = 1; k * w[i] <= weight && k <= cnt[i]; k++) {
dp[weight] = max(dp[weight], dp[weight - k * w[i]] + k * v[i]);
}
}
}
}
```

二维费用背包

题面

有 n 个任务需要完成，完成第 i 个任务需要花费 t[i] 分钟，产生 c[i] 元的开支。

现在有 T 分钟时间， W 元钱来处理这些任务，求最多能完成多少任务。
这道题是很明显的 0-1 背包问题，可是不同的是选一个物品会消耗两种价值（经费、时间），只需在状态中增加一维存放第二种价值即可。
这时候就要注意，再开一维存放物品编号就不合适了，因为容易 MLE。
实现

```
for (int k = 1; k <= n; k++)
for (int i = m; i >= mi; i--)    // 对经费进行一层枚举
for (int j = t; j >= ti; j--)  // 对时间进行一层枚举
dp[i][j] = max(dp[i][j], dp[i - mi][j - ti] + 1);
```

分组背包
题面
有 n 件物品和一个大小为 m 的背包，第 i 个物品的价值为 w[i] ，体积为 v[i] 。同时，每个物品属于一个组，同组内最多只能选择一个物品。求背包能装载物品的最大总价值。
这种题怎么想呢？其实是从「在所有物品中选择一件」变成了「从当前组中选择一件」，于是就对每一组进行一次 0-1 背包就可以了。
再说一说如何进行存储。我们可以将 t_(k,i) 表示第 k 组的第 i 件物品的编号是多少，再用 cnt[k] 表示第 k 组物品有多少个。
实现

```
for (int k = 1; k <= ts; k++)           // 循环每一组
for (int i = m; i >= 0; i--)          // 循环背包容量
for (int j = 1; j <= cnt[k]; j++)  // 循环该组的每一个物品
if (i >= w[t[k][j]])              // 背包容量充足
dp[i] = max(dp[i],
dp[i - w[t[k][j]]] + c[t[k][j]]);  // 像 0-1 背包一样状态转移
```

这里要注意：**一定不能搞错循环顺序**，这样才能保证正确性。
有依赖的背包
题面
金明有 n 元钱，想要买 m 个物品，第 i 件物品的价格为 v[i] ，重要度为 p[i] 。有些物品是从属于某个主件物品的附件，要买这个物品，必须购买它的主件。
目标是让所有购买的物品的 v[i] * p[i] 之和最大。
考虑分类讨论。对于一个主件和它的若干附件，有以下几种可能：只买主件，买主件 + 某些附件。因为这几种可能性只能选一种，所以可以将这看成分组背包。
如果是多叉树的集合，则要先算子节点的集合，最后算父节点的集合。
泛化物品的背包
这种背包，没有固定的费用和价值，它的价值是随着分配给它的费用而定。在背包容量为 V 的背包问题中，当分配给它的费用为 v[i] 时，能得到的价值就是 h(v[i]) 。这时，将固定的价值换成函数的引用即可。
杂项
小优化
根据贪心原理，当费用相同时，只需保留价值最高的；当价值一定时，只需保留费用最低的；当有两件物品 i,j 且 i 的价值大于 j 的价值并且 i 的费用小于 j 的费用时，只需保留 i 。
背包问题变种
输出方案
输出方案其实就是记录下来背包中的某一个状态是怎么推出来的。我们可以用 g_(i,v) 表示第 i 件物品占用空间为 v 的时候是否选择了此物品。然后在转移时记录是选用了哪一种策略（选或不选）。
输出时的伪代码：

```
int v = V;  // 记录当前的存储空间

// 因为最后一件物品存储的是最终状态，所以从最后一件物品进行循环
for (从最后一件循环至第一件) {
if (g[i][v]) {
选了第 i 项物品;
v -= 第 i 项物品的重量;
} else {
未选第 i 项物品;
}
}
```

求方案数

对于给定的一个背包容量、物品费用、其他关系等的问题，求装到一定容量的方案总数。

这种问题就是把求最大值换成求和即可。

例如 0-1 背包问题的转移方程就变成了：

dp[i]= sum (dp[i],dp_(i-c[i]))

初始条件： dp[0]=1

因为当容量为 0 时也有一个方案，即什么都不装。

求最优方案总数

要求最优方案总数，我们要对 0-1 背包里的 dp 数组的定义稍作修改，DP 状态 f_(i,j) 为在只能放前 i 个物品的情况下，容量为 j 的背包「正好装满」所能达到的最大总价值。

这样修改之后，每一种 DP 状态都可以用一个 g_(i,j) 来表示方案数。

f_(i,j) 表示只考虑前 i 个物品时背包体积「正好」是 j 时的最大价值。

g_(i,j) 表示只考虑前 i 个物品时背包体积「正好」是 j 时的方案数。

转移方程：

如果 f_(i,j) = f_(i-1,j) 且 f_(i,j) != f_(i-1,j-v)+w 说明我们此时不选择把物品放入背包更优，方案数由 g_(i-1,j) 转移过来，

如果 f_(i,j) != f_(i-1,j) 且 f_(i,j) = f_(i-1,j-v)+w 说明我们此时选择把物品放入背包更优，方案数由 g_(i-1,j-v) 转移过来，

如果 f_(i,j) = f_(i-1,j) 且 f_(i,j) = f_(i-1,j-v)+w 说明放入或不放入都能取得最优解，方案数由 g_(i-1,j) 和 g_(i-1,j-v) 转移过来。

初始条件：

```
memset(f, 0x3f3f, sizeof(f));  // 避免没有装满而进行了转移
f[0] = 0;
g[0] = 1;  // 什么都不装是一种方案
```

因为背包体积最大值有可能装不满，所以最优解不一定是 f_(m) 。

最后我们通过找到最优解的价值，把 g[j] 数组里取到最优解的所有方案数相加即可。

实现

```
 for (int i = 0; i < N; i++) {
for (int j = V; j >= v[i]; j--) {
int tmp = std::max(dp[j], dp[j - v[i]] + w[i]);
int c = 0;
if (tmp == dp[j]) c += cnt[j];                    // 如果从 dp[j] 转移
if (tmp == dp[j - v[i]] + w[i]) c += cnt[j - v[i]];  // 如果从 dp[j-v[i]] 转移
dp[j] = tmp;
cnt[j] = c;
}
}
int max = 0;  // 寻找最优解
for (int i = 0; i <= V; i++) {
max = std::max(max, dp[i]);
}
int res = 0;
for (int i = 0; i <= V; i++) {
if (dp[i] == max) {
res += cnt[i];  // 求和最优解方案数
}
}
```

例题：

Status

Problem

Tags

P1776

.

宝物筛选

背包DP

P5365

.

英雄联盟

背包DP

P1757

.

通天之分组背包

背包DP

P1064

.

金明的预算方案

背包DP

P1273

.

有线电视网

背包DP

剪枝

树形 DP

动态规划 02 • 树形 DP

在学习树形 DP 之前，我们先要搞清楚一个问题，什么是树？根据图论课上学到的知识我们知道，连通的无圈图称为树。而树我们可以把它近似第看成一个分形结构，这是说我们的树其实是可以递归定义的，树的每个子树也是一颗完整的树，而这种结构就天然地适合递归。

具体来说，在树形动态规划当中，我们一般先算子树再进行合并，在实现上与树的后序遍历相似，都是先遍历子树，遍历完之后将子树的值传给父亲。简单来说我们动态规划的过程大概就是先递归访问所有子树，再在根上合并。
了解了树形动态规划的基本思想后，做一些经典的树形 DP 题型。

经典例题

子树大小

给你一棵有个点的树（号点为根节点），求以为根的子树的大小。
这道题作为我们树形动态规划的引例，当然是非常简单的，只要跑一遍即可求出。
具体细节：

- 设以为根的子树大小，则，其中为子节点。 如果写成伪码，大概长这个样子

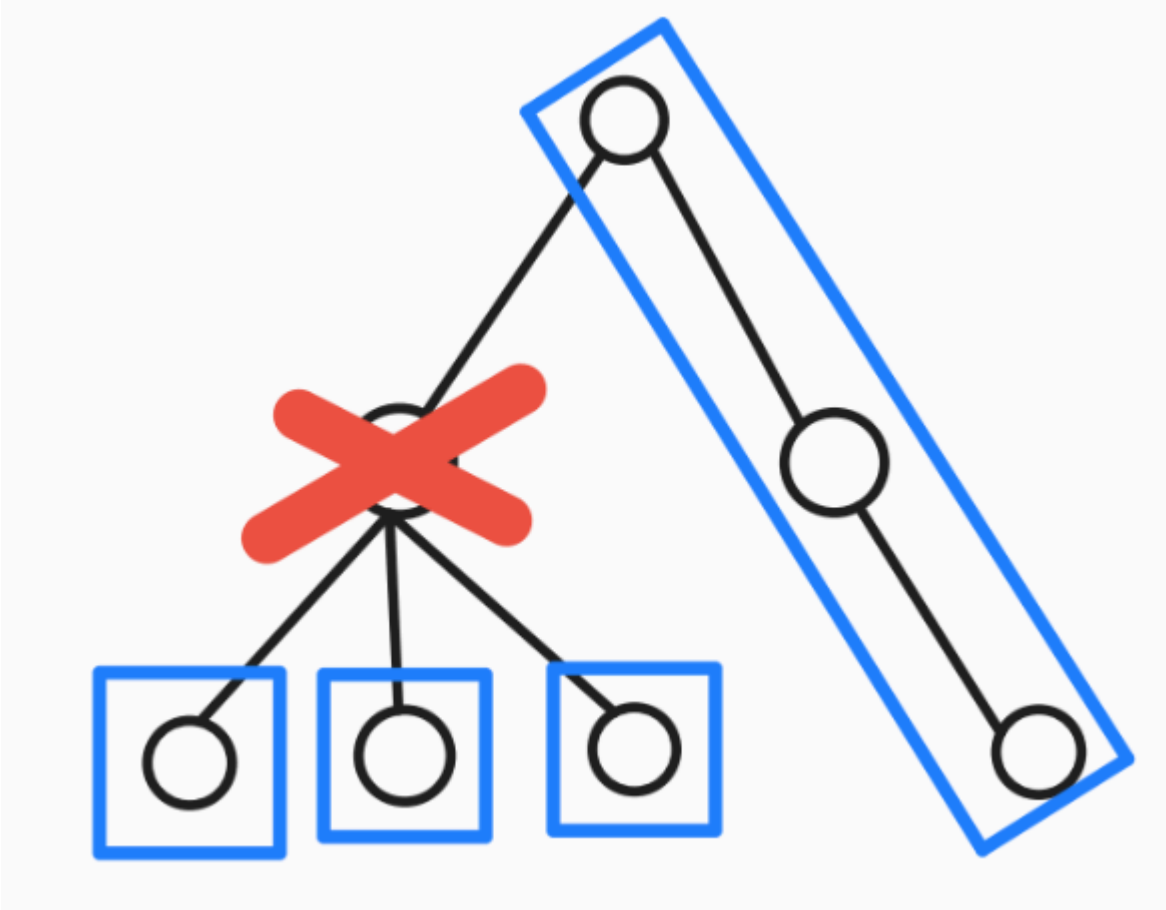
```
void dfs(u){
if (u 是叶子) f[u] = 1 return
for (v 是 u 的儿子){
dfs(v)
f[u] += f[v]
}
f[u] += 1 // 本身
}
```

这就是最简单的树形 DP 了，有没有感受到先遍历子树，遍历完之后将子树的值传给父亲这样的动态规划过程呢？

树的平衡点

给你一个有 n 个点的树，求树的平衡点和删除平衡点后最大子树的节点数。所谓平衡点，指的是树中的一个点，删掉该点，使剩下的若干个连通块中，最大的连通块的块大小最少。
要解决这个问题，我们首先还是先确定状态，现在的问题还很简单，一般就是题目问什么我们就设什么，所以我们先设 F[i] 为删除 i 号点后最大连通块的块大小。然后我们沿用上一题的设 f[i] 为以 i 为根的子树大小，那么很简单就有 F[i] = max(n-f[i],max(f[k])) ，其中 k 是 i 的子节点。这是因为，删除 i 号点后，我们剩下的连通块要么是 i 的子树，要么是 i 的父亲所在的连通块。

> 通俗的来讲，即删除某个节点后，它的儿子就成了独立的连通块，那么最大连通块就是 max(x 的所有儿子连通块最大的 size，n - f[x])



树的平衡点示意图

所以我们只需要先沿用第一题的方法先算出每个点的子树大小，再 DFS 一遍算出每个点的 F[i] ，其值最小的点就是我们树的平衡点。

```
vector<int>e[N];
int ans, idx, f[N];
void dfs(int u, int fa) {
f[u] = 1;
int mx = 0;
for (int v : e[u]) {
if (v == fa)continue;
dfs(v, u);
f[u] += f[v];
mx = max(mx, f[v]);
}
mx = max(mx, n - f[u]);
if (ans > mx) ans = mx, idx = u;
}
```

没有上司的舞会（树的最大独立集）

某大学有 n 个职员，编号为 1 ~ N 。他们之间有从属关系，也就是说他们的关系就像一棵以校长为根的树，父结点就是子结点的直接上司。现在有个周年庆宴会，宴会每邀请来一个职员都会增加一定的快乐指数 a[i] ，但是呢，如果某个职员的直接上司来参加舞会了，那么这个职员就无论如何也不肯来参加舞会了。所以，请你编程计算，邀请哪些职员可以使快乐指数最大，求最大的快乐指数。

我们把题意抽象一下，没有职员会和上司一同参会，也就是说在这棵树上不存在任何一条边使得连接的两个点都来参会，换句话说这道题其实要我们求的是 **树的最大权值的独立集**。

我们设 f(i,0/1) 代表以 i 为根的子树的最优解（第二维的值为 0 代表 i 不参加舞会的情况，1 代表 i 参加舞会的情况）。

对于每个状态，都存在两种决策（其中下面的 x 都是 i 的儿子）：

- 上司不参加舞会时，下属可以参加，也可以不参加，此时有 f(i,0) = sum max f(x,1),f(x,0) ；
- 上司参加舞会时，下属都不会参加，此时有 f(i,1) = sum f(x,0) + a[i] 。

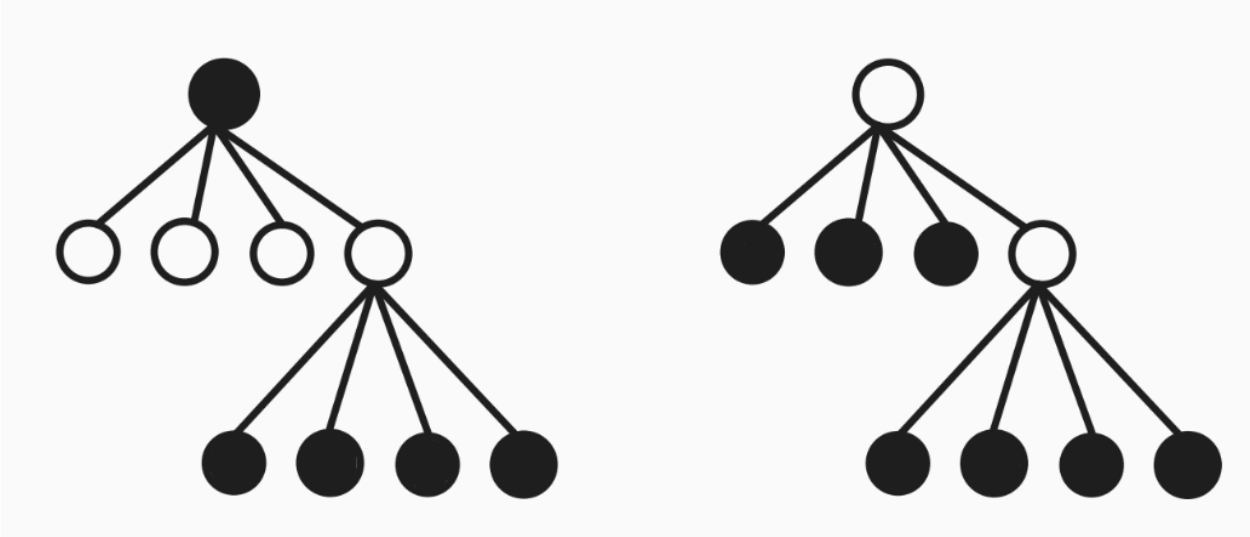


示意图
我们可以通过 DFS，在返回上一层时更新当前结点的最优解。

```
#include <algorithm>
#include <iostream>
using namespace std;

struct edge {
int v, next;
} e[6005];

int head[6005], n, cnt, f[6005][2], ans, is_h[6005], vis[6005];

void addedge(int u, int v) {  // 建图
e[++cnt].v = v;
e[cnt].next = head[u];
head[u] = cnt;
}

void calc(int k) {
vis[k] = 1;
for (int i = head[k]; i; i = e[i].next) {  // 枚举该结点的每个子结点
if (vis[e[i].v]) continue;
calc(e[i].v);
f[k][1] += f[e[i].v][0];
f[k][0] += max(f[e[i].v][0], f[e[i].v][1]);  // 转移方程
}
return;
}

int main() {
cin.tie(nullptr)->sync_with_stdio(false);
cin >> n;
for (int i = 1; i <= n; i++) cin >> f[i][1];
for (int i = 1; i < n; i++) {
int l, k;
cin >> l >> k;
is_h[l] = 1;
addedge(k, l);
}
for (int i = 1; i <= n; i++)
if (!is_h[i]) {  // 从根结点开始 DFS
calc(i);
cout << max(f[i][1], f[i][0]);
return 0;
}
}
```

通常，树形 DP 状态一般都为当前节点的最优解。先 DFS 遍历子树的所有最优解，然后向上传递给子树的父节点来转移，最终根节点的值即为所求的最优解。

例题：

Status

Problem

Tags

P1122

.

最大子树和

树形DP

P2014

.

[CTSC1997] 选课

树形DP

P2015

.

二叉苹果树

树形DP

P2279

.

消防局的设立

树形DP

P3177

.

树上染色

树形DP

背包 DP 进阶

状压 DP

动态规划 03 • 状压 DP

定义

状压 DP 是动态规划的一种，通过将状态压缩为整数来达到优化转移的目的。

例题 1

互不侵犯

在 N * N 的棋盘里面放 K 个国王（ 1 <= N <= 9, 1 <= K <= N * N ），使他们互不攻击，共有多少种摆放方案。

国王能攻击到它上下左右，以及左上左下右上右下八个方向上附近的各一个格子，共 8 个格子。

解释

设 f(i,j,l) 表示前 i 行，第 i 行的状态为 j ，且棋盘上已经放置 l 个国王时的合法方案数。

对于编号为 j 的状态，我们用二进制整数 sit(j) 表示国王的放置情况， sit(j) 的某个二进制位为 0 表示对应位置不放国王，为 1 表示在对应位置上放置国王；用 sta(j) 表示该状态的国王个数，即二进制数 sit(j) 中 1 的个数。例如，如下图所示的状态可用二进制数 100101 来表示（棋盘左边对应二进制低位），则有 sit(j)=100101_((2))=37, sta(j)=3 。



图示

设当前行的状态为 j ，上一行的状态为 x ，可以得到下面的状态转移方程： f(i,j,l) = sum f(i-1,x,l-sta(j)) 。

设上一行的状态编号为 x ，在保证当前行和上一行不冲突的前提下，枚举所有可能的 x 进行转移，转移方程：

f(i,j,l) = sum f(i-1,x,l-sta(j))

实现

```
#include <algorithm>
#include <iostream>
using namespace std;
long long sta[2005], sit[2005], f[15][2005][105];
int n, k, cnt;

void dfs(int x, int num, int cur) {
if (cur >= n) {  // 有新的合法状态
sit[++cnt] = x;
sta[cnt] = num;
return;
}
dfs(x, num, cur + 1);  // cur 位置不放国王
dfs(x + (1 << cur), num + 1,
cur + 2);  // cur 位置放国王，与它相邻的位置不能再放国王
}

bool compatible(int i, int x) {
if (sit[i] & sit[x]) return false;
if ((sit[i] << 1) & sit[x]) return false;
if (sit[i] & (sit[x] << 1)) return false;
return true;
}

int main() {
cin >> n >> k;
dfs(0, 0, 0);  // 先预处理一行的所有合法状态
for (int i = 1; i <= cnt; i++) f[1][j][sta[j]] = 1;
for (int i = 2; i <= n; i++)
for (int j = 1; j <= cnt; j++)
for (int x = 1; x <= cnt; x++) {
if (!compatible(j, x)) continue;  // 排除不合法转移
for (int l = sta[j]; l <= k; l++) f[i][j][l] += f[i - 1][x][l - sta[j]];
}
long long ans = 0;
for (int i = 1; i <= cnt; i++) ans += f[n][i][k];  // 累加答案
cout << ans << endl;
return 0;
}
```

例题 2

过桥

有 n 个人需要过桥，第 i 的人的重量为 w[i] ，过桥用时为 t[i] . 这些人过桥时会分成若干组，只有在某一组的所有人全部过桥后，其余的组才能过桥。桥最大承重为 W ，问这些人全部过桥的最短时间。

100 <= W <= 400 ， 1 <= n <= 16 ， 1 <= t[i] <= 50 ， 10 <= w[i] <= 100 .

解释

我们用 S 表示所有人构成集合的一个子集，设 t(S) 表示 S 中人的最长过桥时间， w(S) 表示 S 中所有人的总重量， f(S) 表示 S 中所有人全部过桥的最短时间，则：

f(空集)=0, f(S)=minlimits_(T 包含于 S;~w(T) <= W)t(T)+f(S - T).

实现

```cpp
#include <iostream>
#include <limits>
#include <vector>
using namespace std;

int main() {
ios::sync_with_stdio(false);
cin.tie(nullptr);
int W, n;
cin >> W >> n;
const int S = (1 << n) - 1;
vector<int> ts(S + 1), ws(S + 1);
for (int j = 0, t, w; j < n; ++j) {
cin >> t >> w;
for (int i = 0; i <= S; ++i)
if (i & (1 << j)) {
ts[i] = max(ts[i], t);
ws[i] += w;
}
}
vector<int> dp(S + 1, numeric_limits<int>::max() / 2);
for (int i = 0; i <= S; ++i) {
if (ws[i] <= W) dp[i] = ts[i];
for (int j = i; j; j = i & (j - 1))
if (ws[i ^ j] <= W) dp[i] = min(dp[i], dp[j] + ts[i ^ j]);
}
cout << dp[S] << '\n';
return 0;
}
```

例题：

Status

Problem

Tags

P2622

.

关灯问题 II

状压DP

P3052

.

Cows in a Skyscraper G

状压DP

B2002

.

Corn Fields G

状压DP

P2157

.

学校食堂

状压DP

P2566

.

围豆豆

状压DP

P3736

.
字符合并
状压DP
树形 DP
数位 DP

动态规划 04 • 数位 DP

本页面将简要介绍数位 DP。

引入

数位是指把一个数字按照个、十、百、千等等一位一位地拆开，关注它每一位上的数字。如果拆的是十进制数，那么每一位数字都是 0~9，其他进制可类比十进制。

数位 DP：用来解决一类特定问题，这种问题比较好辨认，一般具有这几个特征：

1. 要求统计满足一定条件的数的数量（即，最终目的为计数）；
2. 这些条件经过转化后可以使用「数位」的思想去理解和判断；
3. 输入会提供一个数字区间（有时也只提供上界）来作为统计的限制；
4. 上界很大（比如 10^(18) ），暴力枚举验证会超时。

数位 DP 的基本原理：

考虑人类计数的方式，最朴素的计数就是从小到大开始依次加一。但我们发现对于位数比较多的数，这样的过程中有许多重复的部分。例如，从 7000 数到 7999、从 8000 数到 8999、和从 9000 数到 9999 的过程非常相似，它们都是后三位从 000 变到 999，不一样的地方只有千位这一位，所以我们可以把这些过程归并起来，将这些过程中产生的计数答案也都存在一个通用的数组里。此数组根据题目具体要求设置状态，用递推或 DP 的方式进行状态转移。

数位 DP 中通常会利用常规计数问题技巧，比如把一个区间内的答案拆成两部分相减（即 ans_([l, r]) = ans_([0, r])-ans_([0, l - 1]) ）

那么有了通用答案数组，接下来就是统计答案。统计答案可以选择记忆化搜索，也可以选择循环迭代递推。为了不重不漏地统计所有不超过上限的答案，要从高到低枚举每一位，再考虑每一位都可以填哪些数字，最后利用通用答案数组统计答案。

接下来我们具体看几道题目。

例题一

题目大意：给定两个正整数 a,b ，求在 [a,b] 中的所有整数中，每个数码（digit）各出现了多少次。

方法一

解释

发现对于满 i 位的数，所有数字出现的次数都是相同的，故设数组 dp[i] 为满 i 位的数中每个数字出现的次数，此时暂时不处理前导零。则有 dp[i]=10 * dp_(i-1)+10^(i-1) ，这两部分前一个是来自前 i-1 位数字的贡献，后一个是来自第 i 位的数字的贡献。

有了 dp 数组，我们来考虑如何统计答案。将上界按位分开，从高到低枚举，不贴着上界时，后面可以随便取值。贴着上界时，后面就只能取 0 到上界，分两部分分别计算贡献。最后考虑下前导零，第 i 位为前导 0 时，此时 1 到 i-1 位也都是 0 ，也就是多算了将 i-1 位填满的答案，需要额外减去。

实现

```
#include <cstdio>
using namespace std;
constexpr int N = 15;
using ll = long long;
ll l, r, dp[N], mi[N];
ll ans1[N], ans2[N];
int a[N];

void solve(ll n, ll *ans) {
ll tmp = n;
int len = 0;
while (n) a[++len] = n % 10, n /= 10;
for (int i = len; i >= 1; --i) {
for (int j = 0; j < 10; j++) ans[j] += dp[i - 1] * a[i];
for (int j = 0; j < a[i]; j++) ans[j] += mi[i - 1];
tmp -= mi[i - 1] * a[i], ans[a[i]] += tmp + 1;
ans[0] -= mi[i - 1];
}
}

int main() {
scanf("%lld%lld", &l, &r);
mi[0] = 1ll;
for (int i = 1; i <= 13; ++i) {
dp[i] = dp[i - 1] * 10 + mi[i - 1];
mi[i] = 10ll * mi[i - 1];
}
solve(r, ans1), solve(l - 1, ans2);
for (int i = 0; i < 10; ++i) printf("%lld ", ans1[i] - ans2[i]);
return 0;
}
```

方法二

解释

此题也可以使用记忆化搜索。 dp[i] 表示不贴上限、无前导零时，位数为 i 的答案。

详见代码注释

过程

```
#include <cstdio>
#include <cstring>
#include <iostream>
using namespace std;
using ll = long long;
constexpr int N = 50005;
ll a, b;
ll f[15], ksm[15], p[15], now[15];

ll dfs(int u, int x, bool f0,
bool lim) {  // u 表示位数，f0 是否有前导零，lim 是否都贴在上限上
if (!u) {
if (f0) f0 = false;
return 0;
}
if (!lim && !f0 && (~f[u])) return f[u];
ll cnt = 0;
int lst = lim ? p[u] : 9;
for (int i = 0; i <= lst; i++) {  // 枚举这位要填的数字
if (f0 && i == 0)
cnt += dfs(u - 1, x, 1, lim && i == lst);  // 处理前导零
else if (i == x && lim && i == lst)
cnt += now[u - 1] + 1 +
dfs(u - 1, x, 0,
lim && i == lst);  // 此时枚举的前几位都贴在给定的上限上。
else if (i == x)
cnt += ksm[u - 1] + dfs(u - 1, x, 0, lim && i == lst);
else
cnt += dfs(u - 1, x, 0, lim && i == lst);
}
if ((!lim) && (!f0)) f[u] = cnt;  // 只有不贴着上限和没有前导零才能记忆
return cnt;
}

ll gans(ll d, int dig) {
int len = 0;
memset(f, -1, sizeof(f));
while (d) {
p[++len] = d % 10;
d /= 10;
now[len] = now[len - 1] + p[len] * ksm[len - 1];
}
return dfs(len, dig, 1, 1);
}

int main() {
scanf("%lld%lld", &a, &b);
ksm[0] = 1;
for (int i = 1; i <= 12; i++) ksm[i] = ksm[i - 1] * 10ll;
for (int i = 0; i < 9; i++) printf("%lld ", gans(b, i) - gans(a - 1, i));
printf("%lld\n", gans(b, 9) - gans(a - 1, 9));
return 0;
}
```

例题二

题面大意：统计一个区间内数位上不能有 4 也不能有连续的 62 的数有多少。

解释

没有 4 的话在枚举的时候判断一下，不枚举 4 就可以保证状态合法了，所以这个约束没有记忆化的必要，而对于 62 的话，涉及到两位，当前一位是 6 或者不是 6 这两种不同情况计数是不相同的，所以要用状态来记录不同的方案数。 dp_(pos,sta) 表示当前第 pos 位，前一位是否是 6 的状态，这里 sta 只需要取 0 和 1 两种状态就可以了，不是 6 的情况可视为同种，不会影响计数。

实现

```
#include <cstdio>
#include <cstring>
#include <iostream>
using namespace std;
int x, y, dp[15][3], p[50];

void pre() {
memset(dp, 0, sizeof(dp));
dp[0][0] = 1;
for (int i = 1; i <= 10; i++) {
dp[i][0] = dp[i - 1][0] * 9 - dp[i - 1][1];
dp[i][1] = dp[i - 1][0];
dp[i][2] = dp[i - 1][2] * 10 + dp[i - 1][1] + dp[i - 1][0];
}
}

int cal(int x) {
int cnt = 0, ans = 0, tmp = x;
while (x) {
p[++cnt] = x % 10;
x /= 10;
}
bool flag = false;
p[cnt + 1] = 0;
for (int i = cnt; i; i--) {  // 从高到低枚举数位
ans += p[i] * dp[i - 1][2];
if (flag)
ans += p[i] * dp[i - 1][0];
else {
if (p[i] > 4) ans += dp[i - 1][0];
if (p[i] > 6) ans += dp[i - 1][1];
if (p[i] > 2 && p[i + 1] == 6) ans += dp[i][1];
if (p[i] == 4 || (p[i] == 2 && p[i + 1] == 6)) flag = true;
}
}
return tmp - ans;
}

int main() {
pre();
while (~scanf("%d%d", &x, &y)) {
if (!x && !y) break;
x = min(x, y), y = max(x, y);
printf("%d\n", cal(y + 1) - cal(x));
}
return 0;
}
```

例题三

题目大意：给定一个区间 [l,r] ，求其中满足条件 **不含前导 0 且相邻两个数字相差至少为 2** 的数字个数。

解释

首先我们将问题转化成更加简单的形式。设 ans[i] 表示在区间 [1,i] 中满足条件的数的数量，那么所求的答案就是 ans[r]-ans_(l-1) 。

对于一个小于 n 的数，它从高到低肯定出现某一位，使得这一位上的数值小于 n 这一位上对应的数值。而之前的所有位都和 n 上的位相等。

有了这个性质，我们可以定义 f(i,st,op) 表示当前将要考虑的是从高到低的第 i 位，当前该前缀的状态为 st 且前缀和当前求解的数字的大小关系是 op （ op=1 表示等于， op=0 表示小于）时的数字个数。在本题中，这个前缀的状态就是上一位的值，因为当前将要确定的位不能取哪些数只和上一

位有关。在其他题目中，这个值可以是：前缀的数字和，前缀所有数字的 gcd ，该前缀取模某个数的余数，也有两种或多种合用的情况。

写出 **状态转移方程**： f(i,st,op)= sum (k=1 ~ maxx) f(i+1,k,op=1~ and~ k=maxx) (|st-k| >= 2)

这里的 k 就是当前枚举的下一位的值，而 maxx 就是当前能取到的最高位。因为如果 op=1 ，那么你在这一位上取的值一定不能大于求解的数字上该位的值，否则则没有限制。

我们发现，尽管前缀所选择的状态不同，而 f 的三个参数相同，答案就是一样的。为了防止这个答案被计算多次，可以使用记忆化搜索的方式实现。

实现

```
int dfs(int x, int st, int op)  // op=1 =; op=0 <
{
if (!x) return 1;
if (!op && ~f[x][st]) return f[x][st];
int maxx = op ? dim[x] : 9, ret = 0;
for (int i = 0; i <= maxx; i++) {
if (abs(st - i) < 2) continue;
if (st == 11 && i == 0)
ret += dfs(x - 1, 11, op & (i == maxx));
else
ret += dfs(x - 1, i, op & (i == maxx));
}
if (!op) f[x][st] = ret;
return ret;
}

int solve(int x) {
memset(f, -1, sizeof f);
dim.clear();
dim.push_back(-1);
int t = x;
while (x) {
dim.push_back(x % 10);
x /= 10;
}
return dfs(dim.size() - 1, 11, 1);
}
```

例题四

题面大意：假如手写下 [n,m] 之间所有整数，会有多少数看起来和在镜子里看起来一模一样？（ n,m<10^(44), T<10^5 ）

解释

注：由于这里考虑到的镜像，只有 0,1,8 的镜像是自己本身。所以，这里的「一模一样」并不是传统意义上的回文串，而是只含有 0,1,8 的回文串。

首先，在数位 DP 过程中，显然只有 0,1,8 能被选中。

其次，由于数值超过 long long 范围，所以 [n,m]=[1,m]-[1,n-1] 不再适用（高精度比较繁琐），而是需要对 n 是否合法进行判断，得出： [n,m]=[1,m]-[1,n]+check(n) 。

镜像解决了，如何判断回文？

我们需要用一个小数组记录一下之前的值。在未超过一半的长度时，只要不超上限就行；在超过一半的长度时，还需要判断是否和与之「镜面对称」的位相等。

需要额外注意的是，这道题的记忆化部分，不能用 `memset` ，否则会导致超时。

实现

```
int check(char cc[]) {  // n 的特判
int strc = strlen(cc);
for (int i = 0; i < strc; ++i) {
if (!(cc[i] == cc[strc - i - 1] &&
(cc[i] == '1' || cc[i] == '8' || cc[i] == '0')))
return 0ll;
}
return 1ll;
}

// now: 当前位  eff: 有效位  fulc: 是否全顶格  ful0: 是否全0
int dfs(int now, int eff, bool ful0, bool fulc) {
if (now == 0) return 1ll;
if (!fulc && f[now][eff][ful0] != -1)  // 记忆化
return f[now][eff][ful0];

int res = 0, maxk = fulc ? dig[now] : 9;
for (int i = 0; i <= maxk; ++i) {
if (i != 0 && i != 1 && i != 8) continue;
b[now] = i;
if (ful0 && i == 0)  // 全前导 0
res += dfs(now - 1, eff - 1, 1, 0);
else if (now > eff / 2)                                  // 未过半程
res += dfs(now - 1, eff, 0, fulc && (dig[now] == i));  // 已过半程
else if (b[now] == b[eff - now + 1])
res += dfs(now - 1, eff, 0, fulc && (dig[now] == i));
}
if (!fulc) f[now][eff][ful0] = res;
return res;
}

char cc1[100], cc2[100];
int strc, ansm, ansn;

int get(char cc[]) {  // 处理封装
strc = strlen(cc);
for (int i = 0; i < strc; ++i) dig[strc - i] = cc[i] - '0';
return dfs(strc, strc, 1, 1);
}

scanf("%s%s", cc1, cc2);
printf("%lld\n", get(cc2) - get(cc1) + check(cc1));
```

例题五

题面：我们称一个正整数 x 是幸运数，当且仅当它的十进制表示中不包含数字串集合 S 中任意一个元素作为其子串。例如当 S = 22, 333, 0233 时， 233233 是幸运数， 23332333 、 2023320233 、 32233223 不是幸运数。给定 n 和 S ，计算不大于 n 的幸运数个数。答案对 10^9 + 7 取模。

1 <= n<10^(1201)，1 <= m <= 100，1 <= sum *(i = 1)^(m) |s[i]| <= 1500，min*(i = 1)^(m) |s[i]| >= 1 ，其中 |s[i]| 表示字符串 s[i] 的长度。 n 没有前导 0 ，但是 s[i] 可能有前导 0 。

解释

阅读题面发现，如果将数字看成字符串，那么这就是需要完成一个多模匹配，自然而然就想到 AC 自动机。普通数位 DP 中，先从高到低枚举数位，再枚举每一位都填什么，在这道题中，我们也就自然地转化为枚举已经填好的位数，再枚举此时停在 AC 自动机上的哪个节点，然后从当前节点转移到它在 AC 自动机上的子节点。

设 f(i,j,0/1) 表示当前从高到低已经填了 i 位（即在 AC 自动机上走过了 i 条边），此时停在标号为 j 的节点上，当前是否正好贴着上界。

至于题目中的「不包含」条件，只需在 AC 自动机上给每个模式串的结尾节点都打上标记，DP 过程中一旦遇上这些结尾节点就跳过即可。

转移很好想，详见代码主函数部分。

实现

```
#include <cstdio>
#include <cstring>
#include <queue>
using namespace std;
using ll = long long;
constexpr int N = 1505;
constexpr int mod = 1000000007;
int n, m;
char s[N], c[N];
int ch[N][10], fail[N], ed[N], tot, len;

void insert() {
int now = 0;
int l = strlen(s);
for (int i = 0; i < l; ++i) {
if (!ch[now][s[i] - '0']) ch[now][s[i] - '0'] = ++tot;
now = ch[now][s[i] - '0'];
}
ed[now] = 1;
}

queue<int> q;

void build() {
for (int i = 0; i < 10; ++i)
if (ch[0][i]) q.push(ch[0][i]);
while (!q.empty()) {
int u = q.front();
q.pop();
for (int i = 0; i < 10; ++i) {
if (ch[u][i]) {
fail[ch[u][i]] = ch[fail[u]][i], q.push(ch[u][i]),
ed[ch[u][i]] |= ed[fail[ch[u][i]]];
} else
ch[u][i] = ch[fail[u]][i];
}
}
ch[0][0] = 0;
}

ll f[N][N][2], ans;

void add(ll &x, ll y) { x = (x + y) % mod; }

int main() {
scanf("%s", c);
n = strlen(c);
scanf("%d", &m);
for (int i = 1; i <= m; ++i) scanf("%s", s), insert();
build();
f[0][0][1] = 1;
for (int i = 0; i < n; ++i) {
for (int j = 0; j <= tot; ++j) {
if (ed[j]) continue;
for (int k = 0; k < 10; ++k) {
if (ed[ch[j][k]]) continue;
add(f[i + 1][ch[j][k]][0], f[i][j][0]);
if (k < c[i] - '0') add(f[i + 1][ch[j][k]][0], f[i][j][1]);
if (k == c[i] - '0') add(f[i + 1][ch[j][k]][1], f[i][j][1]);
}
}
}
for (int j = 0; j <= tot; ++j) {
if (ed[j]) continue;
add(ans, f[n][j][0]);
```

```
add(ans, f[n][j][1]);
}
printf("%lld\n", ans - 1);
return 0;
}
```

此题可以很好地帮助理解数位 DP 的原理。

例题：

Status

Problem

Tags

P2602

.

数位计数

数位DP

P4999

.

烦人的数学作业

数位DP

P6218

.

Round Numbers S

数位DP

P1831

.

杠杆数

数位DP

P2481

.

代码拍卖会

数位DP

状压 DP

计数 DP

动态规划 05 • 计数 DP

计数 DP 是一种利用类似 DP 的记忆化搜索方法（与在狭义上的 DP，即最优化问题有一定区别），用于解决计数（以及求和）问题。

基础

基本思想

计数问题一般指求一个集合 S 的大小，在 OI 中， S 的大小有时会达到 Theta(n^n) 甚至 Theta(2^(n!)) 的级别（当然，一般会对某一个固定的数取模），其中 n 是问题规模，所以我们不能逐一求出 S 的元素。

如果我们能够将 S 分成若干无交的子集，那么 S 的元素个数就等于这些部分的元素个数的和。如果这些子集的计数恰好与原问题类似，那么我们就可以通过类似动态规划的方法来解决。

例题

给定一个正整数 n ，求有多少个把 n 划分成 k 个正整数的和的方案，位置调换视为不同的划分方案。

需集合 S_(n,k) 为形如 (a[1], …, a[k]) 的正整数组组成的集合，其中 a[1] + … + a[k] = n 。

如果 a[k] 固定，则有如下推导：因为 a[1] + a[2] + … + a[k-1] + a[k] = n ，所以 a[1] +

a[2] + … + a[k-1] = n - a[k] 。根据 S_(n,k) 的定义， (a[1], a[2], …, a[k-1]) 属于 S_(n - a[k], k - 1) 。
由于 a[1], a[2], …, a[k] 是正整数，所以 a[k] 的取值范围是 [1, n - k + 1] 交 mathbb Z 。
因此， S_(n,k) 可以按照 a[k] 被划分，分成 n - k + 1 个子集，其中当 a[k] = i 时，这个子集为：
(L, i) | L 属于 S_(n-i,k-1).
这个子集的元素个数显然等于 S_(n-i,k-1) ，由于 i 的不同，这些子集两两无交。所以：
|S_(n,k)| = sum (i=1 ~ n-k+1) |S_(n-i,k-1)|.
这样我们就可以使用类似 DP 的方法处理它：设 f_(n,k) 为 |S_(n,k)| ，则有状态转移方程：
f_(n,k) = sum (i=1 ~ n-k+1) f_(n-i,k-1).
这样就可以使用 DP 的方法求解了。

与最优化 DP 的异同

可以发现，计数 DP 和最优化 DP 都是在一个范围 Omega 内求一个值（大小值、最优值），这个值通过将 Omega 中的所有元素做一次处理，再对处理值做一次整合得到。
例如，对于 0-1 背包问题， Omega 中的元素为背包内的所有物品组成的集合，对于 Omega 中的一个方案 S ，我们对 S 做一次处理，处理得到的结果 w(S) 为 S 中物品的总价值，对所有得到的处理值，我们取最大值，得到问题的答案。
对于计数问题， Omega 中的元素为要计算元素个数的集合 S ，它的处理是把所有的 S 中元素变为 1 ，然后将这些 1 通过加法的方式汇总起来，因为每一个 S 中元素都对应一个 1 ，所以这样得到的值就是 S 中元素个数。
当汇总操作为最大/最小值时，我们可以将 Omega 分成任意若干个部分，只需这些部分的并为 Omega 即可，无需无交的条件。而计数问题由于不满足这个条件，所以我们需要将 Omega 分成若干个部分，这些部分两两无交，这就是与最优化 DP 的区别。

例题：

Status
Problem
Tags
P3205
.
合唱队
计数DP
P5664
.
Emiya 家今天的饭
计数DP
P1450
.
硬币购物
计数DP
数位 DP
单调队列/栈优化 DP

动态规划 06 · 单调队列/栈优化 DP

引入

前置知识：单调队列、单调栈。
单调队列主要用于维护两端指针单调不减的区间最值，而单调栈则主要用于维护前/后第一个大于/小于当前值的数。

- 求最小值要维护 **单调递增/不减** 的单调队列/单调栈，反之亦然。
- 维护单调递增/递减比较时用 **小于等于/大于等于**，维护单调不减/不增比较时用 **小于/大于**。

单调队列优化具体步骤

- 加入所需元素：向单调队列重复加入元素直到当前元素达到所求区间的右边界，这样就能保证所需元素都在单调队列中。
- 弹出越界队首：单调队列本质上是维护的是所有已插入元素的最值，但我们想要的往往是一个区间最值。于是我们弹出在左边界外的元素，以保证单调队列中的元素都在所求区间中。
- 获取最值：直接取队首作为答案即可。

单调栈优化具体步骤

- 弹出非法栈顶：通过比较当前元素与栈顶的大小，弹出不满足单调栈性质的栈顶。以单调递增的栈（即栈顶最大，维护最小值）为例，将所有大于等于当前元素的栈内元素全部弹出。
- 加入当前元素：将当前元素入栈即可。

单调队列优化多重背包

你有 n 个物品，每个物品重量为 w[i] ，价值为 v[i] ，数量为 k[i] 。你有一个承重上限为 W 的背包，现在要求你在不超过重量上限的情况下选取价值和尽可能大的物品放入背包。求最大价值。

设 f_(i,j) 表示前 i 个物品装入承重为 j 的背包的最大价值，朴素的转移方程为

f_(i,j)=max(k=0 ~ k[i])(f_(i-1,j-k * w[i])+v[i] * k)

时间复杂度 O(W sum k[i]) 。

考虑优化 f[i] 的转移。为方便表述，设 g_(x,y)=f_(i,x * w[i]+y),g'*(x,y)=f*(i-1,x * w[i]+y) ，其中 0 <= y lt w[i] ，则转移方程可以表示为：

g_(x,y)=max(k=0 ~ k[i])(g'_(x-k,y)+v[i] * k)

设 G_(x,y)=g'_(x,y)-v[i] * x 。则方程可以表示为：

g_(x,y)=max(k=0 ~ k[i])(G_(x-k,y))+v[i] * x

这样就转化为一个经典的单调队列优化形式了。 G_(x,y) 可以 O(1) 计算，因此对于固定的 y ，我们可以在 O(floor (W)/(w[i])) 的时间内计算出 g_(x,y) 。因此求出所有 g_(x,y) 的复杂度为 O(floor (W)/(w[i])) * O(w[i])=O(W) 。这样转移的总复杂度就降为 O(nW) 。

在实现的时候，我们需要先枚举 y ，这样才能保证枚举 x 的时候利用单调队列进行优化，而单调队列中存储的是 x-k ，并不存储 k ，这样使用的时候需要通过 `f[last][q.front() * w[i] + y] - q.front() * v[i]` 获取对应的 G_(x-k,y) ，不难发现 x-k 属于 [x - k[i],x] ，因此在枚举 x 的时候，我们需要删除队列中不在这个范围内的元素。

```cpp
#include <array>
#include <deque>
#include <iostream>

constexpr int MAXV = 4e4 + 10;
constexpr int MAXN = 1e2 + 10;

using namespace std;

int n, W, last = 0, now = 1;
array<int, MAXN> v, w, k;
array<array<int, MAXV>, 2> f;
deque<int> q;

int main() {
ios::sync_with_stdio(false);
cin >> n >> W;
for (int i = 1; i <= n; i++) {
cin >> v[i] >> w[i] >> k[i];
}
for (int i = 1; i <= n; i++) {
for (int y = 0; y < w[i]; y++) {
// 清空队列
deque<int>().swap(q);
for (int x = 0; x * w[i] + y <= W; x++) {
// 弹出不在范围的元素
while (!q.empty() && q.front() < x - k[i]) {
q.pop_front();
}
// 保证队列单调
while (!q.empty() && f[last][q.back() * w[i] + y] - q.back() * v[i] <
f[last][x * w[i] + y] - x * v[i]) {
q.pop_back();
}
q.push_back(x);
f[now][x * w[i] + y] =
f[last][q.front() * w[i] + y] - q.front() * v[i] + x * v[i];
}
}
swap(last, now);
}
cout << f[last][W] << endl;
return 0;
}
```

习题

题目大意：城镇中有 n 个位置，有 m 个烟花要放。第 i 个烟花放出的时间记为 t[i] ，放出的位置记为 a[i] 。如果烟花放出的时候，你处在位置 x ，那么你将收获 b[i]-|a[i]-x| 点快乐值。

初始你可在任意位置，你每个单位时间可以移动不大于 d 个单位距离。现在你需要最大化你能获得的快乐值。

设 f_(i,j) 表示在放第 i 个烟花时，你的位置在 j 所能获得的最大快乐值。

写出状态转移方程： f_(i,j)=maxf_(i-1,k)+b[i]-|a[i]-j| ，其中 j-(t[i]-t[i-1]) * d <= k <= j+(t[i]-t[i-1]) * d 。

尝试变形：

由于 max 里出现了一个确定的常量 b[i] ，我们可以将它提到外面去。

f_(i,j)=maxf_(i-1,k)+b[i]-|a[i]-j|=maxf_(i-1,k)-|a[i]-j|+b[i]

如果确定了 i 和 j 的值，那么 |a[i]-j| 的值也是确定的，也可以将这一部分提到外面去。

最后，式子变为：

f_(i,j)=maxf_(i-1,k)-|a[i]-j|+b[i]=maxf_(i-1,k)-|a[i]-j|+b[i]

接下来考虑单调队列优化。由于最终式子中的 max 只和上一状态中连续的一段的最大值有关，所以我们在计算一个新的 i 的状态值时候只需将原来的 f[i-1] 构造成一个单调队列，并维护单调队列，使得其能在均摊 O(1) 的时间复杂度内计算出 maxf_(i-1,k) 的值，从而根据公式计算出 f_(i,j) 的值。

总的时间复杂度为 O(nm) 。

```
#include <algorithm>
#include <cstring>
#include <iostream>
using namespace std;
using ll = long long;

constexpr int MAXN = 150000 + 10;
constexpr int MAXM = 300 + 10;

ll f[2][MAXN];
ll a[MAXM], b[MAXM], t[MAXM];
int n, m, d;

int que[MAXN];

int fl = 1;

void init() {
memset(f, 207, sizeof(f));
memset(que, 0, sizeof(que));
for (int i = 1; i <= n; i++) f[0][i] = 0;
fl = 1;
}

void dp() {
init();
for (int i = 1; i <= m; i++) {
int l = 1, r = 0, k = 1;
for (int j = 1; j <= n;
j++) { // 在这里使用了单调队列的优化，推式子详见上面
for (; k <= min(1ll * n, j + d * (t[i] - t[i - 1])); k++) {
while (l <= r && f[fl ^ 1][que[r]] <= f[fl ^ 1][k]) r--;
que[++r] = k;
}

while (l <= r && que[l] < max(1ll, j - d * (t[i] - t[i - 1]))) l++;
f[fl][j] = f[fl ^ 1][que[l]] - abs(a[i] - j) + b[i];
}

fl ^= 1;
}
}

int main() {
cin >> n >> m >> d;
for (int i = 1; i <= m; i++) cin >> a[i] >> b[i] >> t[i];

// then dp
dp();
ll ans = -1e18;
for (int j = 1; j <= n; j++) ans = max(ans, f[fl ^ 1][j]);
cout << ans << endl;
return 0;
}
```

例题：

Status

Problem
Tags
P6040
.
课后期末考试滑溜滑溜补习班
DP优化
P2569
.
股票交易
DP优化
P3022
.
Odd degrees G
DP优化
P3572
.
PTA-Little Bird
DP优化
计数 DP
四边形不等式优化 DP

动态规划 07 · 四边形不等式优化 DP

四边形不等式优化利用的是状态转移方程中的决策单调性。

基础知识

考虑最简单的情形，我们要解决如下一系列最优化问题。

f(i) = min_(1 <= j <= i) w(j,i) (1 <= i <= n) 1

这里假定成本函数 w(j,i) 可以在 O(1) 时间内计算。

约定

动态规划的状态转移方程经常可以写作一系列最优化问题的形式。以（1）式为例，这些问题含有参数 i ，问题的目标函数和可行域都可以依赖于 i 。每一个问题都是在给定参数 i 时，选取某个可行解 j 来最小化目标函数的取值。为表述方便，下文将参数为 i 的最优化问题简称为「问题 i 」，该最优化问题的可行解 j 称为「决策 j 」，目标函数在最优解处取得的值则称为「状态 f(i) 」。同时，记问题 i 对应的最小最优决策点为 mathopopt(i) 。

在一般的情形下，这些问题总时间复杂度为 O(n^2) 。这是由于对于问题 i ，我们需要考虑所有可能的决策 j 。而在满足决策单调性时，可以有效缩小决策空间，优化总复杂度。

- **决策单调性**：对于任意 i[1] < i[2] ，必然成立 mathopopt(i[1]) <= mathopopt(i[2]) 。

附注

对于问题 i ，最优决策集合未必是一个区间。决策单调性实际可以定义在最优决策集合上。对于集合 A 和 B ，可以定义 A <= B 当且仅当对于任意 a 属于 A 和 b 属于 B ，成立 mina,b 属于 A 和 maxa,b 属于 B 。这蕴含最小（最大）最优决策点的单调性，即此处采取的定义。本文关于最小最优决策点叙述的结论，同样适用于最大最优决策点。但是，存在情形，某更大问题的最小最优决策严格小于另一更小问题的最大最优决策，亦即可能对某些 i[1] < i[2] 成立 mathopoptmax(i[1]) > mathopoptmin(i[2]) ，所以在书写代码时，应保证总是求得最小或最大的最优决策点。

另一方面，拥有相同最小最优决策的问题构成一个区间。这一区间，作为最小最优决策的函数，应严格递增。亦即，给定 j[1] = mathopopt(i[1]) ， j[2] = mathopopt(i[2]) ，如果 j[1] < j[2] ，那么必然有 i[1] < i[2] 。换言之，如果决策 j[1] < j[2] 能够成为最小最优决策的问题区间分别是 [l_(j[1]),r_(j[1])] 和 [l_(j[2]),r_(j[2])] ，那么必然有 r_(j[1]) < l_(j[2]) 。

最常见的判断决策单调性的方法是通过四边形不等式（quadrangle inequality）。

- **四边形不等式**：如果对于任意 a <= b <= c <= d 均成立

w(a,c)+w(b,d) <= w(a,d)+w(b,c)，
则称函数 w 满足四边形不等式（简记为「交叉小于包含」）。若等号永远成立，则称函数 w 满足**四边形恒等式**。
如果没有特别说明，以下都会保证 a <= b <= c <= d 。四边形不等式给出了一个决策单调性的充分不必要条件。
定理1
若 w 满足四边形不等式，则问题 (1) 满足决策单调性。
证明
要证明这一点，可采用反证法。假设对某些 c < d ，成立 a = mathopopt(d) < mathopopt(c) = b 。此时有 a < b <= c < d 。根据最优化条件， w(a,d) <= w(b,d) 且 w(b,c) < w(a,c) ，于是，w(a,d) - w(b,d) <= 0 < w(a,c) - w(b,c) ，这与四边形不等式矛盾。
四边形不等式可以理解在合理的定义域内， w 的二阶混合差分 Delta[i]Delta[j]w(j,i) 非正。
利用决策单调性，有两种常见算法可以将算法复杂度优化到 O(nlog n) 。

分治

要求解所有状态，只需要求解所有最优决策点。为了对所有 1 <= i <= n 求解 mathopopt(i) ，首先计算 mathopopt(n/2) ，而后分别计算 1 <= i < n/2 和 n/2 < i <= n 上的 mathopopt(i) ，注意此时已知前半段的 mathopopt(i) 必然位于 1 和 mathopopt(n/2) 之间（含端点），而后半段的 mathopopt(i) 必然位于 mathopopt(n/2) 和 mathopopt(n) 之间（含端点）。对于两个子区间，也类似处理，直至计算出每个问题的最优决策。在分治的过程中记录搜索的上下边界，就可以保证算法复杂度控制在 O(nlog n) 。递归树层数为 O(log n) ，而每层中，单个决策点至多计算两次，所以总的计算次数是 O(nlog n) 。

```
int w(int j, int i);

void DP(int l, int r, int k_l, int k_r) {
int mid = (l + r) / 2, k = k_l;
// 求状态 f[mid] 的最优决策点
for (int j = k_l; j <= min(k_r, mid - 1); ++j)
if (w(j, mid) < w(k, mid)) k = j;
f[mid] = w(k, mid);
// 根据决策单调性得出左右两部分的决策区间，递归处理
if (l < mid) DP(l, mid - 1, k_l, k);
if (r > mid) DP(mid + 1, r, k, k_r);
}
```

二分队列

注意到对于每个决策点 j ，能使其成为最小最优决策点的问题 i 必然构成一个区间。可以通过单调队列记录到目前为止每个决策点可以解决的问题的区间，这样，问题的最优解自然可以通过队列中记录的决策点计算得到。算法大致如下。

```
int val(int i, int j);
int lt[N], rt[N], f[N];
deque<int> dq;
// 顺次考虑所有问题和决策，下标从 1 开始
for (int j = 1; j <= n; ++j) {
// 出队
if (!dq.empty() && rt[dq.front()] < j) dq.pop_front();
if (!dq.empty()) lt[dq.front()] = j;
// 入队
while (!dq.empty() && val(j, lt[dq.back()]) < val(dq.back(), lt[dq.back()])) {
dq.pop_back();
}
if (dq.empty()) {
lt[j] = i;
rt[j] = n;
dq.emplace_back(j);
} else if (val(j, rt[dq.back()]) >= val(dq.back(), rt[dq.back()])) {
if (rt[dq.back()] < n) {
lt[j] = rt[dq.back()] + 1;
rt[j] = n;
dq.emplace_back(j);
}
} else {
int ll = lt[dq.back()], rr = rt[dq.back()], i = rr;
// 二分
while (ll <= rr) {
int mm = (ll + rr) / 2;
if (val(j, mm) < val(dq.back(), mm)) {
i = mm;
rr = mm - 1;
} else {
ll = mm + 1;
}
}
rt[dq.back()] = i - 1;
lt[j] = i;
rt[j] = n;
dq.emplace_back(j);
}
// 计算
f[j] = val(dq.front(), j);
}
```

掌握这一算法，需要理解如下要点：

- 队列需要记录到目前为止每个可行的决策点 j 和能够解决的问题区间左右端点 l[j] 和 r[j] 构成的 **三元组**。对于给定区间 [l[j],r[j]] 内的问题， j 应该是到目前为止考虑过的决策点中最小最优的（以下简称最优决策）。每时每刻，队列中存储的决策未必是连续的，但是尚未解决的问题 [j,n] 应该是队列中存储的问题区间的不交并。
- 初始时，队列是空的。类似于单调队列，每次考虑下一个决策 j 的时候，都需要进行出队和入队操作。
- **出队**：首先将上一个问题 j-1 从队列中移除。如果队首的决策能够解决的问题的右端点恰为 j-1 ，直接弹出队首；否则，将队首决策能够解决问题的左端点更新为 j 。
- **入队**：要对决策 j 进行入队时，首先比较它和队尾的决策 j’ 。
- 如果对于问题 l_(j’) ，将要入队的决策 j 比已有的决策 j’ 严格更优，即 w(j,l_(j’)) < w(j’,l_(j’)) 时，则弹出队尾的决策 j’ 。此操作持续到队列为空或队尾的决策 j’ 比起 j 对于问题 l_(j’) 更优时为止。
- 如果队列已空，入队 (j,j,n) ，即认为决策 j 是尚未解决的所有问题的最优解。

- 如果队尾决策 j' 对于问题 r_(j') 同样不劣于将入队的决策 j ，那么当 r_(j') < n 时，入队 (j,r_(j')+1,n) ，表示 j 是问题 [r_(j')+1,n] 的最优决策；否则，不需要入队 j ，因为它并不比已有的决策更优。
- 最后的情形是，队尾决策 j' 比起要入队的决策 j 对于问题 l_(j') 严格更优，而对于问题 r_(j') 严格更劣。这说明，存在问题 i 属于 (l_(j'),r_(j')] 使得问题 [l_(j'),i-1] 的最优决策为 j' 且问题 [i,r_(j')] 的最优决策为 j 。因而，需要通过 **二分** 找到最小的 i 属于 [l_(j'),r_(j')] 使得 w(j,i) < w(j',i) ，再将队尾的区间右端点 r_(j') 修改为 i-1 ，并入队 (j,i,n) 。
- 处理完决策 j 后，就已经处理了所有到 j 为止的决策。此时，队首决策就是问题 j 的最优决策，可以记录相应的最优解。

类似于单调队列，每个决策点至多入队一次，出队一次。这里，出队是 O(1) 的，而入队是 O(log n) 的（可能需要二分），所以总的时间复杂度是 O(nlog n) 。

例题

给定一个长度为 n 的序列 a[1],a[2],...,a[n] ，要求对于每一个 1 <= i <= n ，找到最小的非负整数 f[i] 满足

任意 j 属于 [1,n]:a[j] <= a[i] + f[i] - sqrt (|i-j|).

思路

显然，经过不等式变形，我们可以得到待求整数 f[i] = max[j]a[j]+ sqrt (|i-j|)-a[i] 。不妨先考虑 j <= i 的情况（另外一种情况类似），此时我们可以得到状态转移方程：

f[i] = min_(j <= i)-a[j]- sqrt (i-j)+a[i].

根据 - sqrt (x) 的凸性，我们很容易得出（后文将详细描述）函数 w(l, r) = -a_(l) - sqrt (r-l) + a[r] 满足四边形不等式，因此套用上述的算法便可在 O(nlog n) 的时间内解决此题了。

```
#include <algorithm>
#include <cmath>
#include <deque>
#include <iostream>
#define all \
for (auto i : {&q, &l, &r}) (*i)
using namespace std;
int n, a[500009], ans[500009];
deque<int> q, l, r;

double f(int i, int j) { return a[i] + sqrtl(j - i); }

void work() {
all.clear();
for (int i = 0; i < n; ++i) {
if (q.size() && r.front() < i) all.pop_front();  // 队首出队
if (q.size()) l.front() = i;
for (; q.size() && f(q.back(), l.back()) <= f(i, l.back());)  // 队尾出队
all.pop_back();
if (q.empty())  // 入队
q.emplace_back(i), l.emplace_back(i), r.emplace_back(n);
else if (f(q.back(), n) < f(i, n)) {
int ll = l.back(), rr = n, mid;
for (; ll <= rr;) {
mid = (ll + rr) >> 1;
if (f(q.back(), mid) < f(i, mid))
rr = mid - 1;
else
ll = mid + 1;
}
r.back() = rr;
q.emplace_back(i), l.emplace_back(ll), r.emplace_back(n);
}
ans[i] = max(ans[i], (int)(ceil(f(q.front(), i))) - a[i]);
}
}

int main() {
cin >> n;
for (int i = 0; i < n; cin >> a[i++]);
work();
reverse(a, a + n);
reverse(ans, ans + n);
work();
reverse(ans, ans + n);
for (int i = 0; i < n; cout << ans[i++] << '\n');
}
```

区间分拆问题

考虑将某个区间拆分成若干个子区间的问题。形式化地说，将给定区间 [1,n] 拆分成 [a[1],b[1]],...,[a[k],b[k]] ，其中， a[1]=1 ， b[k]=n ，以及 b[i]+1=a[i+1] 对任意 i < k 都成立。对于给定拆分，成本为 sum _(i=1)^(k)w(a[i],b[i]) 。问题要求最小化这一成本。可以列出如下的 1D1D 状态转移方程。

f(i) = min_(1 <= j <= i) f(j-1)+w(j,i) (1 <= i <= n)

这里， f(0)=0 。注意到，只要 w(j,i) 满足四边形不等式， f(j-1)+w(j,i) 必然满足四边形不等式，因为第一项并不包括 j 和 i 的交叉项，在混合差分时会消去。但是由于成本函数依赖于前面的子问题，这一转移只能够顺序计算，所以通常只适合应用二分队列算法。算法复杂度为 O(nlog n) 。

限制区间个数的情形

上述问题可以加强为限制区间个数的情形，即问题指定将区间拆分成 m 个子区间。此时需要将拆分后的区间个数作为转移状态的一维。相应地，有 2D1D 状态转移方程如下。

f(k,i) = min_(1 <= j <= i) f(k-1,j-1)+w(j,i) (1 <= k <= m, 1 <= i <= n) 2
这里， f(0,0)=0 ， f(0,i)=f(k,0)= INF 对任意 1 <= k <= m 和 1 <= i <= n 都成立。和上文同样的道理，这里的 f(k-1,j-1)+w(j,i) 必然满足四边形不等式。此时对于第 i 层的计算，并不再依赖于该层的结果，所以对于每一层，都可以通过分治或者二分队列的方法进行计算，此时算法复杂度为 O(mnlog n) 。
对于这一问题，利用决策单调性，实际上还存在其他的优化算法。第二种优化思路依赖于如下结果。这种优化算法和下文详细描述的 Knuth 优化算法十分相似。
定理2
若 w 满足四边形不等式，则对于问题 (2) 成立 mathopopt(k-1,i) <= mathopopt(k,i) <= mathopopt(k,i+1) 。
证明
第二个不等式只是第 k 层的决策单调性。关键在于第一个不等式。
下证 mathopopt(k,i) <= mathopopt(k+1,i) 。假设有如下两个区间 [1,i] 的分划（逆序标号）：[a[k],d[k]],...,[a[1],d[1]] 和 [b[k+1],c[k+1]],...,[b[1],c[1]] 。其中，每个区间的左端点都是其右端点处对应问题的最小最优决策；同样地，从右向左考虑所有可能的分划，右端点也是左端点对应问题的最小最优决策。例如， d[j] 和 c[j] 分别是将 [a[j],i] 和 [b[j],i] 分成 j 段左起第一个区间右端点的最小最优决策。根据决策单调性，如果 a[j-1] > b[j-1] ，亦即 d[j] > c[j] ，那么必然有 a[j] > b[j] 。由此，如果所证不成立，则有 a[1] > b[1] 。进而可以归纳地证明 a[k] > b[k] 。这显然与所设矛盾。由此得证。
第一个不等式可以另证如下。同样考虑上面证明中的两个分划。如果所证命题不成立，则有 a[1] > b[1] ，但是由于有 a[k] < b[k] ，我们可以找到最小的 j>1 使得 a[j] <= b[j] 。进而，此时有 a[j-1] > b[j-1] ，故 d[j]>c[j] 。我们找到了一组区间满足 a[j] <= b[j] <= c[j] < d[j] 。考虑将这两个分拆重新组合的结果。考虑分拆 [b[k+1],c[k+1]],...,[b[j+1],c[j+1]],[b[j],d[j]],[a[j-1],d[j-1]],...,[a[1],d[1]] ，共 (k+1) 段，于是由前设的最优性可推知，
w(b[k+1],c[k+1])+...+w(b[j+1],c[j+1])+w(b[j],c[j])+w(b[j-1],c[j-1])+...+w(b[1],c[1]) <= w(b[k+1],c[k+1])+...+w(b[j+1],c[j+1])+w(b[j],d[j])+w(a[j-1],d[j-1])+...+w(a[1],d[1]).
同样地，考虑分拆 [a[k],d[k]],...,[a[j+1],d[j+1]],[a[j],c[j]],[b[j-1],c[j-1]],...,[b[1],c[1]] ，共 k 段，则有
w(a[k],d[k])+...+w(a[j+1],d[j+1])+w(a[j],d[j])+w(a[j-1],d[j-1])+...+w(a[1],d[1]) < w(a[k],d[k])+...+w(a[j+1],d[j+1])+w(a[j],c[j])+w(b[j-1],c[j-1])+...+w(b[1],c[1]).
此时，不等号是严格的，因为 a[1] > b[1] ，但是按假设， a[1] 是所有 k 段分拆最末一段的左端点中最小最优的。两个不等式条件相加，得到 w(b[j],c[j]) + w(a[j],d[j]) < w(b[j],d[j]) + w(a[j],c[j]) ，这有悖于四边形不等式。故而原结论得证。
利用这一结果，我们可以限制决策 j 的搜索范围。算法实现时，对 k 正向遍历，对 i 逆向遍历，在之前已确定的上下界范围内暴力搜索 j 就可以保证 O(n(n+m)) 的算法复杂度。
注意
这里算法复杂度不是 O(nm) 的。正确的复杂度计算需要考虑 n * m 维状态矩阵。因为对于问题 (i,k) 只需要考虑 mathopopt(k-1,i) <= j <= mathopopt(k,i+1) 中的决策，所以每条次对角线上（即 i-k 为一定值）的问题所需遍历的决策总数为 O(n) 的。这样的对角线共计 (n+m) 条，故而总的时间复杂度为 O(n(n+m)) 。
最后一种优化方法来源于如下的观察。
定理3
若 w 满足四边形不等式，则问题 (2) 的最优解 g(k):=f(n,k) 是关于 k 的凸函数。
证明
下证 g(k-1) + g(k+1) >= 2g(k) 。为此，考虑长度为 (k-1) 段和 (k+1) 段的最优分划，分别是 [a[1],d[1]],...,[a[k-1],d[k-1]] 和 [b[1],c[1]],...,[b[k+1],c[k+1]] 。取最小的 1 <= j <= k-1 使得 c[j+1] <= d[j] ，其存在性可由 c[k] < n = d[k-1] 推知。根据其最小性得知， b[j+1]

> a[j] 。所以， a[j] < b[j+1] <= c[j+1] <= d[j] 。与上文类似，交换两个现有分拆的后半段，可以得到如下两个区间分拆：
[a[1],d[1]],...,[a[j-1],d[j-1]],[a[j],c[j+1]],[b[j+2],c[j+2]],...,[b[k+1],c[k+1]],
[b[1],c[1]],...,[b[j],c[j]],[b[j+1],d[j]],[a[j+1],d[j+1]],...,[a[k-1],d[k-1]].
两个所得区间都是 k 段的，所以由最优性条件可知
2g(k) <= w(a[1],d[1]) + ... + w(a[j-1],d[j-1]) + w(a[j],c[j+1]) + w(b[j+2],c[j+2]) + ... + w(b[k+1],c[k+1]) + w(b[1],c[1]) + ... + w(b[j],c[j]) + w(b[j+1],d[j]) + w(a[j+1],d[j+1]) + ... + w(a[k-1],d[k-1]) <= w(a[1],d[1]) + ... + w(a[j-1],d[j-1]) + w(a[j],d[j]) + w(a[j+1],d[j+1]) + ... + w(a[k-1],d[k-1]) + w(b[1],c[1]) + ... + w(b[j],c[j]) + w(b[j+1],c[j+1]) + w(b[j+2],c[j+2]) + ... + w(b[k+1],c[k+1]) = g(k-1) + g(k+1).
这里第二个不等式正是四边形不等式。所求凸性由此得证。
这一结论保证了可以通过 wqs 二分（国外称 Alien's trick）的方法解决此问题。具体来说，考虑带参的成本函数 w_(c)(j,i):=w(j,i)+c ，解决不限制区间个数的问题，求得其最优解为 f_(c)(n) 。随着实数 c 递增，相应的最优区间的数目单调递减，故而可以通过二分的方法找到恰使得最优区间个数等于 m 的参数 c ，则原题最优解为 f(n,m) = f_(c)(n)-cm 。这里的实数 c 可以看作区间个数限制的 Lagrange 乘子。该算法的实现有很多细节，可以参考专门介绍 wqs 二分的文章。这一算法的时间复杂度为 O(nlog nlog C) ，这里 C 为某一常数。
对于限制区间个数的区间分拆问题的三种算法，在不同的数据范围时表现各有优劣，需要结合具体的题目选择合适的算法。
例题
高速公路旁边有一些村庄。高速公路表示为整数轴，每个村庄的位置用单个整数坐标标识。没有两个在同样地方的村庄。两个位置之间的距离是其整数坐标差的绝对值。
邮局将建在一些，但不一定是所有的村庄中。为了建立邮局，应选择他们建造的位置，使每个村庄与其最近的邮局之间的距离总和最小。
你要编写一个程序，已知村庄的位置和邮局的数量，计算每个村庄和最近的邮局之间所有距离的最小可能的总和。
思路
每个村庄有其最近的邮局，那么每个邮局也有其管辖的村庄，易知这是一个区间。
考虑把这 n 个村庄分成 m 个区间，再在每个区间中决出一个邮局。
根据数学知识，对于区间 [i,j] ，邮局应该建在第 floor (i+j)/(2) 个村庄处。使用前缀和容易算出 w(i,j) 。
问题转化为限制区间个数的区间分拆问题。可以证明， w 函数满足四边形不等式。直接应用上述优化方法即可。
实现 1，前文第二种优化，复杂度 $O(n(n+m))$

```
#include <iostream>
constexpr int N = 3009;
constexpr int M = 309;
using namespace std;
int n, m, a[N], s[N], g[M][N], p[M][N];

int f(int i, int j) {
int k = (i + j) >> 1;
return a[k] * (k - i + 1) - (s[k] - s[i - 1]) + (s[j] - s[k]) -
a[k] * (j - k);
}

int main() {
cin >> n >> m;
for (int i = 1; i <= n; cin >> a[i], s[i] = s[i - 1] + a[i], ++i);
for (int i = 1; i <= n; ++i) g[0][i] = 1 << 30, p[m + 1][i] = i;
for (int i = 1; i <= m; ++i) p[i][0] = 1;
for (int i = 1; i <= n; ++i)
for (int j = m; j; --j) {
g[j][i] = 1 << 30;
for (int k = p[j][i - 1]; k <= p[j + 1][i]; ++k) {
int tmp = g[j - 1][k - 1] + f(k, i);
if (tmp < g[j][i]) g[j][i] = tmp, p[j][i] = k;
}
}
cout << g[m][n];
}
```

实现 2，wqs 二分，复杂度 O(nlog nlog C)

```
#include <deque>
#include <iostream>
#define all \
for (auto i : {&q, &l, &r}) (*i)
using namespace std;
long long n, m, a[500009], s[500009], u, v, w, sum[500009], cnt[500009];
deque<int> q, l, r;

long long f(int i, int j) {
int k = (i + j) >> 1;
return sum[i - 1] + v + a[k] * (k - i + 1) - (s[k] - s[i - 1]) +
(s[j] - s[k]) - a[k] * (j - k);
}

void work() {
all.clear();
for (int i = 1; i <= n; ++i) {
if (q.size() && r.front() < i) all.pop_front();  // 队首出队
if (q.size()) l.front() = i;
for (; q.size() && f(q.back(), l.back()) >= f(i, l.back());)  // 队尾出队
all.pop_back();
if (q.empty())  // 入队
q.emplace_back(i), l.emplace_back(i), r.emplace_back((int)n);
else if (f(q.back(), n) >= f(i, n)) {
int ll = l.back(), rr = n, mid;
for (; ll <= rr;) {
mid = (ll + rr) >> 1;
if (f(q.back(), mid) >= f(i, mid))
rr = mid - 1;
else
ll = mid + 1;
}
r.back() = rr;
q.emplace_back(i), l.emplace_back(ll), r.emplace_back((int)n);
}
sum[i] = f(q.front(), i);
cnt[i] = cnt[q.front() - 1] + 1;
}
}

int main() {
cin >> n >> m;
for (int i = 1; i <= n; cin >> a[i], s[i] = s[i - 1] + a[i], ++i);
for (w = 2e12; u <= w;) {  // wqs二分
v = (u + w) >> 1;
work();
if (cnt[n] < m)
w = v - 1;
else
u = v + 1;
}
v = w;
work();
cout << sum[n] - m * v;
}
```

区间合并问题

另一类可以通过四边形不等式优化的动态规划问题是区间合并问题，即要将 n 个长度为一的区间[i,i] 两两合并起来，直到得到区间 [1,n] 。每次合并 [j,k] 和 [k+1,i] 时都需要支付成本 w(j,i) 。问题要求找到成本最低的合并方式。对于此类问题，有如下 2D1D 状态转移方程。

f(j,i) = min_(j <= k < i) f(j,k) + f(k+1,i) + w(j,i) (1 <= j< i <= n) 3

这里给定任意初始成本 f(i,i)=w(i,i) 。暴力算法的总复杂度为 O(n^3) ，而当存在决策单调性时，可以优化至 O(n^2) 的算法复杂度。这一算法最早由 Knuth 在解决最优二叉搜索树问题时提出，并由姚储枫进一步研究总结，在国外称为 Knuth's optimization 或 Knuth-Yao speedup。
除了四边形不等式以外，区间合并问题的决策单调性还要求成本函数满足区间包含单调性。

- **区间包含单调性**：如果对于任意 a <= b <= c <= d 均成立

w(b,c) <= w(a,d),
则称函数 w 对于区间包含关系具有单调性。
这实质是成本函数的一阶条件，即 w(j,i) 关于 j 递减，关于 i 递增。
引理1
若 w 满足区间包含单调性和四边形不等式，则状态 f(j,i) 满足四边形不等式。
证明
不妨设 a <= b <= c <= d 。下证 f(a,d) + f(b,c) >= f(a,c) + f(b,d) 。考虑依 d-a 归纳。当 a=b 或 c=d 时，所求即一等式。对于一般的情形，根据 d'=mathopopt(a,d) 的位置分类讨论。
第一种情况， c <= d' 或 d' < b ，即 [b,c] 包含于 [a,d'] 或 [d'+1,d] 之中。
不妨假设 c <= d' ，另一种情形同理。此时有
f(a,d) + f(b,c) = f(a,d') + f(d'+1,d) + w(a,d) + f(b,c)

> = f(a,c) + f(b,d') + f(d'+1,d) + w(a,d) = f(a,c) + f(b,d') + f(d'+1,d) + w(b,d) = f(a,c) + f(b,d).

这里，第一个不等式来自于归纳假设 f(a,c) + f(b,d') <= f(a,d') + f(b,c) ，第二个不等式来自于区间包含单调性 w(b,d) <= w(a,d) ，第三个不等式来自于最优性条件 f(b,d) <= f(b,d') + f(d'+1,d) + w(b,d) 。
第二种情况， b <= d' < c ，即 d' 位于 [b,c] 之中。此时，考虑 c'=mathopopt(b,c) 的位置。
不妨假设 c' <= d' ，即 [b,c'] 包含于 [a,d'] 之中，另一种情形同理。此时有
f(a,d) + f(b,c) = f(a,d') + f(d'+1,d) + w(a,d) + f(b,c') + f(c'+1,c) + w(b,c)

> = f(a,c') + f(c'+1,c) + w(b,c) + f(b,d') + f(d'+1,d) + w(a,d) = f(a,c') + f(c'+1,c) + w(a,c) + f(b,d') + f(d'+1,d) + w(b,d) = f(a,c) + f(b,d).

这里，第一个不等式来自于归纳假设 f(a,c') + f(b,d') <= f(a,d') + f(b,c') ，第二个不等式来自于四边形不等式 w(a,c) + w(b,d) <= w(a,d) + w(b,c) ，第三个不等式来自于 f(a,c) 和 f(b,d) 的最优性条件。
定理 4
若 w 满足区间包含单调性和四边形不等式，则问题 (3) 中最小最优决策 mathopopt(j,i) 满足 mathopopt(j,i-1) <= mathopopt(j,i) <= mathopopt(j+1,i). (j + 1 < i)
证明
引理 1 已经证得 f(j,i) 满足四边形不等式，所以目标函数 f(j,k) + f(k+1,i) + w(j,i) 对于给定 j 作为 (k,i) 的函数满足四边形不等式，所以由定理 1 有， mathopopt(j,i-1) <= mathopopt(j,i) 。注意，不同时含有 (k,i) 的项并不影响四边形不等式成立。类似地，它对于给定 i 作为 (k,j) 的函数也满足四边形不等式，所以 mathopopt(j,i) <= mathopopt(j+1,i) 。即得所证。
利用这一结论，同样可以限制决策点 k 的搜索范围。在这里，正序遍历区间长度 i-j+1 ，再遍历具有同样长度的所有区间 [j,i] ，暴力搜索 mathopopt(j,i-1) 和 mathopopt(j+1,i) 之间的所有 k 求得最优解 f(j,i) 并记录最小最优决策 mathopopt(j,i) 。对于同样长度的所有区间，此算法中决策空间总长度是 O(n) 的，而可能的区间长度的数目同样是 O(n) 的，故而总的算法复杂度为 O(n^2) 的。

```
  for (int len = 2; len <= n; ++len)  // 枚举区间长度
for (int j = 1, i = len; i <= n; ++j, ++i) {
// 枚举长度为 len 的所有区间
f[i][i] = INF;
for (int k = opt[i][i - 1]; k <= opt[i + 1][i]; ++k)
if (f[i][i] > f[i][k] + f[k + 1][i] + w(j, i)) {
f[i][i] = f[i][k] + f[k + 1][i] + w(j, i);  // 更新状态值
opt[j][i] = k;  // 更新（最小）最优决策点
}
}
```

满足四边形不等式的函数类

为了更方便地证明一个函数满足四边形不等式，我们有以下几条性质：

性质 1：若函数 w1 和 w2 均满足四边形不等式（或区间包含单调性），则对于任意 c[1],c[2] >= 0 ，函数 c[1]w[1]+c[2]w[2] 也满足四边形不等式（或区间包含单调性）。

性质 2：若存在函数 f(x) 和 g(x) 使得 w(j,i) = f(j)-g(i) ，则函数 w 满足四边形恒等式。当函数 f 和 g 单调增加时，函数 w 还满足区间包含单调性。

性质 3：设 h(x) 是一个单调增加的凸函数，若函数 w(j,i) 满足四边形不等式并且对区间包含关系具有单调性，则复合函数 h(w(j,i)) 也满足四边形不等式和区间包含单调性。

性质 4：设 h(x) 是一个凸函数，若函数 w(j,i) 满足四边形恒等式并且对区间包含关系具有单调性，则复合函数 h(w(j,i)) 也满足四边形不等式。

首先需要澄清一点，凸函数（Convex Function）的定义在国内教材中有分歧，此处的凸函数指的是下凸函数，即（可微时）一阶导数单调增加的函数。

证明

前两条性质根据定义很容易证明，下面证明第三条性质，性质四的证明过程类似。由于 h(x) 单调，h(w(j,i)) 自然保持对区间包含的单调性。关键在于四边形不等式的证明。

为此，下面考虑 a <= j <= b <= c <= i <= d 上的二阶混合差分。

Delta[i]Delta[j] h(w(j,i)) = h(w(b,d)) - h(w(a,c) + Delta[j]w(j,c) + Delta[i]w(a,i)) + h(w(a,c) + Delta[j]w(j,c) + Delta[i]w(a,i)) - h(w(a,c) + Delta[j]w(j,c)) - h(w(a,c) + Delta[i]w(a,i)) + h(w(a,c)).

这里，根据区间单调性， Delta[i]w(a,i) := w(a,d) - w(a,c) >= 0 和 Delta[j]w(j,c) := w(b,c) - w(a,c) <= 0 。由于 h(x) 具有凸性，对于 t[1],t[2] >= 0 成立 h(x + t[1] - t[2]) - h(x + t[1]) <= h(x - t[2]) - h(x) ，所以后两行必然非正。同时，由于四边形不等式， w(b,d) <= w(a,c) + Delta[j]w(j,c) + Delta[i]w(a,i) = w(b,c) + w(a,d) - w(a,c) ，故而，第一行的差在 h(x) 单调增加的情况下必然也非正。所以，总的二阶混合差分非正。此即四边形不等式。

这一证明实际是如下导数证明的离散版本。

(d ^2)/(d x d y)h(w(x,y)) = h''(w(x,y))(d)/(d x)w(x,y)(d)/(d y)w(x,y) + h'(w(x,y))(d ^2)/(d x d y)w(x,y) <= 0.

这在 h' >= 0 ， h'' >= 0 ， w[x] <= 0 ， w_(y) >= 0 以及 w_(xy) <= 0 的条件下显然成立。其中，区间包含单调性给出了 w 的一阶条件，而四边形不等式给出了其二阶条件。

例题：

Status

Problem

Tags

P1880

.

石子合并

DP优化

P4767

.

邮局加强版
DP优化
单调队列/栈优化 DP
状态设计技巧

动态规划 08 · 状态设计技巧

概述

优化 dp 时，不止可以从转移过程入手，加速转移。有时，也可以从状态定义入手，通过改变设计状态的方式实现复杂度上的优化。

令人比较头疼的是，这类优化大多不具有通用性，即不能很套路地应用于多个题目中。因此，下文将从具体例题出发，力求提供思路上的启发，希望可以对读者有一定帮助。

例 1

题面

给定两个长度分别为 n,m 且仅由小写字母构成的字符串 A,B ，求 A,B 的最长公共子序列。 (n <= 10^6,m <= 10^3)

朴素的解法

定义状态 f_(i,j) 为 A 的前 i 位与 B 的前 j 位最长公共子序列，则有

f_(i,j)= max(f_(i-1,j),f_(i,j-1)) ,A[i] != B[j] f_(i-1,j-1)+1 ,A[i] = B[j]

上述做法的时间复杂度 O(nm) ，无法通过本题。

更优的解法

我们仔细一想，发现了一个性质：最终答案不会超过 m 。

我们又仔细一想，发现 LCS 满足贪心的性质。

更改状态定义 f_(i,j) 为与 B 前 i 位的最长公共子序列长度为 j 的 A 的最短前缀长度（即将朴素做法的答案与第一维状态对调）

可以通过预处理 A 的每一位的下一个 a,b,...,z 的出现位置进行 O(1) 的顺推转移。

复杂度 O(m^2+26n) ，可以通过本题。

例 2

题面

给定一个 n 个点的无权有向图，判断该图是否存在哈密顿回路。 (2 <= n <= 20)

朴素的解法

看到数据范围，我们考虑状压。

设 f_(s,i) 表示从点 1 出发，仅经过点集 s 中的点能否到达点 i 。记 g 为原图的邻接矩阵。则有

f_(s, i) = bigvee_(j 属于 s, j != i)f_s - i, j and g_(j, i) (i 属于 s)

时间复杂度 O(n^2 * 2^n) ，写得好看或许能过，但是并不优美。

更优的解法

上面的状态设计中，每个 dp 值只代表一个 `bool` 值，这让我们觉得有些浪费。

我们可以考虑对于每个状态 s 将 f_(s,1),f_(s,2),…,f_(s,n) 压成一个 `int` ，发现我们可以将邻接矩阵同样压缩后进行 O(1) 转移。

时间复杂度 O(n^2/w * 2^n) ，可以通过这道题，其中 w 为 `int` 的位数。

例题：

Status

Problem

Tags

P4310

.

绝世好题

DP优化

P1284
.
三角形牧场
DP优化
P4095
.
Eden 的新背包问题
DP优化
P3237
.
米特运输
DP优化
四边形不等式优化 DP
模板

动态规划 99・模板

例题：

Status
Problem
Tags
B2002
.
Hello,World!
入门
语法
状态设计技巧
二叉堆

数据结构 01・二叉堆

结构

从二叉堆的结构说起，它是一棵二叉树，并且是完全二叉树，每个结点中存有一个元素（或者说，有个权值）。

堆性质：父亲的权值不小于儿子的权值（大根堆）。同样的，我们可以定义小根堆。本文以大根堆为例。

由堆性质，树根存的是最大值（getmax 操作就解决了）。

过程

插入操作

插入操作是指向二叉堆中插入一个元素，要保证插入后也是一棵完全二叉树。

最简单的方法就是，最下一层最右边的叶子之后插入。

如果最下一层已满，就新增一层。

插入之后可能会不满足堆性质？

向上调整：如果这个结点的权值大于它父亲的权值，就交换，重复此过程直到不满足或者到根。

可以证明，插入之后向上调整后，没有其他结点会不满足堆性质。

向上调整的时间复杂度是 O(log n) 的。

[图示：示意图]（https://cspwiki.oss-cn-beijing.aliyuncs.com/S_04_01_1.svg）

二叉堆的插入

删除操作

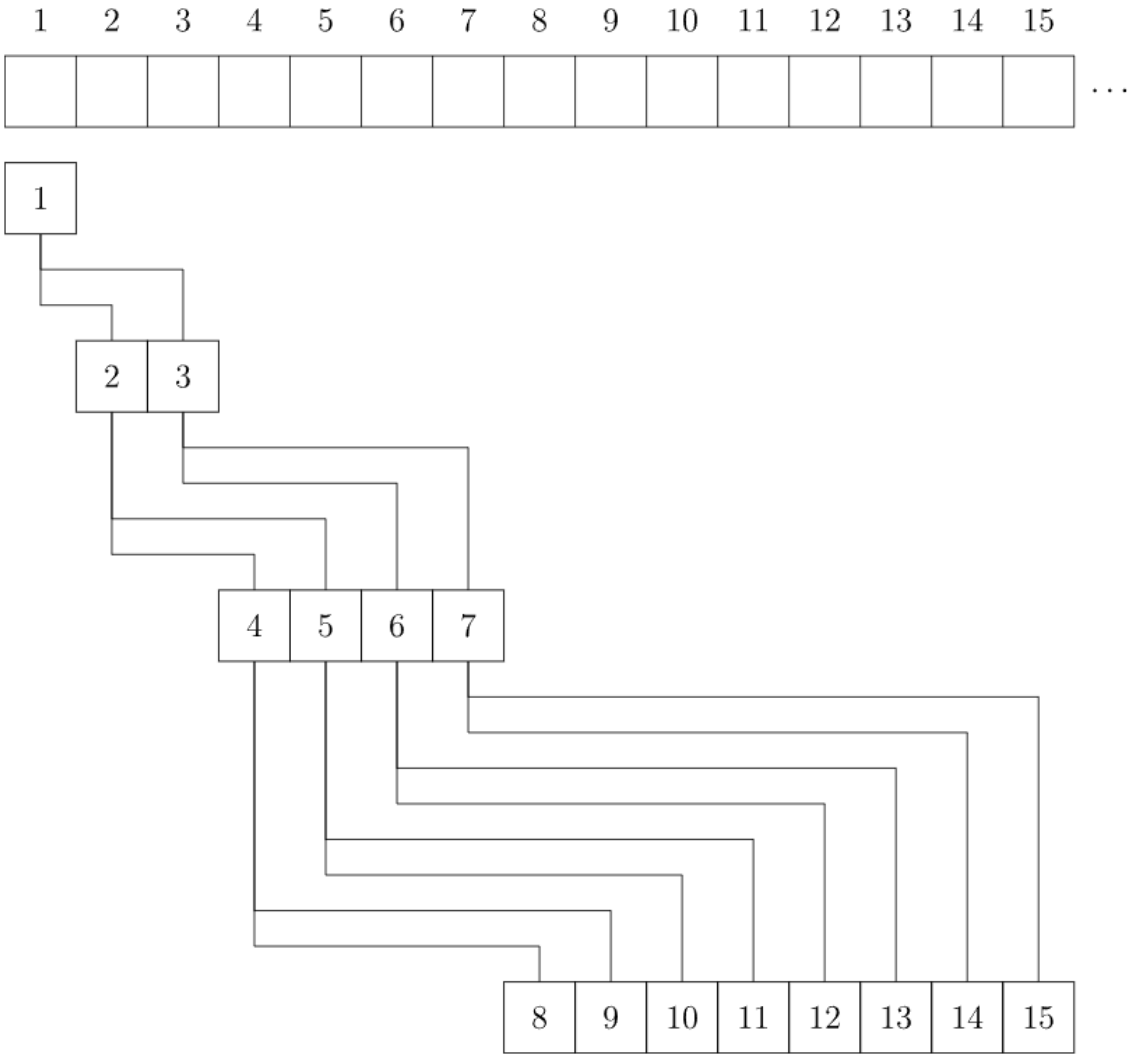
删除操作指删除堆中最大的元素，即删除根结点。
但是如果直接删除，则变成了两个堆，难以处理。
所以不妨考虑插入操作的逆过程，设法将根结点移到最后一个结点，然后直接删掉。
然而实际上不好做，我们通常采用的方法是，把根结点和最后一个结点直接交换。
于是直接删掉（在最后一个结点处的）根结点，但是新的根结点可能不满足堆性质……
向下调整：在该结点的儿子中，找一个最大的，与该结点交换，重复此过程直到底层。
可以证明，删除并向下调整后，没有其他结点不满足堆性质。
时间复杂度 O(log n) 。

增加某个点的权值

很显然，直接修改后，向上调整一次即可，时间复杂度为 O(log n) 。

实现

我们发现，上面介绍的几种操作主要依赖于两个核心：向上调整和向下调整。
考虑使用一个序列 h 来表示堆。 h[i] 的两个儿子分别是 h[2i] 和 h[2i+1] ， 1 是根结点：



结构
参考代码：

```
void up(int x) {
while (x > 1 && h[x] > h[x / 2]) {
std::swap(h[x], h[x / 2]);
x /= 2;
}
}

void down(int x) {
while (x * 2 <= n) {
t = x * 2;
if (t + 1 <= n && h[t + 1] > h[t]) t++;
if (h[t] <= h[x]) break;
std::swap(h[x], h[t]);
x = t;
}
}
```

建堆

考虑这么一个问题，从一个空的堆开始，插入 n 个元素，不在乎顺序。

直接一个一个插入需要 O(n log n) 的时间，有没有更好的方法？

方法一：使用 decreasekey（即，向上调整）

从根开始，按 BFS 序进行。

```
void build_heap_1() {
for (i = 1; i <= n; i++) up(i);
}
```

为啥这么做：对于第 k 层的结点，向上调整的复杂度为 O(k) 而不是 O(log n) 。

总复杂度： log 1 + log 2 + ... + log n = Theta(n log n) 。

方法二：使用向下调整

这时换一种思路，从叶子开始，逐个向下调整

```
void build_heap_2() {
for (i = n; i >= 1; i--) down(i);
}
```

换一种理解方法，每次「合并」两个已经调整好的堆，这说明了正确性。

注意到向下调整的复杂度，为 O(log n - k) ，另外注意到叶节点无需调整，因此可从序列约 n/2 的位置开始调整，可减少部分常数但不影响复杂度。

之所以能 O(n) 建堆，是因为堆性质很弱，二叉堆并不是唯一的。

要是像排序那样的强条件就难说了。

应用

对顶堆

题面

维护一个序列，支持两种操作：

1. 向序列中插入一个元素
2. 输出并删除当前序列的中位数（若序列长度为偶数，则输出较小的中位数）

这个问题可以被进一步抽象成：动态维护一个序列上第 k 大的数， k 值可能会发生变化。

对于此类问题，我们可以使用 **对顶堆** 这一技巧予以解决（可以避免写权值线段树或 BST 带来的繁琐）。

对顶堆由一个大根堆与一个小根堆组成，小根堆维护大值即前 k 大的值（包含第 k 个），大根堆维护小值即比第 k 大数小的其他数。

这两个堆构成的数据结构支持以下操作：

- 维护：当小根堆的大小小于 k 时，不断将大根堆堆顶元素取出并插入小根堆，直到小根堆的大小等于 k ；当小根堆的大小大于 k 时，不断将小根堆堆顶元素取出并插入大根堆，直到小根堆的大小等于 k ；
- 插入元素：若插入的元素大于等于小根堆堆顶元素，则将其插入小根堆，否则将其插入大根堆，然后维护对顶堆；
- 查询第 k 大元素：小根堆堆顶元素即为所求；
- 删除第 k 大元素：删除小根堆堆顶元素，然后维护对顶堆；
- k 值 +1/-1 ：根据新的 k 值直接维护对顶堆。

显然，查询第 k 大元素的时间复杂度是 O(1) 的。由于插入、删除或调整 k 值后，小根堆的大小与期望的 k 值最多相差 1 ，故每次维护最多只需对大根堆与小根堆中的元素进行一次调整，因此，这些操作的时间复杂度都是 O(log n) 的。
示例代码

```
#include <iostream>
#include <queue>
using namespace std;

int main() {
cin.tie(nullptr)->sync_with_stdio(false);
int t, x;
cin >> t;
while (t--) {
// 大根堆，维护前一半元素（存小值）
priority_queue<int, vector<int>, less<int>> a;
// 小根堆，维护后一半元素（存大值）
priority_queue<int, vector<int>, greater<int>> b;
while (cin >> x, x) {
// 若为查询并删除操作，输出并删除大根堆堆顶元素
// 因为这题要求输出中位数中较小者（偶数个数字会存在两个中位数候选）
// 这个和上面的第 k 大讲解有稍许出入，但如果理解了上面的，这个稍微变通下便可理清
if (x == -1) {
cout << a.top() << '\n';
a.pop();
}
// 若为插入操作，根据大根堆堆顶的元素值，选择合适的堆进行插入
else {
if (a.empty() || x <= a.top())
a.push(x);
else
b.push(x);
}
// 对对顶堆进行调整
if (a.size() > (a.size() + b.size() + 1) / 2) {
b.push(a.top());
a.pop();
} else if (a.size() < (a.size() + b.size() + 1) / 2) {
a.push(b.top());
b.pop();
}
}
}
return 0;
}
```

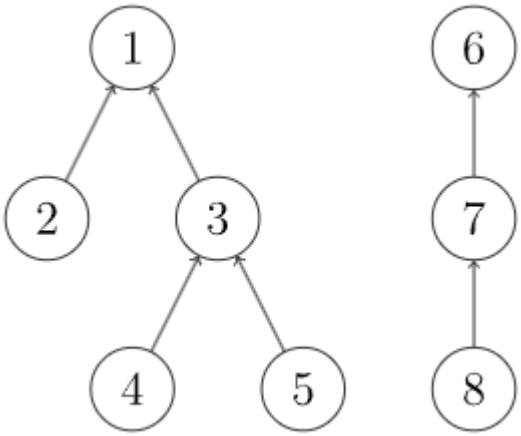
例题：
Status
Problem
Tags

P1090
.
合并果子
数据结构
P1717
.
钓鱼
数据结构
P2085
.
最小函数值
数据结构
P7095
.
不离
数据结构
模板
并查集

数据结构 02 · 并查集

引入

并查集是一种用于管理元素所属集合的数据结构，实现为一个森林，其中每棵树表示一个集合，树中的节点表示对应集合中的元素。



图示

顾名思义，并查集支持两种操作：

- 合并（Union）：合并两个元素所属集合（合并对应的树）
- 查询（Find）：查询某个元素所属集合（查询对应的树的根节点），这可以用于判断两个元素是否属于同一集合

并查集在经过修改后可以支持单个元素的删除、移动；使用动态开点线段树还可以实现可持久化并查集。

并查集无法以较低复杂度实现集合的分离。

初始化

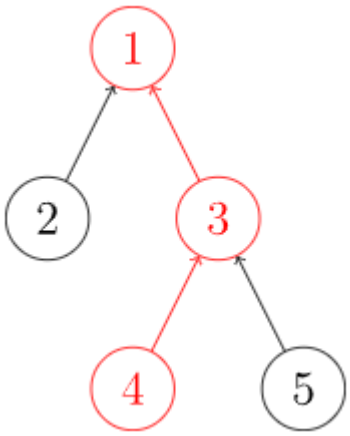
初始时，每个元素都位于一个单独的集合，表示为一棵只有根节点的树。方便起见，我们将根节点的父亲设为自己。

```
struct dsu {
vector<size_t> pa;

explicit dsu(size_t size) : pa(size) { iota(pa.begin(), pa.end(), 0); }
};
```

查询

我们需要沿着树向上移动，直至找到根节点。

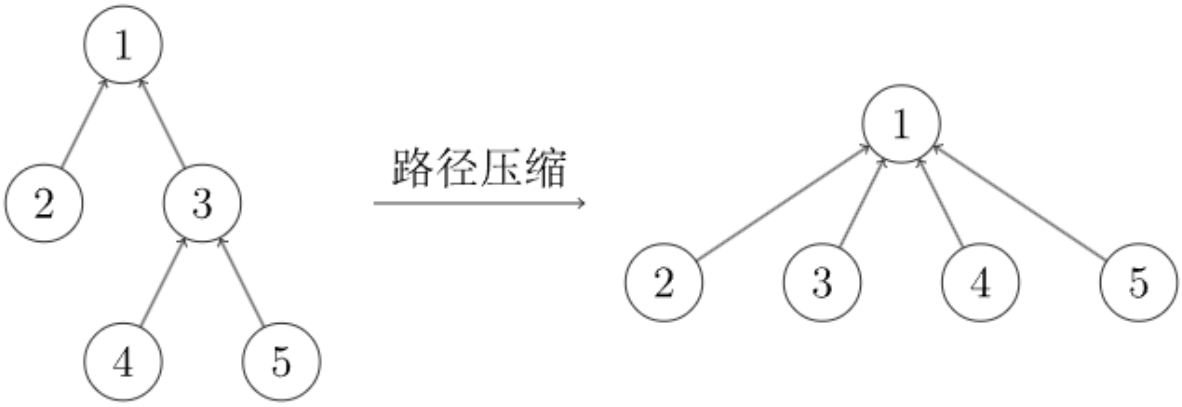


图示

```
size_t dsu::find(size_t x) { return pa[x] == x ? x : find(pa[x]); }
```

路径压缩

查询过程中经过的每个元素都属于该集合，我们可以将其直接连到根节点以加快后续查询。



图示

```
size_t dsu::find(size_t x) { return pa[x] == x ? x : pa[x] = find(pa[x]); }
```

合并

要合并两棵树，我们只需要将一棵树的根节点连到另一棵树的根节点。

[图示：示意图]（https://cspwiki.oss-cn-beijing.aliyuncs.com/S_04_02_4.png）

图示

```
void dsu::unite(size_t x, size_t y) { pa[find(x)] = find(y); }
```

启发式合并

合并时，选择哪棵树的根节点作为新树的根节点会影响未来操作的复杂度。我们可以将节点较少或深度较小的树连到另一棵，以免发生退化。

具体复杂度讨论

由于需要我们支持的只有集合的合并、查询操作，当我们需要将两个集合合二为一时，无论将哪一个集合连接到另一个集合的下面，都能得到正确的结果。但不同的连接方法存在时间复杂度的差异。具体来说，如果我们将一棵点数与深度都较小的集合树连接到一棵更大的集合树下，显然相比于另一种连接方案，接下来执行查找操作的用时更小（也会带来更优的最坏时间复杂度）。

当然，我们不总能遇到恰好如上所述的集合——点数与深度都更小。鉴于点数与深度这两个特征都很容易维护，我们常常从中择一，作为估价函数。而无论选择哪一个，时间复杂度都为 O(malpha(m,n))，具体的证明可参见 References 中引用的论文。

在算法竞赛的实际代码中，即便不使用启发式合并，代码也往往能够在规定时间内完成任务。在 Tarjan 的论文 [^tarjan1984worst] 中，证明了不使用启发式合并、只使用路径压缩的最坏时间复杂度是 O (m log n)。在姚期智的论文 [^yao1985expected] 中，证明了不使用启发式合并、只使用路径压缩，在平均情况下，时间复杂度依然是 O (malpha(m,n))。

如果只使用启发式合并，而不使用路径压缩，时间复杂度为 O(mlog n)。由于路径压缩单次合并可能造成大量修改，有时路径压缩并不适合使用。例如，在可持久化并查集、线段树分治 + 并查集中，一般使用只启发式合并的并查集。

按节点数合并的参考实现：

```
struct dsu {
vector<size_t> pa, size;

explicit dsu(size_t size_) : pa(size_), size(size_, 1) {
iota(pa.begin(), pa.end(), 0);
}

void unite(size_t x, size_t y) {
x = find(x), y = find(y);
if (x == y) return;
if (size[x] < size[y]) swap(x, y);
pa[y] = x;
size[x] += size[y];
}
};
```

删除

要删除一个叶子节点，我们可以将其父亲设为自己。为了保证要删除的元素都是叶子，我们可以预先为每个节点制作副本，并将其副本作为父亲。

```
struct dsu {
vector<size_t> pa, size;

explicit dsu(size_t size_) : pa(size_ * 2), size(size_ * 2, 1) {
iota(pa.begin(), pa.begin() + size_, size_);
iota(pa.begin() + size_, pa.end(), size_);
}

void erase(size_t x) {
--size[find(x)];
pa[x] = x;
}
};
```

移动

与删除类似，通过以副本作为父亲，保证要移动的元素都是叶子。

```
void dsu::move(size_t x, size_t y) {
auto fx = find(x), fy = find(y);
if (fx == fy) return;
pa[x] = fy;
--size[fx], ++size[fy];
}
```

复杂度

时间复杂度

同时使用路径压缩和启发式合并之后，并查集的每个操作平均时间仅为 O(alpha(n)) ，其中 alpha 为阿克曼函数的反函数，其增长极其缓慢，也就是说其单次操作的平均运行时间可以认为是一个很小的常数。

空间复杂度

显然为 O(n) 。

例题：

Status

Problem

Tags

P3367

.

【模板】并查集

数据结构

P1111

.

修复公路

数据结构

P1197

.

星球大战

数据结构

P1551

.

亲戚

数据结构

P1196

.

银河英雄传说

数据结构

二叉堆

单调栈

数据结构 03 • 单调栈

引入

何为单调栈？顾名思义，单调栈即满足单调性的栈结构。与单调队列相比，其只在一端进行进出。

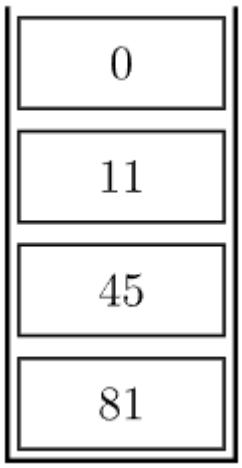
为了描述方便，以下举例及伪代码以维护一个整数的单调递增栈为例。

过程

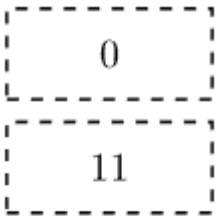
插入

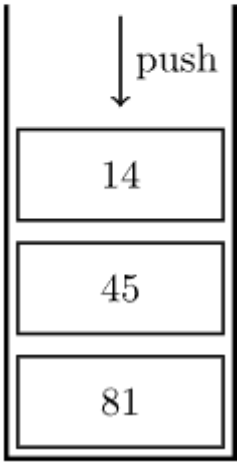
将一个元素插入单调栈时，为了维护栈的单调性，需要在保证将该元素插入到栈顶后整个栈满足单调性的前提下弹出最少的元素。

例如，栈中自顶向下的元素为 0,11,45,81 。



插入元素 14 时为了保证单调性需要依次弹出元素 0,11 ，操作后栈变为 14,45,81 。





用伪代码描述如下：

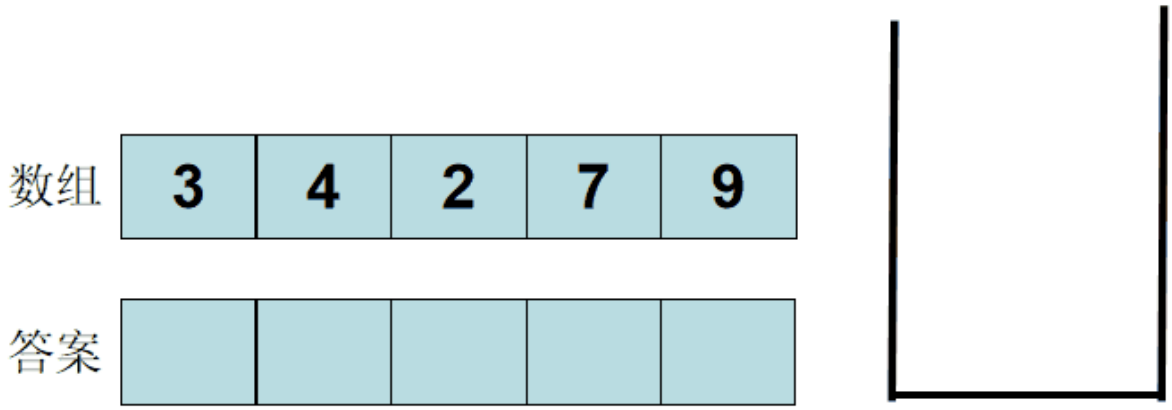
```
insert x
while !sta.empty() && sta.top()<x
sta.pop()
sta.push(x)
```

应用

题面

给出 n 个数组成的数列，请你输出对于每个数，在其左边距离它最近的数，若不存在则输出 -1 。

用单调递增栈，当该元素可以入栈的时候，栈顶元素就是它左侧第一个比它小的元素。




```
#include<iostream>
#include<cstdio>

using namespace std;

const int N = 100010;

int h[N];

int main()
{
int a[N];
int n;
cin >> n;
int tt = 0;

// 单调栈中维护的是一个单调递增的序列
// 一旦入栈时，有元素大于等于它，那就把这些都删掉
// 直至满足栈底到栈顶的元素单调递增。
while(n -- )
{
int m;
scanf("%d", &m);

// 因为是找第一个小的值，不是第一个不大于的值
// 大于等于的都要删除
while(tt && h[tt] >= m) tt--;
// 如果删完了空了，输出 -1
if(!tt) printf("-1 ");
// 不空则，输出栈顶元素
else printf("%d ", h[tt]);//栈顶元素就是左侧第一个比它小的元素。
// 自己入栈
h[++tt] = m;

}

return 0;
}```
```

例题：

Status

Problem

Tags

P5788

.
【模板】单调栈
数据结构
P8094
.
Cow Frisbee S
数据结构
B4273
.
最大的矩形纸片
数据结构
P10334
.
饮料
数据结构
并查集
单调队列

数据结构 04 · 单调队列

引入

在学习单调队列前，让我们先来看一道例题。

给出一个长度为 n 的数组，编程输出每 k 个连续的数中的最大值和最小值。

最暴力的想法很简单，对于每一段 i ~ i+k-1 的序列，逐个比较来找出最大值（和最小值），时间复杂度约为 O(n * k) 。

很显然，这其中进行了大量重复工作，除了开头 k-1 个和结尾 k-1 个数之外，每个数都进行了 k 次比较，而题中 100% 的数据为 n <= 1000000 ，当 k 稍大的情况下，显然会 TLE。

这时所用到的就是单调队列了。

定义

顾名思义，单调队列的重点分为「单调」和「队列」。

「单调」指的是元素的「规律」——递增（或递减）。

「队列」指的是元素只能从队头和队尾进行操作。

例题分析

解释

有了上面「单调队列」的概念，很容易想到用单调队列进行优化。

要求的是每连续的 k 个数中的最大（最小）值，很明显，当一个数进入所要 "寻找" 最大值的范围中时，若这个数比其前面（先进队）的数要大，显然，前面的数会比这个数先出队且不再可能是最大值。

也就是说——当满足以上条件时，可将前面的数 "弹出"，再将该数真正 push 进队尾。

这就相当于维护了一个递减的队列，符合单调队列的定义，减少了重复的比较次数，不仅如此，由于维护出的队伍是查询范围内的且是递减的，队头必定是该查询区域内的最大值，因此输出时只需输出队头即可。

显而易见的是，在这样的算法中，每个数只要进队与出队各一次，因此时间复杂度被降到了 O(N) 。

而由于查询区间长度是固定的，超出查询空间的值再大也不能输出，因此还需要 site 数组记录第 i 个队中的数在原数组中的位置，以弹出越界的队头。

过程

例如我们构造一个单调递增的队列会如下：

原序列为：

```
1 3 -1 -3 5 3 6 7
```

因为我们始终要维护队列保证其 **递增** 的特点，所以会有如下的事情发生：

操作	队列状态
1 入队	`{1}`
3 比 1 大，3 入队	`{1 3}`
-1 比队列中所有元素小，所以清空队列 -1 入队	`{-1}`
-3 比队列中所有元素小，所以清空队列 -3 入队	`{-3}`
5 比 -3 大，直接入队	`{-3 5}`
3 比 5 小，5 出队，3 入队	`{-3 3}`
-3 已经在窗体外，所以 -3 出队；6 比 3 大，6 入队	`{3 6}`
7 比 6 大，7 入队	`{3 6 7}`

示例代码

```
#include <cstdlib>
#include <cstring>
#include <iostream>
constexpr int MAXN = 1000100;
using namespace std;
int q[MAXN], a[MAXN];
int n, k;

void getmin() {  // 得到这个队列里的最小值，直接找到最后的就行了
int head = 0, tail = -1;
for (int i = 1; i < k; i++) {
while (head <= tail && a[q[tail]] >= a[i]) tail--;
q[++tail] = i;
}
for (int i = k; i <= n; i++) {
while (head <= tail && a[q[tail]] >= a[i]) tail--;
q[++tail] = i;
while (q[head] <= i - k) head++;
cout << a[q[head]] << ' ';
}
}

void getmax() {  // 和上面同理
int head = 0, tail = -1;
for (int i = 1; i < k; i++) {
while (head <= tail && a[q[tail]] <= a[i]) tail--;
q[++tail] = i;
}
for (int i = k; i <= n; i++) {
while (head <= tail && a[q[tail]] <= a[i]) tail--;
q[++tail] = i;
while (q[head] <= i - k) head++;
cout << a[q[head]] << ' ';
}
}

int main() {
cin.tie(nullptr)->sync_with_stdio(false);
cin >> n >> k;
for (int i = 1; i <= n; i++) cin >> a[i];
getmin();
cout << '\n';
getmax();
cout << '\n';
return 0;
}
```

Ps. 此处的 "队列" 跟普通队列的一大不同就在于可以从队尾进行操作，STL 中有类似的数据结构 deque。

水滴

给出 N 滴水的坐标， y 表示水滴的高度， x 表示它下落到 x 轴的位置。每滴水以每秒 1 个单位长度的速度下落。你需要把花盆放在 x 轴上的某个位置，使得从被花盆接着的第 1 滴水开始，到被花盆接着的最后 1 滴水结束，之间的时间差至少为 D 。

我们认为，只要水滴落到 x 轴上，与花盆的边沿对齐，就认为被接住。给出 N 滴水的坐标和 D 的大小，请算出最小的花盆的宽度 W 。 1 <= N <= 100000 , 1 <= D <= 1000000, 0 <= x,y <= 10^6

将所有水滴按照 x 坐标排序之后，题意可以转化为求一个 x 坐标差最小的区间使得这个区间内 y 坐标的最大值和最小值之差至少为 D 。我们发现这道题和上一道例题有相似之处，就是都与一个区间内的最大值最小值有关，但是这道题区间的大小不确定，而且区间大小本身还是我们要求的答案。

我们依然可以使用一个递增，一个递减两个单调队列在 R 不断后移时维护 [L,R] 内的最大值和最小值，不过此时我们发现，如果 L 固定，那么 [L,R] 内的最大值只会越来越大，最小值只会越来越

小，所以设 f(R) = max[L,R]-min[L,R] ，则 f(R) 是个关于 R 的递增函数，故 f(R) >= D implies f(r) >= D,Rlt r <= N 。这说明对于每个固定的 L ，向右第一个满足条件的 R 就是最优答案。
所以我们整体求解的过程就是，先固定 L ，从前往后移动 R ，使用两个单调队列维护 [L,R] 的最值。当找到了第一个满足条件的 R ，就更新答案并将 L 也向后移动。随着 L 向后移动，两个单调队列都需及时弹出队头。这样，直到 R 移到最后，每个元素依然是各进出队列一次，保证了 O(n) 的时间复杂度。
示例代码

```
#include <algorithm>
#include <iostream>
using namespace std;
constexpr int N = 100005;
using ll = long long;
int mxq[N], mnq[N];
int D, ans, n, hx, rx, hn, rn;

struct la {
int x, y;

bool operator<(const la &y) const { return x < y.x; }
} a[N];

int main() {
cin.tie(nullptr)->sync_with_stdio(false);
cin >> n >> D;
for (int i = 1; i <= n; ++i) cin >> a[i].x >> a[i].y;
sort(a + 1, a + n + 1);
hx = hn = 1;
ans = 2e9;
int L = 1;
for (int i = 1; i <= n; ++i) {
while (hx <= rx && a[mxq[rx]].y < a[i].y) rx--;
mxq[++rx] = i;
while (hn <= rn && a[mnq[rn]].y > a[i].y) rn--;
mnq[++rn] = i;
while (L <= i && a[mxq[hx]].y - a[mnq[hn]].y >= D) {
ans = min(ans, a[i].x - a[L].x);
L++;
while (hx <= rx && mxq[hx] < L) hx++;
while (hn <= rn && mnq[hn] < L) hn++;
}
}
if (ans < 2e9)
cout << ans << '\n';
else
cout << "-1\n";
return 0;
}
```

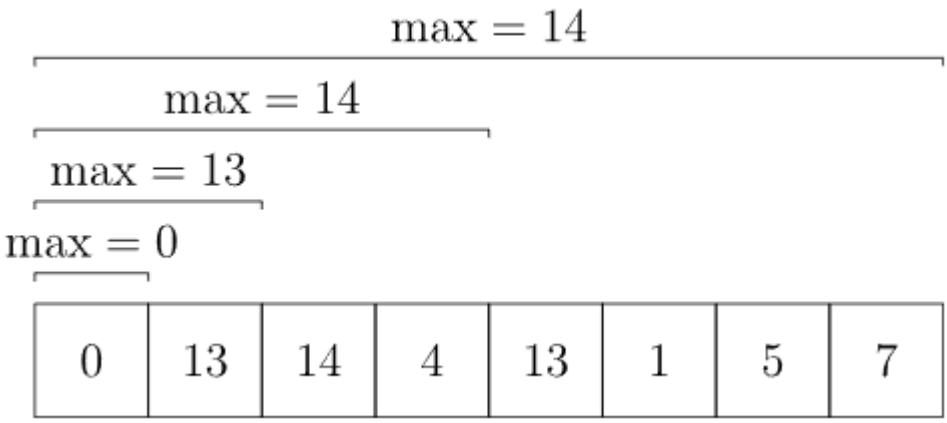
例题：
Status
Problem
Tags
P1886
.
滑动窗口 /【模板】单调队列
数据结构
P1440
.

求m区间内的最小值
数据结构
P2216
.
理想的正方形
数据结构
P2564
.
生日礼物
数据结构
P3400
.
仓鼠窝
数据结构
单调栈
ST 表

数据结构 05 • ST 表

定义

ST 表（Sparse Table，稀疏表）是用于解决 **可重复贡献问题** 的数据结构。



什么是可重复贡献问题？

可重复贡献问题 是指对于运算 opt ，满足 xopt x=x ，则对应的区间询问就是一个可重复贡献问题。例如，最大值有 max(x,x)=x ，gcd 有 gcd(x,x)=x ，所以 RMQ 和区间 GCD 就是一个可重复贡献问题。像区间和就不具有这个性质，如果求区间和的时候采用的预处理区间重叠了，则会导致重叠部分被计算两次，这是我们所不愿意看到的。另外， opt 还必须满足结合律才能使用 ST 表求解。

什么是 RMQ？

RMQ 是英文 Range Maximum/Minimum Query 的缩写，表示区间最大（最小）值。解决 RMQ 问题有很多种方法，可以参考 RMQ 专题。

引入

ST 表模板题

题目大意：给定 n 个数，有 m 个询问，对于每个询问，你需要回答区间 [l,r] 中的最大值。

考虑暴力做法。每次都对区间 [l,r] 扫描一遍，求出最大值。

显然，这个算法会超时。

ST 表

ST 表基于倍增思想，可以做到 Theta(nlog n) 预处理， Theta(1) 回答每个询问。但是不支持修改操作。

基于倍增思想，我们考虑如何求出区间最大值。可以发现，如果按照一般的倍增流程，每次跳 2^i 步的话，询问时的复杂度仍旧是 Theta(log n) ，并没有比线段树更优，反而预处理一步还比线段树慢。

我们发现 max(x,x)=x ，也就是说，区间最大值是一个具有「可重复贡献」性质的问题。即使用来求解的预处理区间有重叠部分，只要这些区间的并是所求的区间，最终计算出的答案就是正确的。

如果手动模拟一下，可以发现我们能使用至多两个预处理过的区间来覆盖询问区间，也就是说询问时的时间复杂度可以被降至 Theta(1) ，在处理有大量询问的题目时十分有效。

具体实现如下：

令 f(i,j) 表示区间 [i,i+2^(j)-1] 的最大值。

显然 f(i,0)=a[i] 。

根据定义式，第二维就相当于倍增的时候「跳了 2^(j)-1 步」，依据倍增的思路，写出状态转移方程： f(i,j)=max(f(i,j-1),f(i+2^(j-1),j-1)) 。

$$\max\left(\boxed{\max(\{a_i,\ldots,a_{i+2^{j-1}-1}\})}\ \boxed{\max(\{a_{i+2^{j-1}},\ldots,a_{i+2^j-1}\})}\right)$$

$$\Vert$$

$$\boxed{\max(\{a_i,\ldots,a_{i+2^j-1}\})}$$

以上就是预处理部分。而对于查询，可以简单实现如下：

对于每个询问 [l,r] ，我们把它分成两部分： [l,l+2^(s)-1] 与 [r-2^(s)+1,r] ，其中 s= floor log2 。两部分的结果的最大值就是回答。

$$\overbrace{\max=\max(\max_l,\max_r)=14}$$

$$\overbrace{\max_r=f(4,2)=13}$$

$$\overbrace{\max_l=f(2,2)=14}$$

0	13	14	4	13	1	5	7

根据上面对于「可重复贡献问题」的论证，由于最大值是「可重复贡献问题」，重叠并不会对区间最大值产生影响。又因为这两个区间完全覆盖了 [l,r] ，可以保证答案的正确性。

示例代码

```
#include <algorithm>
#include <iostream>
using namespace std;
constexpr int MAXN = 2000001;
constexpr int logN = 21;
int f[MAXN][logN + 1], Logn[MAXN + 1];

void pre() {  // 准备工作，初始化
Logn[1] = 0;
Logn[2] = 1;
for (int i = 3; i < MAXN; i++) {
Logn[i] = Logn[i / 2] + 1;
}
}

int main() {
cin.tie(nullptr)->sync_with_stdio(false);
int n, m;
cin >> n >> m;
for (int i = 1; i <= n; i++) cin >> f[i][0];
pre();
for (int j = 1; j <= logN; j++)
for (int i = 1; i + (1 << j) - 1 <= n; i++)
f[i][j] = max(f[i][j - 1], f[i + (1 << (j - 1))][j - 1]);  // ST 表具体实现
for (int i = 1; i <= m; i++) {
int x, y;
cin >> x >> y;
int s = Logn[y - x + 1];
cout << max(f[x][s], f[y - (1 << s) + 1][s]) << '\n';
}
return 0;
}
```

注意点

1. 输入输出数据一般很多，建议开启输入输出优化。
2. 每次用库自带的 log 函数重新计算 log 函数值并不值得，建议进行如下的预处理：

Logn[1] <- 0, Logn[i] <- Logn[(i)/(2)] + 1.

ST 表维护其他信息

除 RMQ 以外，还有其它的「可重复贡献问题」。例如「区间按位与」、「区间按位或」、「区间 GCD」，ST 表都能高效地解决。

需要注意的是，对于「区间 GCD」，ST 表的查询复杂度并没有比线段树更优（令值域为 w ，ST 表的查询复杂度为 Theta(log w) ，而线段树为 Theta(log n+log w) ，且值域一般是大于 n 的），但是 ST 表的预处理复杂度也没有比线段树更劣，而编程复杂度方面 ST 表比线段树简单很多。

如果分析一下，「可重复贡献问题」一般都带有某种类似 RMQ 的成分。例如「区间按位与」就是每一位取最小值，而「区间 GCD」则是每一个质因数的指数取最小值。

总结

ST 表能较好的维护「可重复贡献」的区间信息（同时也应满足结合律），时间复杂度较低，代码量相对其他算法很小。但是，ST 表能维护的信息非常有限，不能较好地扩展，并且不支持修改操作。

例题：

Status

Problem

Tags

P3865

.

【模板】ST 表 && RMQ 问题

数据结构
P3145
.
Splitting the Field G
数据结构
P7167
.
Fountain (Day1)
数据结构
单调队列
树状数组

数据结构 06 · 树状数组

引入

树状数组是一种支持 **单点修改** 和 **区间查询** 的，代码量小的数据结构。

什么是「单点修改」和「区间查询」？

假设有这样一道题：

已知一个数列 a ，你需要进行下面两种操作：

- 给定 x, y ，将 a[x] 自增 y 。
- 给定 l, r ，求解 a[l … r] 的和。

其中第一种操作就是「单点修改」，第二种操作就是「区间查询」。

类似地，还有：「区间修改」、「单点查询」。它们分别的一个例子如下：

- 区间修改：给定 l, r, x ，将 a[l … r] 中的每个数都分别自增 x ；
- 单点查询：给定 x ，求解 a[x] 的值。

注意到，区间问题一般严格强于单点问题，因为对单点的操作相当于对一个长度为 1 的区间操作。

普通树状数组维护的信息及运算要满足 **结合律** 且 **可差分**，如加法（和）、乘法（积）、异或等。

- 结合律：(x o y) o z = x o (y o z) ，其中 o 是一个二元运算符。
- 可差分：具有逆运算的运算，即已知 x o y 和 x 可以求出 y 。

事实上，树状数组能解决的问题是线段树能解决的问题的子集：树状数组能做的，线段树一定能做；线段树能做的，树状数组不一定可以。然而，树状数组的代码要远比线段树短，时间效率常数也更小，因此仍有学习价值。

有时，在差分数组和辅助数组的帮助下，树状数组还可解决更强的 **区间加单点值** 和 **区间加区间和** 问题。

树状数组

初步感受

先来举个例子：我们想知道 a[1 … 7] 的前缀和，怎么做？

一种做法是： a[1] + a[2] + a[3] + a[4] + a[5] + a[6] + a[7] ，需要求 7 个数的和。

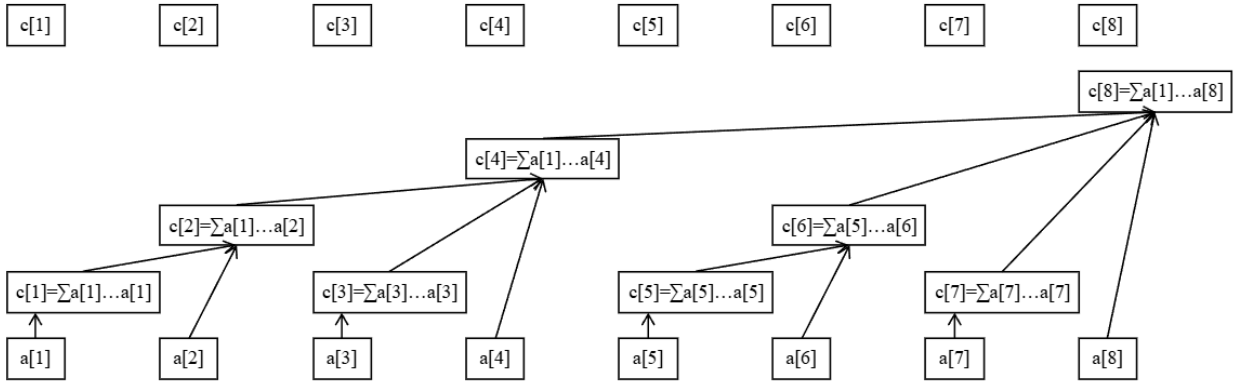
但是如果已知三个数 A ， B ， C ， A = a[1 … 4] 的和， B = a[5 … 6] 的总和， C = a[7 … 7] 的总和（其实就是 a[7] 自己）。你会怎么算？你一定会回答： A + B + C ，只需要求 3 个数的和。

这就是树状数组能快速求解信息的原因：我们总能将一段前缀 [1, n] 拆成 **不多于 log n 段区间**，使得这 log n 段区间的信息是 **已知的**。

于是，我们只需合并这 log n 段区间的信息，就可以得到答案。相比于原来直接合并 n 个信息，效率有了很大的提高。

不难发现信息必须满足结合律，否则就不能像上面这样合并了。

下面这张图展示了树状数组的工作原理：



示意图

最下面的八个方块代表原始数据数组 a 。上面参差不齐的方块（与最上面的八个方块是同一个数组）代表数组 a 的上级—— c 数组。

c 数组就是用来储存原始数组 a 某段区间的和的，也就是说，这些区间的信息是已知的，我们的目标就是把查询前缀拆成这些小区间。

例如，从图中可以看出：

- c[2] 管辖的是 a[1 … 2] ；
- c[4] 管辖的是 a[1 … 4] ；
- c[6] 管辖的是 a[5 … 6] ；
- c[8] 管辖的是 a[1 … 8] ；
- 剩下的 c[x] 管辖的都是 a[x] 自己（可以看做 a[x … x] 的长度为 1 的小区间）。

不难发现， c[x] 管辖的一定是一段右边界是 x 的区间总信息。我们先不关心左边界，先来感受一下树状数组是如何查询的。

举例：计算 a[1 … 7] 的和。

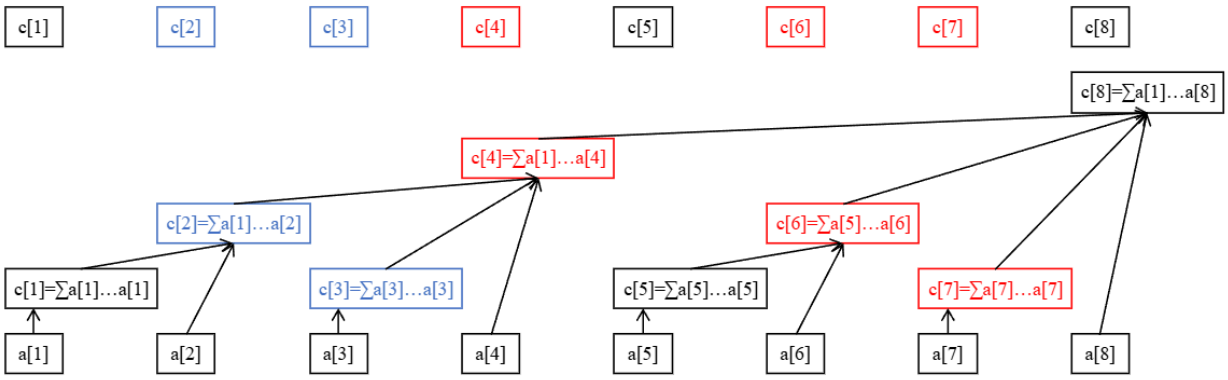
过程：从 c[7] 开始往前跳，发现 c[7] 只管辖 a[7] 这个元素；然后找 c[6] ，发现 c[6] 管辖的是 a[5 … 6] ，然后跳到 c[4] ，发现 c[4] 管辖的是 a[1 … 4] 这些元素，然后再试图跳到 c[0] ，但事实上 c[0] 不存在，不跳了。

我们刚刚找到的 c 是 c[7], c[6], c[4] ，事实上这就是 a[1 … 7] 拆分出的三个小区间，合并得到答案是 c[7] + c[6] + c[4] 。

举例：计算 a[4 … 7] 的和。

我们还是从 c[7] 开始跳，跳到 c[6] 再跳到 c[4] 。此时我们发现它管理了 a[1 … 4] 的和，但是我们不想要 a[1 … 3] 这一部分，怎么办呢？很简单，减去 a[1 … 3] 的和就行了。

那不妨考虑最开始，就将查询 a[4 … 7] 的和转化为查询 a[1 … 7] 的和，以及查询 a[1 … 3] 的和，最终将两个结果作差。



查询 ∑a[4]...a[7] = (∑a[1]...a[7]) - (∑a[1]...a[3])

查询

管辖区间

那么问题来了， cx 管辖的区间到底往左延伸多少？也就是说，区间长度是多少？

树状数组中，规定 c[x] 管辖的区间长度为 2^(k) ，其中：

- 设二进制最低位为第 0 位，则 k 恰好为 x 二进制表示中，最低位的 `1` 所在的二进制位数；
- 2^(k) （ c[x] 的管辖区间长度）恰好为 x 二进制表示中，最低位的 `1` 以及后面所有 `0` 组成的数。

举个例子， c[88] 管辖的是哪个区间？

因为 88_((10))=01011000_((2)) ，其二进制最低位的 `1` 以及后面的 `0` 组成的二进制是 `1000` ，即 8 ，所以 c[88] 管辖 8 个 a 数组中的元素。

因此， c[88] 代表 a[81 … 88] 的区间信息。

我们记 x 二进制最低位 `1` 以及后面的 `0` 组成的数为 lowbit(x) ，那么 c[x] 管辖的区间就是 [x-lowbit(x)+1, x] 。

这里注意： lowbit 指的不是最低位 `1` 所在的位数 k ，而是这个 `1` 和后面所有 `0` 组成的 2^(k) 。

怎么计算 `lowbit` ？根据位运算知识，可以得到 `lowbit(x) = x & -x` 。

lowbit 的原理

将 `x` 的二进制所有位全部取反，再加 1，就可以得到 `-x` 的二进制编码。例如， 6 的二进制编码是 `110` ，全部取反后得到 `001` ，加 `1` 得到 `010` 。

设原先 `x` 的二进制编码是 `(...)10...00` ，全部取反后得到 `[...]01...11` ，加 `1` 后得到 `[...]10...00` ，也就是 `-x` 的二进制编码了。这里 `x` 二进制表示中第一个 `1` 是 `x` 最低位的 `1` 。

`(...)` 和 `[...]` 中省略号的每一位分别相反，所以 `x & -x = (...)10...00 & [...]10...00 = 10...00` ，得到的结果就是 `lowbit` 。

```
int lowbit(int x) {
// x 的二进制中，最低位的 1 以及后面所有 0 组成的数。
// lowbit(0b01011000) == 0b00001000
//          ~~~~^~~~
// lowbit(0b01110010) == 0b00000010
//          ~~~~~~^~
return x & -x;
}
```

区间查询

接下来我们来看树状数组具体的操作实现，先来看区间查询。

回顾查询 a[4 … 7] 的过程，我们是将它转化为两个子过程：查询 a[1 … 7] 和查询 a[1 … 3] 的和，最终作差。

其实任何一个区间查询都可以这么做：查询 a[1 … r] 的和，就是 a[1 … r] 的和减去 a[1 … l - 1] 的和，从而把区间问题转化为前缀问题，更方便处理。
事实上，将有关 l … r 的区间询问转化为 1 … r 和 1 … l - 1 的前缀询问再差分，在竞赛中是一个非常常用的技巧。
那前缀查询怎么做呢？回顾下查询 a[1 … 7] 的过程：

> 从 c[7] 往前跳，发现 c[7] 只管辖 a[7] 这个元素；然后找 c[6] ，发现 c[6] 管辖的是 a[5 … 6] ，然后跳到 c[4] ，发现 c[4] 管辖的是 a[1 … 4] 这些元素，然后再试图跳到 c[0] ，但事实上 c[0] 不存在，不跳了。
> 我们刚刚找到的 c 是 c[7], c[6], c[4] ，事实上这就是 a[1 … 7] 拆分出的三个小区间，合并一下，答案是 c[7] + c[6] + c[4] 。

观察上面的过程，每次往前跳，一定是跳到现区间的左端点的左一位，作为新区间的右端点，这样才能将前缀不重不漏地拆分。比如现在 c[6] 管的是 a[5 … 6] ，下一次就跳到 5 - 1 = 4 ，即访问 c[4] 。
我们可以写出查询 a[1 … x] 的过程：

- 从 c[x] 开始往前跳，有 c[x] 管辖 a[x-lowbit(x)+1 … x] ；
- 令 x <- x - lowbit(x) ，如果 x = 0 说明已经跳到尽头了，终止循环；否则回到第一步。
- 将跳到的 c 合并。

实现时，我们不一定要先把 c 都跳出来然后一起合并，可以边跳边合并。
比如我们要维护的信息是和，直接令初始 ans = 0 ，然后每跳到一个 c[x] 就 ans <- ans + c[x] ，最终 ans 就是所有合并的结果。

```
int getsum(int x) {  // a[1]..a[x] 的和
int ans = 0;
while (x > 0) {
ans = ans + c[x];
x = x - lowbit(x);
}
return ans;
}
```

树状数组与其树形态的性质
在讲解单点修改之前，先讲解树状数组的一些基本性质，以及其树形态来源，这有助于更好理解树状数组的单点修改。
我们约定：

- l(x) = x - lowbit(x) + 1 。即， l(x) 是 c[x] 管辖范围的左端点。
- 对于任意正整数 x ，总能将 x 表示成 s * 2^(k + 1) + 2^(k) 的形式，其中 lowbit(x) = 2^(k) 。
- 下面「 c[x] 和 c[y] 不交」指 c[x] 的管辖范围和 c[y] 的管辖范围不相交，即 [l(x), x] 和 [l(y), y] 不相交。「 c[x] 包含于 c[y] 」等表述同理。

性质 1 ：对于 x <= y ，要么有 c[x] 和 c[y] 不交，要么有 c[x] 包含于 c[y] 。
证明
证明：假设 c[x] 和 c[y] 相交，即 [l(x), x] 和 [l(y), y] 相交，则一定有 l(y) <= x <= y 。
将 y 表示为 s * 2^(k +1) + 2^(k) ，则 l(y) = s * 2^(k + 1) + 1 。所以， x 可以表示为 s * 2^(k +1) + b ，其中 1 <= b <= 2^(k) 。
不难发现 lowbit(x) = lowbit(b) 。又因为 b - lowbit(b) >= 0 ，
所以 l(x) = x - lowbit(x) + 1 = s * 2^(k +1) + b - lowbit(b) +1 >= s * 2^(k +1) + 1 = l(y) ，即 l(y) <= l(x) <= x <= y 。

所以，如果 c[x] 和 c[y] 相交，那么 c[x] 的管辖范围一定完全包含于 c[y] 。

性质 2 ： c[x] 真包含于 c[x + lowbit(x)] 。

证明

证明：设 y = x + lowbit(x) ， x = s * 2^(k + 1) + 2^(k) ，则 y = (s + 1) * 2^(k +1) ，l(x) = s * 2^(k + 1) + 1 。

不难发现 lowbit(y) >= 2^(k + 1) ，所以 l(y) = (s + 1) * 2^(k + 1) - lowbit(y) + 1 <= s * 2^(k +1) + 1= l(x) ，即 l(y) <= l(x) <= x < y 。

所以， c[x] 真包含于 c[x + lowbit(x)] 。

性质 3 ：对于任意 x < y < x + lowbit(x) ，有 c[x] 和 c[y] 不交。

证明

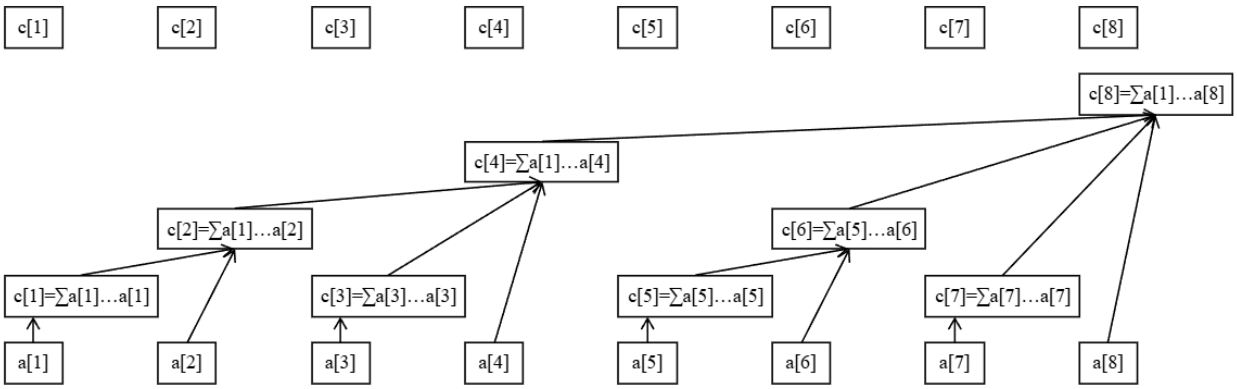
证明：设 x = s * 2^(k + 1) + 2^(k) ，则 y = x + b = s * 2^(k + 1) + 2^(k) + b ，其中 1 <= b < 2^(k) 。

不难发现 lowbit(y) = lowbit(b) 。又因为 b - lowbit(b) >= 0 ，

因此 l(y) = y - lowbit(y) + 1 = x + b - lowbit(b) + 1 > x ，即 l(x) <= x < l(y) <= y 。

所以， c[x] 和 c[y] 不交。

有了这三条性质的铺垫，我们接下来看树状数组的树形态（请忽略 a 向 c 的连边）。



图示

事实上，树状数组的树形态是 x 向 x + lowbit(x) 连边得到的图，其中 x + lowbit(x) 是 x 的父亲。

注意，在考虑树状数组的树形态时，我们不考虑树状数组大小的影响，即我们认为这是一棵无限大的树，方便分析。实际实现时，我们只需用到 x <= n 的 c[x] ，其中 n 是原数组长度。

这棵树天然满足了很多美好性质，下面列举若干（设 fa[u] 表示 u 的直系父亲）：

- u < fa[u] 。
- u 大于任何一个 u 的后代，小于任何一个 u 的祖先。
- 点 u 的 lowbit 严格小于 fa[u] 的 lowbit 。

证明

设 y = x + lowbit(x) ， x = s * 2^(k + 1) + 2^(k) ，则 y = (s + 1) * 2^(k +1) ，不难发现 lowbit(y) >= 2^(k + 1) > lowbit(x) ，证毕。

- 点 x 的高度是 log[2]lowbit(x) ，即 x 二进制最低位 `1` 的位数。

高度的定义

点 x 的高度 h(x) 满足：如果 x mod 2 = 1 ，则 h(x) = 0 ，否则 h(x) = max(h(y)) + 1 ，其中 y 代表 x 的所有儿子（此时 x 至少存在一个儿子 x - 1 ）。

也就是说，一个点的高度恰好比它最高的那个儿子再高 1 。如果一个点没有儿子，它的高度是 0 。

这里引出高度这一概念，是为后面解释复杂度更方便。

- c[u] 真包含于 c[fa[u]] （性质 2 ）。
- c[u] 真包含于 c[v] ，其中 v 是 u 的任一祖先（在上一条性质上归纳）。

- c[u] 真包含 c[v] ，其中 v 是 u 的任一后代（上面那条性质 u ， v 颠倒）。
- 对于任意 v' > u ，若 v' 不是 u 的祖先，则 c[u] 和 c[v'] 不交。

证明

u 和 u 的祖先中，一定存在一个点 v 使得 v < v' < fa[v] ，根据性质 3 得 c[v'] 不相交于 c[v] ，而 c[v] 包含 c[u] ，因此 c[v'] 不交于 c[u] 。

- 对于任意 v < u ，如果 v 不在 u 的子树上，则 c[u] 和 c[v] 不交（上面那条性质 u ， v' 颠倒）。
- 对于任意 v > u ，当且仅当 v 是 u 的祖先， c[u] 真包含于 c[v] （上面几条性质的总结）。这就是树状数组单点修改的核心原理。
- 设 u = s * 2^(k + 1) + 2^(k) ，则其儿子数量为 k = log[2]lowbit(u) ，编号分别为 u - 2^(t)(0 <= t < k) 。
- 举例：假设 k = 3 ， u 的二进制编号为 `...1000` ，则 u 有三个儿子，二进制编号分别为 `...0111` 、 `...0110` 、 `...0100` 。

证明

在一个数 x 的基础上减去 2^(t) ， x 二进制第 t 位会反转，而更低的位保持不变。

考虑 u 的儿子 v ，有 v + lowbit(v) = u ，即 v = u - 2^(t) 且 lowbit(v) = 2^(t) 。设 u = s * 2^(k + 1) + 2^(k) 。

考虑 0 <= t < k， u 的第 t 位及后方均为 0 ，所以 v = u - 2^(t) 的第 t 位变为 1 ，后面仍为 0 ，**满足** lowbit(v) = 2^(t) 。

考虑 t = k，则 v = u - 2^(k) ， v 的第 k 位变为 0 ，**不满足** lowbit(v) = 2^(t) 。

考虑 t > k，则 v = u - 2^(t) ， v 的第 k 位是 1 ，所以 lowbit(v) = 2^(k) ，**不满足** lowbit(v) = 2^(t) 。

- u 的所有儿子对应 c 的管辖区间恰好拼接成 [l(u), u - 1] 。
- 举例：假设 k = 3 ， u 的二进制编号为 `...1000` ，则 u 有三个儿子，二进制编号分别为 `...0111` 、 `...0110` 、 `...0100` 。
- `c[...0100]` 表示 `a[...0001 ~ ...0100]` 。
- `c[...0110]` 表示 `a[...0101 ~ ...0110]` 。
- `c[...0111]` 表示 `a[...0111 ~ ...0111]` 。
- 不难发现上面是三个管辖区间的并集恰好是 `a[...0001 ~ ...0111]` ，即 [l(u), u - 1] 。

证明

u 的儿子总能表示成 u - 2^(t)(0 <= t < k) ，不难发现， t 越小， u - 2^(t) 越大，代表的区间越靠右。我们设 f(t) = u - 2^(t) ，则 f(k - 1), f(k - 2), …, f(0) 分别构成 u 从左到右的儿子。

不难发现 lowbit(f(t)) = 2^(t) ，所以 l(f(t)) = u - 2^(t) - 2^(t) + 1 = u - 2^(t + 1) + 1 。

考虑相邻的两个儿子 f(t + 1) 和 f(t) 。前者管辖区间的右端点是 f(t + 1) = u - 2^(t + 1) ，后者管辖区间的左端点是 l(f(t)) = u - 2^(t + 1) + 1 ，恰好相接。

考虑最左面的儿子 f(k - 1) ，其管辖左边界 l(f(k - 1)) = u - 2^(k) + 1 恰为 l(u) 。

考虑最右面的儿子 f(0) ，其管辖右边界就是 u - 1 。

因此，这些儿子的管辖区间可以恰好拼成 [l(u), u - 1] 。

单点修改

现在来考虑如何单点修改 a[x] 。

我们的目标是快速正确地维护 c 数组。为保证效率，我们只需遍历并修改管辖了 a[x] 的所有 c[y] ，因为其他的 c 显然没有发生变化。

管辖 a[x] 的 c[y] 一定包含 c[x] （根据性质 1 ），所以 y 在树状数组树形态上是 x 的祖先。因此我们从 x 开始不断跳父亲，直到跳得超过了原数组长度为止。
设 n 表示 a 的大小，不难写出单点修改 a[x] 的过程：

- 初始令 x' = x 。
- 修改 c[x'] 。
- 令 x' <- x' + lowbit(x') ，如果 x' > n 说明已经跳到尽头了，终止循环；否则回到第二步。

区间信息和单点修改的种类，共同决定 c[x'] 的修改方式。下面给几个例子：

- 若 c[x'] 维护区间和，修改种类是将 a[x] 加上 p ，则修改方式则是将所有 c[x'] 也加上 p 。
- 若 c[x'] 维护区间积，修改种类是将 a[x] 乘上 p ，则修改方式则是将所有 c[x'] 也乘上 p 。

然而，单点修改的自由性使得修改的种类和维护的信息不一定是同种运算，比如，若 c[x'] 维护区间和，修改种类是将 a[x] 赋值为 p ，可以考虑转化为将 a[x] 加上 p - a[x] 。如果是将 a[x] 乘上 p ，就考虑转化为 a[x] 加上 a[x] * p - a[x] 。
下面以维护区间和，单点加为例给出实现。

```
void add(int x, int k) {
while (x <= n) {  // 不能越界
c[x] = c[x] + k;
x = x + lowbit(x);
}
}
```

建树
也就是根据最开始给出的序列，将树状数组建出来（ c 全部预处理好）。
一般可以直接转化为 n 次单点修改，时间复杂度 Theta(n log n) （复杂度分析在后面）。
比如给定序列 a = (5, 1, 4) 要求建树，直接看作对 a[1] 单点加 5 ，对 a[2] 单点加 1 ，对 a[3] 单点加 4 即可。
复杂度分析
空间复杂度显然 Theta(n) 。
时间复杂度：

- 对于区间查询操作：整个 x <- x - lowbit(x) 的迭代过程，可看做将 x 二进制中的所有 1 ，从低位到高位逐渐改成 0 的过程，拆分出的区间数等于 x 二进制中 1 的数量（即 popcount(x) ）。因此，单次查询时间复杂度是 Theta(log n) ；
- 对于单点修改操作：跳父亲时，访问到的高度一直严格增加，且始终有 x <= n 。由于点 x 的高度是 log[2]lowbit(x) ，所以跳到的高度不会超过 log[2]n ，所以访问到的 c 的数量是 log n 级别。因此，单次单点修改复杂度是 Theta(log n) 。

例题：
Status
Problem
Tags
P3374
.
【模板】树状数组 1
数据结构
P3368
.
【模板】树状数组 2

数据结构
P2068
.
统计和
数据结构
P1966
.
火柴排队
数据结构
P1438
.
无聊的数列
数据结构
ST 表
线段树

数据结构 07 · 线段树

引入

线段树是算法竞赛中常用的用来维护 区间信息 的数据结构。

线段树可以在 O(log N) 的时间复杂度内实现单点修改、区间修改、区间查询（区间求和，求区间最大值，求区间最小值）等操作。

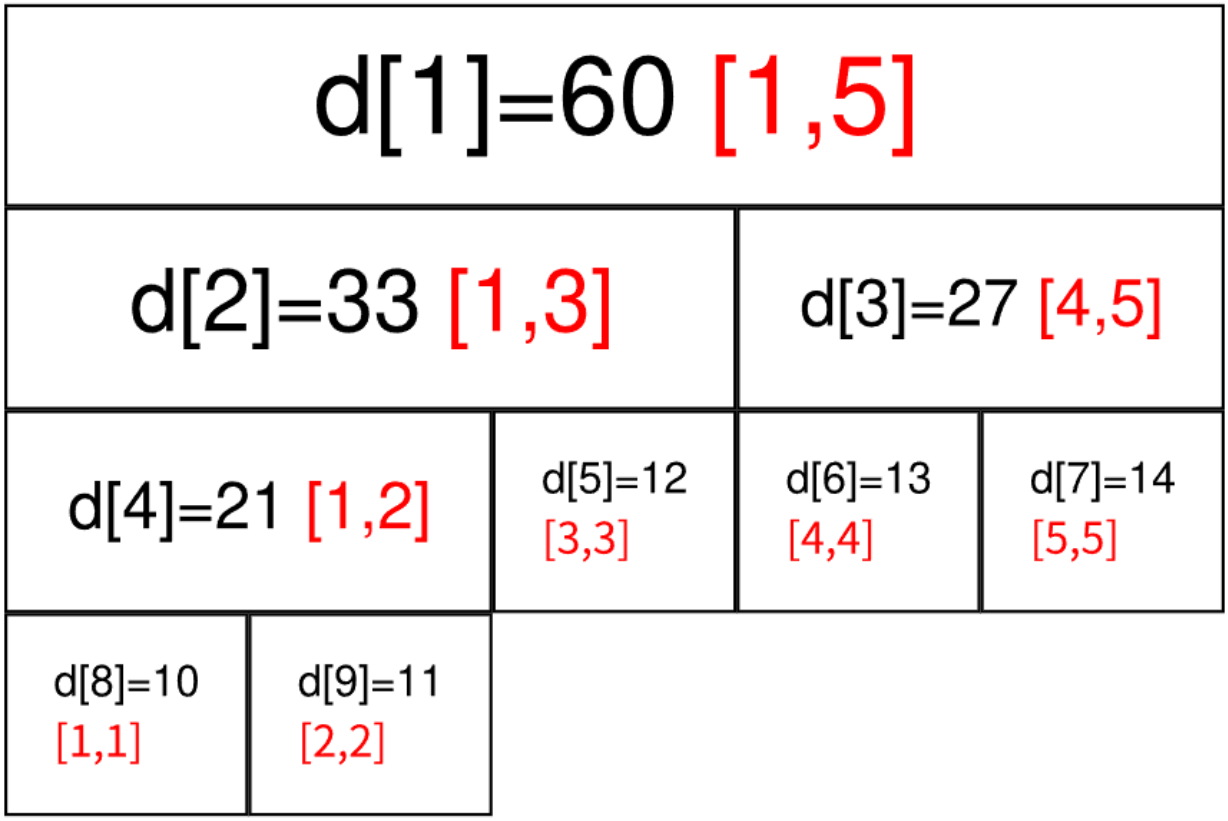
线段树的基本结构与建树

过程

线段树将每个长度不为 1 的区间划分成左右两个区间递归求解，把整个线段划分为一个树形结构，通过合并左右两区间信息来求得该区间的信息。这种数据结构可以方便的进行大部分的区间操作。

有个大小为 5 的数组 a=10,11,12,13,14 ，要将其转化为线段树，有以下做法：设线段树的根节点编号为 1 ，用数组 d 来保存我们的线段树， d[i] 用来保存线段树上编号为 i 的节点的值（这里每个节点所维护的值就是这个节点所表示的区间总和）。

我们先给出这棵线段树的形态，如图所示：



图中每个节点中用红色字体标明的区间，表示该节点管辖的 a 数组上的位置区间。如 d[1] 所管辖的区间就是 [1,5] （ a[1],a[2], ... ,a[5] ），即 d[1] 所保存的值是 a[1]+a[2]+ ... +a[5] ，d[1]=60 表示的是 a[1]+a[2]+ ... +a[5]=60 。

通过观察不难发现， d[i] 的左儿子节点就是 d_(2 * i) ， d[i] 的右儿子节点就是 d_(2 * i+1) 。如果 d[i] 表示的是区间 [s,t] （即 d[i]=a[s]+a[s+1]+ ... +a[t] ）的话，那么 d[i] 的左儿子节点表示的是区间 [s, (s+t)/(2)] ， d[i] 的右儿子表示的是区间 [(s+t)/(2) +1,t] 。

在实现时，我们考虑递归建树。设当前的根节点为 p ，如果根节点管辖的区间长度已经是 1 ，则可以直接根据 a 数组上相应位置的值初始化该节点。否则我们将该区间从中点处分割为两个子区间，分别进入左右子节点递归建树，最后合并两个子节点的信息。

实现

此处给出代码实现，可参考注释理解：

```
void build(int s, int t, int p) {
// 对 [s,t] 区间建立线段树,当前根的编号为 p
if (s == t) {
d[p] = a[s];
return;
}
int m = s + ((t - s) >> 1);
// 移位运算符的优先级小于加减法，所以加上括号
// 如果写成 (s + t) >> 1 可能会超出 int 范围
build(s, m, p * 2), build(m + 1, t, p * 2 + 1);
// 递归对左右区间建树
d[p] = d[p * 2] + d[(p * 2) + 1];
}
```

关于线段树的空间：如果采用堆式存储（ 2p 是 p 的左儿子， 2p+1 是 p 的右儿子），若有 n 个叶子结点，则 d 数组的范围最大为 2^ ceil logn +1 。

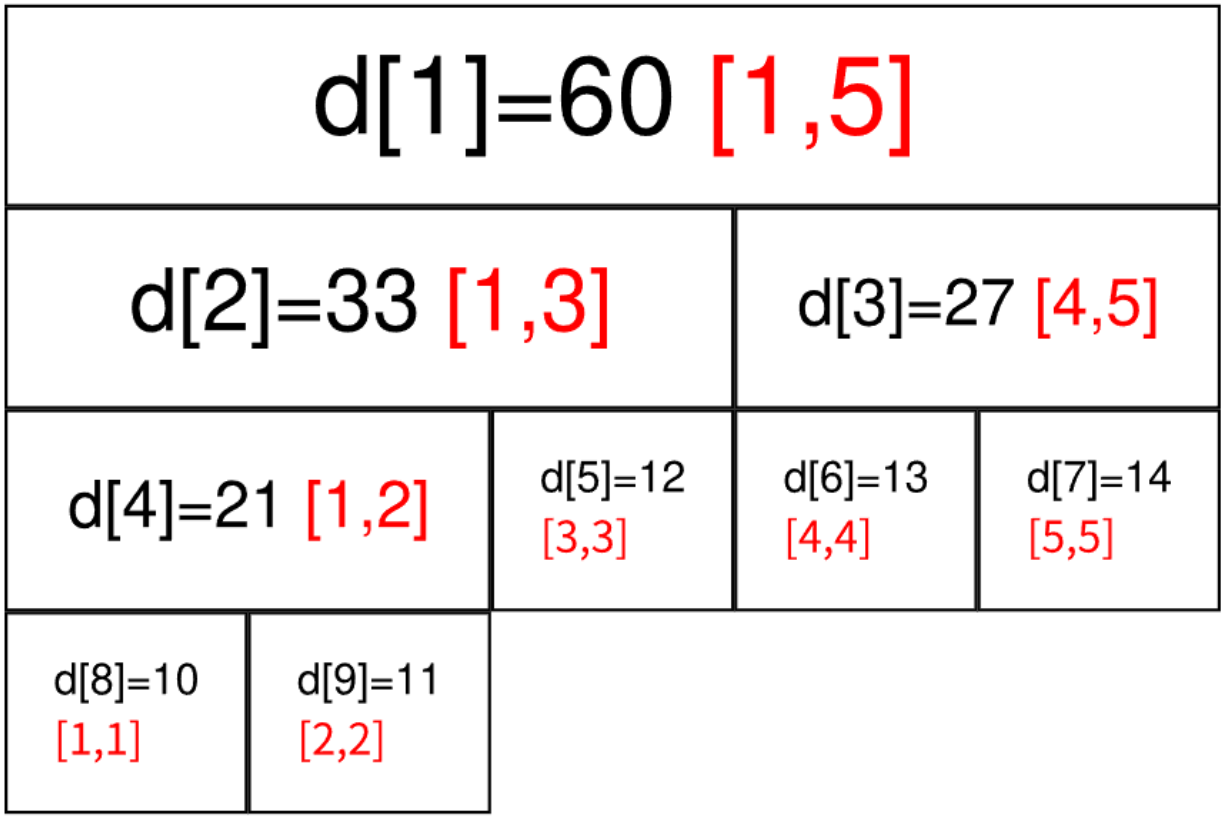
分析：容易知道线段树的深度是 ceil logn 的，则在堆式储存情况下叶子节点（包括无用的叶子节点）数量为 2^ ceil logn 个，又由于其为一棵完全二叉树，则其总节点个数 2^ ceil logn +1-1 。

当然如果你懒得计算的话可以直接把数组长度设为 4n ，因为 (2^ ceil logn +1-1)/(n) 的最大值在 n=2^(x)+1(x 属于 N[+]) 时取到，此时节点数为 2^ ceil logn +1-1=2^(x+2)-1=4n-5 。
而堆式存储存在无用的叶子节点，可以考虑使用内存池管理线段树节点，每当需要新建节点时从池中获取。自底向上考虑，必有每两个底层节点合并为一个上层节点，因此可以类似哈夫曼树地证明，如果有 n 个叶子节点，这样的线段树总共有 2n-1 个节点。其空间效率优于堆式存储，并且是可能的最优情况。

线段树的区间查询

过程

区间查询，比如求区间 [l,r] 的总和（即 a_(l)+a_(l+1)+ ... +a[r] ）、求区间最大值/最小值等操作。



仍然以最开始的图为例，如果要查询区间 [1,5] 的和，那直接获取 d[1] 的值（ 60 ）即可。
如果要查询的区间为 [3,5] ，此时就不能直接获取区间的值，但是 [3,5] 可以拆成 [3,3] 和 [4,5] ，可以通过合并这两个区间的答案来求得这个区间的答案。
一般地，如果要查询的区间是 [1,r] ，则可以将其拆成最多为 O(log n) 个 **极大** 的区间，合并这些区间即可求出 [l,r] 的答案。

实现

此处给出代码实现，可参考注释理解：

```
int getsum(int l, int r, int s, int t, int p) {
// [l, r] 为查询区间, [s, t] 为当前节点包含的区间, p 为当前节点的编号
if (l <= s && t <= r)
return d[p]; // 当前区间为询问区间的子集时直接返回当前区间的和
int m = s + ((t - s) >> 1), sum = 0;
if (l <= m) sum += getsum(l, r, s, m, p * 2);
// 如果左儿子代表的区间 [s, m] 与询问区间有交集, 则递归查询左儿子
if (r > m) sum += getsum(l, r, m + 1, t, p * 2 + 1);
// 如果右儿子代表的区间 [m + 1, t] 与询问区间有交集, 则递归查询右儿子
return sum;
}
```

线段树的区间修改与懒惰标记

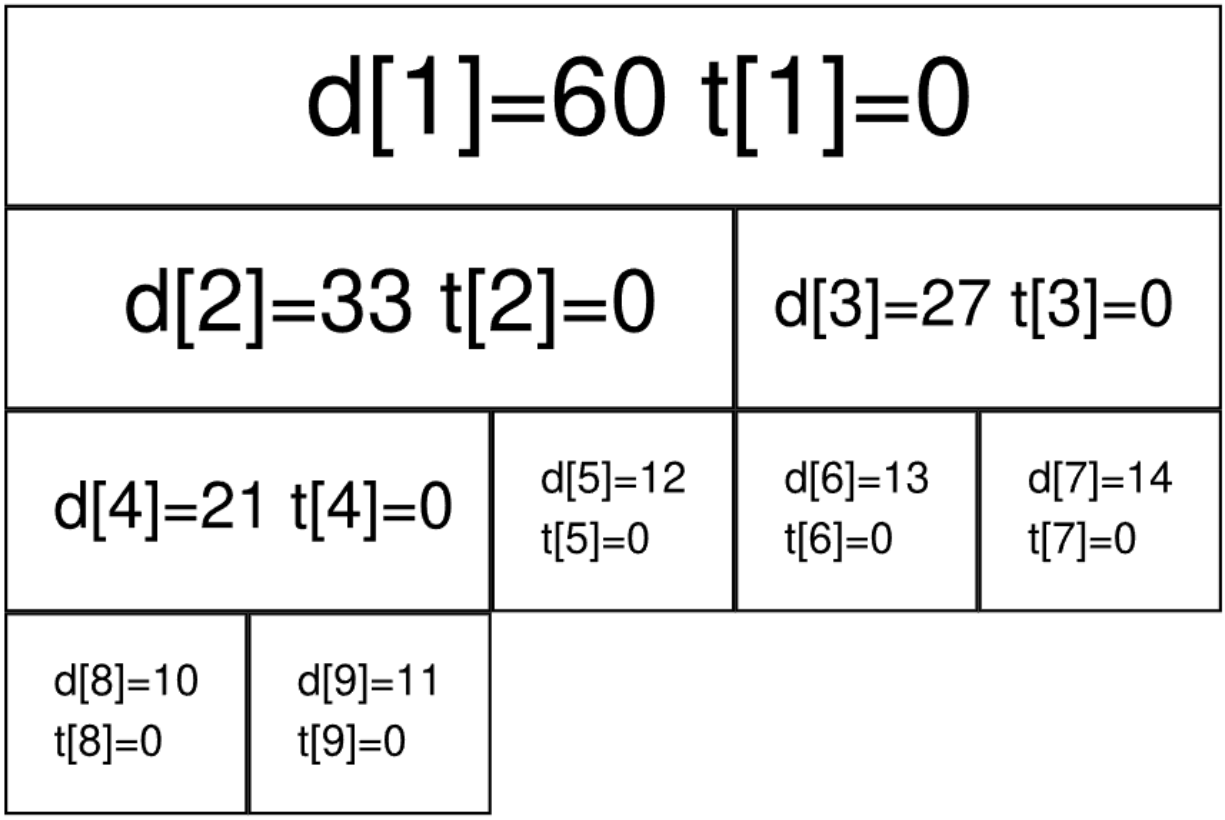
过程

如果要求修改区间［l,r］，把所有包含在区间［l,r］中的节点都遍历一次、修改一次，时间复杂度无法承受。我们这里要引入一个叫做 **「懒惰标记」** 的东西。

懒惰标记，简单来说，就是通过延迟对节点信息的更改，从而减少可能不必要的操作次数。每次执行修改时，我们通过打标记的方法表明该节点对应的区间在某一次操作中被更改，但不更新该节点的子节点的信息。实质性的修改则在下一次访问带有标记的节点时才进行。

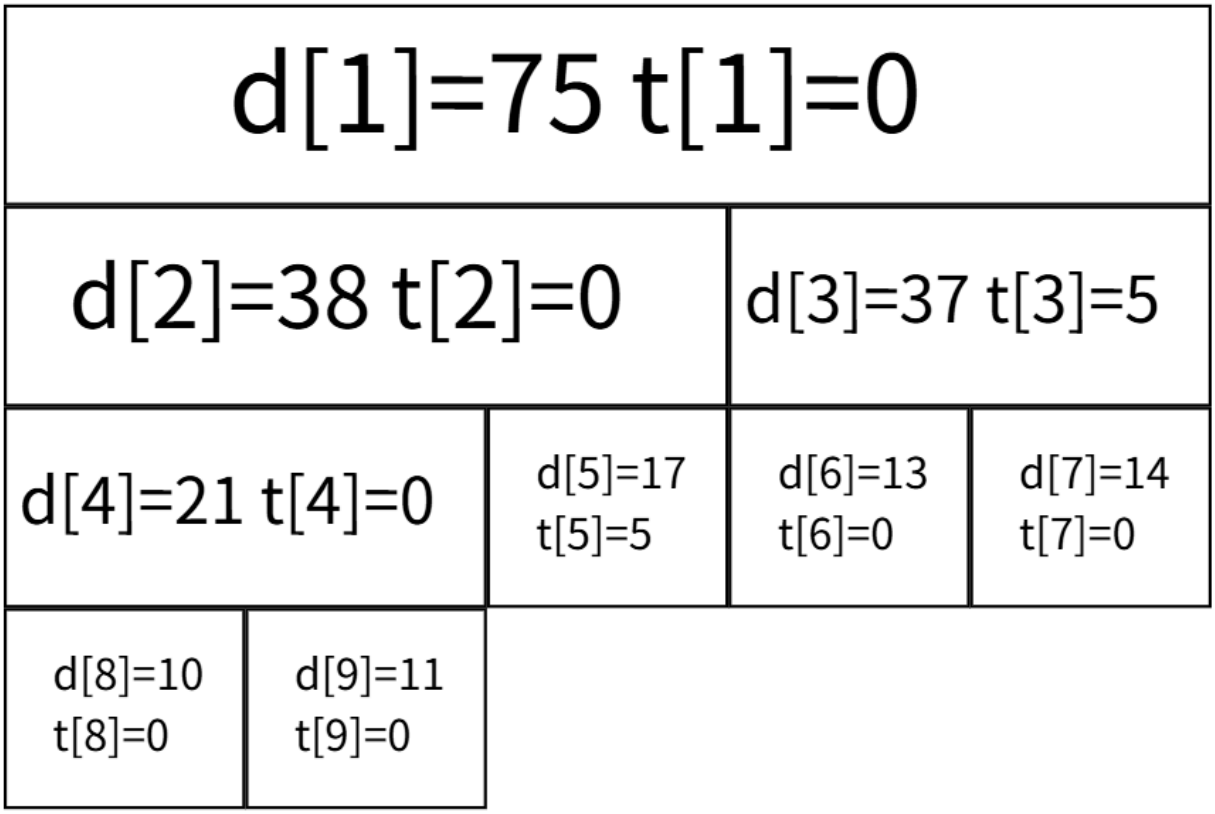
仍然以最开始的图为例，我们将执行若干次给区间内的数加上一个值的操作。我们现在给每个节点增加一个 t[i]，表示该节点带的标记值。

最开始时的情况是这样的（为了节省空间，这里不再展示每个节点管辖的区间）：



现在我们准备给［3,5］上的每个数都加上 5 。根据前面区间查询的经验，我们很快找到了两个极大区间［3,3］和［4,5］（分别对应线段树上的 3 号点和 5 号点）。

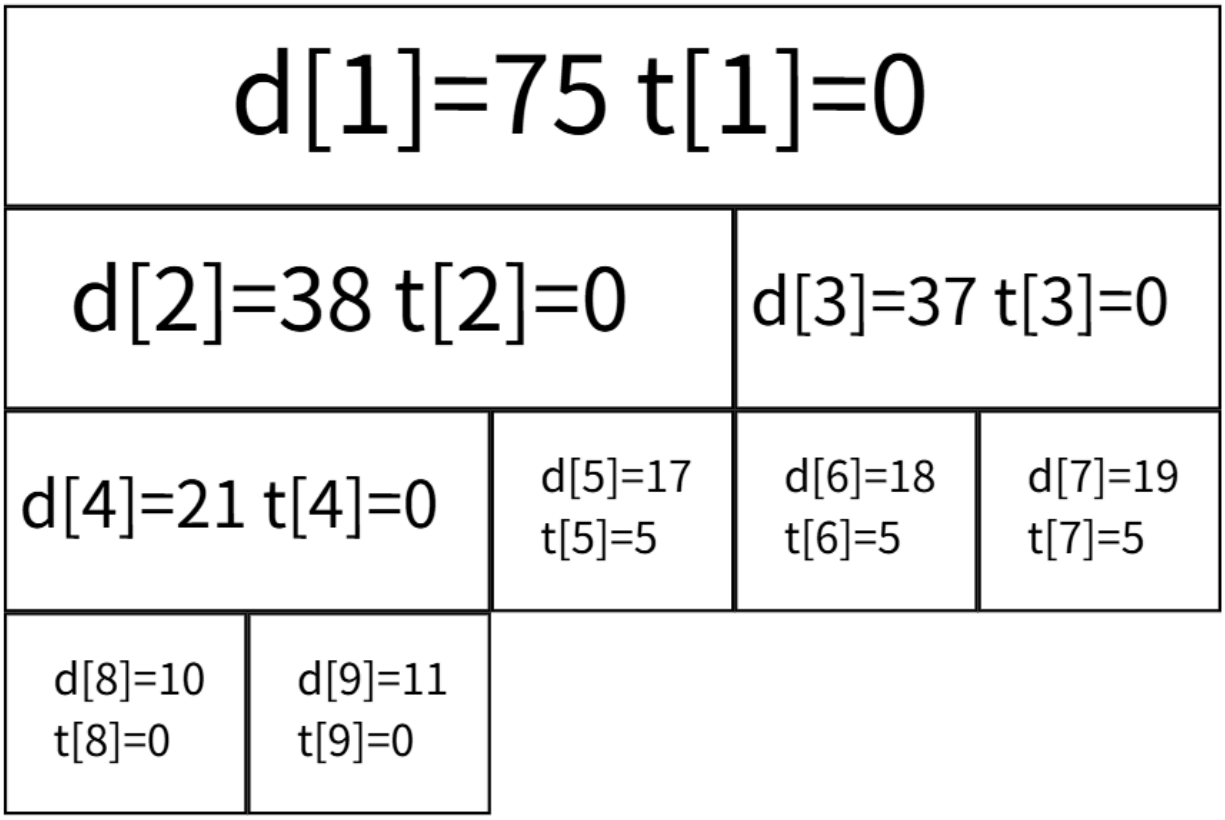
我们直接在这两个节点上进行修改，并给它们打上标记：



我们发现， 3 号节点的信息虽然被修改了（因为该区间管辖两个数，所以 d[3] 加上的数是 5 * 2=10 ），但它的两个子节点却还没更新，仍然保留着修改之前的信息。不过不用担心，虽然修改目前还没进行，但当我们要查询这两个子节点的信息时，我们会利用标记修改这两个子节点的信息，使查询的结果依旧准确。

接下来我们查询一下 [4,4] 区间上各数字的和。

我们通过递归找到 [4,5] 区间，发现该区间并非我们的目标区间，且该区间上还存在标记。这时候就到标记下放的时间了。我们将该区间的两个子区间的信息更新，并清除该区间上的标记。



现在 6 、 7 两个节点的值变成了最新的值，查询的结果也是准确的。

实现

接下来给出在存在标记的情况下，区间修改和查询操作的参考实现。

区间修改（区间加上某个值）：

```
// [l, r] 为修改区间, c 为被修改的元素的变化量, [s, t] 为当前节点包含的区间, p
// 为当前节点的编号
void update(int l, int r, int c, int s, int t, int p) {
// 当前区间为修改区间的子集时直接修改当前节点的值,然后打标记,结束修改
if (l <= s && t <= r) {
d[p] += (t - s + 1) * c, b[p] += c;
return;
}
int m = s + ((t - s) >> 1);
if (b[p] && s != t) {
// 如果当前节点的懒标记非空，则更新当前节点两个子节点的值和懒标记值
d[p * 2] += b[p] * (m - s + 1), d[p * 2 + 1] += b[p] * (t - m);
b[p * 2] += b[p], b[p * 2 + 1] += b[p];  // 将标记下传给子节点
b[p] = 0;                               // 清空当前节点的标记
}
if (l <= m) update(l, r, c, s, m, p * 2);
if (r > m) update(l, r, c, m + 1, t, p * 2 + 1);
d[p] = d[p * 2] + d[p * 2 + 1];
}
```

区间查询（区间求和）：

```
int getsum(int l, int r, int s, int t, int p) {
// [l, r] 为查询区间, [s, t] 为当前节点包含的区间, p 为当前节点的编号
if (l <= s && t <= r) return d[p];
// 当前区间为询问区间的子集时直接返回当前区间的和
int m = s + ((t - s) >> 1);
if (b[p]) {
// 如果当前节点的懒标记非空，则更新当前节点两个子节点的值和懒标记值
d[p * 2] += b[p] * (m - s + 1), d[p * 2 + 1] += b[p] * (t - m);
b[p * 2] += b[p], b[p * 2 + 1] += b[p];  // 将标记下传给子节点
b[p] = 0;                               // 清空当前节点的标记
}
int sum = 0;
if (l <= m) sum = getsum(l, r, s, m, p * 2);
if (r > m) sum += getsum(l, r, m + 1, t, p * 2 + 1);
return sum;
}
```

如果你是要实现区间修改为某一个值而不是加上某一个值的话，代码如下：

```
void update(int l, int r, int c, int s, int t, int p) {
if (l <= s && t <= r) {
d[p] = (t - s + 1) * c, b[p] = c, v[p] = 1;
return;
}
int m = s + ((t - s) >> 1);
// 额外数组储存是否修改值
if (v[p]) {
d[p * 2] = b[p] * (m - s + 1), d[p * 2 + 1] = b[p] * (t - m);
b[p * 2] = b[p * 2 + 1] = b[p];
v[p * 2] = v[p * 2 + 1] = 1;
v[p] = 0;
}
if (l <= m) update(l, r, c, s, m, p * 2);
if (r > m) update(l, r, c, m + 1, t, p * 2 + 1);
d[p] = d[p * 2] + d[p * 2 + 1];
}

int getsum(int l, int r, int s, int t, int p) {
if (l <= s && t <= r) return d[p];
int m = s + ((t - s) >> 1);
if (v[p]) {
d[p * 2] = b[p] * (m - s + 1), d[p * 2 + 1] = b[p] * (t - m);
b[p * 2] = b[p * 2 + 1] = b[p];
v[p * 2] = v[p * 2 + 1] = 1;
v[p] = 0;
}
int sum = 0;
if (l <= m) sum = getsum(l, r, s, m, p * 2);
if (r > m) sum += getsum(l, r, m + 1, t, p * 2 + 1);
return sum;
}
```

例题：

Status

Problem

Tags

P3372

.

【模板】线段树 1

数据结构

P3373

.

【模板】线段树 2

数据结构

P1937

.

Barn Allocation G

数据结构

P2061

.

City Horizon S

数据结构

P2184

.

贪婪大陆

数据结构
P2023
.
维护序列
数据结构
P1533
.
可怜的狗狗
数据结构
P2572
.
序列操作
数据结构
树状数组
二叉搜索树与平衡树

数据结构 08 · 二叉搜索树与平衡树

定义

二叉搜索树是一种二叉树的树形数据结构，其定义如下：

1. 空树是二叉搜索树。
2. 若二叉搜索树的左子树不为空，则其左子树上所有点的附加权值均小于其根节点的值。
3. 若二叉搜索树的右子树不为空，则其右子树上所有点的附加权值均大于其根节点的值。
4. 二叉搜索树的左右子树均为二叉搜索树。

二叉搜索树上的基本操作所花费的时间与这棵树的高度成正比。对于一个有 n 个结点的二叉搜索树中，这些操作的最优时间复杂度为 O(log n) ，最坏为 O(n) 。随机构造这样一棵二叉搜索树的期望高度为 O(log n) 。

过程

二叉搜索树节点的定义

```
struct TreeNode {
int key;
TreeNode* left;
TreeNode* right;
// 维护其他信息，如高度、节点数量等
int size;  // 当前节点为根的子树大小
int count;  // 当前节点的重复数量

TreeNode(int value)
: key(value), size(1), count(1), left(nullptr), right(nullptr) {}
};
```

遍历二叉搜索树

由二叉搜索树的递归定义可得，二叉搜索树的中序遍历权值的序列为非降的序列。时间复杂度为 O(n) 。

遍历一棵二叉搜索树的代码如下：

```
void inorderTraversal(TreeNode* root) {
if (root == nullptr) {
return;
}
inorderTraversal(root->left);
std::cout << root->key << " ";
inorderTraversal(root->right);
}
```

查找最小/最大值

由二叉搜索树的性质可得，二叉搜索树上的最小值为二叉搜索树左链的顶点，最大值为二叉搜索树右链的顶点。时间复杂度为 $O(h)$ 。

```
int findMin(TreeNode* root) {
if (root == nullptr) {
return -1;
}
while (root->left != nullptr) {
root = root->left;
}
return root->key;
}

int findMax(TreeNode* root) {
if (root == nullptr) {
return -1;
}
while (root->right != nullptr) {
root = root->right;
}
return root->key;
}
```

搜索元素

在以 `root` 为根节点的二叉搜索树中搜索一个值为 `value` 的节点。

分类讨论如下：

- 若 `root` 为空，返回 `false` 。
- 若 `root` 的权值等于 `value` ，返回 `true` 。
- 若 `root` 的权值大于 `value` ，在 `root` 的左子树中继续搜索。
- 若 `root` 的权值小于 `value` ，在 `root` 的右子树中继续搜索。

时间复杂度为 $O(h)$ 。

```
bool search(TreeNode* root, int target) {
if (root == nullptr) {
return false;
}
if (root->key == target) {
return true;
} else if (target < root->key) {
return search(root->left, target);
} else {
return search(root->right, target);
}
}
```

插入，删除，修改都需要先在二叉搜索树中进行搜索。

插入一个元素

在以 `root` 为根节点的二叉搜索树中插入一个值为 `value` 的节点。

分类讨论如下：

- 若 `root` 为空，直接返回一个值为 `value` 的新节点。
- 若 `root` 的权值等于 `value`，该节点的附加域该值出现的次数自增 1 。
- 若 `root` 的权值大于 `value`，在 `root` 的左子树中插入权值为 `value` 的节点。
- 若 `root` 的权值小于 `value`，在 `root` 的右子树中插入权值为 `value` 的节点。

时间复杂度为 O(h) 。

```
TreeNode* insert(TreeNode* root, int value) {
if (root == nullptr) {
return new TreeNode(value);
}
if (value < root->key) {
root->left = insert(root->left, value);
} else if (value > root->key) {
root->right = insert(root->right, value);
} else {
root->count++;  // 节点值相等，增加重复数量
}
root->size = root->count + (root->left ? root->left->size : 0) +
(root->right ? root->right->size : 0);  // 更新节点的子树大小
return root;
}
```

删除一个元素

在以 `root` 为根节点的二叉搜索树中删除一个值为 `value` 的节点。

先在二叉搜索树中搜索权值为 `value` 的节点，分类讨论如下：

- 若该节点的附加 `count` 大于 1 ，只需要减少 `count` 。
- 若该节点的附加 `count` 为 1 ：
- 若 `root` 为叶子节点，直接删除该节点即可。
- 若 `root` 为链节点，即只有一个儿子的节点，返回这个儿子。
- 若 `count` 有两个非空子节点，一般是用它左子树的最大值（左子树最右的节点）或右子树的最小值（右子树最左的节点）代替它，然后将它删除。

时间复杂度 O(h) 。

方法使用 `root = remove(root, 1)` 表示删除根节点为 `root` 树中值为 1 的节点，并返回新的根节点。

```
// 此处返回值为删除 value 后的新 root
TreeNode* remove(TreeNode* root, int value) {
if (root == nullptr) {
return root;
}
if (value < root->key) {
root->left = remove(root->left, value);
} else if (value > root->key) {
root->right = remove(root->right, value);
} else {
if (root->count > 1) {
root->count--;  // 节点重复数量大于1，减少重复数量
} else {
if (root->left == nullptr) {
TreeNode* temp = root->right;
delete root;
return temp;
} else if (root->right == nullptr) {
TreeNode* temp = root->left;
delete root;
return temp;
} else {
TreeNode* successor = findMinNode(root->right);
root->key = successor->key;
root->count = successor->count;  // 更新重复数量
// 当 successor->count > 1时，也应该删除该节点，否则
// 后续的删除只会减少重复数量
successor->count = 1;
root->right = remove(root->right, successor->key);
}
}
}
// 继续维护size，不写成 --root->size;
// 是因为value可能不在树中，从而可能未发生删除
root->size = root->count + (root->left ? root->left->size : 0) +
(root->right ? root->right->size : 0);
return root;
}

// 此处以右子树的最小值为例
TreeNode* findMinNode(TreeNode* root) {
while (root->left != nullptr) {
root = root->left;
}
return root;
}
```

求元素的排名

排名定义为将数组元素升序排序后第一个相同元素之前的数的个数加一。

查找一个元素的排名，首先从根节点跳到这个元素，若向右跳，答案加上左儿子节点个数加当前节点重复的数个数，最后答案加上终点的左儿子子树大小加一。

时间复杂度 O(h) 。

```
int queryRank(TreeNode* root, int v) {
if (root == nullptr) return 0;
if (root->key == v) return (root->left ? root->left->size : 0) + 1;
if (root->key > v) return queryRank(root->left, v);
return queryRank(root->right, v) + (root->left ? root->left->size : 0) +
root->count;
}
```

查找排名为 k 的元素

在一棵子树中，根节点的排名取决于其左子树的大小。

- 若其左子树的大小大于等于 k ，则该元素在左子树中；
- 若其左子树的大小在区间 [k-count,k-1] （ `count` 为当前结点的值的出现次数）中，则该元素为子树的根节点；
- 若其左子树的大小小于 k-count ，则该元素在右子树中。

时间复杂度 O(h) 。

```
int querykth(TreeNode* root, int k) {
if (root == nullptr) return -1;  // 或者根据需求返回其他合适的值
if (root->left) {
if (root->left->size >= k) return querykth(root->left, k);
if (root->left->size + root->count >= k) return root->key;
} else {
if (k <= root->count) return root->key;
}
return querykth(root->right,
k - (root->left ? root->left->size : 0) - root->count);
}
```

平衡树简介

使用搜索树的目的之一是缩短插入、删除、修改和查找（插入、删除、修改都包括查找操作）节点的时间。

关于查找效率，如果一棵树的高度为 h ，在最坏的情况，查找一个关键字需要对比 h 次，查找时间复杂度（也为平均查找长度 ASL，Average Search Length）不超过 O(h) 。一棵理想的二叉搜索树所有操作的时间可以缩短到 O(log n) （n 是节点总数）。

然而 O(log n) 的时间复杂度仅为理想情况。在最坏情况下，搜索树有可能退化为链表。想象一棵每个结点只有右孩子的二叉搜索树，那么它的性质就和链表一样，所有操作（增删改查）的时间是 O(n) 。

可以发现操作的复杂度与树的高度 h 有关。由此引出了平衡树，通过一定操作维持树的高度（平衡性）来降低操作的复杂度。

平衡性的定义

关于一棵搜索树是否「**平衡**」，不同的平衡树中对「**平衡**」有着不同的定义。比如以 T 为根节点的二叉搜索树，左子树和右子树的高度相差很大，或者左子树的节点个数远大于右子树的节点个数，这棵树显然不具有平衡性。

对于二叉搜索树来说，常见的平衡性的定义是指：以 T 为根节点的树，每一个结点的左子树和右子树高度差最多为 1。

- Splay 树中，对于任意节点的访问操作（搜索、插入还是删除），都会将被访问的节点移动到树的根节点位置。
- AVL 树每个节点 N 维护以 N 为根节点的树的高度信息。AVL 树对平衡性的定义：如果 T 是一棵 AVL 树，当且仅当左右子树也是 AVL 树，且 |height(T->left) - height(T->right)| <= 1 。
- Size Balanced Tree 每个节点 N 维护以 N 为根节点的树中节点个数 `size` 。对平衡性的定义：任意节点的 `size` 不小于其兄弟节点（Sibling）的所有子节点（Nephew）的 `size` 。
- B 树对平衡性的定义：每个节点应该保持在一个预定义的范围内的关键字数量。

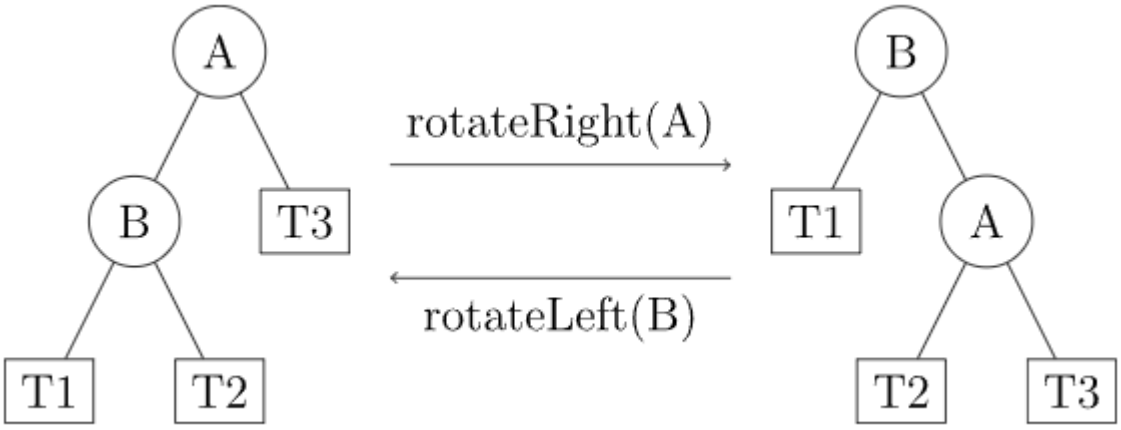
此外，对于拥有同样元素值集合的搜索树，平衡状态可能是不唯一的。也就是说，可能两棵不同的搜索树，含有的元素值集合相同，并且都是平衡的。

平衡的调整过程

对不满足平衡条件的搜索树进行调整操作，可以使不平衡的搜索树重新具有平衡性。

关于二叉平衡树，平衡的调整操作分为包括 **左旋（Left Rotate 或者 zag）** 和 **右旋（Right Rotate 或者 zig）** 两种。由于二叉平衡树在调整时需要保证中序遍历序列不变。这两种操作均不

改变中序遍历序列。
在这里先介绍右旋，右旋也称为「右单旋转」或「LL 平衡旋转」。对于结点 A 的右旋操作是指：将 A 的左孩子 B 向右上旋转，代替 A 成为根节点，将 A 结点向右下旋转成为 B 的右子树的根结点，B 的原来的右子树变为 A 的左子树。



右旋操作只改变了三组结点关联，相当于对三组边进行循环置换一下，因此需要暂存一个结点再进行轮换更新。
对于右旋操作一般的更新顺序是：暂存 B 结点（新的根节点），让 A 的左孩子指向 B 的右子树 T2，再让 B 的右孩子指针指向 A ，最后让 A 的父结点指向暂存的 B 。
完全同理，有对应的左旋操作，也称为「左单旋转」或「RR 平衡旋转」。左旋操作与右旋操作互为镜像。
下面给出左旋和右旋的代码。

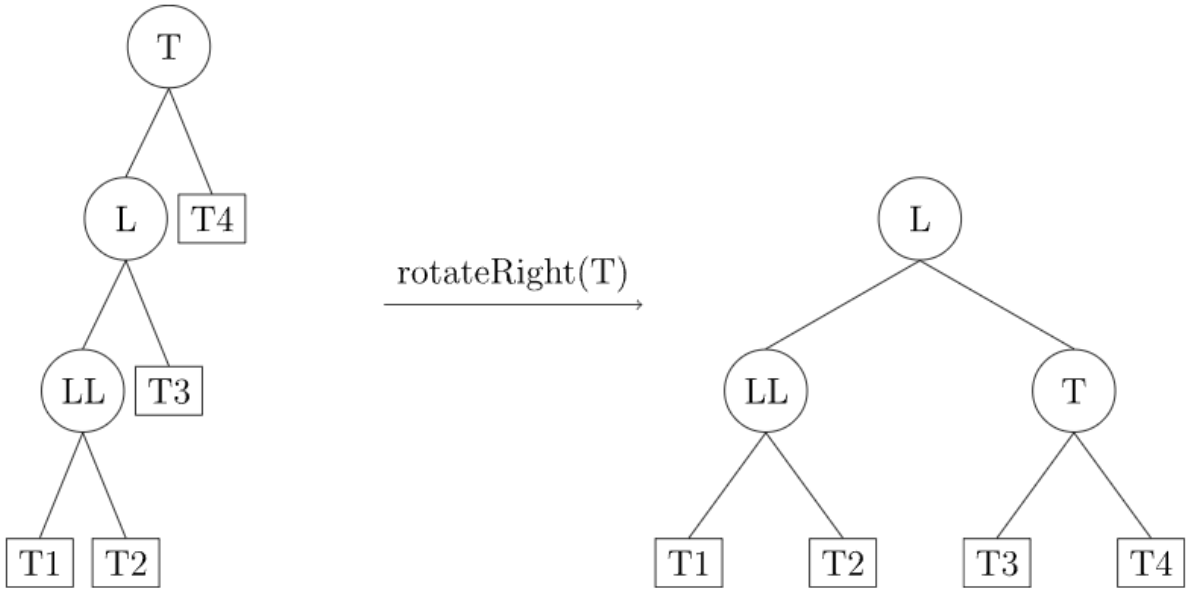
```
TreeNode* rotateLeft(TreeNode* root) {
TreeNode* newRoot = root->right;
root->right = newRoot->left;
newRoot->left = root;
// 更新相关节点的信息
updateHeight(root);
updateHeight(newRoot);
return newRoot;  // 返回新的根节点
}

TreeNode* rotateRight(TreeNode* root) {
TreeNode* newRoot = root->left;
root->left = newRoot->right;
newRoot->right = root;
updateHeight(root);
updateHeight(newRoot);
return newRoot;
}
```

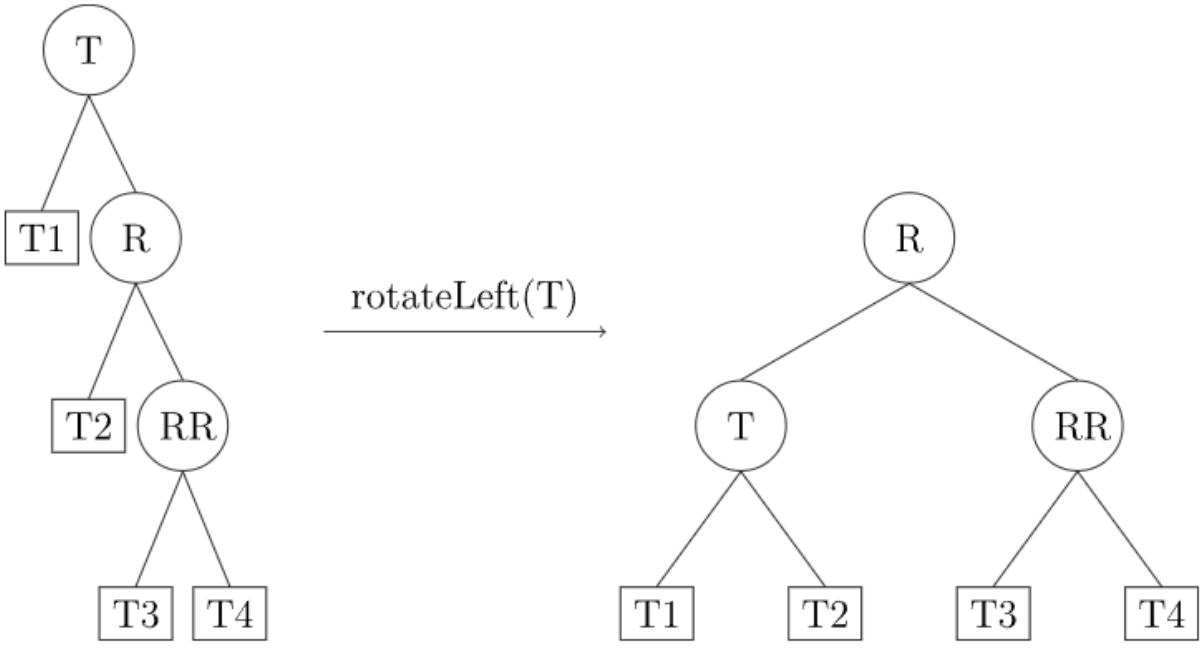
对于这段示例代码，在调用时需要保存 `root` 的父节点 `pre`。方法返回指向新的根节点的指针，只需要将 `pre` 指向新的根节点即可。

四种平衡性破坏的情况

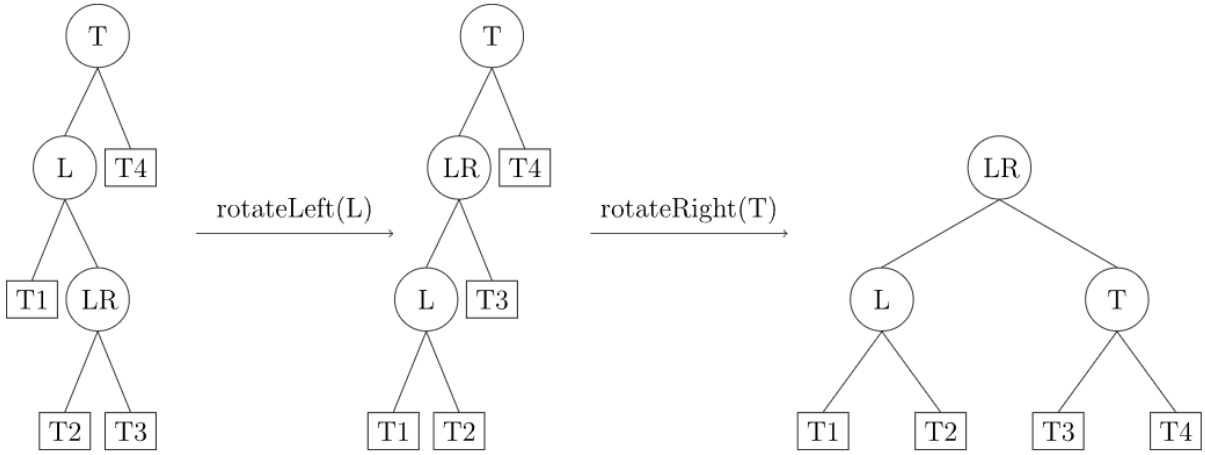
虽然不同的二叉平衡树的定义有所区别，不同二叉平衡树区别只在于节点维护的信息不同，以及旋转调整后节点更新的信息不同。二叉平衡树平衡性被破坏的情况只有以下四种。进行平衡性调整的操作只包括左旋和右旋。以下先介绍四种情况，再对不同的二叉平衡树进行对比。
LL 型：T 的左孩子的左子树过长导致平衡性破坏。
调整方式：右旋节点 T。



RR 型：与 LL 型类似，T 的右孩子的右子树过长导致平衡性破坏。
调整方式：左旋节点 T。

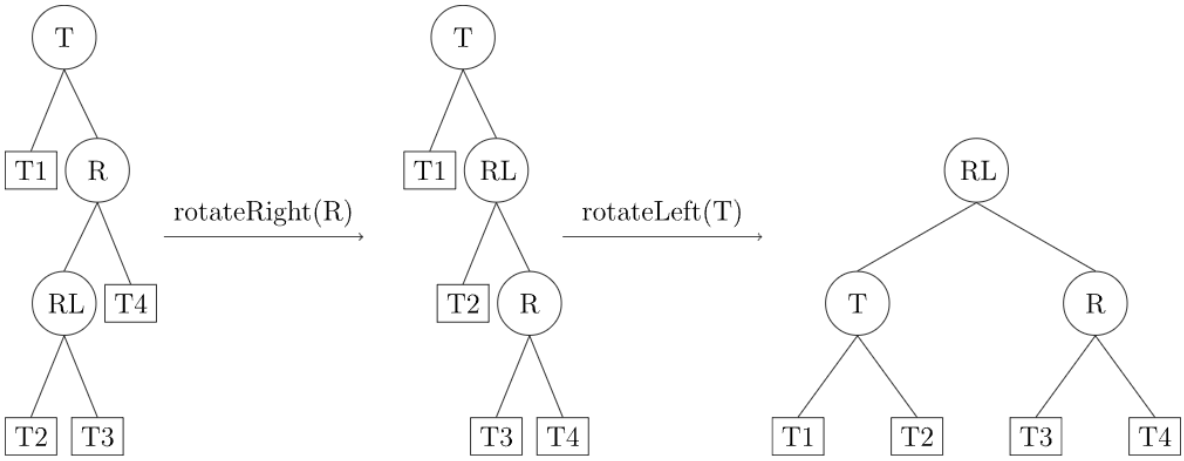


LR 型：T 的左孩子的右子树过长导致平衡性破坏。
调整方式：先左旋节点 L，成为 LL 型，再右旋节点 T。



RL 型：与 LR 型类似，T 的右孩子的左子树过长导致平衡性破坏。

调整方式：先右旋节点 R，成为 RR 型，再左旋节点 T。



线段树

Splay 树

数据结构 09 • Splay 树

本页面将简要介绍如何用 Splay 维护二叉查找树。

定义

Splay 树，或 **伸展树**，是一种平衡二叉查找树，它通过 **伸展（splay）操作** 不断将某个节点旋转到根节点，使得整棵树仍然满足二叉查找树的性质，能够在均摊 O(log N) 时间内完成插入、查找和删除操作，并且保持平衡而不至于退化为链。

Splay 树由 Daniel Sleator 和 Robert Tarjan 于 1985 年发明。

基本结构与操作

本节讨论 Splay 树的基本结构和它的核心操作，其中最为重要的是伸展操作。

Splay 树是一棵二叉查找树，查找某个值时满足性质：左子树任意节点的值 < 根节点的值 < 右子树任意节点的值。

维护信息

本文使用数组模拟指针来实现 Splay 树，需要维护如下信息：

rt	id	fa[i]	ch[i][0/1]	val[i]	cnt[i]	sz[i]
根节点编号	已使用节点个数	父亲	左右儿子编号	节点权值	权值出现次数	子树大小

初始化时，所有信息都置零即可。

辅助操作

首先是一些简单的辅助操作：

- `dir(x)`：判断节点 x 是父亲节点的左儿子还是右儿子；
- `push_up(x)`：在改变节点位置后，根据子节点信息更新节点 x 的信息。

```
bool dir(int x) { return x == ch[fa[x]][1]; }

void push_up(int x) { sz[x] = cnt[x] + sz[ch[x][0]] + sz[ch[x][1]]; }
```

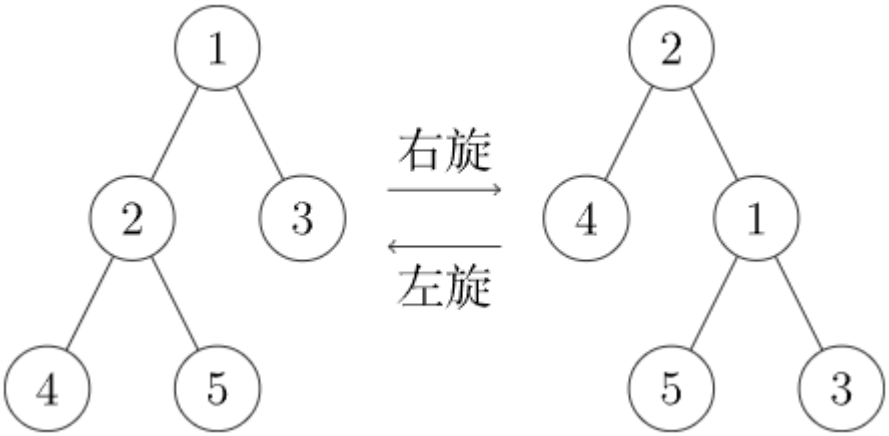
旋转操作

为了使 Splay 保持平衡，需要进行旋转操作。旋转的作用是将某个节点上移一个位置。

旋转需要保证：

- 整棵 Splay 的中序遍历不变（不能破坏二叉查找树的性质）；
- 受影响的节点维护的信息依然正确有效；
- `rt` 必须指向旋转后的根节点。

在 Splay 中旋转分为两种：左旋和右旋。



观察图示可知，如果要通过旋转将节点 x （左旋时的 1 和右旋时的 2 ）上移，则旋转的方向由该节点是其父节点的左节点还是右节点唯一确定。因此，实现旋转操作时，只需要将要上移的节点 x 传入即可。

具体分析旋转步骤：（假设需要上移的节点为 x ，以右旋为例）

1. 首先，记录节点 x 的父节点 y ，以及 y 的父节点 z （可能为空），并记录 x 是 y 的左子节点还是右子节点；
2. 按照旋转后的树中自下向上的顺序，依次更新 y 的左子节点为 x 的右子节点， x 的右子节点为 y ，以及若 z 非空， z 的子节点为 x ；
3. 按照同样的顺序，依次更新当前 y 的左子节点（若存在）的父节点为 y ， y 的父节点为 x ，以及 x 的父节点为 z ；
4. 自下而上维护节点信息。

```
void rotate(int x) {
int y = fa[x], z = fa[y];
bool r = dir(x);
ch[y][r] = ch[x][!r];
ch[x][!r] = y;
if (z) ch[z][dir(y)] = x;
if (ch[y][r]) fa[ch[y][r]] = y;
fa[y] = x;
fa[x] = z;
push_up(y);
push_up(x);
}
```

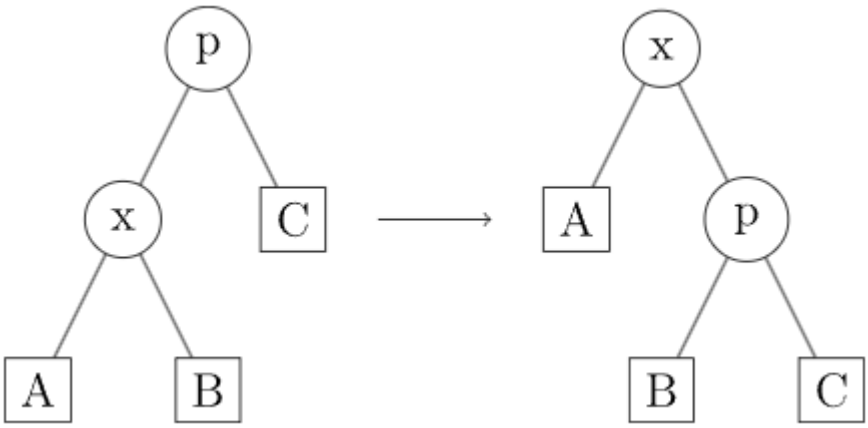
在所有函数的实现时，都应注意不要修改节点 0 的信息。

伸展操作

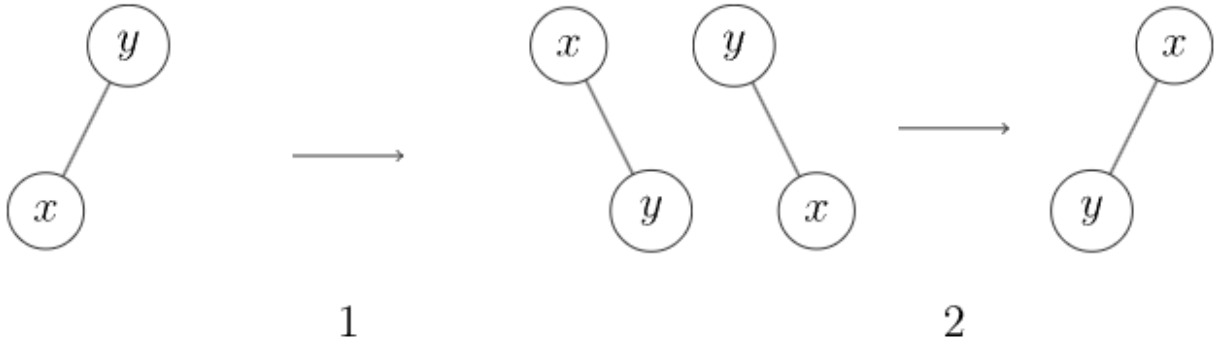
Splay 树要求每访问一个节点 x 后都要强制将其旋转到根节点。该操作也称为伸展操作。

设刚访问的节点为 x 。要做伸展操作，就是要对 x 做一系列的 **伸展步骤**。每次对 x 做一次伸展步骤， x 到根节点的距离都会更近。定义 p 为 x 的父节点。伸展步骤有三种：

1. `zig`: 在 p 是根节点时操作。Splay 树会根据 x 和 p 间的边旋转。`zig` 存在是用于处理奇偶校验问题，仅当 x 在伸展操作开始时具有奇数深度时作为伸展操作的最后一步执行。

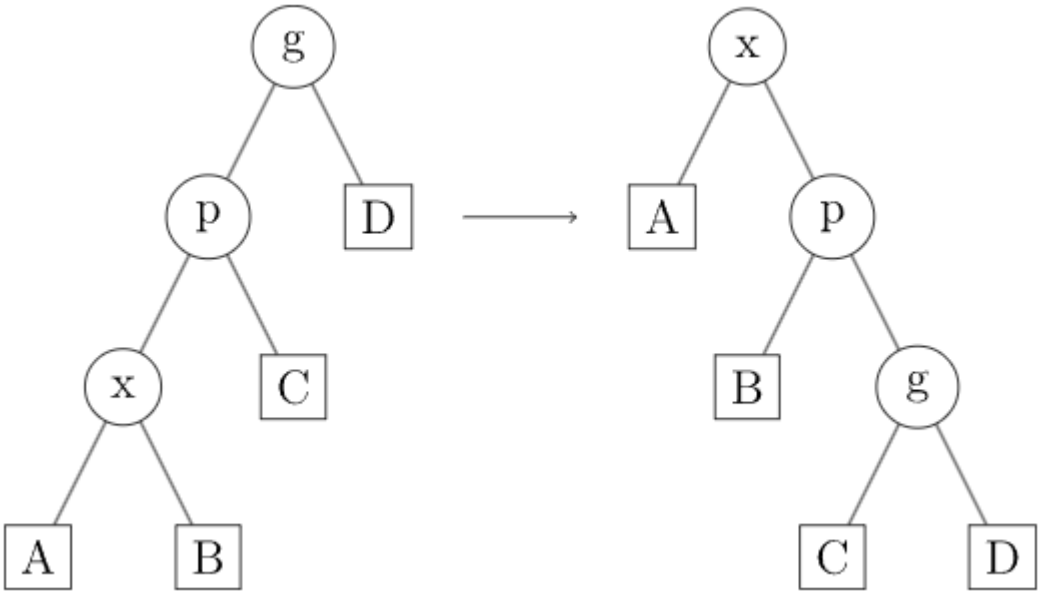


即直接将 x 右旋或左旋（图 1, 2）。

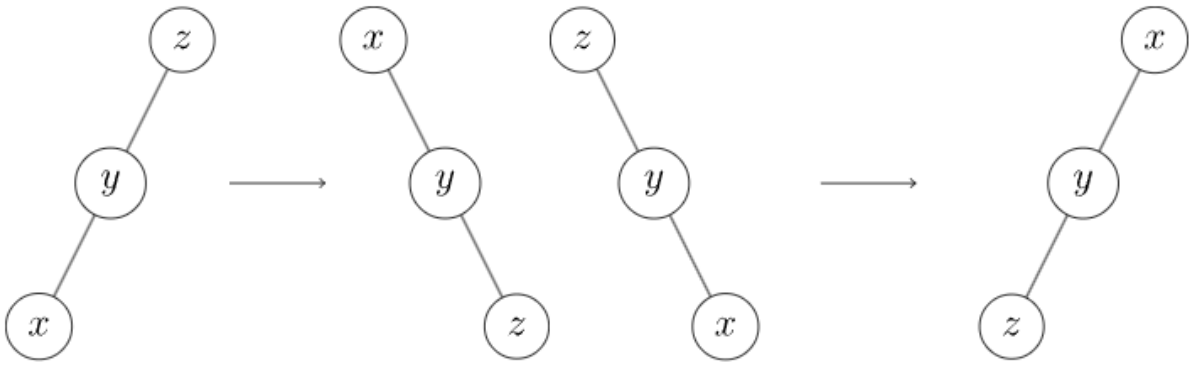


1 2

1. zig-zig: 在 p 不是根节点且 x 和 p 都是右侧子节点或都是左侧子节点时操作。下方例图显示了 x 和 p 都是左侧子节点时的情况。Splay 树首先按照连接 p 与其父节点 g 边旋转，然后按照连接 x 和 p 的边旋转。

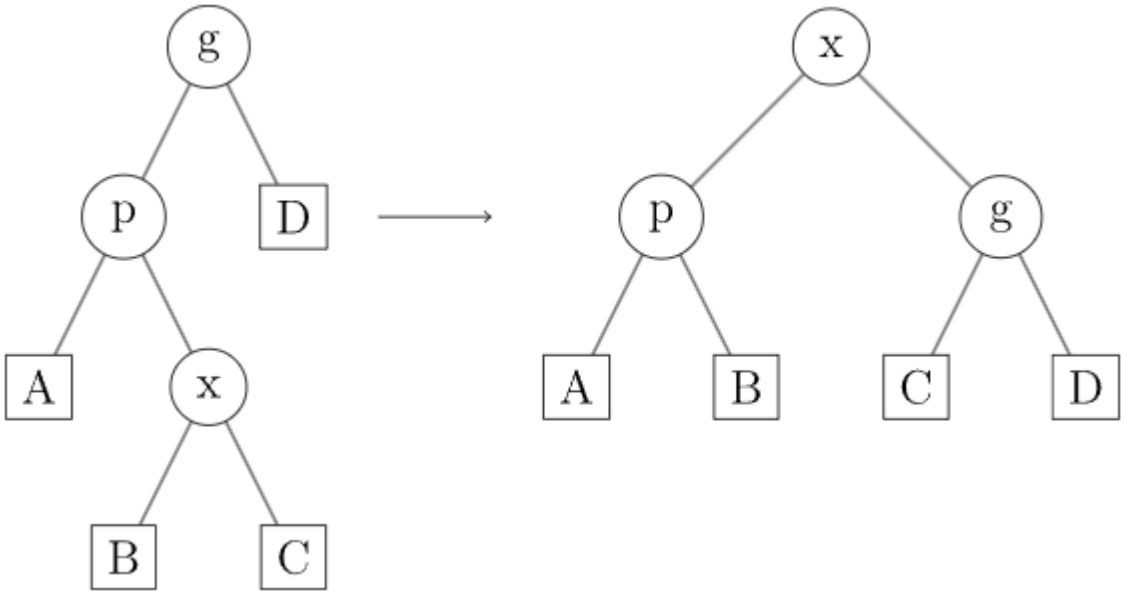


即首先将 p 右旋或左旋，然后将 x 右旋或左旋（图 3, 4）。

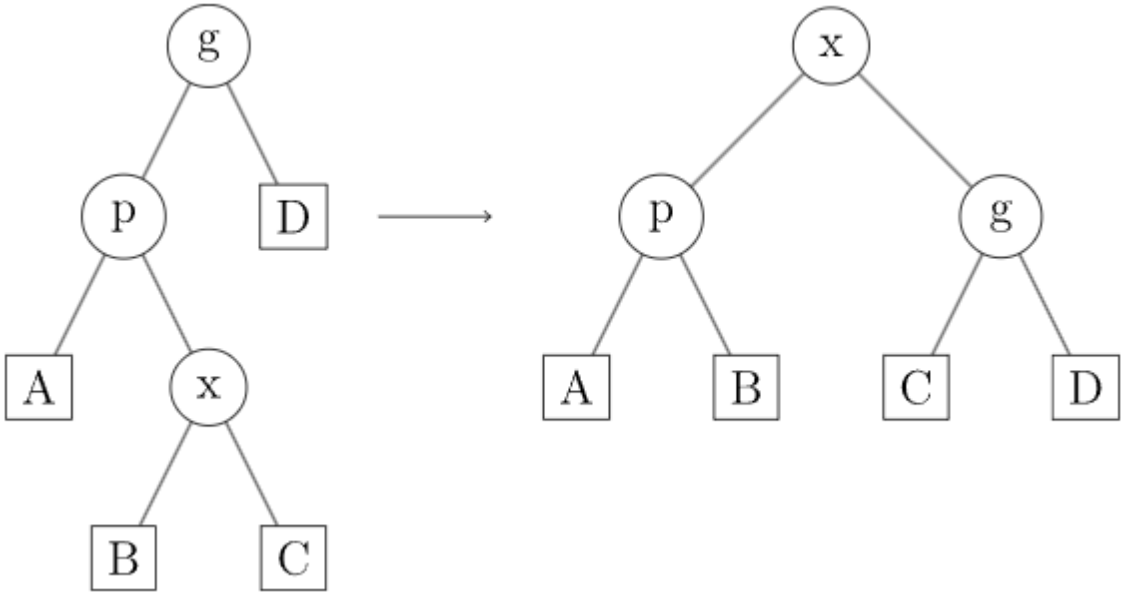


3 4

1. zig-zag: 在 p 不是根节点且 x 和 p 一个是右侧子节点一个是左侧子节点时操作。Splay 树首先按 p 和 x 之间的边旋转，然后按 x 和 g 新生成的结果边旋转。



即将 x 先左旋再右旋或先右旋再左旋（图 5, 6）。



请读者尝试自行模拟 6 种旋转情况，以理解伸展操作的基本思想。

比较三种伸展步骤可知，要区分此时应使用哪种操作，关键是要判断 x 是否是根节点的子节点，以及 x 和它父节点是否在各自的父节点同侧。

此处提供的实现，可以指定任意根节点 z ，并将它的子树内任意节点 x 上移至 z 处：

1. 首先记录根节点 z 的父节点 w ，从而可以利用 `fa[x] == w` 判断 x 已经位于根结点处；
2. 记录 x 当前的父节点 y ，如果 y 和 w 相同，说明 x 已经到达根节点；
3. 否则，利用 `fa[y] == w` 判断 y 是否是根节点。如果是，直接做 zig 操作将 x 旋转；如果不是，利用 `dir(x) == dir(y)` 判断使用 zig-zig 还是 zig-zag，前者先旋转 y 再旋转 x ，后者直接旋转两次 x 。

```
void splay(int& z, int x) {
int w = fa[z];
for (int y; (y = fa[x]) != w; rotate(x)) {
if (fa[y] != w) rotate(dir(x) == dir(y) ? y : x);
}
z = x;
}
```

伸展操作是 Splay 树的核心操作，也是它的时间复杂度能够得到保证的关键步骤。请务必保证每次向下访问节点后，都进行一次伸展操作。

另外，伸展操作会将当前节点 x 到根节点 z 的路径上的所有节点信息自下而上地更新一遍。正是因为这一点，才可以修改非根节点，再通过伸展操作将它上移至根来完成整个树的信息更新。

时间复杂度

对大小为 n 的 Splay 树做 m 次伸展操作的复杂度是 O((n+m)log n) 的，单次均摊复杂度是 O(log n) 的。

平衡树操作

本节讨论基于 Splay 树实现平衡树的常见操作的方法。其中，较为重要的是按照值或排名查找元素，它们可以将某个特定的元素找到，并上移至根节点处，以便后续处理。

作为例子，本节将讨论模板题目普通平衡树的实现。

按照值查找

作为二叉查找树，可以通过值 v 查找到相应的节点，只需要将待查找的值 v 和当前节点的值比较即可，找到后将该元素上移至根部即可。

应注意，经常存在树中不存在相应的节点的情形。对于这种情形，要记录最后一个访问的节点（即实现中的 y ），并将 y 上移至根部。此时，节点 y 存储的值必然要么是所有小于 v 的元素中最大的（即 v 的前驱），要么是所有大于 v 的元素中最小的（即 v 的后继）。这是因为查找过程保证，左子树总是存储小于 v 的值，而右子树总是存储大于 v 的值。

```
void find(int& z, int v) {
int x = z, y = fa[x];
for (; x && val[x] != v; x = ch[y = x][v > val[x]]);
splay(z, x ? x : y);
}
```

该实现允许指定任何节点 z 作为根节点，并在它的子树内按值查找。

按照排名访问

因为记录了子树大小信息，所以 Splay 树还可以通过排名访问元素，即查找树中第 k 小的元素。

设 k 为剩余排名，具体步骤如下：

- 如果左子树非空且剩余排名 k 不大于左子树的大小，那么向左子树查找；
- 否则，如果 k 不大于左子树加上根的大小，那么根节点就是要寻找的；
- 否则，将 k 减去左子树的和根的大小，继续向右子树查找；
- 将最终找到的元素上移至根部。

```
void loc(int& z, int k) {
int x = z;
for (;;) {
if (sz[ch[x][0]] >= k) {
x = ch[x][0];
} else if (sz[ch[x][0]] + cnt[x] >= k) {
break;
} else {
k -= sz[ch[x][0]] + cnt[x];
x = ch[x][1];
}
}
splay(z, x);
}
```

该实现需要保证排名 k 不超过根 z 处的树大小。
模板题目中操作 4 要求按照排名返回值，直接调用该方法，并返回值即可。

```
int find_kth(int k) {
if (k > sz[rt]) return -1;
loc(rt, k);
return val[rt];
}
```

合并操作

有些时候需要合并两棵 Splay 树。
设两棵树的根节点分别为 x 和 y ，那么为了保证结果仍是二叉查找树，需要要求 x 树中的最大值小于 y 树中的最小值。这条件通常都可以满足，因为两棵树往往是从更大的子树中分裂出的。
合并操作如下：

- 如果 x 和 y 其中之一或两者都为空树，直接返回不为空的那一棵树的根节点或空树；
- 否则，通过 `loc(y, 1)` 将 y 树中的最小值上移至根 y 处，再将它的左节点（此时必然为空）设置为 x ，并更新节点信息，返回节点 y 。

```
int merge(int x, int y) {
if (!x || !y) return x | y;
loc(y, 1);
ch[y][0] = x;
fa[x] = y;
push_up(y);
return y;
}
```

分裂操作类似。因而，Splay 树可以模拟无旋 treap 的思路做各种操作，包括区间操作。后文会介绍更具有 Splay 树风格的区间操作处理方法。

插入操作

插入操作是一个比较复杂的过程。具体步骤如下：（假设插入的值为 v ）

- 类似按值查找的过程，根据 v 向下查找到存储 v 的节点或者空节点，过程中记录父节点 y ；
- 如果存在存储 v 的节点 x ，直接更新信息，否则就新建节点 x ；
- 做伸展操作，将最后一个节点 x 上移至根部。

```
void insert(int v) {
int x = rt, y = 0;
for (; x && val[x] != v; x = ch[y = x][v > val[x]]);
if (x) {
++cnt[x];
++sz[x];
} else {
x = ++id;
val[x] = v;
cnt[x] = sz[x] = 1;
fa[x] = y;
if (y) ch[y][v > val[y]] = x;
}
splay(rt, x);
}
```

该实现允许直接向空树内插入值。若不想处理空树，可以在树中提前插入哑节点。

删除操作

删除操作也是一个比较复杂的操作。具体步骤如下：（假设删除的值为 v ）

- 首先按照值 v 查找存储它的节点，并上移至根部；
- 如果不存在存储它的节点，直接返回；（上一步已经做了伸展操作）
- 否则，更新节点信息；
- 如果得到的根节点为空节点，就合并左右子树作为新的根节点，注意合并前需要更新两个子树的根的父节点为空。

```
bool remove(int v) {
find(rt, v);
if (!rt || val[rt] != v) return false;
--cnt[rt];
--sz[rt];
if (!cnt[rt]) {
int x = ch[rt][0];
int y = ch[rt][1];
fa[x] = fa[y] = 0;
rt = merge(x, y);
}
return true;
}
```

查询排名

直接按照值 v 访问节点（并上移至根），然后返回相应的值即可。

注意，当 v 不存在时，方法 `find(rt, v)` 返回的根和 v 的大小关系无法确定，需要单独讨论。

```
int find_rank(int v) {
find(rt, v);
return sz[ch[rt][0]] + (val[rt] < v ? cnt[rt] : 0) + 1;
}
```

查询前驱

前驱定义为小于 v 的最大的数。具体步骤如下：

- 按照值 v 访问节点（并上移至根部）；
- 如果根部的值小于 v ，那么它必然是最大的那个，直接返回；
- 否则，在左子树中找到最大值，并上移至根部。

最后一步相当于直接调用 `loc(ch[rt][0], cnt[ch[rt][0]])` ，只是省去了不必要的判断。

```
int find_prev(int v) {
find(rt, v);
if (rt && val[rt] < v) return val[rt];
int x = ch[rt][0];
if (!x) return -1;
for (; ch[x][1]; x = ch[x][1]);
splay(rt, x);
return val[rt];
}
```

该实现允许前驱不存在，此时返回 -1 。

查询后继

后继定义为大于 x 的最小的数。查询方法和前驱类似，只是将左子树的最大值换成了右子树的最小值，即调用 `loc(ch[rt][1], 1)` 。

```
int find_next(int v) {
find(rt, v);
if (rt && val[rt] > v) return val[rt];
int x = ch[rt][1];
if (!x) return -1;
for (; ch[x][0]; x = ch[x][0]);
splay(rt, x);
return val[rt];
}
```

参考实现

本节的最后，给出模板的参考实现。

```
#include <iostream>

constexpr int N = 2e6;
int id, rt;
int fa[N], val[N], cnt[N], sz[N], ch[N][2];

bool dir(int x) { return x == ch[fa[x]][1]; }

void push_up(int x) { sz[x] = cnt[x] + sz[ch[x][0]] + sz[ch[x][1]]; }

void rotate(int x) {
int y = fa[x], z = fa[y];
bool r = dir(x);
ch[y][r] = ch[x][!r];
ch[x][!r] = y;
if (z) ch[z][dir(y)] = x;
if (ch[y][r]) fa[ch[y][r]] = y;
fa[y] = x;
fa[x] = z;
push_up(y);
push_up(x);
}

void splay(int& z, int x) {
int w = fa[z];
for (int y; (y = fa[x]) != w; rotate(x)) {
if (fa[y] != w) rotate(dir(x) == dir(y) ? y : x);
}
z = x;
}

void find(int& z, int v) {
int x = z, y = fa[x];
for (; x && val[x] != v; x = ch[y = x][v > val[x]]);
splay(z, x ? x : y);
}

void loc(int& z, int k) {
int x = z;
for (;;) {
if (sz[ch[x][0]] >= k) {
x = ch[x][0];
} else if (sz[ch[x][0]] + cnt[x] >= k) {
break;
} else {
k -= sz[ch[x][0]] + cnt[x];
x = ch[x][1];
}
}
splay(z, x);
}

int merge(int x, int y) {
if (!x || !y) return x | y;
loc(y, 1);
ch[y][0] = x;
fa[x] = y;
push_up(y);
return y;
}

void insert(int v) {
int x = rt, y = 0;
for (; x && val[x] != v; x = ch[y = x][v > val[x]]);
if (x) {
```

```
++cnt[x];
++sz[x];
} else {
x = ++id;
val[x] = v;
cnt[x] = sz[x] = 1;
fa[x] = y;
if (y) ch[y][v > val[y]] = x;
}
splay(rt, x);
}

bool remove(int v) {
find(rt, v);
if (!rt || val[rt] != v) return false;
--cnt[rt];
--sz[rt];
if (!cnt[rt]) {
int x = ch[rt][0];
int y = ch[rt][1];
fa[x] = fa[y] = 0;
rt = merge(x, y);
}
return true;
}

int find_rank(int v) {
find(rt, v);
return sz[ch[rt][0]] + (val[rt] < v ? cnt[rt] : 0) + 1;
}

int find_kth(int k) {
if (k > sz[rt]) return -1;
loc(rt, k);
return val[rt];
}

int find_prev(int v) {
find(rt, v);
if (rt && val[rt] < v) return val[rt];
int x = ch[rt][0];
if (!x) return -1;
for (; ch[x][1]; x = ch[x][1]);
splay(rt, x);
return val[rt];
}

int find_next(int v) {
find(rt, v);
if (rt && val[rt] > v) return val[rt];
int x = ch[rt][1];
if (!x) return -1;
for (; ch[x][0]; x = ch[x][0]);
splay(rt, x);
return val[rt];
}

int main() {
int n;
std::cin >> n;
for (; n; --n) {
int op, x;
std::cin >> op >> x;
switch (op) {
case 1:
```

```
insert(x);
break;
case 2:
remove(x);
break;
case 3:
std::cout << find_rank(x) << '\n';
break;
case 4:
std::cout << find_kth(x) << '\n';
break;
case 5:
std::cout << find_prev(x) << '\n';
break;
case 6:
std::cout << find_next(x) << '\n';
break;
}
}
return 0;
}
```

例题：

Status

Problem

Tags

P3369

.

【模板】普通平衡树

数据结构

P1486

.

郁闷的出纳员

数据结构

二叉搜索树与平衡树

Treap

数据结构 10 • Treap

前置知识：朴素二叉搜索树，堆基础。

简介

Treap（树堆）是一种 **弱平衡** 的 **二叉搜索树**。

Treap 的结点除了被维护的 **权值**（ val ）之外，还附加了一个随机的 **优先级**（ priority ）。其中，权值满足二叉搜索树性质，优先级满足堆性质（小根堆或大根堆）。

其中，二叉搜索树的性质是指：

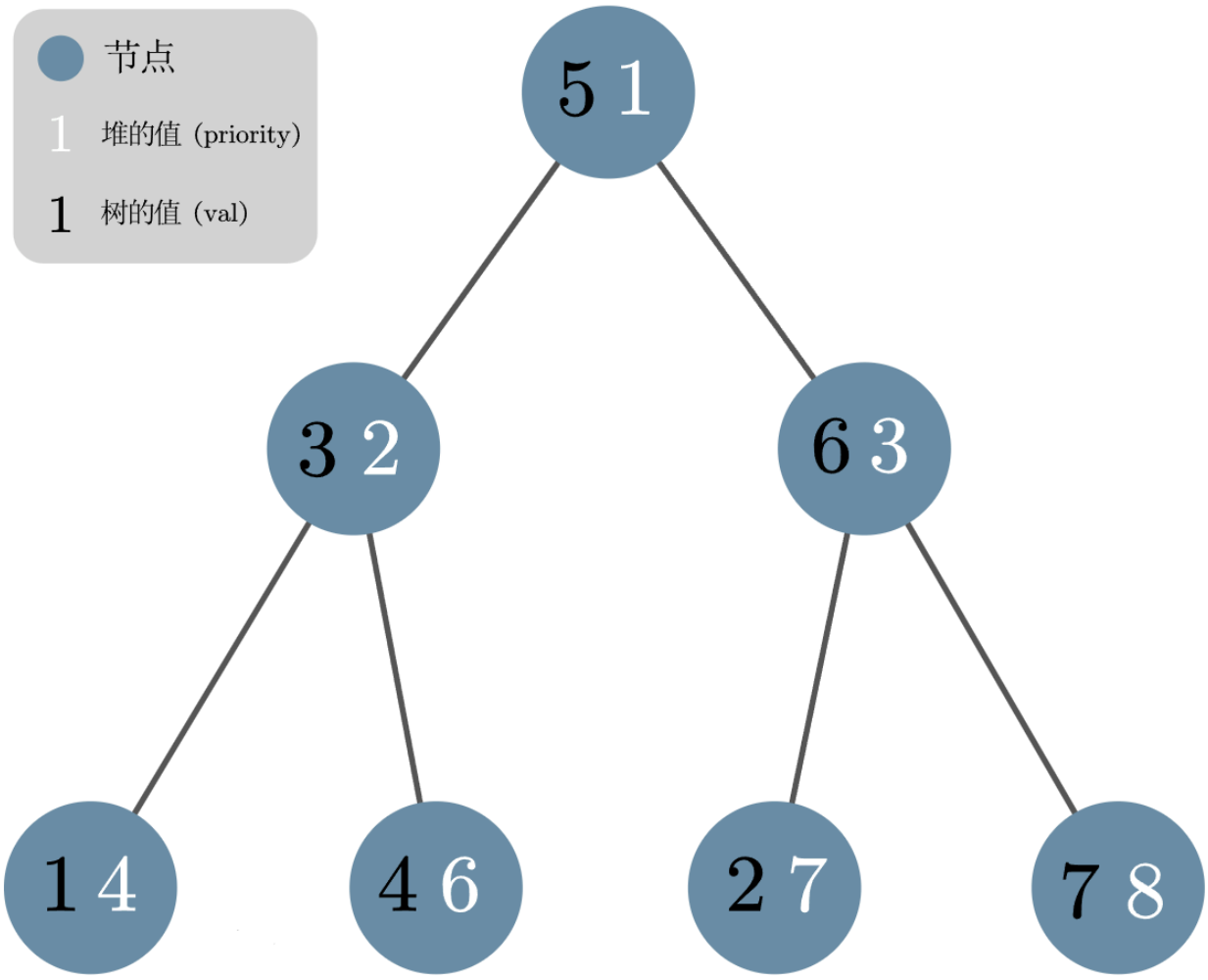
- 左子节点的权值（ val ）比父节点小。
- 右子节点的权值（ val ）比父节点大。

堆的性质是：

- 子节点优先级（ priority ）比父节点大或小（取决于是小根堆还是大根堆）。

不难看出，如果用的是同一个值，那么这两种数据结构在组合后会变成一条链，所以我们再在搜索树的基础上，引入一个给堆的值 priority 。对于 val 值，我们维护搜索树的性质，对于 priority 值，我们维护堆的性质。其中 priority 这个值是随机给出的。

下图就是一个 Treap 的例子（这里使用的是小根堆，即根节点的优先级最小）。



那我们为什么需要大费周章的去让这个数据结构符合树和堆的性质，并且随机给出堆的值呢？

要理解这个，首先需要理解朴素二叉搜索树的问题。在给朴素搜索树插入一个新节点时，我们需要从这个搜索树的根节点开始递归，如果新节点比当前节点小，那就向左递归，反之亦然。

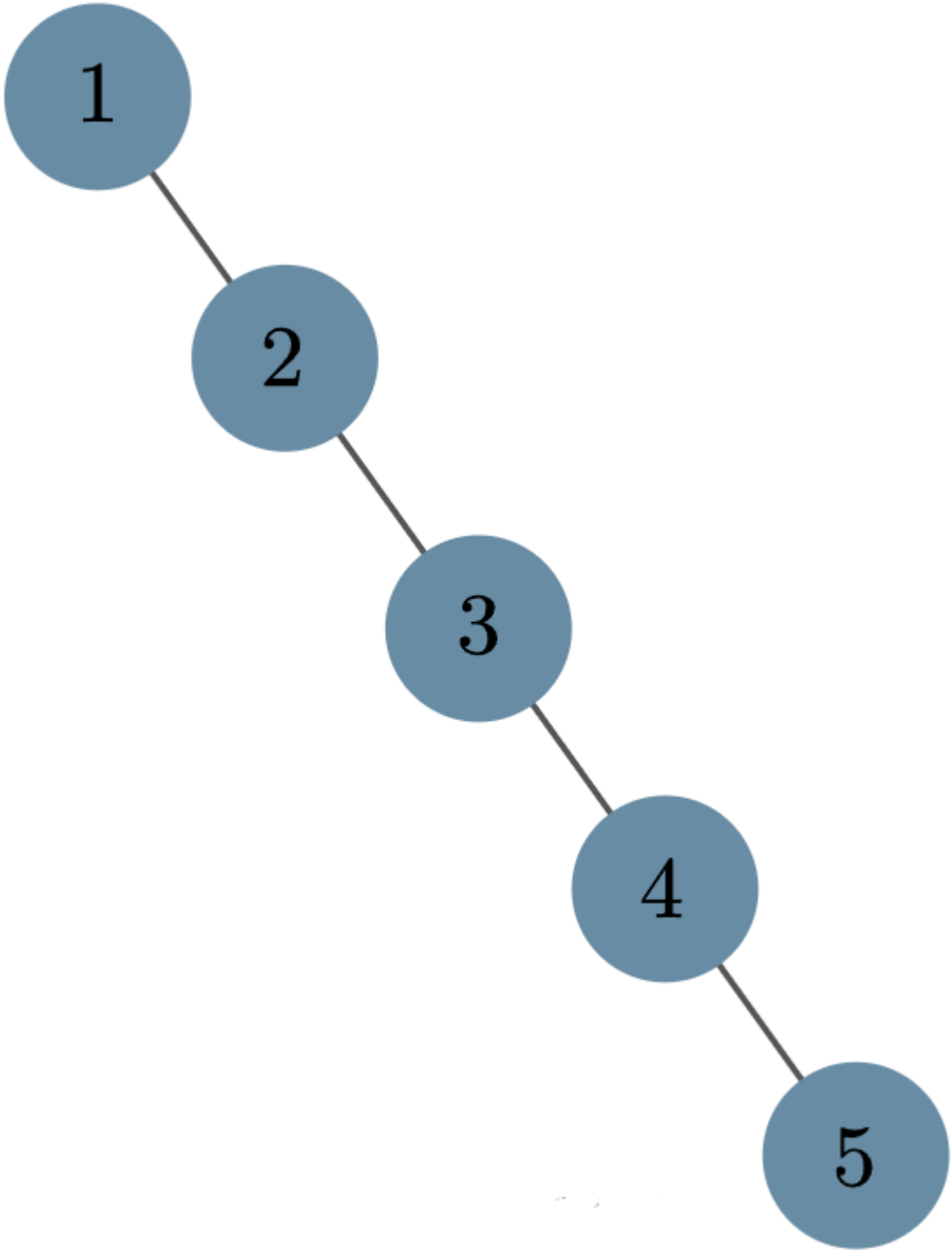
最后当发现当前节点没有子节点时，就根据新节点的值的大小，让新节点成为当前节点的左或右子节点。

如果插入结点的权值是随机的（换言之，是随机插入的），那这个朴素搜索树的高度较小（接近 log n ，其中 n 为结点数），而每一层的节点数较多，即它的形状会非常的「胖」。上图的 Treap 就是一个例子。因此此时的任意操作复杂度都将会是 O(log n) 左右。

不过，这只是在随机情况下的复杂度，如果我们按照下面这个非常有序的顺序给一个朴素的搜索树插入节点：

```
1 2 3 4 5
```

那么这棵树将会退化成链，即变得非常「瘦长」（每次插入的节点都比前面的大，所以都被安排到右子节点了）：



不难看出，查询的复杂度也从 O(log n) 变成了 O(n) .

而 treap 为了解决这个问题、达到一个较为「平衡」的状态，通过维护随机的优先级满足堆性质，「打乱」了节点的插入顺序，从而让二叉搜索树达到了理想的复杂度，避免了退化成链的问题。

Treap 复杂度的证明

由于 treap 各种操作的复杂度都和所操作的结点的深度有关，我们首先证明，所有结点的期望高度都是 O(log n) .

记号约定

为了方便表述，我们约定：

- n 是节点个数。
- Treap 结点中满足二叉搜索树性质的称为 **权值**，满足堆性质的（也就是随机的）称为 **优先级**。不妨设优先级满足小根堆性质。
- x[k] 表示权值第 k 小的结点。

- X_(i,j) 表示集合 x[i],x[i+1],...,x[j-1],x[j] ，即按权值升序排列后第 i 个到第 j 个的结点构成的集合。
- dep(x) 表示结点 x 的深度。规定根节点的深度是 0 .
- Y_(i,j) 是一个指示器随机变量，当 x[i] 是 x[j] 的祖先时值为 1 ，否则为 0 . 特别地，Y_(i,i)=0 .
- Pr(A) 表示事件 A 发生的概率。

树高的证明

由于结点 x[i] 的深度等于它祖先的个数，因此有

dep(x[i])= sum *(k=1)^nY*(k,i)

那么根据期望的线性性，有

E(dep(x[i]))=E(sum *(k=1)^nY*(k,i))= sum *(k=1)^nE(Y*(k,i))

由于 Y_(k,i) 是指示器随机变量，它的期望就等于它为 1 的概率，因此

E(dep(x[i]))= sum *(k=1)^nPr(Y*(k,i)=1)

我们先证明引理： Y_(i,j)=1 当且仅当 x[i] 的优先级是 X_(i,j) 中最小的。

引理的证明

证明：考虑分类讨论 x[i] 和 x[j] 的情况。

1. 若 x[i] 是根节点：由于优先级满足小根堆性质， x[i] 的优先级最小，并且对于任意的 x[j] ， x[i] 都是 x[j] 的祖先。
2. 若 x[j] 是根节点：同理， x[j] 优先级最小，因此 x[i] 不是 X_(i,j) 中优先级最小的；同时 x[i] 也不是 x[j] 的祖先。
3. 若 x[i] 和 x[j] 在根节点的两个子树中（一左一右），那么根节点 r 属于 X_(i,j) . 因此 x[i] 的优先级不可能是 X_(i,j) 中最小的（因为根节点的比它小）。同时，由于 x[i] 和 x[j] 分属两个子树， x[i] 也不是 x[j] 的祖先。
4. 若 x[i] 和 x[j] 在根节点的同一个子树中，此时可以将这个子树单独拿出来作为一棵新的 treap，递归进行上面的证明即可。 square

那么根据引理，深度的期望可以转化成

E(dep(x[i]))= sum *(k=1)^nPr(x[k]=min X*(i,k) and k != i)

又因为结点的优先级是随机的，我们假定集合 X_(i,j) 中任何一个结点的优先级最小的概率都相同，那么

E(dep(x[i])) = sum *(k=1)^nPr(x[k]=min X*(i,k) and k != i) = sum (k=1 ~ n)Pr(x[k]=min X_(i,k))-1 = sum *(k=1)^n(1)/(|i-k|+1)-1 = sum (k=1 ~ i-1)(1)/(i-k+1)+ sum* (k=i+1)^n(1)/(k-i+1) = sum *(j=2)^i(1)/(j)+ sum (j=2 ~ n-i+1)(1)/(j) <= 2 sum* (j=2)^n(1)/(j) < 2 sum _(j=2)^n int [j-1]^(j)(1)/(x)mathrm dx =2 int [1]^n(1)/(x)mathrm dx=2ln n=O(log n)

因此每个结点的期望高度都是 O(log n) .

而朴素的二叉搜索树的操作的复杂度均是 O(h) ，同时 treap 维护堆性质的复杂度也是 O(h) 的，因此 treap 各种操作的期望复杂度都是 O(log n) .

期望复杂度的感性理解

首先，我们需要认识到一个节点的 priority 属性是和它所在的层数有直接关联的。再回忆堆的性质：

- 子节点值（ priority ）比父节点大或小（取决于是小根堆还是大根堆）

我们发现层数低的节点，比如整个树的根节点，它的 priority 属性也会更小（在小根堆中）。并且，在朴素的搜索树中，先被插入的节点，也更有可能会有比较小的层数。我们可以把这个 priority 属性和被插入的顺序关联起来理解，这样，也就理解了为什么 treap 可以把节点插入的顺序通过 priority 打乱。

给 treap 插入新节点时，需要同时维护树和堆的性质。其中，搜索树的性质可以在插入时维护，而堆性质的维护则有两种处理方法，分别是旋转和分裂、合并。使用这两种方法的 treap 被分别称为 **旋转 treap** 和 **无旋 treap**。

旋转 treap

旋转 treap 维护平衡的方式为旋转，和 AVL 树的旋转操作类似，分为 **左旋** 和 **右旋**。即在满足二叉搜索树的条件下根据堆的优先级对 treap 进行平衡操作。

旋转 treap 在做普通平衡树题的时候，是所有平衡树中常数较小的。

下面的讲解中的代码用指针实现了旋转 treap，文末附有数组形式的完整实现。

代码中的 `rank` 代表前面讲的优先级（ priority 属性），该属性满足的是小根堆性质。

节点结构

```
struct Node {
Node *ch[2]; // 两个子节点的地址
int val, rank;
int rep_cnt; // 当前这个值（val）重复出现的次数
int siz;     // 以当前节点为根的子树大小

Node(int val) : val(val), rep_cnt(1), siz(1) {
ch[0] = ch[1] = nullptr;
rank = rand();
// 注意初始化的时候，rank 是随机给出的
}

void upd_siz() {
// 用于旋转和删除过后，重新计算 siz 的值
siz = rep_cnt;
if (ch[0] != nullptr) siz += ch[0]->siz;
if (ch[1] != nullptr) siz += ch[1]->siz;
}
};
```

旋转

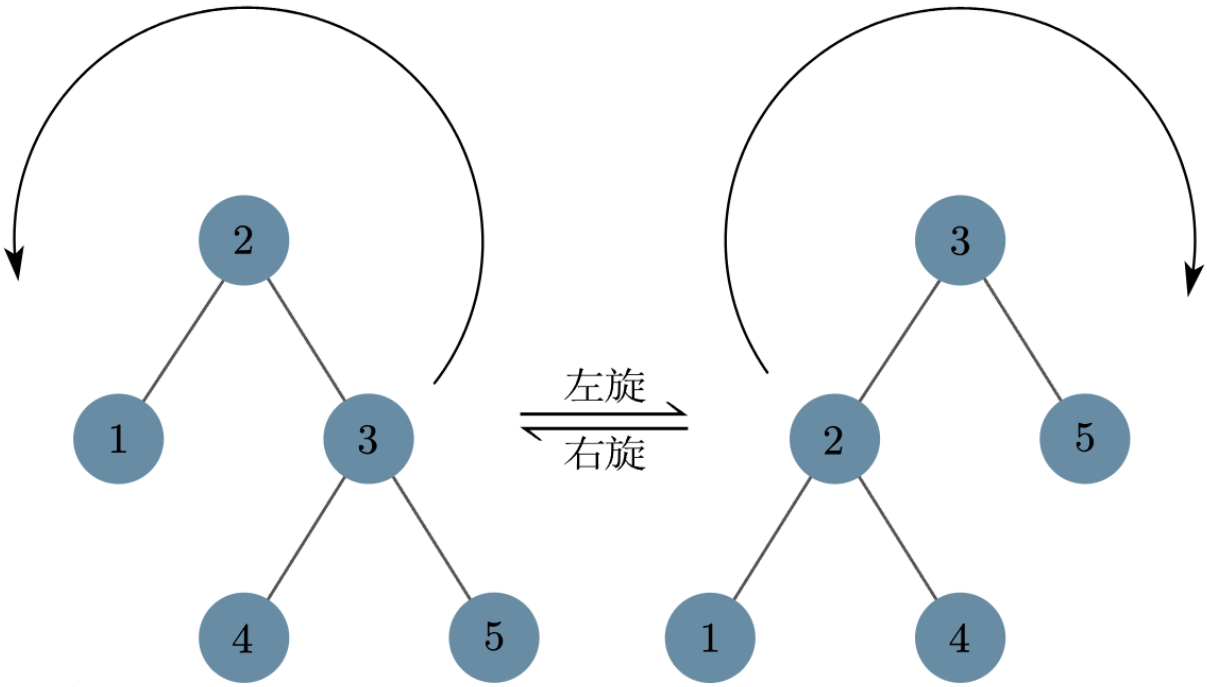
旋转操作是 treap 的一个非常重要的操作，主要用来在保持 treap 树性质的同时，调整不同节点的层数，以达到维护堆性质的作用。

旋转操作的左旋和右旋可能不是特别容易区分，以下是两个较为明显的特点：

旋转操作的含义：

- 在不影响搜索树性质的前提下，把和旋转方向相反的子树变成根节点（如左旋，就是把右子树变成根节点）
- 不影响性质，并且在旋转过后，跟旋转方向相同的子节点变成了原来的根节点（如左旋，旋转完之后的左子节点是旋转前的根节点）

左旋和右旋操作是相互的，如下图。




```
enum rot_type { LF = 1, RT = 0 };

void  rotate(Node *&cur
rot_type dir) {  // dir 参数代表旋转的方向 0 为右旋，1 为左旋
// 注意传进来的 cur 是指针的引用，也就是改了这个
// cur，变量是跟着一起改的，如果这个 cur 是别的 树的子节点，根据 ch
// 找过来的时候，也是会找到这里的

// 以下的代码解释的均是左旋时的情况
Node *tmp = cur->ch[dir];  // 让 C 变成根节点，
// 这里的 tmp
// 是一个临时的节点指针，指向成为新的根节点的节点

/* 左旋，也就是让右子节点变成根节点
*          A                    C
*         / \                  / \
*        B   C     ----->     A   E
*           / \              / \
*          D   E            B   D
*/
cur->ch[dir] = tmp->ch[!dir];     // 让 A 的右子节点变成 D
tmp->ch[!dir] = cur;              // 让 C 的左子节点变成 A
cur->upd_siz(), tmp->upd_siz();   // 更新大小信息
cur = tmp;  // 最后把临时储存 C 树的变量赋值到当前根节点上（注意 cur 是引用）
}
```

插入

类似普通二叉搜索树的插入，但是需要在插入的过程中通过旋转来维护优先级的堆性质。

```
void _insert(Node *&cur, int val) {
if (cur == nullptr) {
// 没这个节点直接新建
cur = new Node(val);
return;
} else if (val == cur->val) {
// 如果有这个值相同的节点，就把重复数量加一
cur->rep_cnt++;
cur->siz++;
} else if (val < cur->val) {
// 维护搜索树性质，val 比当前节点小就插到左边，反之亦然
_insert(cur->ch[0], val);
if (cur->ch[0]->rank < cur->rank) {
// 小根堆中，上面节点的优先级一定更小
// 因为新插的左子节点比父节点小，现在需要让左子节点变成父节点
rotate(cur, RT);  // 注意前面的旋转性质，要把左子节点转上来，需要右旋
}
cur->upd_siz();  // 插入之后大小会变化，需要更新
} else {
_insert(cur->ch[1], val);
if (cur->ch[1]->rank < cur->rank) {
rotate(cur, LF);
}
cur->upd_siz();
}
}
```

删除

主要就是分类讨论，不同的情况有不同的处理方法，删完了树的大小会有变化，要注意更新。并且如果要删的节点有左子树和右子树，就要考虑删除之后让谁来当父节点（维护 rank 小的节点在上面）。

```
void _del(Node *&cur, int val) {
if (val > cur->val) {
_del(cur->ch[1], val);
// 值更大就在右子树，反之亦然
cur->upd_siz();
} else if (val < cur->val) {
_del(cur->ch[0], val);
cur->upd_siz();
} else {
if (cur->rep_cnt > 1) {
// 如果要删除的节点是重复的，可以直接把重复值减小
cur->rep_cnt--, cur->siz--;
return;
}
uint8_t state = 0;
state |= (cur->ch[0] != nullptr);
state |= ((cur->ch[1] != nullptr) << 1);
// 00 都无，01 有左无右，10，无左有右，11 都有
Node *tmp = cur;
switch (state) {
case 0:
delete cur;
cur = nullptr;
// 没有任何子节点，就直接把这个节点删了
break;
case 1:  // 有左无右
cur = tmp->ch[0];
// 把根变成左儿子，然后把原来的根节删了，注意这里的 tmp 是从 cur
// 复制的，而 cur 是引用
delete tmp;
break;
case 2:  // 有右无左
cur = tmp->ch[1];
delete tmp;
break;
case 3:
rot_type dir = cur->ch[0]->rank < cur->ch[1]->rank
? RT
: LF;  // dir 是 rank 更小的那个儿子
_rotate(cur, dir);  // 这里的旋转可以把优先级更小的儿子转上去，rt 是 0，
// 而 lf 是 1，刚好跟实际的子树下标反过来
_del(
cur->ch[!dir],
val);  // 旋转完成后原来的根节点就在旋方向那边，所以需要
// 继续把这个原来的根节点删掉
// 如果说要删的这个节点是在整个树的「上层的」，那我们会一直通过这
// 里的旋转操作，把它转到没有子树了（或者只有一个），再删掉它。
cur->upd_siz();
// 删除会造成大小改变
break;
}
}
}
```

根据值查询排名

操作含义：查询以 cur 为根节点的子树中，val 这个值的大小的排名（该子树中小于 val 的节点的个数 + 1）

```
int _query_rank(Node *cur, int val) {
int less_siz = cur->ch[0] == nullptr ? 0 : cur->ch[0]->siz;
// 这个树中小于 val 的节点的数量
if (val == cur->val)
// 如果这个节点就是要查的节点
return less_siz + 1;
else if (val < cur->val) {
if (cur->ch[0] != nullptr)
return _query_rank(cur->ch[0], val);
else
return 1;  // 如果左子树是空的，说比最小的节点还要小，那这个数字就是最小的
} else {
if (cur->ch[1] != nullptr)
// 如果要查的值比这个节点大，那这个节点的左子树以及这个节点自身肯定都比要查的值小
// 所以要加上这两个值，再加上往右边找的结果
// （以右子树为根的子树中，val 这个值的大小的排名）
return less_siz + cur->rep_cnt + _query_rank(cur->ch[1], val);
else
return cur->siz + 1;
// 没有右子树的话直接整个树 + 1 相当于 less_siz + cur->rep_cnt + 1
}
}
```

根据排名查询值

要根据排名查询值，我们首先要知道如何判断要查的节点在树的哪个部分：

以下是一个判断方法的表：

左子树	根节点/当前节点	右子树
排名 <= 左子树的大小	排名 > 左子树的大小，并且 <= 左子树的大小 + 根节点的重复次数	排名 > 左子树的大小 + 根节点的重复次数

注意如果在右子树，递归的时候需要对原来的 `rank` 进行处理。递归的时候就相当去查，在右子树中为这个排名的值，为了把排名转换成基于右子树的，需要把原来的 `rank` 减去左子树的大小和根节点的重复次数。

可以把所有节点想象成一个排好序的数组，或者数轴（如下），

1 -> |左子树的节点|根节点|右子树的节点| -> n

^

要查的排名

下转换成基于右子树的排名

1 -> |右子树的节点| -> n

^

要查的排名

这里的转换方法就是直接把排名减去左子树的大小和根节点的重复数量。

```
int _query_val(Node *cur, int rank) {
// 查询树中第 rank 大的节点的值
int less_siz = cur->ch[0] == nullptr ? 0 : cur->ch[0]->siz;
// less_siz 是左子树的大小
if (rank <= less_siz)
return _query_val(cur->ch[0], rank);
else if (rank <= less_siz + cur->rep_cnt)
return cur->val;
else
return _query_val(cur->ch[1], rank - less_siz - cur->rep_cnt);  // 见前文
}
```

查询第一个比 val 小的节点

注意这里使用了一个类中的全局变量，`q_prev_tmp`。

这个值是只有在 val 比当前节点值大的时候才会被更改的，所以返回这个变量就是返回 val 最后一次比当前节点的值大，之后就是更小了。

```
int _query_prev(Node *cur, int val) {
if (val <= cur->val) {
// 还是比 val 大，所以往左子树找
if (cur->ch[0] != nullptr) return _query_prev(cur->ch[0], val);
} else {
// 只有能进到这个 else 里，才会更新 q_prev_tmp 的值
q_prev_tmp = cur->val;
// 当前节点已经比 val 小了，但是不确定是否是最大的，所以要到右子树继续找
if (cur->ch[1] != nullptr) _query_prev(cur->ch[1], val);
// 接下来的递归可能不会更改 q_prev_tmp
// 了，那就直接返回这个值，总之返回的就是最后一次进到 这个 else 中的
// cur->val
return q_prev_tmp;
}
return NIL;
}
```

查询第一个比 val 大的节点

跟前一个很相似，只是大于小于号换了一下。

```
int _query_nex(Node *cur, int val) {
if (val >= cur->val) {
if (cur->ch[1] != nullptr) return _query_nex(cur->ch[1], val);
} else {
q_nex_tmp = cur->val;
if (cur->ch[0] != nullptr) _query_nex(cur->ch[0], val);
return q_nex_tmp;
}
return NIL;
}
```

完整代码

以下为 treap 的模板代码：

```
#include <iostream>

constexpr int MAXN = 100005;
constexpr int INF = 1 << 30;

int n;

struct treap {  // 直接维护成数据结构，可以直接用
int l[MAXN], r[MAXN], val[MAXN], rnd[MAXN], size_[MAXN], w[MAXN];
int sz, ans, rt;

void pushup(int x) { size_[x] = size_[l[x]] + size_[r[x]] + w[x]; }

void lrotate(int &k) {
int t = r[k];
r[k] = l[t];
l[t] = k;
size_[t] = size_[k];
pushup(k);
k = t;
}

void rrotate(int &k) {
int t = l[k];
l[k] = r[t];
r[t] = k;
size_[t] = size_[k];
pushup(k);
k = t;
}

void insert(int &k, int x) {  // 插入
if (!k) {
sz++;
k = sz;
size_[k] = 1;
w[k] = 1;
val[k] = x;
rnd[k] = rand();
return;
}
size_[k]++;
if (val[k] == x) {
w[k]++;
} else if (val[k] < x) {
insert(r[k], x);
if (rnd[r[k]] < rnd[k]) lrotate(k);
} else {
insert(l[k], x);
if (rnd[l[k]] < rnd[k]) rrotate(k);
}
}

bool del(int &k, int x) {  // 删除节点
if (!k) return false;
if (val[k] == x) {
if (w[k] > 1) {
w[k]--;
size_[k]--;
return true;
}
if (l[k] == 0 || r[k] == 0) {
k = l[k] + r[k];
return true;
} else if (rnd[l[k]] < rnd[r[k]]) {
```

```
rrotate(k);
return del(k, x);
} else {
lrotate(k);
return del(k, x);
}
} else if (val[k] < x) {
bool succ = del(r[k], x);
if (succ) size_[k]--;
return succ;
} else {
bool succ = del(l[k], x);
if (succ) size_[k]--;
return succ;
}
}

int queryrank(int k, int x) {
if (!k) return 0;
if (val[k] == x)
return size_[l[k]] + 1;
else if (x > val[k]) {
return size_[l[k]] + w[k] + queryrank(r[k], x);
} else
return queryrank(l[k], x);
}

int querynum(int k, int x) {
if (!k) return 0;
if (x <= size_[l[k]])
return querynum(l[k], x);
else if (x > size_[l[k]] + w[k])
return querynum(r[k], x - size_[l[k]] - w[k]);
else
return val[k];
}

void querypre(int k, int x) {
if (!k) return;
if (val[k] < x)
ans = k, querypre(r[k], x);
else
querypre(l[k], x);
}

void querysub(int k, int x) {
if (!k) return;
if (val[k] > x)
ans = k, querysub(l[k], x);
else
querysub(r[k], x);
}
} T;

using std::cin;
using std::cout;

int main() {
cin.tie(nullptr)->sync_with_stdio(false);
srand(123);
cin >> n;
int opt, x;
for (int i = 1; i <= n; i++) {
cin >> opt >> x;
if (opt == 1)
```

```
T.insert(T.rt, x);
else if (opt == 2)
T.del(T.rt, x);
else if (opt == 3) {
cout << T.queryrank(T.rt, x) << '\n';
} else if (opt == 4) {
cout << T.querynum(T.rt, x) << '\n';
} else if (opt == 5) {
T.ans = 0;
T.querypre(T.rt, x);
cout << T.val[T.ans] << '\n';
} else if (opt == 6) {
T.ans = 0;
T.querysub(T.rt, x);
cout << T.val[T.ans] << '\n';
}
}
return 0;
}
```

例题：

Status

Problem

Tags

P6136

.

【模板】普通平衡树（数据加强版）

数据结构

P2286

.

宠物收养场

数据结构

P2042

.

维护数列

数据结构

Splay 树

分块思想

数据结构 11 • 分块思想

简介

分块本质上是一种思想，而不是一种数据结构。

从 NOIP 到 NOI 到 IOI，各种难度的分块思想都有出现。

分块的基本思想是，通过对原数据的适当划分，并在划分后的每一个块上预处理部分信息，从而较一般的暴力算法取得更优的时间复杂度。

分块的时间复杂度主要取决于分块的块长，一般可以通过均值不等式求出某个问题下的最优块长，以及相应的时间复杂度。

分块是一种很灵活的思想，相较于树状数组和线段树，分块的优点是通用性更好，可以维护很多树状数组和线段树无法维护的信息。

当然，分块的缺点是渐进意义的复杂度，相较于线段树和树状数组不够好。

不过在大多数问题上，分块仍然是解决这些问题的一个不错选择。

下面是几个例子。

区间和

例题

给定一个长度为 n 的序列 a[i] ，需要执行 n 次操作。操作分为两种：

1. 给 a_(l) ~ a[r] 之间的所有数加上 x ；
2. 求 sum _(i=l)^(r) a[i] 。 1 <= n <= 5 * 10^4

我们将序列按每 s 个元素一块进行分块，并记录每块的区间和 b[i] 。

underbracea[1], a[2], …, a[s]*(b[1]), underbracea[s+1], …, a[2s]*(b[2]), …, underbracea_((s-1) * s+1), …, a[n]*b((n)/(s))

最后一个块可能是不完整的（因为 n 很可能不是 s 的倍数），但是这对于我们的讨论来说并没有太大影响。

首先看查询操作：

- 若 l 和 r 在同一个块内，直接暴力求和即可，因为块长为 s ，因此最坏复杂度为 O(s) 。
- 若 l 和 r 不在同一个块内，则答案由三部分组成：以 l 开头的不完整块，中间几个完整块，以 r 结尾的不完整块。对于不完整的块，仍然采用上面暴力计算的方法，对于完整块，则直接利用已经求出的 b[i] 求和即可。这种情况下，最坏复杂度为 O((n)/(s)+s) 。

接下来是修改操作：

- 若 l 和 r 在同一个块内，直接暴力修改即可，因为块长为 s ，因此最坏复杂度为 O(s) 。
- 若 l 和 r 不在同一个块内，则需要修改三部分：以 l 开头的不完整块，中间几个完整块，以 r 结尾的不完整块。对于不完整的块，仍然是暴力修改每个元素的值（别忘了更新区间和 b[i] ），对于完整块，则直接修改 b[i] 即可。这种情况下，最坏复杂度和仍然为 O((n)/(s)+s) 。

利用均值不等式可知，当 (n)/(s)=s ，即 s= sqrt (n) 时，单次操作的时间复杂度最优，为 O(sqrt (n)) 。

```
#include <cmath>
#include <iostream>
using namespace std;
int id[50005], len;
// id 表示块的编号，len=sqrt(n)  即上述题解中的 s, sqrt 的时候时间复杂度最优
long long a[50005], b[50005], s[50005];

// a 数组表示数据数组，b 数组记录每个块的整体赋值情况，类似于 lazy_tag, s
// 表示块内元素总和
void add(int l, int r, long long x) {  // 区间加法
int sid = id[l], eid = id[r];
if (sid == eid) {  // 在一个块中
for (int i = l; i <= r; i++) a[i] += x, s[sid] += x;
return;
}
for (int i = l; id[i] == sid; i++) a[i] += x, s[sid] += x;
for (int i = sid + 1; i < eid; i++)
b[i] += x, s[i] += len * x;  // 更新区间和数组（完整的块）
for (int i = r; id[i] == eid; i--) a[i] += x, s[eid] += x;
// 以上两行不完整的块直接简单求和，就 OK
}

long long query(int l, int r, long long p) {  // 区间查询
int sid = id[l], eid = id[r];
long long ans = 0;
if (sid == eid) {  // 在一个块里直接暴力求和
for (int i = l; i <= r; i++) ans = (ans + a[i] + b[sid]) % p;
return ans;
}
for (int i = l; id[i] == sid; i++) ans = (ans + a[i] + b[sid]) % p;
for (int i = sid + 1; i < eid; i++) ans = (ans + s[i]) % p;
for (int i = r; id[i] == eid; i--) ans = (ans + a[i] + b[eid]) % p;
// 和上面的区间修改是一个道理
return ans;
}

int main() {
int n;
cin >> n;
len = sqrt(n);  // 均值不等式可知复杂度最优为根号 n
for (int i = 1; i <= n; i++) {  // 题面要求
cin >> a[i];
id[i] = (i - 1) / len + 1;
s[id[i]] += a[i];
}
for (int i = 1; i <= n; i++) {
int op, l, r, c;
cin >> op >> l >> r >> c;
if (op == 0)
add(l, r, c);
else
cout << query(l, r, c + 1) << endl;
}
return 0;
}
```

区间和 2

上一个做法的复杂度是 Omega(1) , O(sqrt (n)) 。

我们在这里介绍一种 O(sqrt (n)) - O(1) 的算法。

为了 O(1) 询问，我们可以维护各种前缀和。

然而在有修改的情况下，不方便维护，只能维护单个块内的前缀和。

以及整块作为一个单位的前缀和。

每次修改 O(T+(n)/(T)) 。
询问：涉及三部分，每部分都可以直接通过前缀和得到，时间复杂度 O(1) 。
对询问分块
同样的问题，现在序列长度为 n ，有 m 个操作。
如果操作数量比较少，我们可以把操作记下来，在询问的时候加上这些操作的影响。
假设最多记录 T 个操作，则修改 O(1) ，询问 O(T) 。
T 个操作之后，重新计算前缀和， O(n) 。
总复杂度： O(mT+n(m)/(T)) 。
T= sqrt (n) 时，总复杂度 O(m sqrt (n)) 。
其他问题
分块思想也可以应用于其他整数相关问题：寻找零元素的数量、寻找第一个非零元素、计算满足某个性质的元素个数等等。
还有一些问题可以通过分块来解决，例如维护一组允许添加或删除数字的集合，检查一个数是否属于这个集合，以及查找第 k 大的数。要解决这个问题，必须将数字按递增顺序存储，并分割成多个块，每个块中包含 sqrt (n) 个数字。每次添加或删除一个数字时，必须通过在相邻块的边界移动数字来重新分块。
一种很有名的离线算法莫队算法，也是基于分块思想实现的。
例题：
Status
Problem
Tags
P3870
.
开关
数据结构
P4109
.
定价
数据结构
P2801
.
教主的魔法
数据结构
Treap
莫队算法

数据结构 12 · 莫队算法

莫队算法简介
莫队算法是由莫涛提出的算法。在莫涛提出莫队算法之前，莫队算法已经在 Codeforces 的高手圈里小范围流传，但是莫涛是第一个对莫队算法进行详细归纳总结的人。莫涛提出莫队算法时，只分析了普通莫队算法，但是经过 OIer 和 ACMer 的集体智慧改造，莫队有了多种扩展版本。
莫队算法可以解决一类离线区间询问问题，适用性极为广泛。同时将其加以扩展，便能轻松处理树上路径询问以及支持修改操作。
形式
假设 n=m ，那么对于序列上的区间询问问题，如果从 [l,r] 的答案能够 O(1) 扩展到 [l-1,r],[l+1,r],[l,r+1],[l,r-1] （即与 [l,r] 相邻的区间）的答案，那么可以在 O(n sqrt (n)) 的复杂度内求出所有询问的答案。

解释

离线后排序，顺序处理每个询问，暴力从上一个区间的答案转移到下一个区间答案（一步一步移动即可）。

排序方法

对于区间 [l,r] ，以 l 所在块的编号为第一关键字， r 为第二关键字从小到大排序。

实现

```
void move(int pos, int sign) {
// update nowAns
}

void solve() {
BLOCK_SIZE = int(ceil(pow(n, 0.5)));
sort(querys, querys + m);
for (int i = 0; i < m; ++i) {
const query &q = querys[i];
while (l > q.l) move(--l, 1);
while (r < q.r) move(++r, 1);
while (l < q.l) move(l++, -1);
while (r > q.r) move(r--, -1);
ans[q.id] = nowAns;
}
}
```

复杂度分析

以下的情况在 n 和 m 同阶的前提下讨论。

首先是分块这一步，这一步的时间复杂度是 O(sqrt (n)• sqrt (n)log sqrt (n)+nlog n)=O(nlog n) ；

接着就到了莫队算法的精髓了，下面我们用通俗易懂的初中方法来证明它的时间复杂度是 O(n sqrt (n)) ；

证明

证：令每一块中 L 的最大值为 max[1],max[2],max[3], ... , max_(ceil sqrt (n)) 。

由第一次排序可知， max[1] <= max[2] <= ... <= max_(ceil sqrt (n)) 。

显然，对于每一块暴力求出第一个询问的时间复杂度为 O(n) 。

考虑最坏的情况，在每一块中， R 的最大值均为 n ，每次修改操作均要将 L 由 max_(i - 1) 修改至 max[i] 或由 max[i] 修改至 max_(i - 1) 。

考虑 R ：因为 R 在块中已经排好序，所以在同一块修改完它的时间复杂度为 O(n) 。对于所有块就是 O(n sqrt (n)) 。

重点分析 L ：因为每一次改变的时间复杂度都是 O(max[i]-max[i-1]) 的，所以在同一块中时间复杂度为 O(sqrt (n)•(max[i]-max[i-1])) 。

将每一块 L 的时间复杂度合在一起，可以得到：

对于 L 的总时间复杂度为

O(sqrt (n)(max[1]-1)+ sqrt (n)(max[2]-max[1])+ sqrt (n)(max[3]-max[2])+...+ sqrt (n)(max_(ceil sqrt (n))-max_(ceil sqrt (n) -1))) = phantom O(sqrt (n)•(max[1]-1+max[2]-max[1]+max[3]-max[2]+...+max_(ceil sqrt (n) -1)-max_(ceil sqrt (n) -2)+max_(ceil sqrt (n))-max_(ceil sqrt (n) -1))) = phantom O(sqrt (n)•(max_(ceil sqrt (n) -1)))

（裂项求和）

由题可知 max_(ceil sqrt (n)) 最大为 n ，所以 L 的总时间复杂度最坏情况下为 O(n sqrt (n)) 。

综上所述，莫队算法的时间复杂度为 O(n sqrt (n)) ；

但是对于 m 的其他取值，如 m<n ，分块方式需要改变才能变的更优。

怎么分块呢？

我们设块长度为 S ，那么对于任意多个在同一块内的询问，挪动的距离就是 n ，一共 (n)/(S) 个块，移动的总次数就是 (n^2)/(S) ，移动可能跨越块，所以还要加上一个 mS 的复杂度，总复杂度为 O((n^2)/(S)+mS) ，我们要让这个值尽量小，那么就要将这两个项尽量相等，发现 S 取 (n)/(sqrt (m)) 是最优的，此时复杂度为 O((n^2)/((n)/(sqrt (m)))+m((n)/(sqrt (m))))=O(n sqrt (m)) 。

事实上，如果块长度的设定不准确，则莫队的时间复杂度会受到很大影响。例如，如果 m 与 sqrt (n) 同阶，并且块长误设为 sqrt (n) ，则可以很容易构造出一组数据使其时间复杂度为 O(n sqrt (n)) 而不是正确的 O(n) 。

莫队算法看起来十分暴力，很大程度上是因为莫队算法的分块排序方法看起来很粗糙。我们会想到通过看上去更精细的排序方法对所有区间排序。一种方法是把所有区间 [l, r] 看成平面上的点 (l, r) ，并对所有点建立曼哈顿最小生成树，每次沿着曼哈顿最小生成树的边在询问之间转移答案。这样看起来可以改善莫队算法的时间复杂度，但是实际上对询问分块排序的方法的时间复杂度上界已经是最优的了。

假设 n, m 同阶且 n 是完全平方数。我们考虑形如 a sqrt (n), b sqrt (n) 的区间，这样的区间一共有 n 个。如果把所有的区间看成平面上的点，则两点之间的曼哈顿距离恰好为两区间的转移代价，并且任意两个区间之间的最小曼哈顿距离为 sqrt (n) ，所以处理所有询问的时间复杂度最小为 O(n sqrt (n)) 。其它情况的数据构造方法与之类似。

莫队算法还有一个特点：当 n 不变时， m 越大，处理每次询问的平均转移代价就越小。一些其他的离线算法也具有同样的特点（如求 LCA 的 Tarjan 算法），但是莫队算法的平均转移代价随 m 的变化最明显。

例题 & 代码

「国家集训队」小 Z 的袜子

题目大意：

有一个长度为 n 的序列 c[i] 。现在给出 m 个询问，每次给出两个数 l,r ，从编号在 l 到 r 之间的数中随机选出两个不同的数，求两个数相等的概率。

过程

思路：莫队算法模板题。

对于区间 [l,r] ，以 l 所在块的编号为第一关键字， r 为第二关键字从小到大排序。

然后从序列的第一个询问开始计算答案，第一个询问通过直接暴力算出，复杂度为 O(n) ，后面的询问在前一个询问的基础上得到答案。

具体做法：

对于区间 [i,i] ，由于区间只有一个元素，我们很容易就能知道答案。然后一步一步从当前区间（已知答案）向下一个区间靠近。

我们设 col[i] 表示当前颜色 i 出现了多少次， ans 当前共有多少种可行的配对方案（有多少种可以选到一双颜色相同的袜子），表示然后每次移动的时候更新答案——设当前颜色为 k ，如果是增长区间就是 ans 加上 C(col[k]+1,2)-C(col[k],2) ，如果是缩短就是 ans 减去 C(col[k],2)-C(col[k]-1,2) 。

而这个询问的答案就是 (ans)/(C(r-l+1,2)) 。

这里有个优化： C(a,2)=(a (a-1))/(2) 。

所以 C(a+1,2)-C(a,2)=((a+1) a)/(2)-(a (a-1))/(2)=(a)/(2)· (a+1-a+1)=(a)/(2)· 2=a 。

所以 C(col[k]+1,2)-C(col[k],2)=col[k] 。

算法总复杂度： O(n sqrt (n))

下面的代码中 `deno` 表示答案的分母 (denominator)， `nume` 表示分子 (numerator)， `sqn` 表示块的大小： sqrt (n) ， `arr` 是输入的数组， `node` 是存储询问的结构体， `tab` 是询问序列（排序后的）， `col` 同上所述。

注意：由于 `++l` 和 `--r` 的存在，下面代码中的移动区间的 4 个 while 循环的位置很关键，不能随意改变它们之间的位置关系。

关于四个循环位置的讨论
莫队区间的移动过程，就相当于加入了 [1,r] 的元素，并删除了 [1,l-1] 的元素。因此，

- 对于 l <= r 的情况， [1,l-1] 的元素相当于被加入了一次又被删除了一次， [1,r] 的元素被加入一次， [r+1,+ INF) 的元素没有被加入。这个区间是合法区间。
- 对于 l=r+1 的情况， [1,r] 的元素相当于被加入了一次又被删除了一次， [r+1,+ INF) 的元素没有被加入。这时这个区间表示空区间。
- 对于 l>r+1 的情况，那么 [r+1,l-1] （这个区间非空）的元素被删除了一次但没有被加入，因此这个元素被加入的次数是负数。

因此，如果某时刻出现 l>r+1 的情况，那么会存在一个元素，它的加入次数是负数。这在某些题目会出现问题，例如我们如果用一个 `set` 维护区间中的所有数，就会出现「需要删除 `set` 中不存在的元素」的问题。
代码中的四个 while 循环一共有 4!=24 种排列顺序。不妨设第一个循环用于操作左端点，就有以下 12 种排列（另外 12 种是对称的）。下表列出了这 12 种写法的正确性，还给出了错误写法的反例。

循环顺序	正确性	反例或注释
`l--,l++,r--,r++`	错误	l<r<l'<r'
`l--,l++,r++,r--`	错误	l<r<l'<r'
`l--,r--,l++,r++`	错误	l<r<l'<r'
`l--,r--,r++,l++`	正确	证明较繁琐
`l--,r++,l++,r--`	正确	
`l--,r++,r--,l++`	正确	
`l++,l--,r--,r++`	错误	l<r<l'<r'
`l++,l--,r++,r--`	错误	l<r<l'<r'
`l++,r++,l--,r--`	错误	l<r<l'<r'
`l++,r++,r--,l--`	错误	l<r<l'<r'
`l++,r--,l--,r++`	错误	l<r<l'<r'
`l++,r--,r++,l--`	错误	l<r<l'<r'

全部 24 种排列中只有 6 种是正确的，其中有 2 种的证明较繁琐，这里只给出其中 4 种的证明。
这 4 种正确写法的共同特点是，前两步先扩大区间（ `l--` 或 `r++` ），后两步再缩小区间（ `l++` 或 `r--` ）。这样写，前两步是扩大区间，可以保持 l <= r+1 ；执行完前两步后， l <= l' <= r' <= r 一定成立，再执行后两步只会把区间缩小到 [l',r'] ，依然有 l <= r+1 ，因此这样写是正确的。
实现

```
#include <algorithm>
#include <cmath>
#include <iostream>
using namespace std;
constexpr int N = 50005;
int n, m, maxn;
int c[N];
long long sum;
int cnt[N];
long long ans1[N], ans2[N];

struct query {
int l, r, id;

bool operator<(const query &x) const {  // 重载<运算符
if (l / maxn != x.l / maxn) return l < x.l;
return (l / maxn) & 1 ? r < x.r : r > x.r;
}
} a[N];

void add(int i) {
sum += cnt[i];
cnt[i]++;
}

void del(int i) {
cnt[i]--;
sum -= cnt[i];
}

long long gcd(long long a, long long b) { return b ? gcd(b, a % b) : a; }

int main() {
cin.tie(nullptr)->sync_with_stdio(false);
cin >> n >> m;
maxn = sqrt(n);
for (int i = 1; i <= n; i++) cin >> c[i];
for (int i = 0; i < m; i++) cin >> a[i].l >> a[i].r, a[i].id = i;
sort(a, a + m);
for (int i = 0, l = 1, r = 0; i < m; i++) {  // 具体实现
if (a[i].l == a[i].r) {
ans1[a[i].id] = 0, ans2[a[i].id] = 1;
continue;
}
while (l > a[i].l) add(c[--l]);
while (r < a[i].r) add(c[++r]);
while (l < a[i].l) del(c[l++]);
while (r > a[i].r) del(c[r--]);
ans1[a[i].id] = sum;
ans2[a[i].id] = (long long)(r - l + 1) * (r - l) / 2;
}
for (int i = 0; i < m; i++) {
if (ans1[i] != 0) {
long long g = gcd(ans1[i], ans2[i]);
ans1[i] /= g, ans2[i] /= g;
} else
ans2[i] = 1;
cout << ans1[i] << '/' << ans2[i] << '\n';
}
return 0;
}
```

普通莫队的优化

过程

我们看一下下面这组数据

```
// 设块的大小为 2 (假设)
1 1
2 100
3 1
4 100
```

手动模拟一下可以发现，r 指针的移动次数大概为 300 次，我们处理完第一个块之后， l = 2, r = 100 ，此时只需要移动两次 l 指针就可以得到第四个询问的答案，但是我们却将 r 指针移动到 1 来获取第三个询问的答案，再移动到 100 获取第四个询问的答案，这样多了九十几次的指针移动。我们怎么优化这个地方呢？这里我们就要用到奇偶化排序。

什么是奇偶化排序？奇偶化排序即对于属于奇数块的询问，r 按从小到大排序，对于属于偶数块的排序，r 从大到小排序，这样我们的 r 指针在处理完这个奇数块的问题后，将在返回的途中处理偶数块的问题，再向 n 移动处理下一个奇数块的问题，优化了 r 指针的移动次数，一般情况下，这种优化能让程序快 30% 左右。

实现

排序代码：

```
struct node {
int l, r, id;

bool operator<(const node &x) const {
if (l / unit != x.l / unit) return l < x.l;
// 注意下面两行不能写小于（大于）等于，否则会出错（详见下面的小细节）
if ((l / unit) & 1) return r < x.r;
return r > x.r;
}
};
```

细节

如果使用 `sort` 比较两个结构体，不能出现 a < b 和 b < a 同时为真的情况，否则会运行错误。

例题：

Status

Problem

Tags

P1494

.

小 Z 的袜子

数据结构

P1903

.

数颜色 / 维护队列

数据结构

P2464

.

郁闷的小 J

数据结构

P2709

.

小B的询问

数据结构

分块思想

图论 01 • 最小生成树

定义

设 G = (V, E, w) 为一个连通无向带权图，其中：

- V 是顶点集，
- E 是边集，
- w: E -> R 是边权函数（赋予每条边一个实数权重）。

生成树（Spanning Tree）是 G 的一个子图 T = (V, E_(T))，满足：

1. T 包含 G 的所有顶点（即顶点集与 G 相同）；
2. T 是一棵树（连通、无环，且恰好有 |V| - 1 条边）。

若生成树 T 的所有边权之和 w(T) = sum *(e 属于 E*(T)) w(e) 在 G 的所有生成树中**最小**，则称 T 为 G 的 **最小生成树**。

补充说明

1. **存在性：** 当且仅当图 G 是连通图时，最小生成树存在。对于非连通图，可通过**最小生成森林**（Minimum Spanning Forest）覆盖各连通分量。
2. **唯一性：** 最小生成树不一定唯一。若图中存在多条权值相同的边，可能存在不同的最小生成树，但它们的边权总和必定相等。

Kruskal 算法

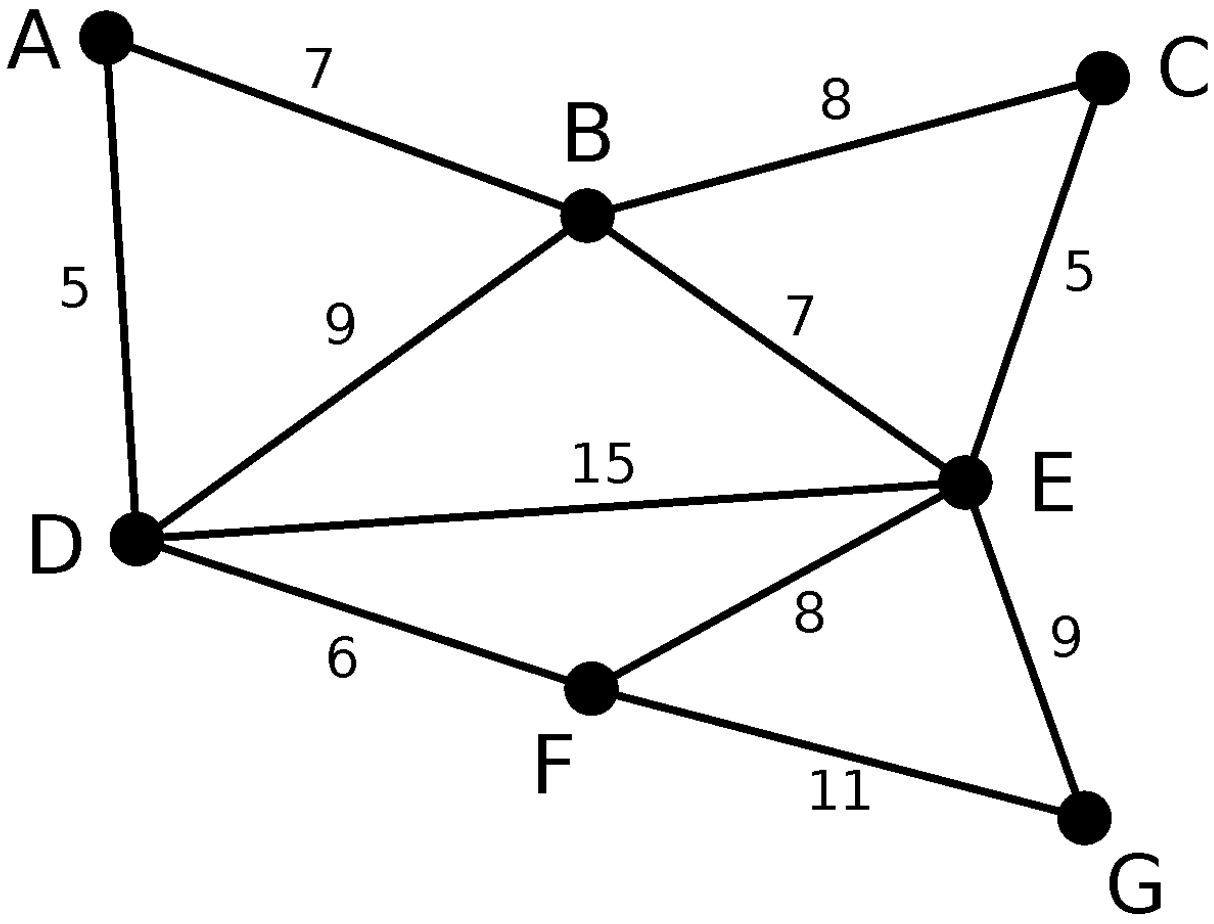
Kruskal 算法是一种常见并且好写的最小生成树算法，由 Kruskal 发明。该算法的基本思想是从小到大加入边，是个贪心算法。

前置知识

Kruskal 算法基于并查集来实现。

实现

图示：



伪代码：

1 Input. The edges of the graph e , where each element in e is (u, v, w) denoting that there is an edge between u and v weighted w . 2 Output. The edges of the MST of the input graph. 3 Method. 4 result <- 空集 5 sort e into nondecreasing order by weight w 6 for each (u, v, w) in the sorted e 7 if u and v are not connected in the union-find set 8 connect u and v in the union-find set 9 result <- result bigcup (u, v, w) 10 return result

算法虽简单，但需要相应的数据结构来支持……具体来说，维护一个森林，查询两个结点是否在同一棵树中，连接两棵树。

抽象一点地说，维护一堆 **集合**，查询两个元素是否属于同一集合，合并两个集合。

其中，查询两点是否连通和连接两点可以使用并查集维护。

如果使用 O(mlog m) 的排序算法，并且使用 O(malpha(m, n)) 或 O(mlog n) 的并查集，就可以得到时间复杂度为 O(mlog m) 的 Kruskal 算法。

证明

思路很简单，为了造出一棵最小生成树，我们从最小边权的边开始，按边权从小到大依次加入，如果某次加边产生了环，就扔掉这条边，直到加入了 n-1 条边，即形成了一棵树。

证明：使用归纳法，证明任何时候 K 算法选择的边集都被某棵 MST 所包含。

基础：对于算法刚开始时，显然成立（最小生成树存在）。

归纳：假设某时刻成立，当前边集为 F ，令 T 为这棵 MST，考虑下一条加入的边 e 。

如果 e 属于 T ，那么成立。

否则， T+e 一定存在一个环，考虑这个环上不属于 F 的另一条边 f （一定只有一条）。

首先， f 的权值一定不会比 e 小，不然 f 会在 e 之前被选取。

然后， f 的权值一定不会比 e 大，不然 T+e-f 就是一棵比 T 还优的生成树了。
所以， T+e-f 包含了 F ，并且也是一棵最小生成树，归纳成立。

例题

口袋的天空

有 n 朵云，你要将它们连成 k 个棉花糖，将 X[i] 云朵和 Y[i] 连接起来需要 L[i] 的代价，求最小代价。

```
#include <algorithm>
#include <iostream>
using namespace std;

int fa[1010];  // 定义父亲
int n, m, k;

struct edge {
int u, v, w;
};

int l;
edge g[10010];

void add(int u, int v, int w) {
l++;
g[l].u = u;
g[l].v = v;
g[l].w = w;
}

// 标准并查集
int findroot(int x) { return fa[x] == x ? x : fa[x] = findroot(fa[x]); }

void Merge(int x, int y) {
x = findroot(x);
y = findroot(y);
fa[x] = y;
}

bool cmp(edge A, edge B) { return A.w < B.w; }

// Kruskal 算法
void kruskal() {
int tot = 0;  // 存已选了的边数
int ans = 0;  // 存总的代价
for (int i = 1; i <= m; i++) {
int xr = findroot(g[i].u), yr = findroot(g[i].v);
if (xr != yr) {  // 如果父亲不一样
Merge(xr, yr);  // 合并
tot++;          // 边数增加
ans += g[i].w;  // 代价增加
}
if (tot >= (n - k)) {  // 检查选的边数是否满足 k 个棉花糖
cout << ans << '\n';
return;
}
}
cout << "No Answer\n";  // 无法连成
}

int main() {
cin >> n >> m >> k;
for (int i = 1; i <= n; i++) {  // 初始化
fa[i] = i;
}
for (int i = 1; i <= m; i++) {
int u, v, w;
cin >> u >> v >> w;
add(u, v, w);  // 添加边
}
sort(g + 1, g + m + 1, cmp);  // 先按边权排序
kruskal();
```

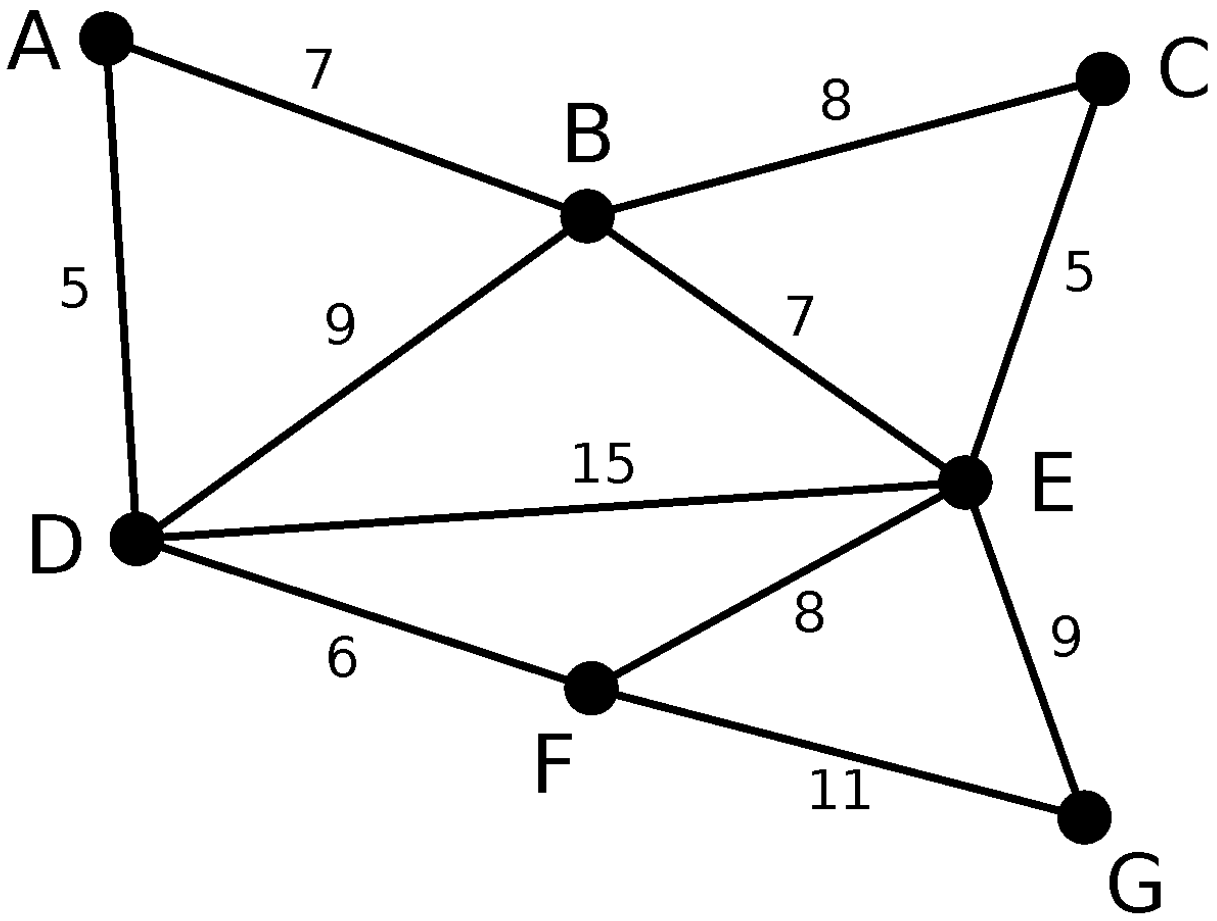
```
return 0;
}
```

Prim 算法

Prim 算法是另一种常见并且好写的最小生成树算法。该算法的基本思想是从一个结点开始，不断加点（而不是 Kruskal 算法的加边）。

实现

图示：



具体来说，每次要选择距离最小的一个结点，以及用新的边更新其他结点的距离。

其实跟 Dijkstra 算法一样，每次找到距离最小的一个点，可以暴力找也可以用堆维护。

堆优化的方式类似 Dijkstra 的堆优化，但如果使用二叉堆等不支持 O(1) decrease-key 的堆，复杂度就不优于 Kruskal，常数也比 Kruskal 大。所以，一般情况下都使用 Kruskal 算法，在稠密图尤其是完全图上，暴力 Prim 的复杂度比 Kruskal 优，但 不一定 实际跑得更快。

暴力： O(n^2+m) 。

二叉堆： O((n+m) log n) 。

Fib 堆： O(n log n + m) 。

伪代码：

```
1 Input. The nodes of the graph V ; the function g(u, v) which
means the weight of the edge (u, v); the function adj(v) which
means the nodes adjacent to v.
2 Output. The sum of weights of the MST of the input graph.
3 Method.
4 result <- 0
```

```
5 choose an arbitrary node in V to be the root
6 dis(root) <- 0
7 for each node v 属于 (V-root)
8 dis(v) <- INF
9 rest <- V
10 while rest != 空集
11 cur <- the node with the minimum dis in rest
12 result <- result+dis(cur)
13 rest <- rest-cur
14 for each node v 属于 adj(cur)
15 dis(v) <- min(dis(v), g(cur, v))
16 return result
```

注意：上述代码只是求出了最小生成树的权值，如果要输出方案还需要记录每个点的 dis 代表的是哪条边。

```cpp
// 使用二叉堆优化的 Prim 算法。
#include <cstring>
#include <iostream>
#include <queue>
using namespace std;
constexpr int N = 5050, M = 2e5 + 10;

struct E {
int v, w, x;
} e[M * 2];

int n, m, h[N], cnte;

void adde(int u, int v, int w) { e[++cnte] = E{v, w, h[u]}, h[u] = cnte; }

struct S {
int u, d;
};

bool operator<(const S &x, const S &y) { return x.d > y.d; }

priority_queue<S> q;
int dis[N];
bool vis[N];

int res = 0, cnt = 0;

void Prim() {
memset(dis, 0x3f, sizeof(dis));
dis[1] = 0;
q.push({1, 0});
while (!q.empty()) {
if (cnt >= n) break;
int u = q.top().u, d = q.top().d;
q.pop();
if (vis[u]) continue;
vis[u] = true;
++cnt;
res += d;
for (int i = h[u]; i; i = e[i].x) {
int v = e[i].v, w = e[i].w;
if (w < dis[v]) {
dis[v] = w, q.push({v, w});
}
}
}
}

int main() {
cin >> n >> m;
for (int i = 1, u, v, w; i <= m; ++i) {
cin >> u >> v >> w, adde(u, v, w), adde(v, u, w);
}
Prim();
if (cnt == n)
cout << res;
else
cout << "No MST.";
return 0;
}
```

证明

从任意一个结点开始，将结点分成两类：已加入的，未加入的。

每次从未加入的结点中，找一个与已加入的结点之间边权最小值最小的结点。

然后将这个结点加入，并连上那条边权最小的边。
重复 n-1 次即可。
证明：还是说明在每一步，都存在一棵最小生成树包含已选边集。
基础：只有一个结点的时候，显然成立。
归纳：如果某一步成立，当前边集为 F ，属于 T 这棵 MST，接下来要加入边 e 。
如果 e 属于 T ，那么成立。
否则考虑 T+e 中环上另一条可以加入当前边集的边 f 。
首先， f 的权值一定不小于 e 的权值，否则就会选择 f 而不是 e 了。
然后， f 的权值一定不大于 e 的权值，否则 T+e-f 就是一棵更小的生成树了。
因此， e 和 f 的权值相等， T+e-f 也是一棵最小生成树，且包含了 F 。
例题：
Status
Problem
Tags
P3366
.
【模板】最小生成树
图论
P1194
.
买礼物
图论
P1265
.
公路修建
图论
P1340
.
兽径管理
图论
P2115
.
Sabotage G
图论
P2573
.
滑雪
图论
P3104
.
Counting Friends G
图论
莫队算法
Floyd 最短路算法

图论 02 • Floyd 最短路算法
最短路径定义

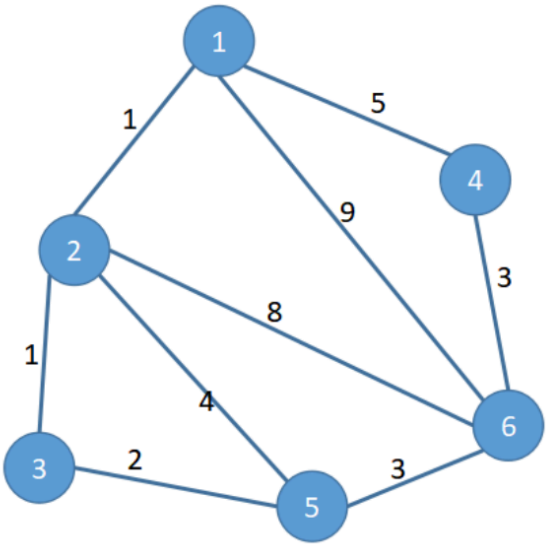
在图论中，**最短路径（Shortest Path）**是指在**带权图**（有权值的图）中，从一个顶点（起点）到另一个顶点（终点）的路径中，边权之和最小的路径。严格定义如下：
设 G = (V, E, w) 为一个带权图（可以是有向图或无向图），其中：

- V 是顶点集，
- E 是边集，
- w: E -> R 是边权函数（赋予每条边一个实数权重，可为正、负或零）。

对于顶点 u, v 属于 V ，从 u 到 v 的一条**路径** P = (v[0], v[1], …, v[k]) （其中 v[0] = u ，v[k] = v ）的**权重**（或长度）为该路径所有边权之和：
w(P) = sum _(i=1)^(k) w(v[i-1], v[i]).
若存在一条路径 P^ ，*使得对于所有从 u 到 v 的路径 P ，都有 w(P^) <= w(P)* ，则称 P^ *为从 u 到 v 的最短路径**。
Floyd 算法
Floyd 算法又称为插点法，是一种利用动态规划的思想寻找给定的加权图中多源点之间最短路径的算法。
作用：求任意两个结点之间的最短路。
复杂度比较高，但是常数小，容易实现（只有三个 `for` ）。
适用于任何图，不管有向无向，边权正负，但是最短路必须存在。（不能有负环）
实现
我们定义一个数组 `f[k][x][y]` ，表示只允许经过结点 1 到 k （也就是说，在子图 V'=1, 2, …, k 中的路径，注意， x 与 y 不一定在这个子图中），结点 x 到结点 y 的最短路长度。
很显然， `f[n][x][y]` 就是结点 x 到结点 y 的最短路长度（因为 V'=1, 2, …, n 即为 V 本身，其表示的最短路径就是所求路径）。
接下来考虑如何求出 `f` 数组的值。
`f[0][x][y]` ： x 与 y 的边权，或者 0 ，或者 + INF （ `f[0][x][y]` 什么时候应该是 + INF ？当 x 与 y 间有直接相连的边的时候，为它们的边权；当 x = y 的时候为零，因为到本身的距离为零；当 x 与 y 没有直接相连的边的时候，为 + INF ）。
`f[k][x][y] = min(f[k-1][x][y], f[k-1][x][k]+f[k-1][k][y])` （ `f[k-1][x][y]` ，为不经过 k 点的最短路径，而 `f[k-1][x][k]+f[k-1][k][y]` ，为经过了 k 点的最短路）。
上面两行都显然是对的，所以说这个做法空间是 O(N^3) ，我们需要依次增加问题规模（ k 从 1 到 n ），判断任意两点在当前问题规模下的最短路。

```
for (k = 1; k <= n; k++) {
for (x = 1; x <= n; x++) {
for (y = 1; y <= n; y++) {
f[k][x][y] = min(f[k - 1][x][y], f[k - 1][x][k] + f[k - 1][k][y]);
}
}
}
```

图示
1，从任意一条单边路径开始。所有两点之间的距离是边的权，如果两点之间没有边相连，则权为无穷大。
现在我们要寻找到每两点之间的最小路径，如下图所示。
首先把图转化为一个二维数组，这个二维数组就是每条边的权重。权重的大小就是他们之间的路径距离。

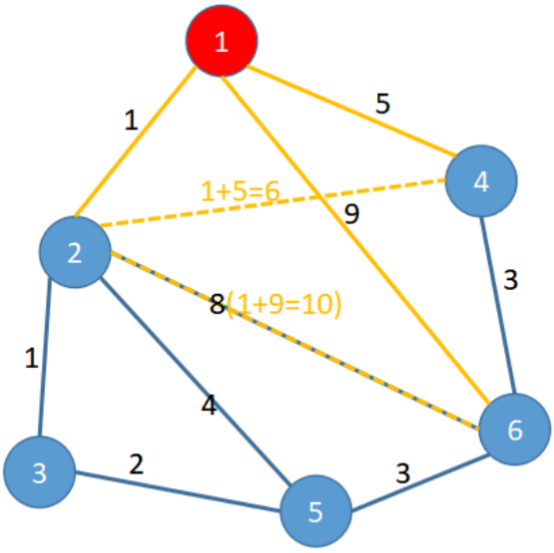


0	1	∞	5	∞	9
1	0	1	∞	4	8
∞	1	0	∞	2	∞
5	∞	∞	0	∞	3
∞	4	2	∞	0	3
9	8	∞	3	3	0

2. 加点

第二步就是循环加点，这里我们首先加入点①。加入①后，本部相通的②与④之间就有了一条路径，长度为 （1+5）= 6 < INF ，所以就要进行更新。

加入①后还影响了②与⑥的连接，他们之间又多了一条路径，这时也需要比较 1 + 9 < 8，所以②与⑥的最短路径保持原来的不变，不进行更新。



0	1	∞	5	∞	9
1	0	1	6	4	8
∞	1	0	∞	2	∞
5	6	∞	0	∞	3
∞	4	2	∞	0	3
9	8	∞	3	3	0

继续加入节点②,由于②本身的基础连接比较多，所以影响的路径更多，这里在图中就不一一的详细标明。

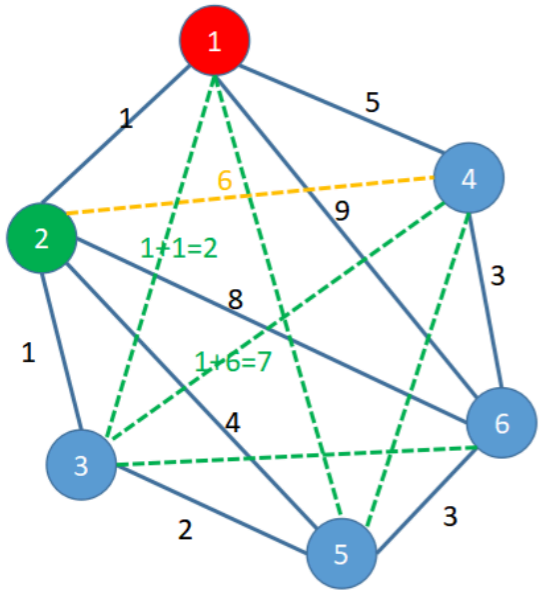
①与③ = 1+1 //2

③与④ = 1+6 //7

③与⑥ = 1+8 //9

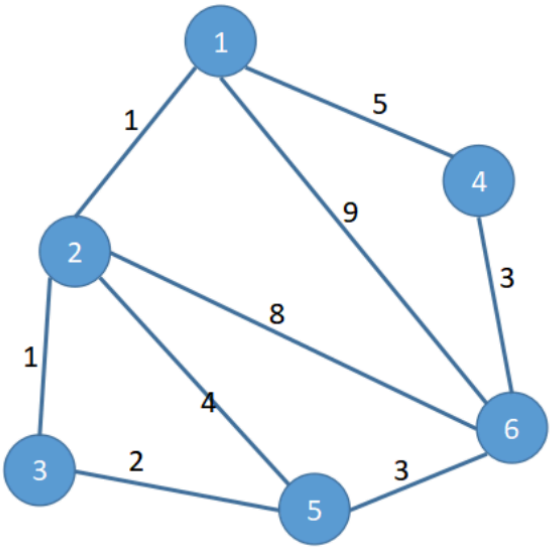
....

除了这些原来未连通的新增的路径外，不要忘记若有原路径的需要比较（当然，程序是不是忘记的，毕竟是一个 for 循环）。



0	1	2	5	5	9
1	0	1	6	4	8
2	1	0	7	2	9
5	6	7	0	10	3
5	4	2	10	0	3
9	8	9	3	3	0

重复上面的操作，知道每个点都加入后，那么最终得到的就是一个每个点亮亮之间最短的路径了。



0	1	2	5	4	7
1	0	1	6	3	6
2	1	0	7	2	5
5	6	7	0	6	3
4	3	2	6	0	3
7	6	5	3	3	0

优化

因为第一维对结果无影响，我们可以发现数组的第一维是可以省略的，于是可以直接改成 `f[x][y] = min(f[x][y], f[x][k]+f[k][y])` 。

证明第一维对结果无影响

对于给定的 `k` ，当更新 `f[k][x][y]` 时，涉及的元素总是来自 `f[k-1]` 数组的第 `k` 行和第 `k` 列。然后我们可以发现，对于给定的 `k` ，当更新 `f[k][k][y]` 或 `f[k][x][k]` ，总是不会发生数值更新，因为按照公式 `f[k][k][y] = min(f[k-1][k][y], f[k-1][k][k]+f[k-1][k][y])` ，`f[k-1][k][k]` 为 0，因此这个值总是 `f[k-1][k][y]` ，对于 `f[k][x][k]` 的证明类似。

因此，如果省略第一维，在给定的 `k` 下，每个元素的更新中使用到的元素都没有在这次迭代中更新，因此第一维的省略并不会影响结果。

```
for (k = 1; k <= n; k++) {
for (x = 1; x <= n; x++) {
for (y = 1; y <= n; y++) {
f[x][y] = min(f[x][y], f[x][k] + f[k][y]);
}
}
}
```

综上时间复杂度是 O(N^3) ，空间复杂度是 O(N^2) 。

例题：

Status

Problem

Tags

P3416

.

Moocast S

图论

P8186

.

Redistributing Gifts S

图论

P5839

.

Moortal Cowmbat G

图论

最小生成树

Bellman-Ford 与 SPFA 算法

图论 03 • Bellman-Ford 与 SPFA 算法

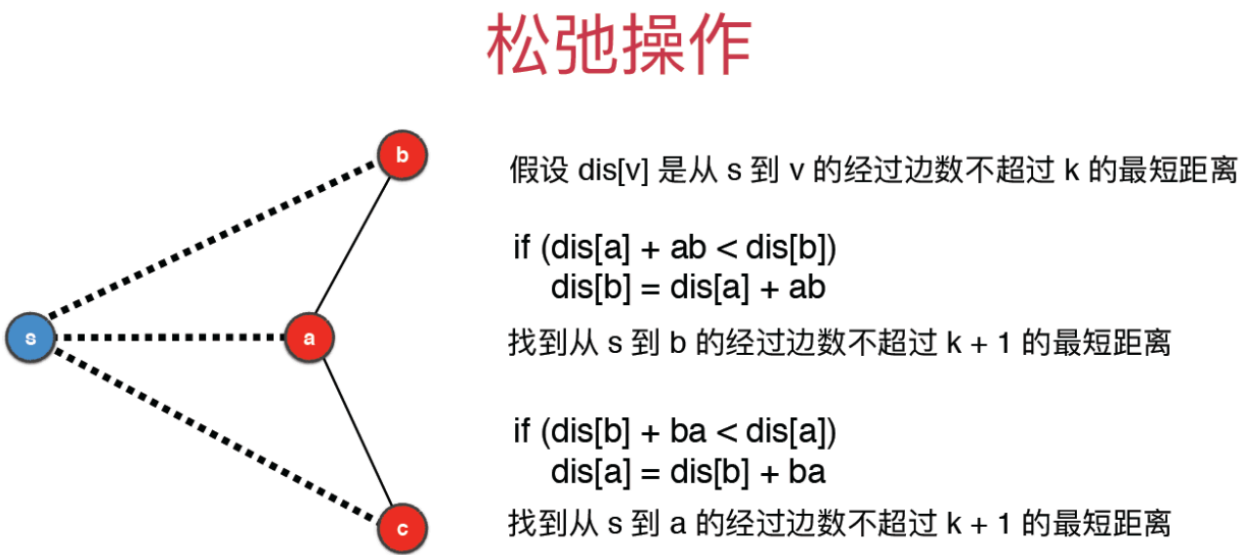
Bellman－Ford 算法

Bellman－Ford 算法是一种基于松弛（relax）操作的最短路算法，可以求出有负权的图的最短路，并可以对最短路不存在的情况进行判断。

在国内 OI 界，你可能听说过的「SPFA」，就是 Bellman－Ford 算法的一种实现。

过程

先介绍 Bellman－Ford 算法要用到的松弛操作（Dijkstra 算法也会用到松弛操作）。



对于边 (u,v) ，松弛操作对应下面的式子： dis(v) = min(dis(v), dis(u) + w(u, v)) 。

这么做的含义是显然的：我们尝试用 S -> u -> v （其中 S -> u 的路径取最短路）这条路径去更新 v 点最短路的长度，如果这条路径更优，就进行更新。

Bellman－Ford 算法所做的，就是不断尝试对图上每一条边进行松弛。我们每进行一轮循环，就对图上所有的边都尝试进行一次松弛操作，当一次循环中没有成功的松弛操作时，算法停止。

每次循环是 O(m) 的，那么最多会循环多少次呢？

在最短路存在的情况下，由于一次松弛操作会使最短路的边数至少 +1 ，而最短路的边数最多为 n-1 ，因此整个算法最多执行 n-1 轮松弛操作。故总时间复杂度为 O(nm) 。
但还有一种情况，如果从 S 点出发，抵达一个负环时，松弛操作会无休止地进行下去。注意到前面的论证中已经说明了，对于最短路存在的图，松弛操作最多只会执行 n-1 轮，因此如果第 n 轮循环时仍然存在能松弛的边，说明从 S 点出发，能够抵达一个负环。
负环判断中存在的常见误区
需要注意的是，以 S 点为源点跑 Bellman－Ford 算法时，如果没有给出存在负环的结果，只能说明从 S 点出发不能抵达一个负环，而不能说明图上不存在负环。
因此如果需要判断整个图上是否存在负环，最严谨的做法是建立一个超级源点，向图上每个节点连一条权值为 0 的边，然后以超级源点为起点执行 Bellman－Ford 算法。
实现

```
struct Edge {
int u, v, w;
};

vector<Edge> edge;

int dis[MAXN], u, v, w;
constexpr int INF = 0x3f3f3f3f;

bool bellmanford(int n, int s) {
memset(dis, 0x3f, (n + 1) * sizeof(int));
dis[s] = 0;
bool flag = false;  // 判断一轮循环过程中是否发生松弛操作
for (int i = 1; i <= n; i++) {
flag = false;
for (int j = 0; j < edge.size(); j++) {
u = edge[j].u, v = edge[j].v, w = edge[j].w;
if (dis[u] == INF) continue;
// 无穷大与常数加减仍然为无穷大
// 因此最短路长度为 INF 的点引出的边不可能发生松弛操作
if (dis[v] > dis[u] + w) {
dis[v] = dis[u] + w;
flag = true;
}
}
// 没有可以松弛的边时就停止算法
if (!flag) {
break;
}
}
// 第 n 轮循环仍然可以松弛时说明 s 点可以抵达一个负环
return flag;
}
```

队列优化：SPFA
即 Shortest Path Faster Algorithm。
很多时候我们并不需要那么多无用的松弛操作。
很显然，只有上一次被松弛的结点，所连接的边，才有可能引起下一次的松弛操作。
那么我们用队列来维护「哪些结点可能会引起松弛操作」，就能只访问必要的边了。
SPFA 也可以用于判断 s 点是否能抵达一个负环，只需记录最短路经过了多少条边，当经过了至少 n 条边时，说明 s 点可以抵达一个负环。

```
struct edge {
int v, w;
};

vector<edge> e[MAXN];
int dis[MAXN], cnt[MAXN], vis[MAXN];
queue<int> q;

bool spfa(int n, int s) {
memset(dis, 0x3f, (n + 1) * sizeof(int));
dis[s] = 0, vis[s] = 1;
q.push(s);
while (!q.empty()) {
int u = q.front();
q.pop(), vis[u] = 0;
for (auto ed : e[u]) {
int v = ed.v, w = ed.w;
if (dis[v] > dis[u] + w) {
dis[v] = dis[u] + w;
cnt[v] = cnt[u] + 1;  // 记录最短路经过的边数
if (cnt[v] >= n) return false;
// 在不经过负环的情况下，最短路至多经过 n - 1 条边
// 因此如果经过了多于 n 条边，一定说明经过了负环
if (!vis[v]) q.push(v), vis[v] = 1;
}
}
}
return true;
}
```

虽然在大多数情况下 SPFA 跑得很快，但其最坏情况下的时间复杂度为 O(nm) ，将其卡到这个复杂度也是不难的，所以考试时要谨慎使用（在没有负权边时最好使用 Dijkstra 算法，在有负权边且题目中的图没有特殊性质时，若 SPFA 是标算的一部分，题目不应当给出 Bellman-Ford 算法无法通过的数据范围）。

例题：

Status

Problem

Tags

P3385

.

【模板】负环

图论

P3371

.

【模板】单源最短路径（弱化版）

图论

P2850

.

Wormholes G

图论

Floyd 最短路算法

Dijkstra 算法

图论 04 • Dijkstra 算法

Dijkstra 算法

Dijkstra（/’dikstra/或/’deikstra/）算法由荷兰计算机科学家 E. W. Dijkstra 于 1956 年发现，1959 年公开发表。是一种求解 **非负权图** 上单源最短路径的算法。

过程

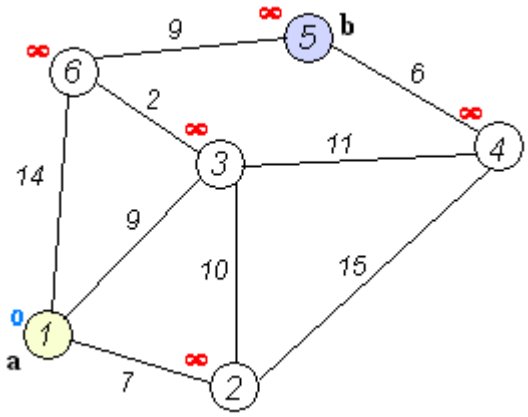
将结点分成两个集合：已确定最短路长度的点集（记为 S 集合）的和未确定最短路长度的点集（记为 T 集合）。一开始所有的点都属于 T 集合。

初始化 dis(s)=0 ，其他点的 dis 均为 + INF 。

然后重复这些操作：

1. 从 T 集合中，选取一个最短路长度最小的结点，移到 S 集合中。
2. 对那些刚刚被加入 S 集合的结点的所有出边执行松弛操作。

直到 T 集合为空，算法结束。



时间复杂度

朴素的实现方法为每次 2 操作执行完毕后，直接在 T 集合中暴力寻找最短路长度最小的结点。2 操作总时间复杂度为 O(m) ，1 操作总时间复杂度为 O(n^2) ，全过程的时间复杂度为 O(n^2 + m) = O(n^2) 。

可以用堆来优化这一过程：每成功松弛一条边 (u,v) ，就将 v 插入堆中（如果 v 已经在堆中，直接执行 Decrease-key），1 操作直接取堆顶结点即可。共计 O(m) 次 Decrease-key， O(n) 次 pop，选择不同堆可以取到不同的复杂度，参考堆页面。堆优化能做到的最优复杂度为 O(nlog n+m) ，能做到这一复杂度的有斐波那契堆等。

特别地，可以使用优先队列维护，此时无法执行 Decrease-key 操作，但可以通过每次松弛时重新插入该结点，且弹出时检查该结点是否已被松弛过，若是则跳过，复杂度 O(mlog n) ，优点是实现较简单。

这里的堆也可以用线段树来实现，复杂度为 O(mlog n) ，在一些特殊的非递归线段树实现下，该做法常数比堆更小。并且线段树支持的操作更多，在一些特殊图问题上只能用线段树来维护。

在稀疏图中， m = O(n) ，堆优化的 Dijkstra 算法具有较大的效率优势；而在稠密图中， m = O(n^2) ，这时候使用朴素实现更优。

实现

这里同时给出 O(n^2) 的暴力做法实现和 O(m log m) 的优先队列做法实现。

朴素实现

```
struct edge {
int v, w;
};

vector<edge> e[MAXN];
int dis[MAXN], vis[MAXN];

void dijkstra(int n, int s) {
memset(dis, 0x3f, (n + 1) * sizeof(int));
dis[s] = 0;
for (int i = 1; i <= n; i++) {
int u = 0, mind = 0x3f3f3f3f;
for (int j = 1; j <= n; j++)
if (!vis[j] && dis[j] < mind) u = j, mind = dis[j];
vis[u] = true;
for (auto ed : e[u]) {
int v = ed.v, w = ed.w;
if (dis[v] > dis[u] + w) dis[v] = dis[u] + w;
}
}
}
```

优先队列实现

```
struct edge {
int v, w;
};

struct node {
int dis, u;

bool operator>(const node& a) const { return dis > a.dis; }
};

vector<edge> e[MAXN];
int dis[MAXN], vis[MAXN];
priority_queue<node, vector<node>, greater<node>> q;

void dijkstra(int n, int s) {
memset(dis, 0x3f, (n + 1) * sizeof(int));
memset(vis, 0, (n + 1) * sizeof(int));
dis[s] = 0;
q.push({0, s});
while (!q.empty()) {
int u = q.top().u;
q.pop();
if (vis[u]) continue;
vis[u] = 1;
for (auto ed : e[u]) {
int v = ed.v, w = ed.w;
if (dis[v] > dis[u] + w) {
dis[v] = dis[u] + w;
q.push({dis[v], v});
}
}
}
}
```

例题：

Status

Problem

Tags

P4779

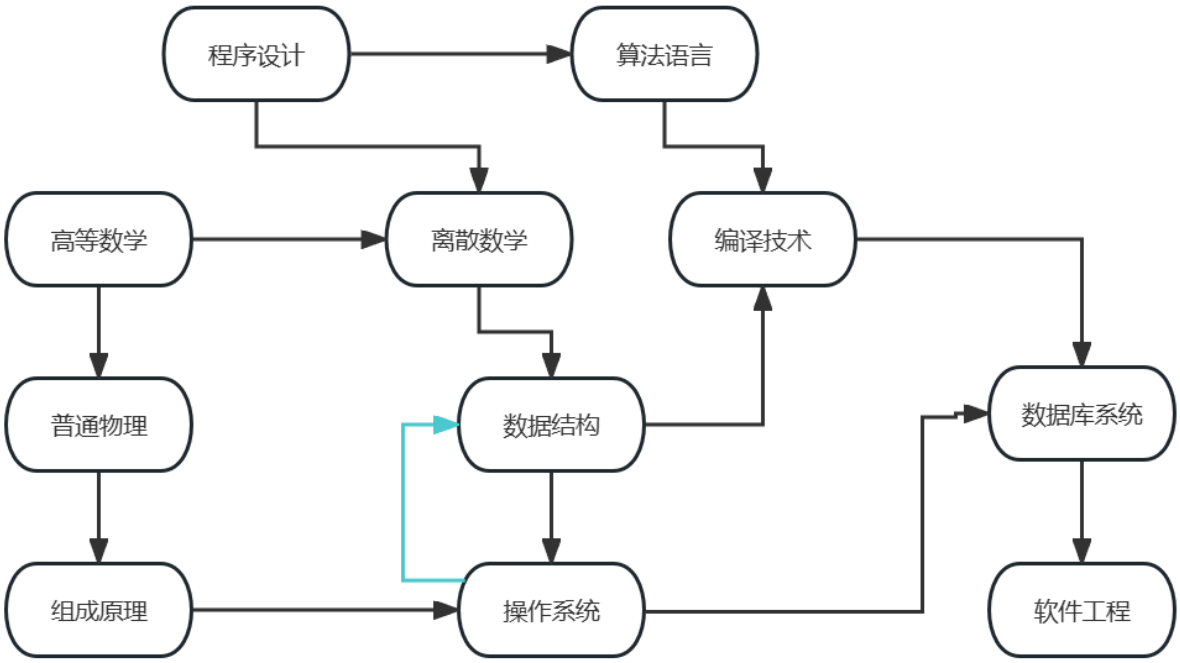
.
【模板】单源最短路径（标准版）
图论
P1119
.
灾后重建
图论
P1462
.
通往奥格瑞玛的道路
图论
P1608
.
路径统计
图论
P1772
.
物流运输
图论
P2371
.
墨墨的等式
图论
Bellman-Ford 与 SPFA 算法
拓扑排序与 Kahn 算法

图论 05 • 拓扑排序与 Kahn 算法

定义

拓扑排序（Topological sorting）要解决的问题是如何给一个有向无环图的所有节点排序。

我们可以拿大学每学期排课的例子来描述这个过程，比如学习大学课程中有：「程序设计」，「算法语言」，「高等数学」，「离散数学」，「编译技术」，「普通物理」，「数据结构」，「数据库系统」等。按照例子中的排课，当我们想要学习「数据结构」的时候，就必须先学会「离散数学」，学习完这门课后就获得了学习「编译技术」的前置条件。当然，「编译技术」还有一个更加前的课程「算法语言」。这些课程就相当于几个顶点 u ，顶点之间的有向边 (u,v) 就相当于学习课程的顺序。教务处安排这些课程，使得在逻辑关系符合的情况下排出课表，就是拓扑排序的过程。



但是如果某一天排课的老师打瞌睡了，说想要学习数据结构，还得先学操作系统，而操作系统的前置课程又是数据结构，那么到底应该先学哪一个（不考虑同时学习的情况）？在这里，数据结构和操作系统间就出现了一个环，显然同学们现在没办法弄清楚自己需要先学什么了，也就没办法进行拓扑排序了。因为如果有向图中存在环路，那么我们就没办法进行拓扑排序。

因此我们可以说在一个 DAG（有向无环图）中，我们将图中的顶点以线性方式进行排序，使得对于任何的顶点 u 到 v 的有向边 (u,v)，都可以有 u 在 v 的前面。

还有给定一个 DAG，如果从 i 到 j 有边，则认为 j 依赖于 i 。如果 i 到 j 有路径（ i 可达 j ），则称 j 间接依赖于 i 。

拓扑排序的目标是将所有节点排序，使得排在前面的节点不能依赖于排在后面的节点。

Kahn 算法

构造拓扑序列步骤

1. 从图中选择一个入度为零的点。
2. 输出该顶点，从图中删除此顶点及其所有的出边。

重复上面两步，直到所有顶点都输出，拓扑排序完成，或者图中不存在入度为零的点，此时说明图是有环图，拓扑排序无法完成，陷入死锁。

过程

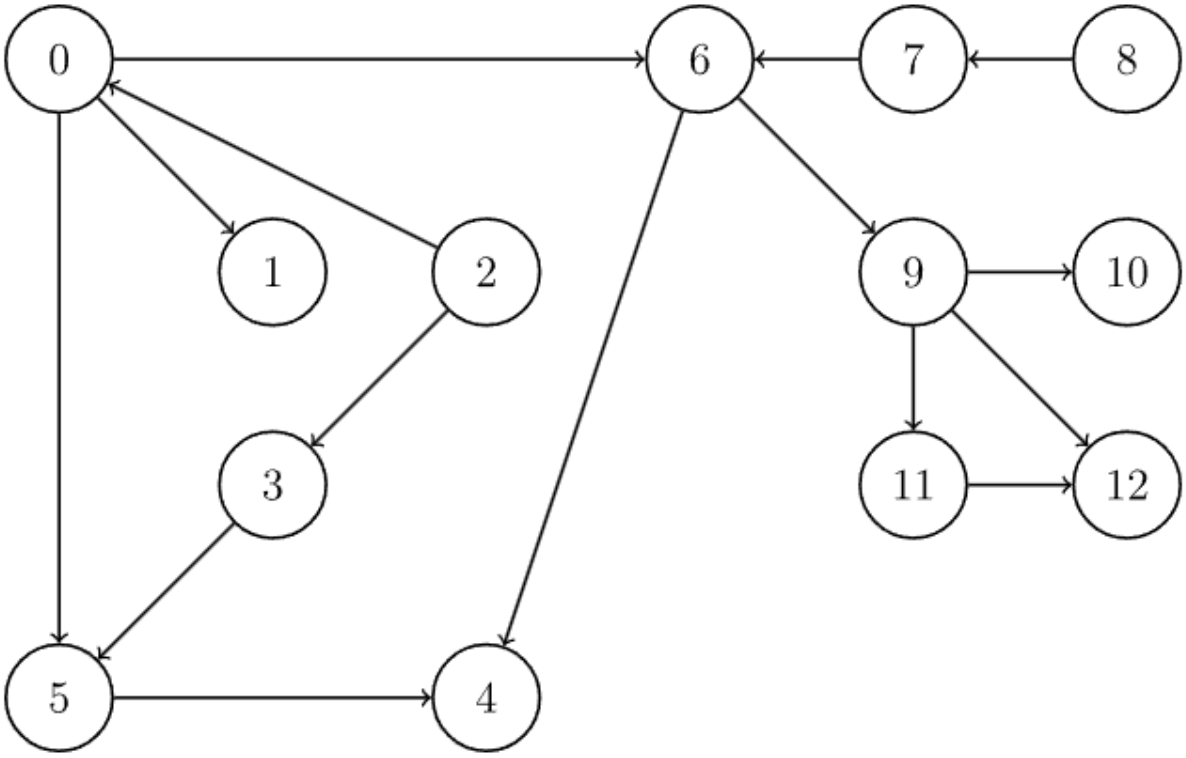
初始状态下，集合 S 装着所有入度为 0 的点， L 是一个空列表。

每次从 S 中取出一个点 u （可以随便取）放入 L ，然后将 u 的所有边 (u, v[1]), (u, v[2]), (u, v[3]) ... 删除。对于边 (u, v) ，若将该边删除后点 v 的入度变为 0 ，则将 v 放入 S 中。

不断重复以上过程，直到集合 S 为空。检查图中是否存在任何边，如果有，那么这个图一定有环路，否则返回 L ， L 中顶点的顺序就是构造拓扑序列的结果。

代码的核心是维持一个入度为 0 的顶点的集合。

可以参考该图



对其排序的结果就是：2 -> 8 -> 0 -> 3 -> 7 -> 1 -> 5 -> 6 -> 9 -> 4 -> 11 -> 10 -> 12

时间复杂度

假设这个图 G = (V, E) 在初始化入度为 0 的集合 S 的时候就需要遍历整个图，并检查每一条边，因而有 O(E+V) 的复杂度。然后对该集合进行操作，显然也是需要 O(E+V) 的时间复杂度。

因而总的时间复杂度就有 O(E+V)

实现

```
int n, m;
vector<int> G[MAXN];
int in[MAXN];  // 存储每个结点的入度

bool toposort() {
vector<int> L;
queue<int> S;
for (int i = 1; i <= n; i++)
if (in[i] == 0) S.push(i);
while (!S.empty()) {
int u = S.front();
S.pop();
L.push_back(u);
for (auto v : G[u]) {
if (--in[v] == 0) {
S.push(v);
}
}
}
if (L.size() == n) {
for (auto i : L) cout << i << ' ';
return true;
}
return false;
}
```

例题：

Status

Problem

Tags
B3644
.
【模板】拓扑排序 / 家谱树
图论
P1347
.
排序
图论
P1038
.
神经网络
图论
P1983
.
车站分级
图论
Dijkstra 算法
欧拉路

图论 06 • 欧拉路

欧拉路径定义

通过图中所有边恰好一次且行遍所有顶点（允许多次经过同一个点）的通路称为欧拉路径。即一笔画。

如果这条路径的起点和终点重合，那么就是欧拉回路。

如何判断图是否有欧拉回路或者欧拉路径？

无向图：因为欧拉路径中，除了起点与终点以外，任意点的“进”“出”次数相等，所以除了两个点为奇点（度数为奇数的点）（终点和起点）以外，其它点的度数均为偶数。

如果是欧拉回路，奇点的个数应该为 0。

有向图：欧拉路径中，最多只有两个点的入度不等于出度。起点出度比入度大 1，终点入度比出度大 1。

如果是欧拉回路，所有点的入度 = 出度。

Hierholzer 算法

Hierholzer 算法用于寻找欧拉回路，在找不到欧拉回路的情况下会找到欧拉路径。

算法流程：

对于无向图：

1、判断奇点数。奇点数若为 0 则任意指定起点，奇点数若为 2 则指定起点为奇点。

2、开始递归函数 Hierholzer(x)：

循环寻找与 x 相连的边 x -> u:

删除 x -> u

删除 u -> x

Hierholzer(u);

回溯时将 x 插入答案队列之中

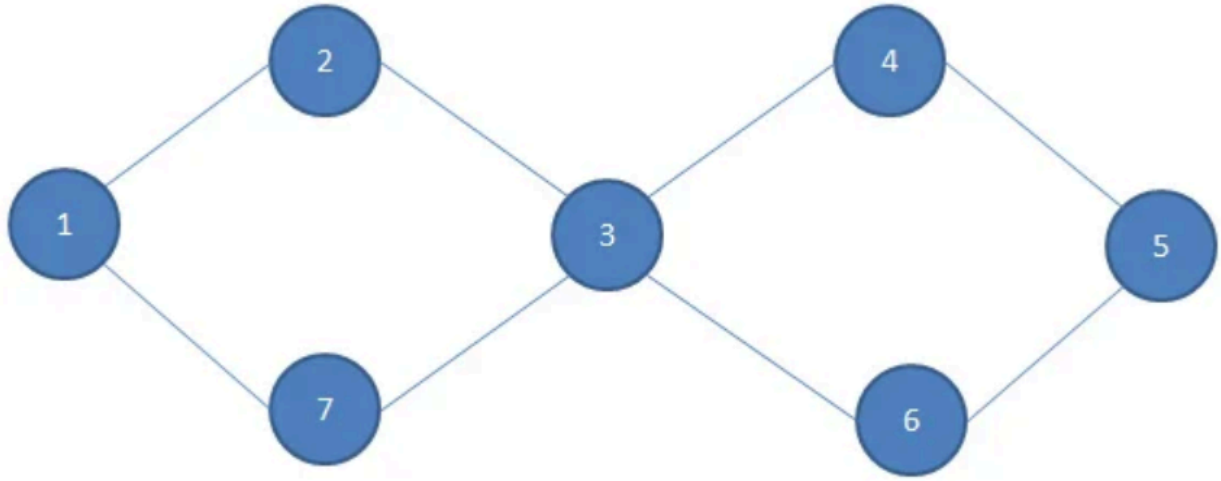
3、倒序输出答案队列

对于有向图：

1、判断顶点入度与出度：

若所有顶点入度等于出度 -> 任意指定起点；

若恰有一个顶点出度=入度 +1（起点），另一顶点入度=出度 +1（终点） -> 指定起点为前者；
其他情况 -> 不存在欧拉路径/回路。
2、开始递归函数 Hierholzer(x)：
循环寻找与 x 相连的出边 x -> u：
删除出边 x -> u（从邻接表中移除 u）；
Hierholzer(u)；
回溯时将 x 插入答案队列之中。
3、倒序输出答案队列。
示例



对于该图，算法的执行流程如下：
step1: 找到该图没有奇点，从 1 开始进行 Hierholzer 算法。
step2: 删边 1 -> 2 递归到 2
step3: 删边 2 -> 3 递归到 3
step4: 删边 3 -> 7 递归到 7
step5: 删边 7 -> 1 递归到 1
step6: 1 无边，1 加入队列，返回
step7: 7 加入队列，返回
step8: 删边 3 -> 4 递归到 4
step9: 删边 4 -> 5 递归到 5
step10: 删边 5 -> 6 递归到 6
step11: 删边 6 -> 3 递归到 3
step12: 3 加入队列，返回
step13: 6 加入队列，返回
step14: 5 加入队列，返回
step15: 4 加入队列，返回
step16: 3 加入队列，返回
step17: 2 加入队列，返回
step18: 1 加入队列，返回
答案队列为：1 7 3 6 5 4 3 2 1。反向输出即为答案。
示例代码

```
#include<bits/stdc++.h> //万能头文件
using namespace std;

struct node{
int u;
bool vis;  //记录是否被访问过
};
int n,m;

vector<node> v[100010];
int in[100010],st[100010]; //in 表示每个结点入度与出度的差（即入读 - 出度）

stack<int> s;  //记录答案

void dfs(int x){
for(int i=0;i<v[x].size();i=max(i+1,st[x])){
if(v[x][i].vis) continue;
v[x][i].vis=1;
st[x]=i+1;
dfs(v[x][i].u);
}
s.push(x);
}

bool cmp(node a,node b){
return a.u<b.u;
}

int main(){

cin>>n>>m;
for(int i=1,x,y;i<=m;i++){
cin>>x>>y;
v[x].push_back(node{y,0});
in[y]++;
in[x]--;
}

int fb=0,fe=0,pb=1,pe;
for(int i=1;i<=n;i++){  //判断是否存在欧拉路径
if(in[i]>1||in[i]<-1) {
printf("No");
return 0;
}
if(in[i]==1) fe++,pe=i;
if(in[i]==-1) fb++,pb=i;
if(fb>1||fe>1) {
printf("No");
return 0;
}
}

for(int i=1;i<=n;i++) sort(v[i].begin(),v[i].end(),cmp);

dfs(pb);

while(s.size()){
printf("%d ",s.top());
s.pop();
}
return 0;
}
```

例题：

Status
Problem
Tags
P7771
.
【模板】欧拉路径
图论
P1333
.
瑞瑞的木棍
图论
P1341
.
无序字母对
图论
P2731
.
骑马修栅栏
图论
拓扑排序与 Kahn 算法
强连通分量与 Tarjan 算法

图论 07 • 强连通分量与 Tarjan 算法

定义

强连通的定义是：有向图 G 强连通是指，G 中任意两个结点连通。
强连通分量（Strongly Connected Components，SCC）的定义是：极大的强连通子图。
这里要介绍的是如何来求强连通分量。

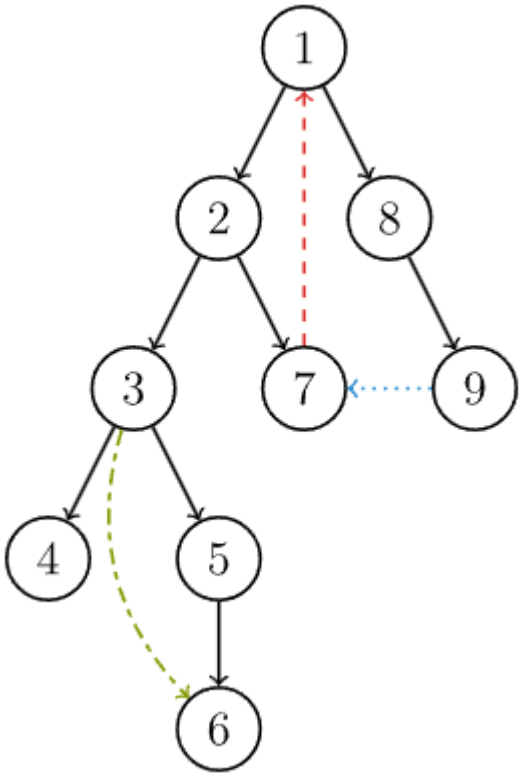
Tarjan 算法

引入

Robert E. Tarjan（罗伯特 • 塔扬，1948~），生于美国加州波莫纳，计算机科学家。
Tarjan 发明了很多算法和数据结构。不少他发明的算法都以他的名字命名，以至于有时会让人混淆几种不同的算法。比如求各种连通分量的 Tarjan 算法，求 LCA（Lowest Common Ancestor，最近公共祖先）的 Tarjan 算法。并查集、Splay、Toptree 也是 Tarjan 发明的。
我们这里要介绍的是在有向图中求强连通分量的 Tarjan 算法。

DFS 生成树

在介绍该算法之前，先来了解 DFS 生成树，我们以下面的有向图为例：



有向图的 DFS 生成树主要有 4 种边（不一定全部出现）：

1. 树边（tree edge）：示意图中以黑色边表示，每次搜索找到一个还没有访问过的结点的时候就形成了一条树边。
2. 反祖边（back edge）：示意图中以红色边表示（即 7 -> 1 ），也被叫做回边，即指向祖先结点的边。
3. 横叉边（cross edge）：示意图中以蓝色边表示（即 9 -> 7 ），它主要是在搜索的时候遇到了一个已经访问过的结点，但是这个结点 **并不是** 当前结点的祖先。
4. 前向边（forward edge）：示意图中以绿色边表示（即 3 -> 6 ），它是在搜索的时候遇到子树中的结点的时候形成的。

我们考虑 DFS 生成树与强连通分量之间的关系。

如果结点 u 是某个强连通分量在搜索树中遇到的第一个结点，那么这个强连通分量的其余结点肯定是在搜索树中以 u 为根的子树中。结点 u 被称为这个强连通分量的根。

反证法：假设有个结点 v 在该强连通分量中但是不在以 u 为根的子树中，那么 u 到 v 的路径中肯定有一条离开子树的边。但是这样的边只可能是横叉边或者反祖边，然而这两条边都要求指向的结点已经被访问过了，这就和 v 不在以 u 为根的子树中矛盾了。得证。

Tarjan 算法求强连通分量

Tarjan 算法基于对图进行深度优先搜索。我们视每个连通分量为搜索树中的一棵子树，在搜索过程中，维护一个栈，每次把搜索树中尚未处理的节点加入栈中。

在 Tarjan 算法中为每个结点 u 维护了以下几个变量：

1. dfn_(u) ：深度优先搜索遍历时结点 u 被搜索的次序。
2. low_(u) ：在 u 的子树中能够回溯到的最早的已经在栈中的结点。设以 u 为根的子树为 Subtree_(u) 。 low_(u) 定义为以下结点的 dfn 的最小值： Subtree_(u) 中的结点；从 Subtree_(u) 通过一条不在搜索树上的边能到达的结点。

一个结点的子树内结点的 dfn 都大于该结点的 dfn。

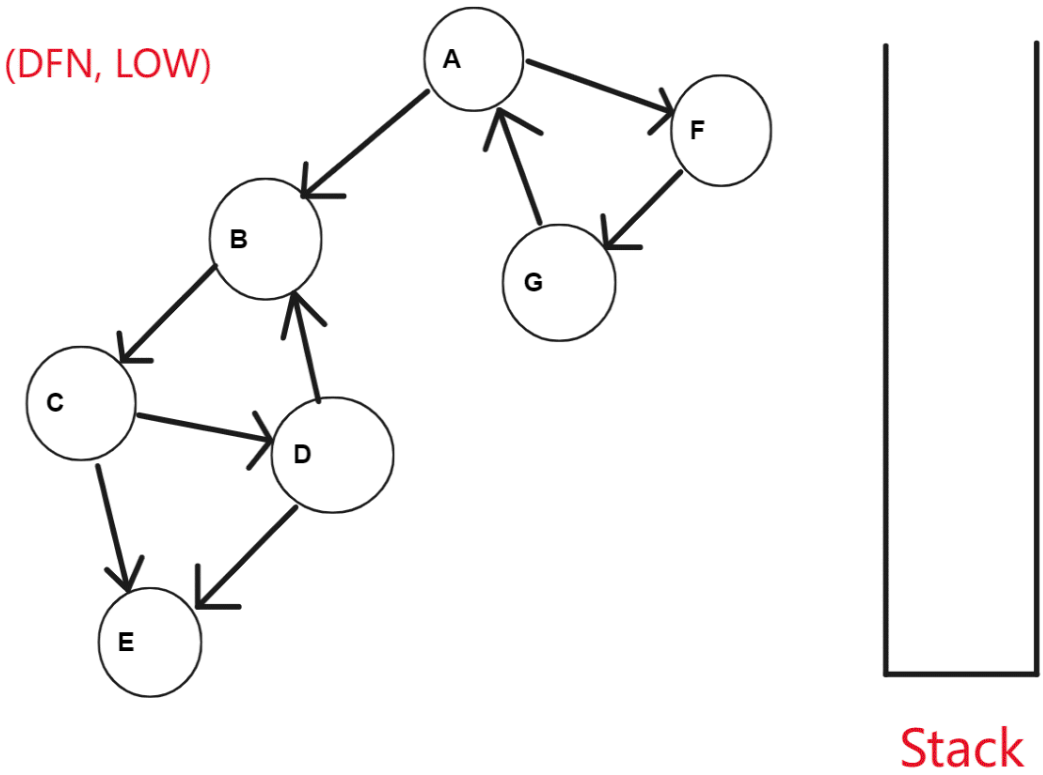
从根开始的一条路径上的 dfn 严格递增，low 严格非降。

按照深度优先搜索算法搜索的次序对图中所有的结点进行搜索，维护每个结点的 `dfn` 与 `low` 变量，且让搜索到的结点入栈。每当找到一个强连通元素，就按照该元素包含结点数目让栈中元素出栈。在搜索过程中，对于结点 u 和与其相邻的结点 v （ v 不是 u 的父节点）考虑 3 种情况：

1. v 未被访问：继续对 v 进行深度搜索。在回溯过程中，用 low_(v) 更新 low_(u) 。因为存在从 u 到 v 的直接路径，所以 v 能够回溯到的已经在栈中的结点， u 也一定能够回溯到。
2. v 被访问过，已经在栈中：根据 low 值的定义，用 dfn_(v) 更新 low_(u) 。
3. v 被访问过，已不在栈中：说明 v 已搜索完毕，其所在连通分量已被处理，所以不用对其做操作。

将上述算法写成伪代码：

```
TARJAN SEARCH(int u)
vis[u]=true
low[u]=dfn[u]=++dfncnt
push u to the stack
for each (u,v) then do
if v hasn't been searched then
TARJAN SEARCH(v) // 搜索
low[u]=min(low[u],low[v]) // 回溯
else if v has been in the stack then
low[u]=min(low[u],dfn[v])
```



对于一个连通分量图，我们很容易想到，在该连通图中有且仅有一个 u 使得 dfn_(u)=low_(u) 。该结点一定是在深度遍历的过程中，该连通分量中第一个被访问过的结点，因为它的 dfn 和 low 值最小，不会被该连通分量中的其他结点所影响。

因此，在回溯的过程中，判定 dfn_(u)=low_(u) 是否成立，如果成立，则栈中 u 及其上方的结点构成一个 SCC。

实现

```
int dfn[N], low[N], dfncnt, s[N], in_stack[N], tp;
int scc[N], sc;  // 结点 i 所在 SCC 的编号
int sz[N];       // 强连通 i 的大小

void tarjan(int u) {
low[u] = dfn[u] = ++dfncnt, s[++tp] = u, in_stack[u] = 1;
for (int i = h[u]; i; i = e[i].nex) {
const int &v = e[i].t;
if (!dfn[v]) {
tarjan(v);
low[u] = min(low[u], low[v]);
} else if (in_stack[v]) {
low[u] = min(low[u], dfn[v]);
}
}
if (dfn[u] == low[u]) {
++sc;
do {
scc[s[tp]] = sc;
sz[sc]++;
in_stack[s[tp]] = 0;
} while (s[tp--] != u);
}
}
```

时间复杂度 O(n + m) 。

例题：

Status

Problem

Tags

P3387

.

【模板】缩点

图论

P1262

.

间谍网络

图论

P2002

.

消息扩散

图论

P2169

.

正则表达式

图论

P1073

.

最优贸易

图论

欧拉路

图论 08 • 割点与割边

割点

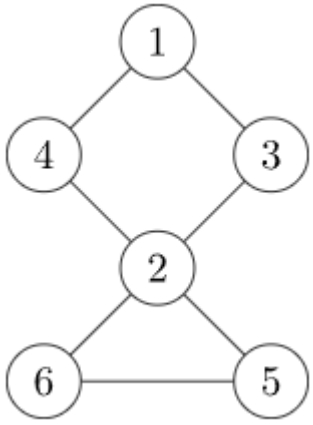
定义

对于一个无向图，如果把一个点删除后这个图的极大连通分量数增加了，那么这个点就是这个图的割点（又称割顶）。

过程

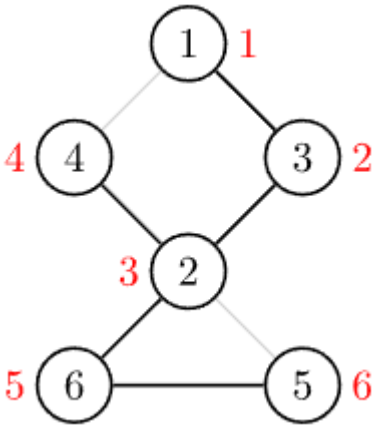
如果我们尝试删除每个点，并且判断这个图的连通性，那么复杂度会特别的高。所以要介绍一个常用的算法：Tarjan。

对于下图：



很容易的看出割点是 2，而且这个图仅有这一个割点。

首先，我们按照 DFS 序给他打上时间戳（访问的顺序）。



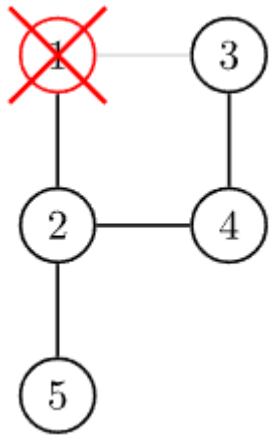
这些信息被我们保存在一个叫做 `dfn` 的数组中。

还需要另外一个数组 `low`，用它来存储不经过其父亲能到达的最小的时间戳。

例如 `low[2]` 是 1，`low[5]` 和 `low[6]` 是 3。

然后我们开始 DFS，我们判断某个点是否是割点的根据是：对于某个顶点 u ，如果存在至少一个顶点 v （ u 的儿子），使得 low_(v) >= dfn_(u) ，即不能回到祖先，那么 u 点为割点。

此根据惟独不适用于搜索的起始点，其需要特殊考虑：若该点不是割点，则其他路径亦能到达全部结点，因此从起始点只「向下搜了一次」，即在搜索树内仅有一个子结点。如果在搜索树内有两个及以上的儿子，那么他一定是割点了（设想上图从 2 开始搜索，搜索树内应有两个子结点：3 或 4 及 5 或 6）。如果只有一个儿子，那么把它删掉，不会有任何的影响。比如下面这个图，此处形成了一个环。



我们在访问 1 的儿子时候，假设先 DFS 到了 2，然后标记用过，然后递归往下，来到了 4，4 又来到了 3，当递归回溯的时候，会发现 3 已经被访问过了，所以不是割点。

更新 `low` 的伪代码如下：

```
1 if v is a son of u 2 low_(u) = min(low_(u), low_(v)) 3 else 4 low_(u) = min(low_(u), dfn_(v))
```

例题

模板题：【模板】割点（割顶）

示例代码

```
#include <iostream>
#include <vector>
using namespace std;
int n, m;  // n: 点数 m: 边数
int dfn[100001], low[100001], idx, res;
// dfn: 记录每个点的时间戳
// low: 能不经过父亲到达最小的编号 idx: 时间戳 res: 答案数量
bool vis[100001], flag[100001];  // flag: 答案 vis: 标记是否重复
vector<int> edge[100001];        // 存图用的

void Tarjan(int u, int fa) {  // u 当前点的编号，fa 自己爸爸的编号
vis[u] = true;                // 标记
low[u] = dfn[u] = ++idx;      // 打上时间戳
int child = 0;                // 每一个点儿子数量
for (const auto &v : edge[u]) {  // 访问这个点的所有邻居（C++11）
if (!vis[v]) {
child++;                      // 多了一个儿子
Tarjan(v, u);                 // 继续
low[u] = min(low[u], low[v]);  // 更新能到的最小节点编号
if (fa != u && low[v] >= dfn[u] && !flag[u]) {  // 主要代码
// 如果不是自己，且不通过父亲返回的最小点符合割点的要求，并且没有被标记过
// 要求即为：删了父亲连不上去了，即为最多连到父亲
flag[u] = true;
res++;  // 记录答案
}
} else if (v != fa) {
// 如果这个点不是自己的父亲，更新能到的最小节点编号
low[u] = min(low[u], dfn[v]);
}
}
// 主要代码，自己的话需要 2 个儿子才可以
if (fa == u && child >= 2 && !flag[u]) {
flag[u] = true;
res++;  // 记录答案
}
}

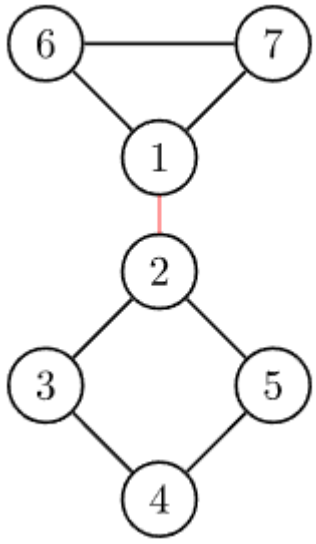
int main() {
cin >> n >> m;                // 读入数据
for (int i = 1; i <= m; i++) {  // 注意点是从 1 开始的
int x, y;
cin >> x >> y;
edge[x].push_back(y);
edge[y].push_back(x);
}  // 使用 vector 存图
for (int i = 1; i <= n; i++)  // 因为 Tarjan 图不一定连通
if (!vis[i]) {
idx = 0;        // 时间戳初始为 0
Tarjan(i, i);  // 从第 i 个点开始，父亲为自己
}
cout << res << endl;
for (int i = 1; i <= n; i++)
if (flag[i]) cout << i << " ";  // 输出结果
return 0;
}
```

割边（无重边时）

定义

对于一个无向图，如果删掉一条边后图中的连通分量数增加了，则称这条边为桥或者割边。严谨来说，就是：假设有连通图 G=V,E ， e 是其中一条边（即 e 属于 E ），如果 G-e 是不连通的，则边 e 是图 G 的一条割边（桥）。

比如说，下图中，



红色的边就是割边。

过程

和割点差不多，只要改一处： low_(v)>dfn_(u) 就可以了，而且不需要考虑根节点的问题。

割边是和是不是根节点没关系的，原来我们求割点的时候是指点 v 是不可能不经过父节点 u 为回到祖先节点（包括父节点），所以顶点 u 是割点。如果 low_(v)=dfn_(u) 表示还可以回到父节点，如果顶点 v 不能回到祖先也没有另外一条回到父亲的路，那么 u-v 这条边就是割边。

实现

下面代码实现了对 **无重边** 的无向图求割边，其中，当 `isbridge[x]` 为真时，`(father[x],x)` 为一条割边。

```
int low[MAXN], dfn[MAXN], idx;
bool isbridge[MAXN];
vector<int> G[MAXN];
int cnt_bridge;
int father[MAXN];

void tarjan(int u, int fa) {
father[u] = fa;
low[u] = dfn[u] = ++idx;
for (const auto &v : G[u]) {
if (!dfn[v]) {
tarjan(v, u);
low[u] = min(low[u], low[v]);
if (low[v] > dfn[u]) {
isbridge[v] = true;
++cnt_bridge;
}
} else if (v != fa) {
low[u] = min(low[u], dfn[v]);
}
}
}
```

割边（有重边时）

然而，上述无重边时的做法在有重边的无向图上是有问题的。

因为两节点间可能不止有一条边，此时它们都不会是桥。

过程

一种思路是将参数 `fa` 改为刚刚走过的边的编号（每条边的编号一致）即可，即将「不用父节点更新」改为「不用来时的边更新」。

另一种更简单的思路是设立一个标记判断是否已有一条边抵达父节点，标记后再访问到父节点时正常更新。
下面代码实现了对可能 **有重边** 的无向图求割边。

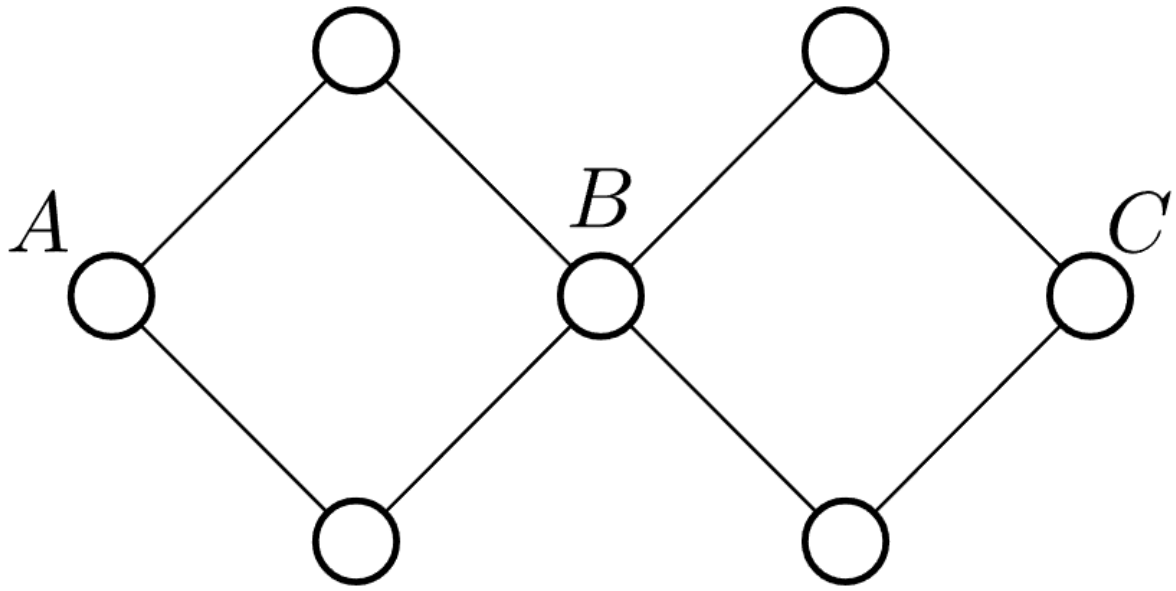
```
int low[MAXN], dfn[MAXN], idx;
bool isbridge[MAXN];
vector<int> G[MAXN];
int cnt_bridge;
int father[MAXN];

void tarjan(int u, int fa) {
bool flag = false;
father[u] = fa;
low[u] = dfn[u] = ++idx;
for (const auto &v : G[u]) {
if (!dfn[v]) {
tarjan(v, u);
low[u] = min(low[u], low[v]);
if (low[v] > dfn[u]) {
isbridge[v] = true;
++cnt_bridge;
}
} else {
if (v != fa || flag)
low[u] = min(low[u], dfn[v]);
else
flag = true;
}
}
}
```

例题：
Status
Problem
Tags
B2002
.
【模板】割点（割顶）
图论
强连通分量与 Tarjan 算法
双连通分量

图论 09 • 双连通分量

定义
在一张连通的无向图中，对于两个点 u 和 v ，如果无论删去哪条边（只能删去一条）都不能使它们不连通，我们就说 u 和 v **边双连通**。
在一张连通的无向图中，对于两个点 u 和 v ，如果无论删去哪个点（只能删去一个，且不能删 u 和 v 自己）都不能使它们不连通，我们就说 u 和 v **点双连通**。
边双连通具有传递性，即，若 x,y 边双连通， y,z 边双连通，则 x,z 边双连通。
点双连通 **不** 具有传递性，反例如下图， A,B 点双连通， B,C 点双连通，而 A,C **不** 点双连通。



对于一个无向图中的 **极大** 边双连通的子图，我们称这个子图为一个 **边双连通分量**。
对于一个无向图中的 **极大** 点双连通的子图，我们称这个子图为一个 **点双连通分量**。

DFS 生成树

对于一张连通的无向图，我们可以从任意一点开始 DFS，得到原图的一棵 DFS 生成树（以开始 DFS 的那个点为根），这棵生成树上的边称作 **树边**，不在生成树上的边称作 **非树边**。
由于 DFS 的性质，我们可以保证所有非树边连接的两个点在生成树上都满足其中一个是另一个的祖先。
DFS 的代码如下：

```
void DFS(int n) {
visited[n] = true;
for (int to : edge[n])
if (!visited[to]) DFS(to);
}
```

边双连通分量

例题
对于一个 n 个节点 m 条无向边的图，请输出其边双连通分量的个数，并且输出每个边双连通分量。

Tarjan 算法 1

用 Tarjan 求双连通分量过程与求强连通分量类似，可以先阅读强连通分量的 Tarjan 算法。
我们考虑先求出所有的桥，再 DFS 求出边双连通分量。
求桥可参见割点和桥的桥部分。
时间复杂度 O(n+m) 。

```cpp
#include <algorithm>
#include <iostream>
#include <vector>

using namespace std;
constexpr int N = 5e5 + 5, M = 2e6 + 5;
int n, m, ans;
int tot = 1, hd[N];

struct edge {
int to, nt;
} e[M << 1];

void add(int u, int v) { e[++tot].to = v, e[tot].nt = hd[u], hd[u] = tot; }

void uadd(int u, int v) { add(u, v), add(v, u); }

bool bz[M << 1];
int bcc_cnt, dfn[N], low[N], vis_bcc[N];
vector<vector<int>> bcc;

void tarjan(int x, int in) {
dfn[x] = low[x] = ++bcc_cnt;
for (int i = hd[x]; i; i = e[i].nt) {
int v = e[i].to;
if (dfn[v] == 0) {
tarjan(v, i);
if (dfn[x] < low[v]) bz[i] = bz[i ^ 1] = true;
low[x] = min(low[x], low[v]);
} else if (i != (in ^ 1))
low[x] = min(low[x], dfn[v]);
}
}

void dfs(int x, int id) {
vis_bcc[x] = id, bcc[id - 1].push_back(x);
for (int i = hd[x]; i; i = e[i].nt) {
int v = e[i].to;
if (vis_bcc[v] || bz[i]) continue;
dfs(v, id);
}
}

int main() {
cin.tie(nullptr)->sync_with_stdio(false);
cin >> n >> m;
int u, v;
for (int i = 1; i <= m; i++) {
cin >> u >> v;
if (u == v) continue;
uadd(u, v);
}
for (int i = 1; i <= n; i++)
if (dfn[i] == 0) tarjan(i, 0);
for (int i = 1; i <= n; i++)
if (vis_bcc[i] == 0) {
bcc.push_back(vector<int>());
dfs(i, ++ans);
}
cout << ans << '\n';
for (int i = 0; i < ans; i++) {
cout << bcc[i].size();
for (int j = 0; j < bcc[i].size(); j++) cout << ' ' << bcc[i][j];
cout << '\n';
}
```

```
return 0;
}
```

Tarjan 算法 2

我们先总结出一个重要的性质，在无向图中，DFS 生成树上的边不是树边就只有非树边。

我们联系一下求强连通分量的方法，在无向图中只要一个分量没有桥，那么在 DFS 生成树上，它的所有点都在同一个强连通分量中。

反过来，在 DFS 生成树上的一个强连通分量，在原无向图中是边双连通分量。

可以发现，求边双连通分量的过程实际上就是求强连通分量的过程。

时间复杂度 O(n+m) 。

```cpp
#include <algorithm>
#include <iostream>
#include <stack>
#include <vector>

using namespace std;
constexpr int N = 5e5 + 5, M = 2e6 + 5;
int n, m;

struct edge {
int to, nt;
} e[M << 2];

int hd[N << 1], tot = 1;

void add(int u, int v) { e[++tot] = edge{v, hd[u]}, hd[u] = tot; }

void uadd(int u, int v) { add(u, v), add(v, u); }

int bcc_cnt, sum;
int dfn[N], low[N];
bool vis[N];
vector<vector<int>> ans;
stack<int> st;

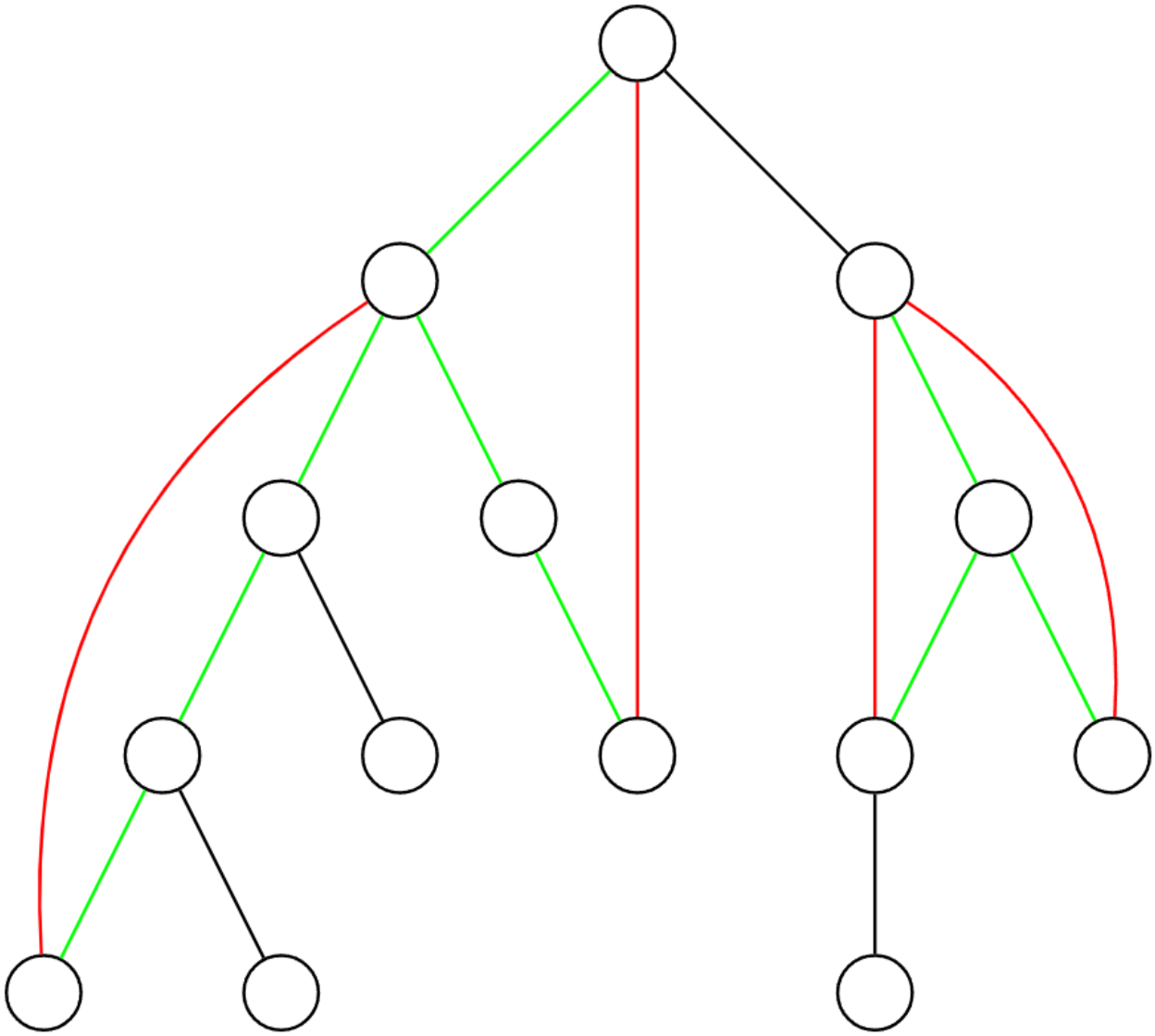
void tarjan(int u, int in) {
low[u] = dfn[u] = ++bcc_cnt;
st.push(u), vis[u] = true;
for (int i = hd[u]; i; i = e[i].nt) {
int v = e[i].to;
if (i == (in ^ 1)) continue;
if (!dfn[v])
tarjan(v, i), low[u] = min(low[u], low[v]);
else if (vis[v])
low[u] = min(low[u], dfn[v]);
}
if (dfn[u] == low[u]) {
vector<int> t;
t.push_back(u);
while (st.top() != u)
t.push_back(st.top()), vis[st.top()] = false, st.pop();
st.pop(), ans.push_back(t);
}
}

int main() {
cin.tie(nullptr)->sync_with_stdio(false);
cin >> n >> m;
int u, v;
for (int i = 1; i <= m; i++) {
cin >> u >> v;
if (u != v) uadd(u, v);
}
for (int i = 1; i <= n; i++)
if (!dfn[i]) tarjan(i, 0);
cout << ans.size() << '\n';
for (int i = 0; i < ans.size(); i++) {
cout << ans[i].size() << ' ';
for (int j = 0; j < ans[i].size(); j++) cout << ans[i][j] << ' ';
cout << '\n';
}
return 0;
}
```

差分算法

和 Tarjan 算法 1 类似，我们先求出所有的桥，再差分求出边双连通分量。

首先，对原图进行 DFS。



如上图所示，黑色与绿色边为树边，红色边为非树边。每一条非树边的两个端点都唯一对应了树上的一条由树边构成的简单路径，我们说这条非树边 **覆盖** 了这条简单路径上所有的边。

在图中，绿色的树边 **至少** 被一条非树边覆盖，黑色的树边不被 **任何** 非树边覆盖。

显然，**非树边** 和 **绿色的树边** 一定不是桥，**黑色的树边** 一定是桥。

首先考虑一个暴力的做法，对于每一条非树边，都逐个地将它覆盖的每一条树边置成绿色，时间复杂度为 O(nm) 。

考虑用差分优化。对于每一条非树边，在其树上深度较小的端点处打上 `-1` 标记，在其树上深度较大的端点处打上 `+1` 标记，然后 O(n) 求出每个点的子树内部的标记和。

对于一个点 u ，其子树内部的标记之和等于覆盖了 u 和 fa_(u) 之间的树边的非树边数量。若这个值等于 0 ，则 u 和 fa_(u) 之间的树边是 **桥**。

再用 DFS 求出边双连通分量。

时间复杂度 O(n+m) 。

```cpp
#include <cstring>
#include <iostream>
#include <vector>
using namespace std;
#define min(x, y) (x < y ? x : y)
#define max(x, y) (x > y ? x : y)
constexpr int N = 5e5 + 5, M = 2e6 + 5;
int n, m;

struct edge {
int to, nt;
} e[M << 1];

int hd[N], tot;

void add(int u, int v) { e[++tot] = {v, hd[u]}, hd[u] = tot; }

void uadd(int u, int v) { add(u, v), add(v, u); }  // 链式前向星

using ll = long long;
#define P(x, y) ((ll)min(x, y) * N + (ll)max(x, y))
constexpr int hmod = 1e5 + 7;

struct hash {
vector<ll> v1[hmod];
vector<short> v2[hmod];

short &operator[](ll x) {
int y = x % hmod;
for (int i = 0; i < v1[y].size(); i++)
if (v1[y][i] == x) return v2[y][i];
v1[y].push_back(x), v2[y].push_back(0);
return v2[y].back();
}
} re, be;  // 用 vector 实现 hash 表，因为本题时空限制均比较紧

// re 判断是否有重边，be 记录边是不是桥

// #define P(x, y) {min(x, y), max(x, y)}
// using pii = pair<int, int>;
// map<pii, int> re, be; // 不紧时可以用 map 实现 hash 表

int dep[N], bz[N], sum[N];  // 记录深度，差分值，子树查分和
int vis[N], fa[N];          // 记录是否访问过，分组编号

void dfs(int x, int pre) {  // 计算每个点的深度和单点查分
if (dep[x] < dep[pre]) bz[x]++, bz[pre]--;  // 回到祖先，更改差分值
if (dep[x]) return;
dep[x] = dep[pre] + 1;
for (int i = hd[x]; i; i = e[i].nt) dfs(e[i].to, x);
}

int dfs2(int x, int pre) {  // 处理子树查分
if (vis[x] == 1) return sum[x];
vis[x] = 1, sum[x] = bz[x];
for (int i = hd[x]; i; i = e[i].nt) {
int v = e[i].to;
if (dep[v] > dep[x] && !vis[v]) sum[x] += dfs2(v, x);
}
if (sum[x] == 0 && re[P(x, pre)] == 1) be[P(x, pre)] = 1;  // 记录桥
return sum[x];
}

int cnt;
vector<int> ans[N];
```

```
void dfs3(int x) {  // 计算双连通分量
if (fa[x]) return;
ans[cnt].push_back(x); fa[x] = cnt;
for (int i = hd[x]; i; i = e[i].nt) {
int v = e[i].to;
if (be[P(x, v)] != 1) dfs3(v);  // 不是桥，递归处理子树
}
}

int main() {
cin.tie(nullptr)->sync_with_stdio(false);
cin >> n >> m;
int u, v;
for (int i = 1; i <= m; i++) cin >> u >> v, uadd(u, v), re[P(u, v)]++;
for (int i = 1; i <= n; i++)
if (!dep[i]) dfs(i, 0);
for (int i = 1; i <= n; i++)
if (!vis[i]) dfs2(i, 0);
for (int i = 1; i <= n; i++)
if (!fa[i]) cnt++, dfs3(i);
cout << cnt << '\n';
for (int i = 1; i <= cnt; i++) {
cout << ans[i].size() << ' ';
for (int j = 0; j < ans[i].size(); j++) cout << ans[i][j] << ' ';
cout << '\n';
}
return 0;
}
```

点双连通分量

题面

对于一个 n 个节点 m 条无向边的图，请输出其点双连通分量的个数，并且输出每个点双连通分量。

Tarjan 算法

需要先学习割点，可以先参见割点和桥的割点部分。

先给出两个性质：

1. 两个点双最多只有一个公共点，且一定是割点。
2. 对于一个点双，它在 DFS 搜索树中 dfn 值最小的点一定是割点或者树根。

我们根据第二个性质，分类讨论：

1. 当这个点为割点时，它一定是点双连通分量的根，因为一旦包含它的父节点，他仍然是割点。
2. 当这个点为树根时：
3. 有两个及以上子树，它是一个割点。
4. 只有一个子树，它是一个点双连通分量的根。
5. 它没有子树，视作一个点双。

```cpp
#include <iostream>
#include <vector>
using namespace std;
constexpr int N = 5e5 + 5, M = 2e6 + 5;
int n, m;

struct edge {
int to, nt;
} e[M << 1];

int hd[N], tot = 1;

void add(int u, int v) { e[++tot] = edge{v, hd[u]}, hd[u] = tot; }

void uadd(int u, int v) { add(u, v), add(v, u); }

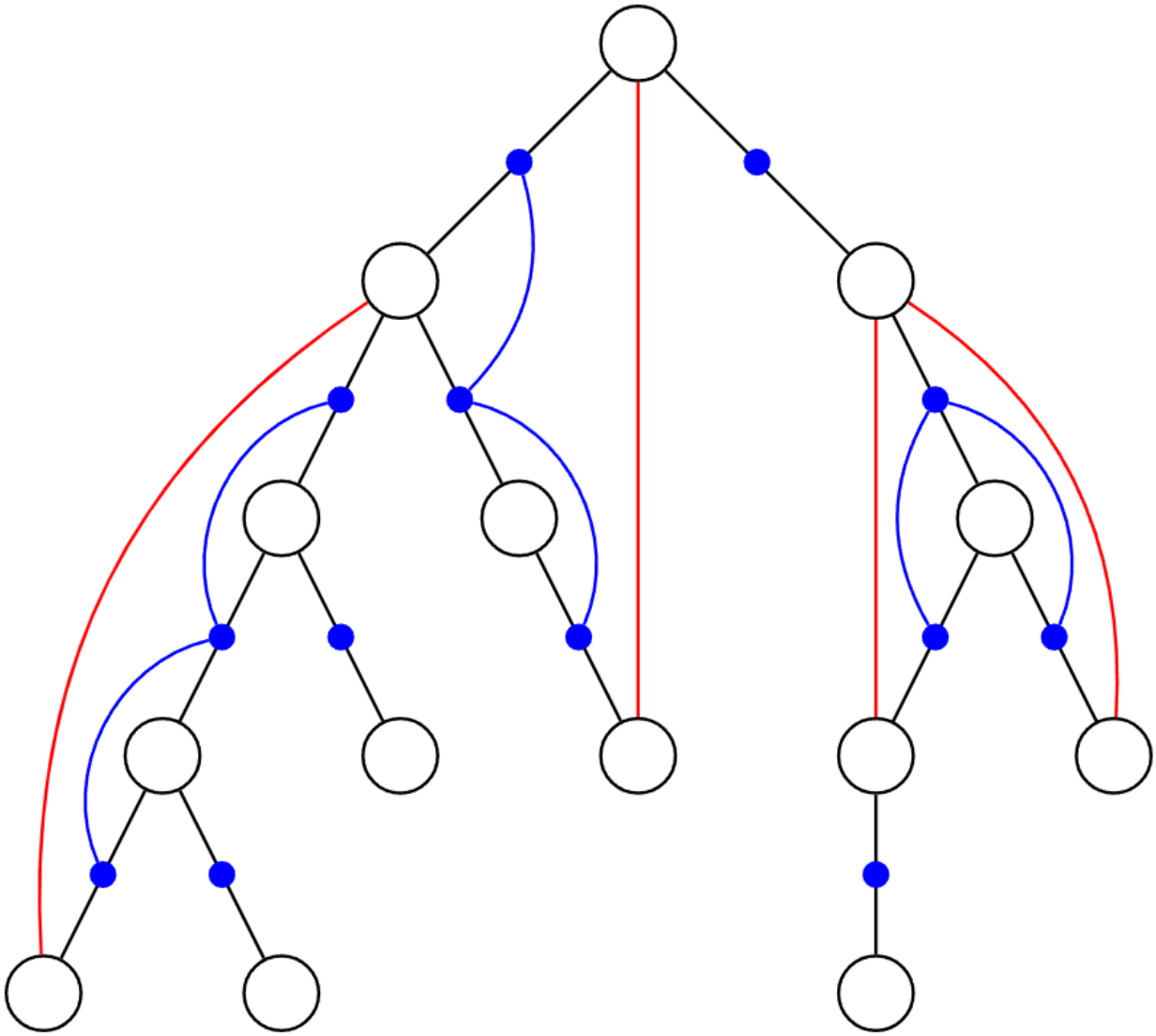
int ans;
int dfn[N], low[N], bcc_cnt;
int sta[N], top, cnt;
bool cut[N];
vector<int> dcc[N];
int root;

void tarjan(int u) {
dfn[u] = low[u] = ++bcc_cnt, sta[++top] = u;
if (u == root && hd[u] == 0) {
dcc[++cnt].push_back(u);
return;
}
int f = 0;
for (int i = hd[u]; i; i = e[i].nt) {
int v = e[i].to;
if (!dfn[v]) {
tarjan(v);
low[u] = min(low[u], low[v]);
if (low[v] >= dfn[u]) {
if (++f > 1 || u != root) cut[u] = true;
cnt++;
do dcc[cnt].push_back(sta[top--]);
while (sta[top + 1] != v);
dcc[cnt].push_back(u);
}
} else
low[u] = min(low[u], dfn[v]);
}
}

int main() {
cin.tie(nullptr)->sync_with_stdio(false);
cin >> n >> m;
int u, v;
for (int i = 1; i <= m; i++) {
cin >> u >> v;
if (u != v) uadd(u, v);
}
for (int i = 1; i <= n; i++)
if (!dfn[i]) root = i, tarjan(i);
cout << cnt << '\n';
for (int i = 1; i <= cnt; i++) {
cout << dcc[i].size() << ' ';
for (int j = 0; j < dcc[i].size(); j++) cout << dcc[i][j] << ' ';
cout << '\n';
}
```

```
return 0;
}
```

差分算法



如上图所示，黑色边为树边，红色边为非树边，每一条非树边的两个端点都唯一对应了树上由树边构成的的一条简单路径。

考虑一张新图，新图中的每一个点对应原图中的每一条树边（在图中用蓝色点表示）。对于原图中的每一条非树边，将这条非树边对应的树上简单路径中的所有边在新图中对应的蓝点连成一个连通块（在图中用蓝色的边体现出来）。

这样，一个点若 **不是** 割点，当且仅当与其相连的所有边在新图中对应的蓝点都 **属于** 同一个连通块。

两个点 **是** 点双连通，当且仅当它们在原图的树上路径中的所有边在新图中对应的蓝点都 **属于** 同一个连通块，即图中的每个蓝点构成的连通块都是一个点双连通分量。

蓝点间的连通关系可以用与求边双连通时用到的差分类似的方法维护，时间复杂度 $O(n+m)$ 。

例题：

Status

Problem

Tags

P8435

.

【模板】点双连通分量

图论
P3225
.
矿场搭建
图论
割点与割边
树的直径与重心

图论 10 • 树的直径与重心

树的直径定义

树上任意两节点之间最长的简单路径即为树的「直径」。

引入

显然，一棵树可以有多条直径，他们的长度相等。

可以用两次 DFS 或者树形 DP 的方法在 O(n) 时间求出树的直径。

例题

给定一棵 n 个节点的树，求其直径的长度。 1 <= n <= 10^4 。

做法 1. 两次 DFS

过程

首先从任意节点 y 开始进行第一次 DFS，到达距离其最远的节点，记为 z ，然后再从 z 开始做第二次 DFS，到达距离 z 最远的节点，记为 z’ ，则 delta(z,z’) 即为树的直径。

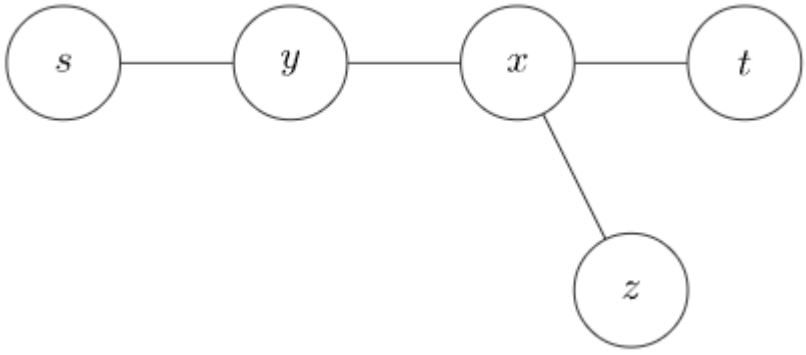
显然，如果第一次 DFS 到达的节点 z 是直径的一端，那么第二次 DFS 到达的节点 z’ 一定是直径的一端。我们只需证明在任意情况下， z 必为直径的一端。

定理：在一棵树上，从任意节点 y 开始进行一次 DFS，到达的距离其最远的节点 z 必为直径的一端。

证明

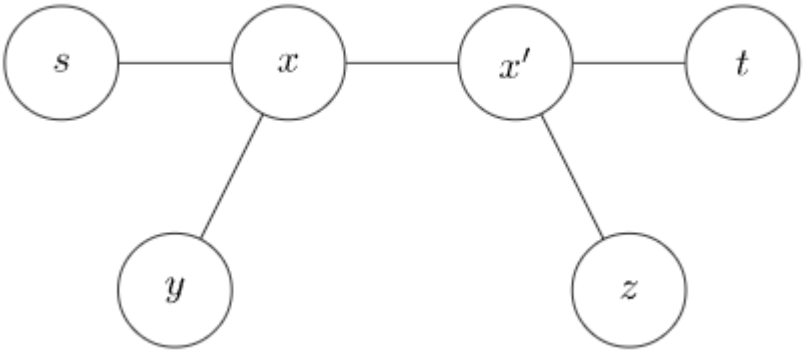
使用反证法。记出发节点为 y 。设真实的直径是 delta(s,t) ，而从 y 进行的第一次 DFS 到达的距离其最远的节点 z 不为 t 或 s 。共分三种情况：

- 若 y 在 delta(s,t) 上：



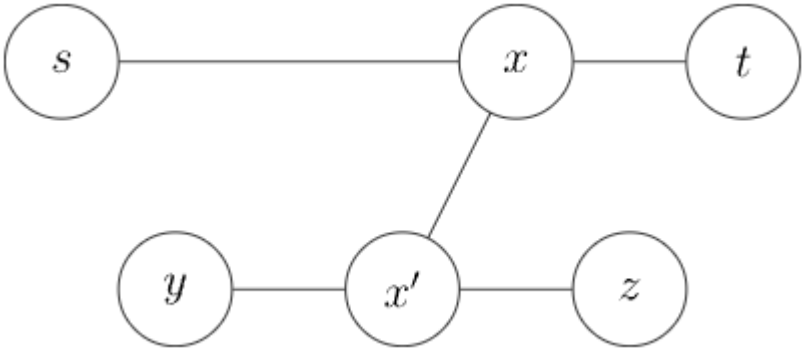
有 delta(y,z) > delta(y,t) Longrightarrow delta(x,z) > delta(x,t) Longrightarrow delta(s,z) > delta(s,t) ，与 delta(s,t) 为树上任意两节点之间最长的简单路径矛盾。

- 若 y 不在 delta(s,t) 上，且 delta(y,z) 与 delta(s,t) 存在重合路径：



有 delta(y,z) > delta(y,t) Longrightarrow delta(x,z) > delta(x,t) Longrightarrow delta(s,z) > delta(s,t) ，与 delta(s,t) 为树上任意两节点之间最长的简单路径矛盾。

- 若 y 不在 delta(s,t) 上，且 delta(y,z) 与 delta(s,t) 不存在重合路径：



有 delta(y,z) > delta(y,t) Longrightarrow delta(x',z) > delta(x',t) Longrightarrow delta(x,z) > delta(x,t) Longrightarrow delta(s,z) > delta(s,t) ，与 delta(s,t) 为树上任意两节点之间最长的简单路径矛盾。

综上，三种情况下假设均会产生矛盾，故原定理得证。

负权边

上述证明过程建立在所有路径均不为负的前提下。如果树上存在负权边，则上述证明不成立。故若存在负权边，则无法使用两次 DFS 的方式求解直径。

实现

代码实现如下。

```
constexpr int N = 10000 + 10;

int n, c, d[N];
vector<int> E[N];

void dfs(int u, int fa) {
for (int v : E[u]) {
if (v == fa) continue;
d[v] = d[u] + 1;
if (d[v] > d[c]) c = v;
dfs(v, u);
}
}

int main() {
scanf("%d", &n);
for (int i = 1; i < n; i++) {
int u, v;
scanf("%d %d", &u, &v);
E[u].push_back(v), E[v].push_back(u);
}
dfs(1, 0);
d[c] = 0, dfs(c, 0);
printf("%d\n", d[c]);
return 0;
}
```

如果需要求出一条直径上所有的节点，则可以在第二次 DFS 的过程中，记录每个点的前序节点，即可从直径的一端一路向前，遍历直径上所有的节点。

做法 2. 树形 DP

过程 1

我们记录当 1 为树的根时，每个节点作为子树的根向下，所能延伸的最长路径长度 d[1] 与次长路径（与最长路径无公共边）长度 d[2] ，那么直径就是对于每一个点，该点 d[1] + d[2] 能取到的值中的最大值。

树形 DP 可以在存在负权边的情况下求解出树的直径。

实现 1

代码实现如下：

```
constexpr int N = 10000 + 10;

int n, d = 0;
int d1[N], d2[N];
vector<int> E[N];

void dfs(int u, int fa) {
d1[u] = d2[u] = 0;
for (int v : E[u]) {
if (v == fa) continue;
dfs(v, u);
int t = d1[v] + 1;
if (t > d1[u])
d2[u] = d1[u], d1[u] = t;
else if (t > d2[u])
d2[u] = t;
}
d = max(d, d1[u] + d2[u]);
}

int main() {
scanf("%d", &n);
for (int i = 1; i < n; i++) {
int u, v;
scanf("%d %d", &u, &v);
E[u].push_back(v), E[v].push_back(u);
}
dfs(1, 0);
printf("%d\n", d);
return 0;
}
```

如果需要求出一条直径上所有的节点，则可以在 DP 的过程中，记录下每个节点能向下延伸的最长路径与次长路径（定义同上）所对应的子节点，在求 d 的同时记下对应的节点 u ，使得 d = d[1][u] + d[2][u] ，即可分别沿着从 u 开始的最长路径的次长路径对应的子节点一路向某个方向（对于无根树，虽然这里指定了 1 为树的根，但仍需记录每点跳转的方向；对于有根树，一路向上跳即可），遍历直径上所有的节点。

过程 2

这里提供一种只使用一个数组进行的树形 DP 方法。

我们定义 dp[u] ：以 u 为根的子树中，从 u 出发的最长路径。那么容易得出转移方程： dp[u] = max(dp[u], dp[v] + w(u, v)) ，其中的 v 为 u 的子节点， w(u, v) 表示所经过边的权重。

对于树的直径，实际上是可以通过枚举从某个节点出发不同的两条路径相加的最大值求出。因此，在 DP 求解的过程中，我们只需要在更新 dp[u] 之前，计算 d = max(d, dp[u] + dp[v] + w(u, v)) 即可算出直径 d 。

实现 2

代码实现如下：

```
constexpr int N = 10000 + 10;

int n, d = 0;
int dp[N];
vector<int> E[N];

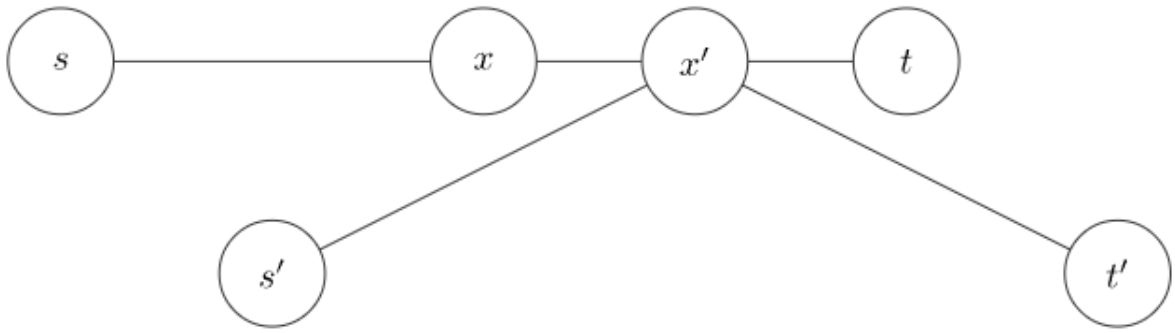
void dfs(int u, int fa) {
for (int v : E[u]) {
if (v == fa) continue;
dfs(v, u);
d = max(d, dp[u] + dp[v] + 1);
dp[u] = max(dp[u], dp[v] + 1);
}
}

int main() {
scanf("%d", &n);
for (int i = 1; i < n; i++) {
int u, v;
scanf("%d %d", &u, &v);
E[u].push_back(v), E[v].push_back(u);
}
dfs(1, 0);
printf("%d\n", d);
return 0;
}
```

性质

若树上所有边边权均为正，则树的所有直径中点重合

证明：使用反证法。设两条中点不重合的直径分别为 delta(s,t) 与 delta(s',t') ，中点分别为 x 与 x' 。显然， delta(s,x) = delta(x,t) = delta(s',x') = delta(x',t') 。



有 delta(s,t') = delta(s,x) + delta(x,x') + delta(x',t') > delta(s,x) + delta(x,t) = delta(s,t) ，与 delta(s,t) 为树上任意两节点之间最长的简单路径矛盾，故性质得证。

树的重心定义

如果在树中选择某个节点并删除，这棵树将分为若干棵子树，统计子树节点数并记录最大值。取遍树上所有节点，使此最大值取到最小的节点被称为整个树的重心。

（这里以及下文中的「子树」若无特殊说明都是指无根树的子树，即包括「向上」的那棵子树，并且不包括整棵树自身。）

性质

- 树的重心如果不唯一，则至多有两个，且这两个重心相邻。
- 以树的重心为根时，所有子树的大小都不超过整棵树大小的一半。
- 树中所有点到某个点的距离和中，到重心的距离和是最小的；如果有两个重心，那么到它们的距离和一样。
- 把两棵树通过一条边相连得到一棵新的树，那么新的树的重心在连接原来两棵树的重心的路径上。

- 在一棵树上添加或删除一个叶子，那么它的重心最多只移动一条边的距离。

求法

在 DFS 中计算每个子树的大小，记录「向下」的子树的最大大小，利用总点数 - 当前子树（这里的子树指有根树的子树）的大小得到「向上」的子树的大小，然后就可以依据定义找到重心了。

```
// 这份代码默认节点编号从 1 开始，即 i ∈ [1,n]
int size[MAXN];  // 这个节点的「大小」（所有子树上节点数 + 该节点）
weight[MAXN];  // 这个节点的「重量」，即所有子树「大小」的最大值
centroid[2];  // 用于记录树的重心（存的是节点编号）

void GetCentroid(int cur, int fa) {  // cur 表示当前节点 (current)
size[cur] = 1;
weight[cur] = 0;
for (int i = head[cur]; i != -1; i = e[i].nxt) {
if (e[i].to != fa) {  // e[i].to 表示这条有向边所通向的节点。
GetCentroid(e[i].to, cur);
size[cur] += size[e[i].to];
weight[cur] = max(weight[cur], size[e[i].to]);
}
}
weight[cur] = max(weight[cur], n - size[cur]);
if (weight[cur] <= n / 2) {  // 依照树的重心的定义统计
centroid[centroid[0] != 0] = cur;
}
}
```

例题：

Status
Problem
Tags
P3304
.
直径
图论
P1395
.
会议
图论
P2491
.
[SDOI2011] 消防
图论
双连通分量
最近公共祖先 LCA

图论 11 • 最近公共祖先 LCA

定义

最近公共祖先简称 LCA（Lowest Common Ancestor）。两个节点的最近公共祖先，就是这两个点的公共祖先里面，离根最远的那个。

为了方便，我们记某点集 S=v[1],v[2],…,v[n] 的最近公共祖先为 LCA(v[1],v[2],…,v[n]) 或 LCA(S) 。

性质

1. LCA(u)=u ；

2. u 是 v 的祖先，当且仅当 LCA(u,v)=u ；
3. 如果 u 不为 v 的祖先并且 v 不为 u 的祖先，那么 u,v 分别处于 LCA(u,v) 的两棵不同子树中；
4. 前序遍历中， LCA(S) 出现在所有 S 中元素之前，后序遍历中 LCA(S) 则出现在所有 S 中元素之后；
5. 两点集并的最近公共祖先为两点集分别的最近公共祖先的最近公共祖先，即 LCA(A 并 B)=LCA(LCA(A), LCA(B)) ；
6. 两点的最近公共祖先必定处在树上两点间的最短路上；
7. d(u,v)=h(u)+h(v)-2h(LCA(u,v)) ，其中 d 是树上两点间的距离， h 代表某点到树根的距离。

求法

朴素算法

过程

可以每次找深度比较大的那个点，让它向上跳。显然在树上，这两个点最后一定会相遇，相遇的位置就是想要求的 LCA。

或者先向上调整深度较大的点，令他们深度相同，然后再共同向上跳转，最后也一定会相遇。

性质

朴素算法预处理时需要 dfs 整棵树，时间复杂度为 O(n) ，单次查询时间复杂度为 Theta(n) 。如果树满足随机性质，则时间复杂度与这种随机树的期望高度有关。

倍增算法

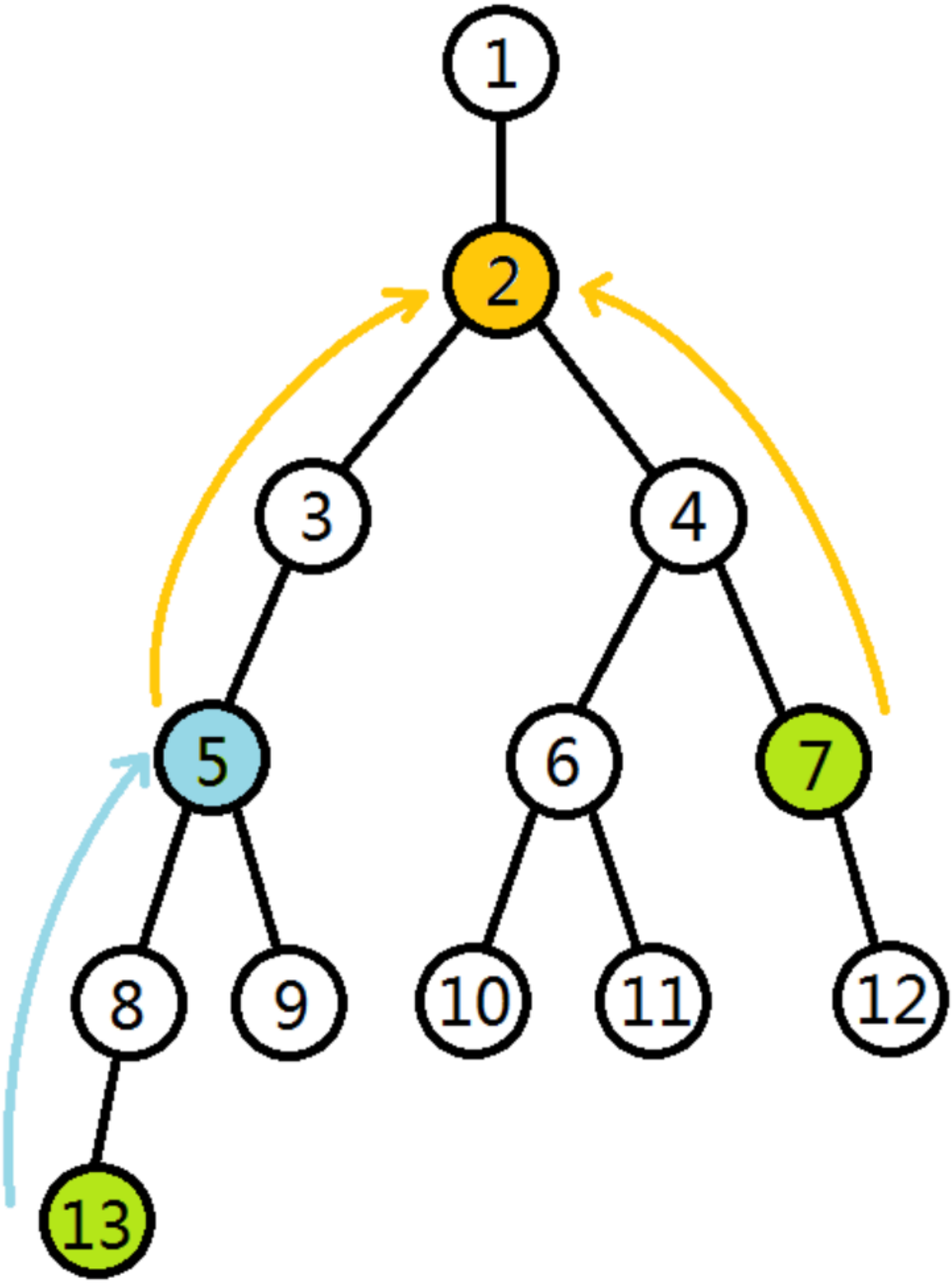
过程

倍增算法是最经典的 LCA 求法，他是朴素算法的改进算法。通过预处理 fa_(x,i) 数组，游标可以快速移动，大幅减少了游标跳转次数。 fa_(x,i) 表示点 x 的第 2^i 个祖先。 fa_(x,i) 数组可以通过 dfs 预处理出来。

现在我们看看如何优化这些跳转：

在调整游标的第一阶段中，我们要将 u,v 两点跳转到同一深度。我们可以计算出 u,v 两点的深度之差，设其为 y 。通过将 y 进行二进制拆分，我们将 y 次游标跳转优化为「 y 的二进制表示所含 `1` 的个数」次游标跳转。

在第二阶段中，我们从最大的 i 开始循环尝试，一直尝试到 0 （包括 0 ），如果 fa_(u,i)not=fa_(v,i) ，则 ugetsfa_(u,i),vgetsfa_(v,i) ，那么最后的 LCA 为 fa_(u,0) 。



复杂度

倍增算法的预处理时间复杂度为 O(n log n) ，单次查询时间复杂度为 O(log n) 。

另外倍增算法可以通过交换 `fa` 数组的两维使较小维放在前面。这样可以减少 cache miss 次数，提高程序效率。

倍增求 LCA 模板

```
#include <cstring>
#include <iostream>
#include <vector>

constexpr int MXN = 40005;
using namespace std;
vector<int> v[MXN];
vector<int> w[MXN];

int fa[MXN][31], cost[MXN][31], dep[MXN];
int n, m;
int a, b, c;

// dfs，用来为 lca 算法做准备，接受两个参数：dfs 起始节点和它的父亲节点。
void dfs(int root, int fno) {
// 初始化：第 2^0 = 1 个祖先就是它的父亲节点，dep 也比父亲节点多 1。
fa[root][0] = fno;
dep[root] = dep[fa[root][0]] + 1;
// 初始化：其他的祖先节点：第 2^i 的祖先节点是第 2^(i-1) 的祖先节点的第
// 2^(i-1) 的祖先节点。
for (int i = 1; i < 31; ++i) {
fa[root][i] = fa[fa[root][i - 1]][i - 1];
cost[root][i] = cost[fa[root][i - 1]][i - 1] + cost[root][i - 1];
}
// 遍历子节点来进行 dfs。
int sz = v[root].size();
for (int i = 0; i < sz; ++i) {
if (v[root][i] == fno) continue;
cost[v[root][i]][0] = w[root][i];
dfs(v[root][i], root);
}
}

// lca，用倍增算法算取 x 和 y 的 lca 节点。
int lca(int x, int y) {
// 令 y 比 x 深。
if (dep[x] > dep[y]) swap(x, y);
// 令 y 和 x 在一个深度。
int tmp = dep[y] - dep[x], ans = 0;
for (int i = 0; tmp; ++i, tmp >>= 1)
if (tmp & 1) ans += cost[y][i], y = fa[y][i];
// 如果这个时候 y == x，那么 x，y 就都是它们自己的祖先。
if (y == x) return ans;
// 不然的话，找到第一个不是它们祖先的两个点
for (int i = 30; i >= 0 && y != x; --j) {
if (fa[x][i] != fa[y][i]) {
ans += cost[x][j] + cost[y][j];
x = fa[x][i];
y = fa[y][j];
}
}
// 返回结果。
ans += cost[x][0] + cost[y][0];
return ans;
}

void Solve() {
cin.tie(nullptr)->sync_with_stdio(false);
// 初始化表示祖先的数组 fa，代价 cost 和深度 dep。
memset(fa, 0, sizeof(fa));
memset(cost, 0, sizeof(cost));
memset(dep, 0, sizeof(dep));
// 读入树，节点数一共有 n 个，查询 m 次，每一次查找两个节点的 lca 点。
cin >> n >> m;
// 初始化树边和边权
```

```
for (int i = 1; i <= n; ++i) {
v[i].clear();
w[i].clear();
}
for (int i = 1; i < n; ++i) {
cin >> a >> b >> c;
v[a].push_back(b);
v[b].push_back(a);
w[a].push_back(c);
w[b].push_back(c);
}
// 为了计算 lca 而使用 dfs。
dfs(1, 0);
for (int i = 0; i < m; ++i) {
cin >> a >> b;
cout << lca(a, b) << '\n';
}
}

int main() {
cin.tie(nullptr)->sync_with_stdio(false);
int T;
cin >> T;
while (T--) Solve();
return 0;
}
```

例题：

Status

Problem

Tags

P3379

.

【模板】最近公共祖先（LCA）

图论

P1351

.

联合权值

图论

P1967

.

货车运输

图论

P3950

.

部落冲突

图论

树的直径与重心

树上前缀和与差分

图论 12 · 树上前缀和与差分

树上前缀和定义

设 sum[i] 表示结点 i 到根节点的权值总和。

然后：

- 若是点权， x,y 路径上的和为 sum[x] + sum_(y) - sum_(l)ca - sum_(fa_(l)ca) 。

- 若是边权， x,y 路径上的和为 sum[x] + sum_(y) - 2cdotsum_(lca) 。 LCA 的求法参见最近公共祖先。

差分

解释

差分是一种和前缀和相对的策略，可以当做是求和的逆运算。

这种策略的定义是令 b[i]=a[i]-a[i-1] i 属于 [2,n]

a[1] i=1

性质

- a[i] 的值是 b[i] 的前缀和，即 a[n]= sum limits_(i=1)^nb[i]
- 计 算 a[i] 的 前 缀 和 sum= sum limits_(i=1)^na[i]= sum limits_(i=1)^n sum limits_(j=1)^ib[j]= sum limits_(i=1)^n(n-i+1)b[i]

它可以维护多次对序列的一个区间加上一个数，并在最后询问某一位的数或是多次询问某一位的数。

注意修改操作一定要在查询操作之前。

示例

譬如使 [l,r] 中的每个数加上一个 k ，即

b_(l) <- b_(l) + k,b_(r + 1) <- b_(r + 1) - k

其中 b_(l)+k=a_(l)+k-a_(l-1) ， b[r+1]-k=a[r+1]-(a[r]+k)

最后做一遍前缀和就好了。

树上差分

树上差分可以理解为对树上的某一段路径进行差分操作，这里的路径可以类比一维数组的区间进行理解。例如在对树上的一些路径进行频繁操作，并且询问某条边或者某个点在经过操作后的值的时候，就可以运用树上差分思想了。

树上差分通常会结合最近公共祖先来进行考察。树上差分又分为 **点差分** 与 **边差分**，在实现上会稍有不同。

点差分

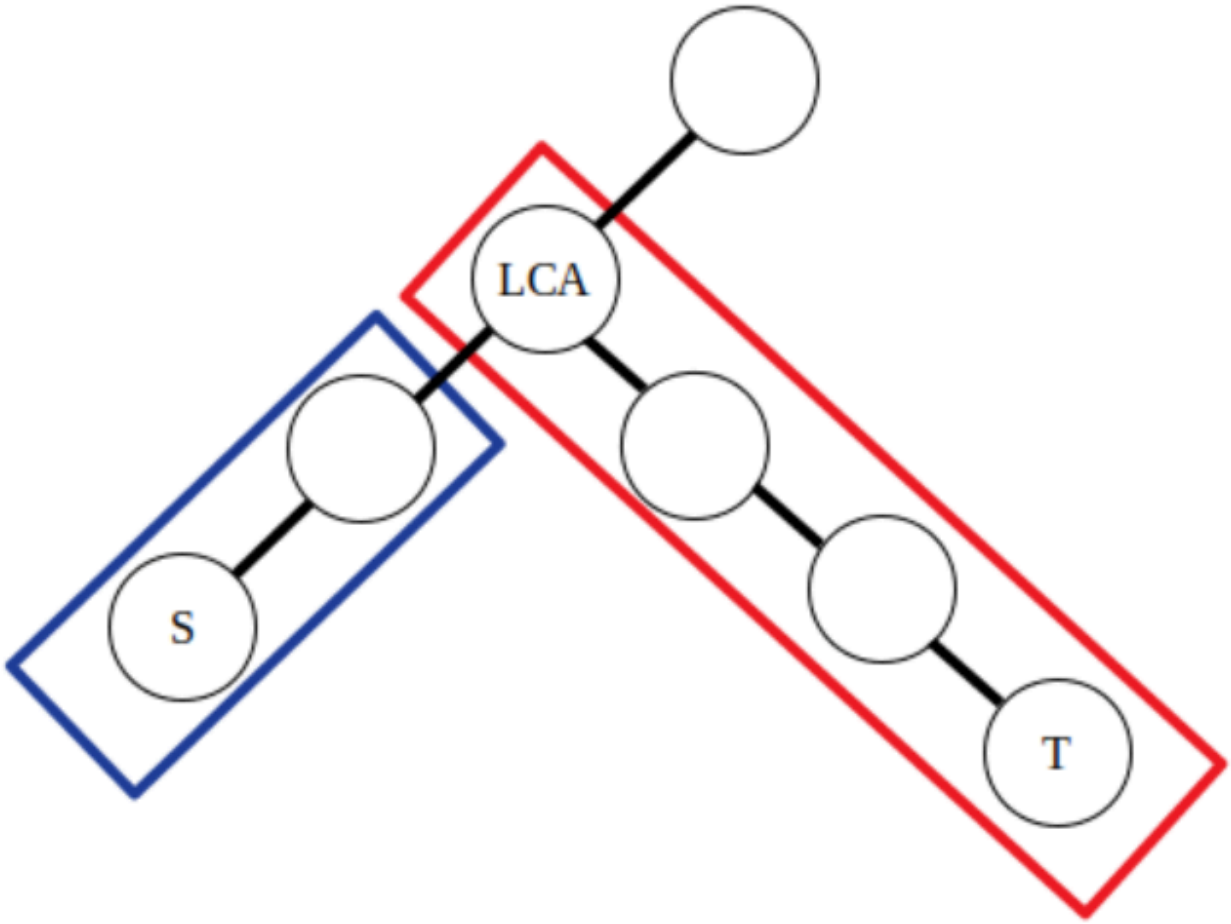
举例：对树上的一些路径 delta(s[1],t[1]), delta(s[2],t[2]), delta(s[3],t[3])… 进行访问，问一条路径 delta(s,t) 上的点被访问的次数。

对于一次 delta(s,t) 的访问，需要找到 s 与 t 的公共祖先，然后对这条路径上的点进行访问（点的权值加一），若采用 DFS 算法对每个点进行访问，由于有太多的路径需要访问，时间上承受不了。

这里进行差分操作：

d[s] <- d[s]+1 d_(lca) <- d_(lca)-1 d[t] <- d[t]+1 d_(f(lca)) <- d_(f(lca))-1

其中 f(x) 表示 x 的父亲节点， d[i] 为点权 a[i] 的差分数组。

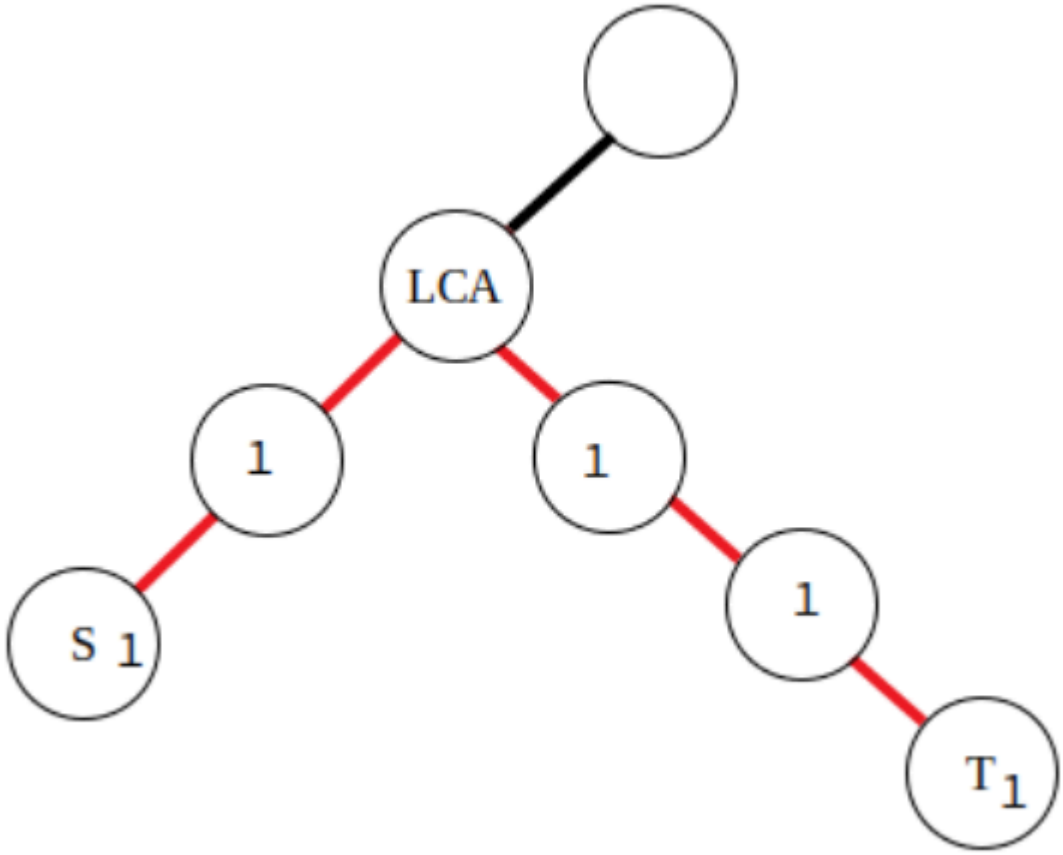


可以认为公式中的前两条是对蓝色方框内的路径进行操作，后两条是对红色方框内的路径进行操作。不妨令 lca 左侧的直系子节点为 left 。那么有 d_(lca)-1=a_(lca)-(a_(left)+1) ，d_(f(lca))-1=a_(f(lca))-(a_(lca)+1) 。可以发现实际上点差分的操作和上文一维数组的差分操作是类似的。

边差分

若是对路径中的边进行访问，就需要采用边差分策略了，使用以下公式：

d[s] <- d[s]+1 d[t] <- d[t]+1 d_(lca) <- d_(lca)-2



由于在边上直接进行差分比较困难，所以将本来应当累加到红色边上的值向下移动到附近的点里，那么操作起来也就方便了。对于公式，有了点差分的理解基础后也不难推导，同样是对两段区间进行差分。

例题

完整题面：Max Flow P

FJ 给他的牛棚的 N(2 <= N <= 50,000) 个隔间之间安装了 N-1 根管道，隔间编号从 1 到 N 。所有隔间都被管道连通了。

FJ 有 K(1 <= K <= 100,000) 条运输牛奶的路线，第 i 条路线从隔间 s[i] 运输到隔间 t[i] 。一条运输路线会给它的两个端点处的隔间以及中间途径的所有隔间带来一个单位的运输压力，你需要计算压力最大的隔间的压力是多少。

解题思路

需要统计每个点经过了多少次，那么就用树上差分将每一次的路径上的点加一，可以很快得到每个点经过的次数。这里采用倍增法计算 LCA，最后对 DFS 遍历整棵树，在回溯时对差分数组求和就能求得答案了。

参考代码

```
#include <algorithm>
#include <iostream>
using namespace std;

constexpr int MAXN = 50010;

struct node {
int to, next;
} edge[MAXN << 1];

int fa[MAXN][30], head[MAXN << 1];
int power[MAXN];
int depth[MAXN], lg[MAXN];
int n, k, ans = 0, tot = 0;

void add(int x, int y) {  // 加边
edge[++tot].to = y;
edge[tot].next = head[x];
head[x] = tot;
}

void dfs(int now, int father) {  // dfs 求最大压力
fa[now][0] = father;
depth[now] = depth[father] + 1;
for (int i = 1; i <= lg[depth[now]]; ++i)
fa[now][i] = fa[fa[now][i - 1]][i - 1];
for (int i = head[now]; i; i = edge[i].next)
if (edge[i].to != father) dfs(edge[i].to, now);
}

int lca(int x, int y) {  // 求 LCA, 最近公共祖先
if (depth[x] < depth[y]) swap(x, y);
while (depth[x] > depth[y]) x = fa[x][lg[depth[x] - depth[y]] - 1];
if (x == y) return x;
for (int k = lg[depth[x]] - 1; k >= 0; k--) {
if (fa[x][k] != fa[y][k]) x = fa[x][k], y = fa[y][k];
}
return fa[x][0];
}

// 用 dfs 求最大压力, 回溯时将子树的权值加上
void get_ans(int u, int father) {
for (int i = head[u]; i; i = edge[i].next) {
int to = edge[i].to;
if (to == father) continue;
get_ans(to, u);
power[u] += power[to];
}
ans = max(ans, power[u]);
}

int main() {
cin.tie(nullptr)->sync_with_stdio(false);
cin >> n >> k;
int x, y;
for (int i = 1; i <= n; i++) {
lg[i] = lg[i - 1] + (1 << lg[i - 1] == i);
}
for (int i = 1; i <= n - 1; i++) {  // 建图
cin >> x >> y;
add(x, y);
add(y, x);
}
dfs(1, 0);
int s, t;
```

```
for (int i = 1; i <= k; i++) {
cin >> s >> t;
int ancestor = lca(s, t);
// 树上差分
power[s]++;
power[t]++;
power[ancestor]--;
power[fa[ancestor][0]]--;
}
get_ans(1, 0);
cout << ans << '\n';
return 0;
}
```

例题：

Status

Problem

Tags

P8805

.

机房

图论

P8201

.

生活在树上

图论

最近公共祖先 LCA

最长路

图论 13 · 最长路

最长路

最长路是与最短路相对应的概念，最短路是指路径长度最短的路径，最长路则是路径长度最长的路径。

最长路的求解方法主要有两种：SPFA 算法和拓扑排序算法。拓扑排序算法的执行效率通常显著高于 SPFA 算法。需要说明的是，尽管 SPFA 算法存在容易被特殊数据构造卡时间的情况，但该算法在处理负边权问题上具有独特价值，建议学习者掌握该算法。

SPFA 算法

SPFA 算法能够处理包含负边权的图结构。由于负数的特性，数值越小其绝对值越大，因此可以将最长路问题转化为最短路问题求解。具体方法为：在存储边权时取其相反数，通过最短路算法求解后，再将结果取相反数即可得到最长路的数值。

拓扑排序算法

拓扑排序算法的应用存在一定限制，仅适用于有向无环图，无法直接处理无向图等结构。该算法需要结合动态规划（DP）思想实现，具体思路可通过代码实现进行理解。以下通过例题进行讲解：

题面：最长路

SPFA 求最长路示例代码

```
#include <bits/stdc++.h>
using namespace std;
int n,m,u,v,w,tot;
int dis[510010],vis[510010],head[510010];

struct node {
int to,net,val;
} e[510010];

inline void add(int u,int v,int w) {
e[++tot].to=v;
e[tot].net=head[u];
e[tot].val=w;
head[u]=tot;
}
//链式前向星建边
inline void spfa() {
queue<int> q;
for(register int i=1;i<=n;i++) dis[i]=20050206;
dis[1]=0;
vis[1]=1;
q.push(1);
while(!q.empty()) {
int x=q.front();
q.pop();
vis[x]=0;
for(register int i=head[x];i;i=e[i].net) {
int v=e[i].to;
if(dis[v]>dis[x]+e[i].val) {
dis[v]=dis[x]+e[i].val;
if(!vis[v]) {
vis[v]=1;
q.push(v);
}
}
}
}//正常跑最短路

int main() {
scanf("%d%d",&n,&m);
for(register int i=1;i<=m;i++) {
scanf("%d%d%d",&u,&v,&w);
add(u,v,-w);//存一个相反数
}
spfa();
if(dis[n]==20050206) puts("-1"); //到不了就 -1
else printf("%d",-dis[n]);//记得存回来
return 0;
}
```

拓扑排序求最长路示例代码

```
#include<bits/stdc++.h>
using namespace std;
const int MAXN=2*5*1e4;
int n,m;
struct edge{
int net,to,w;
}e[MAXN];
int head[MAXN],tot;
void add(int x,int y,int z){
e[++tot].net=head[x];
e[tot].to=y;
e[tot].w=z;
head[x]=tot;
}
//链式前向星建边
bool v[MAXN];
//用来标记是否可以从 1 走到这个点
//因为是 1 到 n，所以如果不能从 1 开始走
//说明不满足条件，没有这条最长路
int ru[MAXN];
int ans[MAXN];
queue<int>q;
void toop(){
for(register int i=1;i<=n;i++){
if(ru[i]==0) q.push(i);
}//入度为 0 的进队
while(!q.empty()){
int x=q.front();
q.pop();//出队
for(register int i=head[x];i;i=e[i].net){
int y=e[i].to,z=e[i].w;
ru[y]--;//入度--
if(v[x]==true){
ans[y]=max(ans[y],ans[x]+z);
v[y]=true;
}//如果这个节点能从 1 走到，说明它的边可以走
//更新最长路
if(ru[y]==0) q.push(y);//进队
}
}
}
int main(){
scanf("%d%d",&n,&m);
for(register int i=1;i<=m;i++){
int u,v,w;
scanf("%d%d%d",&u,&v,&w);
add(u,v,w);
ru[v]++;
}//建边，入度++
v[1]=true;//1 肯定自己能走
ans[n]=-1;//初始值为 -1，方便输出
toop();//拓扑排序求最长路
cout<<ans[n];
return 0;
}
```

例题：

Status

Problem

Tags

P1807

.

最长路
图论
树上前缀和与差分
单源次短路径

图论 14 • 单源次短路径

非严格次短路

在理解最短路和最长路的概念后，次短路的概念可定义为：除最短路之外的最短路径（允许与最短路长度相等）。

本文仅介绍一种常用方法，A*算法可用于求解 K 短路问题，因复杂度较高暂不展开。次短路的求解依赖于最短路的结果，需首先计算出最短路。

次短路的长度必然大于或等于最短路长度。通过分析可知，次短路至少存在一条边不属于最短路径（可能与最短路径无交集）。基于此特性，求解次短路的思路为：记录最短路径中的边，依次删除最短路径中的每条边后重新计算最短路，通过比较所有结果得到次短路。以下通过例题具体说明：

例题：集合位置

该问题为次短路的典型模板题。解题步骤如下：

1、首先构建图结构（注意使用双精度浮点型存储边权及结果，并进行精度处理）

2、然后执行“计算最短路 -> 记录路径 -> 枚举删除路径边并重新计算最短路 -> 处理结果”的流程。

需注意特殊情况处理：若存在多条最短路径，删除部分边后仍可能得到最短路；若起点与终点间仅有一条简单路径（即最短路），删除边后可能导致无法到达终点，此时次短路无解。

```
#include<bits/stdc++.h>
using namespace std;
const int MAXN=1e7+50;
const double INF=200500305;
int n,m;
int x[MAXN],y[MAXN];
struct node{
int net,to,from;
double w;
}e[MAXN];
int head[MAXN],tot;
void add(int u,int v,double w){
e[++tot].net=head[u];
e[tot].to=v;
e[tot].from=u;
//这里的 from 和 to 表示这一条边的两个端点
//在后面的程序中用来比较求次短路
e[tot].w=w;
head[u]=tot;
}
//链式前向星建边
double d[MAXN];
int bian[MAXN]; //记录最短路
bool v[MAXN];
inline bool ok(int i,int j){
if(min(e[i].to,e[i].from)==min(e[j].to,e[j].from)&&max(e[i].to,e[i].from)==max(e[j].to,e[j].fro
m))return 0;
return 1;
}//这一坨长长的东西用来判断是不是我这次要删掉的边
void dij(int s,int p){ //p 用来表示删除哪一条边
priority_queue<pair<double,int> >q;
for(register int i=1;i<=n;i++) d[i]=INF,v[i]=false;
d[s]=0; //初始化
q.push(make_pair(0,s));
while(!q.empty()){
int x=q.top().second;
q.pop();
if(v[x]==true) continue;
v[x]=true;
for(register int i=head[x];i;i=e[i].net)
if(p==-1||ok(i,p)){  //如果是第一次跑最短路就记录路径，如果是该边被删去就不跑
int y=e[i].to;
double z=e[i].w;
if(d[y]>d[x]+z){
d[y]=d[x]+z;
if(p==-1)bian[y]=i; //第一次跑最短路记录路径
q.push(make_pair(-d[y],y));
}
}
}
double Min(double x,double y){
if(x<=y) return x;
return y;
} //c++ 自带的 min 不支持 double 类型的比较
int main(){
scanf("%d%d",&n,&m);
for(register int i=1;i<=n;i++) scanf("%d%d",&x[i],&y[i]);
for(register int i=1;i<=m;i++){
int u,v;
double w;
scanf("%d%d",&u,&v);
w=(double)sqrt((x[u]-x[v])*(x[u]-x[v])+(y[u]-y[v])*(y[u]-y[v]));
add(u,v,w);
add(v,u,w);
```

```
}//建双向变
dij(1,-1); //第一次跑最短路不删边
int t=n; //用 t 来代替 n，遍历最短路的边
double ans=INF;
while(t!=1){
int i=bian[t];
dij(1,i);
ans=min(ans,d[n]); //取一个更小的答案表示次短路
t=e[bian[t]].from; //遍历最短路的路径
}
printf("%.2lf",ans); //输出答案
return 0;
}
```

严格次短路

使用非严格次短路的算法求解此题时会出现错误，因该问题要求严格次短路（长度必须严格大于最短路）。

严格次短路的求解需确保结果严格大于最短路长度。

具体方法为：预处理双向最短路，使用 d1 数组存储起点到各点的最短路长度，d2 数组存储各点到终点的最短路长度。通过枚举所有边，结合双向最短路数据计算严格次短路。

```
#include<bits/stdc++.h>
#define M 1000051
#define N 5005
using namespace std;
int n,m;
struct node{
int net,to,z;
}e[M];
int head[N],tot;
queue<int>q;
int d1[N],d2[N]; //同上
bool v[N];
void add(int x,int y,int z){
e[++tot].net=head[x];
e[tot].to=y;
e[tot].z=z;
head[x]=tot;
}
void spfas(int s){
for(register int i=1;i<=n;i++){
d1[i]=20040915,v[i]=false;
}
d1[s]=0;
v[s]=true;
q.push(s);
while(!q.empty()){
int x=q.front();
q.pop();
v[x]=false;
for(register int i=head[x];i;i=e[i].net){
int y=e[i].to,z=e[i].z;
if(d1[y]>d1[x]+z){
d1[y]=d1[x]+z;
if(v[y]==false){
v[y]=true;
q.push(y);
}
}
}
}
void spfae(int s){
for(register int i=1;i<=n;i++){
d2[i]=20040915,v[i]=false;
}
d2[s]=0;
v[s]=true;
q.push(s);
while(!q.empty()){
int x=q.front();
q.pop();
v[x]=false;
for(register int i=head[x];i;i=e[i].net){
int y=e[i].to,z=e[i].z;
if(d2[y]>d2[x]+z){
d2[y]=d2[x]+z;
if(v[y]==false){
v[y]=true;
q.push(y);
}
}
}
}
int main(){
```

```
scanf("%d%d" &n &m);
for(register int i=1;i<=m;i++){
int x y z;
scanf("%d%d%d",&x,&y,&z);
add(x y z);
add(y,x,z);
}
spfas(1); //预处理 d1
spfae(n); //预处理 d2
int minn=d1[n] now=20040915;
//记录最短路，并将次短路改为极大值
for(register int i=1;i<=n;i++){
for(register int j=head[i];j;j=e[j].net){ //枚举每一个点的出边
if(d1[i]+d2[e[j].to]+e[j].z>minn) now=min(now,d1[i]+d2[e[j].to]+e[j].z);
//如果比最短路大，那么更新次短路
}
}
cout<<now;
return 0;
}
```

例题：

Status

Problem

Tags

P2176

.

RoadBlock S

图论

最长路

Prufer 序列与 Caley 公式

图论 15 • Prufer 序列与 Caley 公式

本文介绍 Prüfer 序列 (Prüfer code)，这是一种将带标号的树用一个唯一的整数序列表示的方法。使用 Prüfer 序列可以证明凯莱 Caley 公式。并且我们也会讲解如何计算在一个图中加边使图连通的方案数。

注意：我们不考虑含有 1 个结点的树。

Prüfer 序列

引入

Prüfer 序列可以将一个带标号 n 个结点的树用 [1,n] 中的 n-2 个整数表示。你也可以把它理解为完全图的生成树与数列之间的双射。常用组合计数问题中。

对树建立 Prüfer 序列

Prüfer 是这样建立的：每次选择一个编号最小的叶结点并删掉它，然后在序列中记录下它连接到的那个结点。重复 n-2 次后就只剩下两个结点，算法结束。

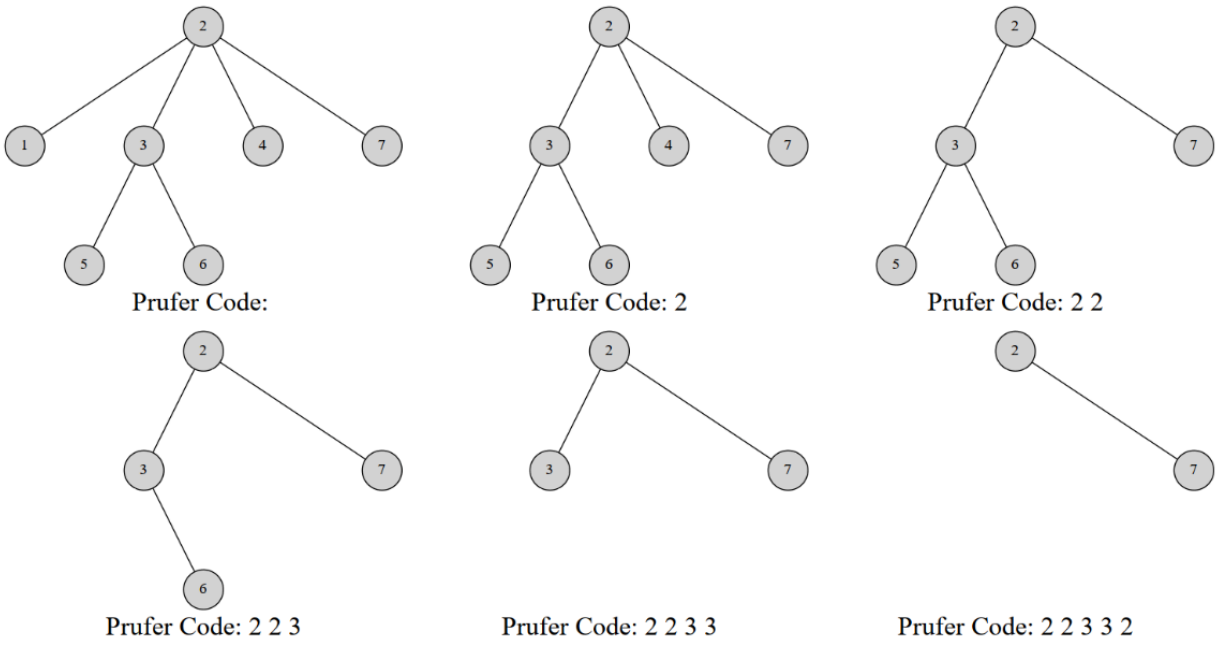
显然使用堆可以做到 O(nlog n) 的复杂度

```
//注意本代码节点从0开始标号
vector<vector<int>> adj;

vector<int> pruefer_code() {
int n = adj.size();
set<int> leafs;
vector<int> degree(n);
vector<bool> killed(n);
for (int i = 0; i < n; i++) {
degree[i] = adj[i].size();
if (degree[i] == 1) leafs.insert(i);
}

vector<int> code(n - 2);
for (int i = 0; i < n - 2; i++) {
int leaf = *leafs.begin();
leafs.erase(leafs.begin());
killed[leaf] = true;
int v;
for (int u : adj[leaf])
if (!killed[u]) v = u;
code[i] = v;
if (--degree[v] == 1) leafs.insert(v);
}
return code;
}
```

例如，这是一棵 7 个结点的树的 Prüfer 序列构建过程：



最终的序列就是 2,2,3,3,2 。

当然，也有一个线性的构造算法。

Prüfer 序列的线性构造算法

线性构造的本质就是维护一个指针指向我们将要删除的结点。首先发现，叶结点数是非严格单调递减的，删去一个叶结点，叶结点总数要么不变要么减 1。

于是我们考虑这样一个过程：维护一个指针 p 。初始时 p 指向编号最小的叶结点。同时我们维护每个结点的度数，方便我们知道在删除结点的时侯是否产生新的叶结点。操作如下：

1. 删除 p 指向的结点，并检查是否产生新的叶结点。
2. 如果产生新的叶结点，假设编号为 x ，我们比较 p,x 的大小关系。如果 x>p ，那么不做其他操作；否则就立刻删除 x ，然后检查删除 x 后是否产生新的叶结点，重复 2 步骤，直到未产生新

节点或者新节点的编号 >p 。

3. 让指针 p 自增直到遇到一个未被删除叶结点为止；

正确性

循环上述操作 n-2 次，就完成了序列的构造。接下来考虑算法的正确性。

p 是当前编号最小的叶结点，若删除 p 后未产生叶结点，我们就只能去寻找下一个叶结点；若产生了叶结点 x ：

- 如果 x>p ，则反正 p 往后扫描都会扫到它，于是不做操作；
- 如果 x<p ，因为 p 原本就是编号最小的，而 x 比 p 还小，所以 x 就是当前编号最小的叶结点，优先删除。删除 x 继续这样的考虑直到没有更小的叶结点。

算法复杂度分析，发现每条边最多被访问一次（在删度数的时侯），而指针最多遍历每个结点一次，因此复杂度是 O(n) 的。

实现

```
//同样以 0 为起点
vector<vector<int>> adj;
vector<int> parent;

void dfs(int v) {
for (int u : adj[v]) {
if (u != parent[v]) parent[u] = v, dfs(u);
}
}

vector<int> pruefer_code() {
int n = adj.size();
parent.resize(n), parent[n - 1] = -1;
dfs(n - 1);

int ptr = -1;
vector<int> degree(n);
for (int i = 0; i < n; i++) {
degree[i] = adj[i].size();
if (degree[i] == 1 && ptr == -1) ptr = i;
}

vector<int> code(n - 2);
int leaf = ptr;
for (int i = 0; i < n - 2; i++) {
int next = parent[leaf];
code[i] = next;
if (--degree[next] == 1 && next < ptr) {
leaf = next;
} else {
ptr++;
while (degree[ptr] != 1) ptr++;
leaf = ptr;
}
}
return code;
}
```

Prüfer 序列的性质

1. 在构造完 Prüfer 序列后原树中会剩下两个结点，其中一个一定是编号最大的点 n 。
2. 每个结点在序列中出现的次数是其度数减 1 。（没有出现的就是叶结点）

用 Prüfer 序列重建树

重建树的方法是类似的。根据 Prüfer 序列的性质，我们可以得到原树上每个点的度数。然后你也可以得到编号最小的叶结点，而这个结点一定与 Prüfer 序列的第一个数连接。然后我们同时删掉这两个结点的度数。

讲到这里也许你已经知道该怎么做了。每次我们选择一个度数为 1 的最小的结点编号，与当前枚举到的 Prüfer 序列的点连接，然后同时减掉两个点的度。到最后我们剩下两个度数为 1 的点，其中一个是结点 n 。就把它们建立连接。使用堆维护这个过程，在减度数的过程中如果发现度数减到 1 就把这个结点添加到堆中，这样做的复杂度是 O(nlog n) 的。

```
vector<pair<int, int>> pruefer_decode(vector<int> const& code) {
int n = code.size() + 2;
vector<int> degree(n, 1);
for (int i : code) degree[i]++;

set<int> leaves;
for (int i = 0; i < n; i++)
if (degree[i] == 1) leaves.insert(i);

vector<pair<int, int>> edges;
for (int v : code) {
int leaf = *leaves.begin();
leaves.erase(leaves.begin());

edges.emplace_back(leaf, v);
if (--degree[v] == 1) leaves.insert(v);
}
edges.emplace_back(*leaves.begin(), n - 1);
return edges;
}
```

线性时间重建树

同线性构造 Prüfer 序列的方法。在删度数的时侯会产生新的叶结点，于是判断这个叶结点与指针 p 的大小关系，如果更小就优先考虑它。

实现

```
vector<pair<int, int>> pruefer_decode(vector<int> const& code) {
int n = code.size() + 2;
vector<int> degree(n, 1);
for (int i : code) degree[i++];

int ptr = 0;
while (degree[ptr] != 1) ptr++;
int leaf = ptr;

vector<pair<int, int>> edges;
for (int v : code) {
edges.emplace_back(leaf, v);
if (--degree[v] == 1 && v < ptr) {
leaf = v;
} else {
ptr++;
while (degree[ptr] != 1) ptr++;
leaf = ptr;
}
}
edges.emplace_back(leaf, n - 1);
return edges;
}
```

通过这些过程其实可以理解，Prüfer 序列与带标号无根树建立了双射关系。

Cayley 公式 (Cayley's formula)

完全图 K[n] 有 n^(n-2) 棵生成树。
怎么证明？方法很多，但是用 Prüfer 序列证是很简单的。任意一个长度为 n-2 的值域 [1,n] 的整数序列都可以通过 Prüfer 序列双射对应一个生成树，于是方案数就是 n^(n-2) 。

图连通方案数

Prüfer 序列可能比你想得还强大。它能创造比凯莱公式更通用的公式。比如以下问题：

> 一个 n 个点 m 条边的带标号无向图有 k 个连通块。我们希望添加 k-1 条边使得整个图连通。求方案数。

证明

设 s[i] 表示每个连通块的数量。我们对 k 个连通块构造 Prüfer 序列，然后你发现这并不是普通的 Prüfer 序列。因为每个连通块的连接方法很多。不能直接淦就设啊。于是设 d[i] 为第 i 个连通块的度数。由于度数之和是边数的两倍，于是 sum _(i=1)^(k)d[i]=2k-2 。则对于给定的 d 序列构造 Prüfer 序列的方案数是
C(k-2,d[1]-1,d[2]-1,...,d[k]-1)=((k-2)!)/((d[1]-1)!(d[2]-1)!...(d[k]-1)!)
对于第 i 个连通块，它的连接方式有 s[i]^(d[i]) 种，因此对于给定 d 序列使图连通的方案数是
C(k-2,d[1]-1,d[2]-1,...,d[k]-1)• prod _(i=1)^(k)s[i]^(d[i])
现在我们要枚举 d 序列，式子变成
sum *d[i] >= 1 ，* *sum* (i=1)^(k)d[i]=2k-2C(k-2,d[1]-1,d[2]-1,...,d[k]-1)• prod _(i=1)^(k)s[i]^(d[i])
好的这是一个非常不喜闻乐见的式子。但是别慌！我们有多元二项式定理：
(x[1] + ⋯ + x_(m))^(p) = sum *substackc[i] >= 0 ，* *sum* (i=1)^(m) c[i] = p C(p,c[1], c[2], ... ,c_(m))• prod _(i=1)^(m)x[i]^(c[i])
那么我们对原式做一下换元，设 e[i]=d[i]-1 ，显然 sum _(i=1)^(k)e[i]=k-2 ，于是原式变成
sum *e[i] >= 0 ，* *sum* (i=1)^(k)e[i]=k-2C(k-2,e[1],e[2],...,e[k])• prod _(i=1)^(k)s[i]^(e[i]+1)
化简得到
(s[1]+s[2]+...+s[k])^(k-2)• prod _(i=1)^(k)s[i]
答案即
n^(k-2)• prod _(i=1)^(k)s[i]

例题：

Status
Problem
Tags
P6086
.
【模板】Prufer 序列
图论
P9619
.
生成树
图论
P4981
.
父子
图论
单源次短路径
字符串基础知识及常用术语

字符串 01 • 字符串基础知识及常用术语

定义

字符集

一个 **字符集** Sigma 是一个建立了全序关系的集合，也就是说， Sigma 中的任意两个不同的元素 alpha 和 beta 都可以比较大小，要么 alpha<beta ，要么 beta<alpha 。字符集 Sigma 中的元素称为字符。

字符串

一个 **字符串** S 是将 n 个字符顺次排列形成的序列， n 称为 S 的长度，表示为 |S| 。

如果字符串下标从 1 开始计算， S 的第 i 个字符表示为 S[i] ；

如果字符串下标从 0 开始计算， S 的第 i 个字符表示为 S[i-1] 。

子串

字符串 S 的 **子串** S[i..j]，i <= j ，表示 S 串中从 i 到 j 这一段，也就是顺次排列 S[i],S[i+1],…,S[j] 形成的字符串。

有时也会用 S[i..j] ， i>j 来表示空串。

子序列

字符串 S 的 **子序列** 是从 S 中将若干元素提取出来并不改变相对位置形成的序列，即 S[p[1]],S[p[2]],…,S[p[k]] ， 1 <= p[1]< p[2]<...< p[k] <= |S| 。

后缀

后缀 是指从某个位置 i 开始到整个串末尾结束的一个特殊子串。字符串 S 的从 i 开头的后缀表示为 Suffix(S,i) ，也就是 Suffix(S,i)=S[i..|S|-1] 。

真后缀 指除了 S 本身的 S 的后缀。

举例来说，字符串 `abcabcd` 的所有后缀为 `{d, cd, bcd, abcd, cabcd, bcabcd, abcabcd}` ，而它的真后缀为 `{d, cd, bcd, abcd, cabcd, bcabcd}` 。

前缀

前缀 是指从串首开始到某个位置 i 结束的一个特殊子串。字符串 S 的以 i 结尾的前缀表示为 Prefix(S,i) ，也就是 Prefix(S,i)=S[0..i] 。

真前缀 指除了 S 本身的 S 的前缀。

举例来说，字符串 `abcabcd` 的所有前缀为 `{a, ab, abc, abca, abcab, abcabc, abcabcd}` ，而它的真前缀为 `{a, ab, abc, abca, abcab, abcabc}` 。

字典序

以第 i 个字符作为第 i 关键字进行大小比较，空字符小于字符集内任何字符（即： a< aa ）。

回文串

回文串 是正着写和倒着写相同的字符串，即满足任意 1 <= i <= |s|，s[i]=s[|s|+1-i] 的 s 。

汉明距离

汉明距离 是两个等长字符串之间的距离，它表示两个长度相同的字符串对应位字符不同的数量。

我们可以简单的认为对两个串进行异或运算，结果为 1 的数量就是两个串的汉明距离。

字符串的存储

- 使用 `char` 数组存储，用空字符 `\0` 表示字符串的结尾（C 风格字符串）。
- 使用 C++ 标准库提供的 `string` 类。
- 字符串常量可以用字符串字面量（用双引号括起来的字符串）表示。

Prufer 序列与 Caley 公式

前缀函数与字符串匹配

字符串 02 • 前缀函数与字符串匹配

字符串前缀和后缀定义

关于字符串前缀、真前缀，后缀、真后缀的定义详见字符串基础

前缀函数

定义

给定一个长度为 n 的字符串 s ，其 **前缀函数** 被定义为一个长度为 n 的数组 pi 。

其中 pi[i] 的定义是：

1. 如果子串 s[0… i] 有一对相等的真前缀与真后缀： s[0… k-1] 和 s[i - (k - 1) … i] ，那么 pi[i] 就是这个相等的真前缀（或者真后缀，因为它们相等）的长度，也就是 pi[i]=k ；
2. 如果不止有一对相等的，那么 pi[i] 就是其中最长的那一对的长度；
3. 如果没有相等的，那么 pi[i]=0 。

简单来说 pi[i] 就是，子串 s[0… i] 最长的相等的真前缀与真后缀的长度。

用数学语言描述如下：

pi[i] = max_(k = 0 … i)k: s[0 … k - 1] = s[i - (k - 1) … i]

特别地，规定 pi[0]=0 。

过程

举例来说，对于字符串 `abcabcd` ，

pi[0]=0 ，因为 `a` 没有真前缀和真后缀，根据规定为 0

pi[1]=0 ，因为 `ab` 无相等的真前缀和真后缀

pi[2]=0 ，因为 `abc` 无相等的真前缀和真后缀

pi[3]=1 ，因为 `abca` 只有一对相等的真前缀和真后缀： `a` ，长度为 1

pi[4]=2 ，因为 `abcab` 相等的真前缀和真后缀只有 `ab` ，长度为 2

pi[5]=3 ，因为 `abcabc` 相等的真前缀和真后缀只有 `abc` ，长度为 3

pi[6]=0 ，因为 `abcabcd` 无相等的真前缀和真后缀

同理可以计算字符串 `aabaaab` 的前缀函数为 [0, 1, 0, 1, 2, 2, 3] 。

计算前缀函数的朴素算法

过程

一个直接按照定义计算前缀函数的算法流程：

- 在一个循环中以 i = 1 -> n - 1 的顺序计算前缀函数 pi[i] 的值（ pi[0] 被赋值为 0 ）。
- 为了计算当前的前缀函数值 pi[i] ，我们令变量 j 从最大的真前缀长度 i 开始尝试。
- 如果当前长度下真前缀和真后缀相等，则此时长度为 pi[i] ，否则令 j 自减 1，继续匹配，直到 j=0 。
- 如果 j = 0 并且仍没有任何一次匹配，则置 pi[i] = 0 并移至下一个下标 i + 1 。 具体实现如下： `// 注：`

```
// string substr (size_t pos = 0, size_t len = npos) const;
vector<int> prefix_function(string s) {
int n = (int)s.length();
vector<int> pi(n);
for (int i = 1; i < n; i++)
for (int j = i; j >= 0; j--)
if (s.substr(0, j) == s.substr(i - j + 1, j)) {
pi[i] = j;
break;
}
return pi;
}
```

显见该算法的时间复杂度为 O(n^3) ，具有很大的改进空间。

计算前缀函数的高效算法

第一个优化

第一个重要的观察是 相邻的前缀函数值至多增加 1。

参照下图所示，只需如此考虑：当取一个尽可能大的 pi[i+1] 时，必然要求新增的 s[i+1] 也与之对应的字符匹配，即 s[i+1]=s[pi[i]] ，此时 pi[i+1] = pi[i]+1 。

underbraceoverbraces[0] ~ s[1] ~ s[2]^(pi[i] = 3) ~ s[3]*(pi[i+1] = 4)* ~ … ~ *underbraceoverbraces[i-2] ~ s[i-1] ~ s[i]^(pi[i] = 3) ~ s[i+1]*(pi[i+1] = 4)

所以当移动到下一个位置时，前缀函数的值要么增加一，要么维持不变，要么减少。

此时的改进的算法为：

```
vector<int> prefix_function(string s) {
int n = (int)s.length();
vector<int> pi(n);
for (int i = 1; i < n; i++)
for (int j = pi[i - 1] + 1; j >= 0; j--)  // improved: j=i => j=pi[i-1]+1
if (s.substr(0, j) == s.substr(i - j + 1, j)) {
pi[i] = j;
break;
}
return pi;
}
```

在这个初步改进的算法中，在计算每个 pi[i] 时，最好的情况是第一次字符串比较就完成了匹配，也就是说基础的字符串比较次数是 `n-1` 次。

而由于存在 `j = pi[i-1]+1` （ `pi[0]=0` ）对于最大字符串比较次数的限制，可以看出每次只有在最好情况才会为字符串比较次数的上限积累 1，而每次超过一次的字符串比较消耗的是之后次数的增长空间。

由此我们可以得出字符串比较次数最多的一种情况：至少 `1` 次字符串比较次数的消耗和最多 `n-2` 次比较次数的积累，此时字符串比较次数为 `n-1 + n-2 = 2n-3` 。

可见经过此次优化，计算前缀函数只需要进行 O(n) 次字符串比较，总复杂度降为了 O(n^2) 。

第二个优化

在第一个优化中，我们讨论了计算 pi[i+1] 时的最好情况： s[i+1]=s[pi[i]] ，此时 pi[i+1] = pi[i]+1 。现在让我们沿着这个思路走得更远一点：讨论当 s[i+1] != s[pi[i]] 时如何跳转。

[图示：图片描述]（https://cspwiki.oss-cn-beijing.aliyuncs.com/S_06_01_1.svg）

如上图所示，失配时，我们希望找到对于子串 s[0… i] ，仅次于 pi[i] 的第二长度 j ，使得在位置 i 的前缀性质仍得以保持，也即 s[0 … j - 1] = s[i - j + 1 … i] ：

overbraceunderbraces[0] ~ s[1j] ~ s[2] ~ s[3]^(pi[i]) ~ … ~ overbraces[i-3] ~ s[i-2] ~ underbraces[i-1] ~ s[ij]^(pi[i]) ~ s[i+1]

如果我们找到了这样的长度 j ，那么仅需要再次比较 s[i + 1] 和 s[j] 。如果它们相等，那么就有 pi[i + 1] = j + 1 。否则，我们需要找到子串 s[0… i] 仅次于 j 的第二长度 j^((2)) ，使得前缀性质得以保持，如此反复，直到 j = 0 。如果 s[i + 1] != s[0] ，则 pi[i + 1] = 0 。第二次比较的示意图如下所示

[图示：图片描述]（https://cspwiki.oss-cn-beijing.aliyuncs.com/S_06_01_2.svg）

观察上图可以发现，因为 s[0… pi[i]-1] = s[i-pi[i]+1… i] ，所以对于 s[0… i] 的第二长度 j ，有这样的性质：

s[0 … j - 1] = s[i - j + 1 … i]= s[pi[i]-j… pi[i]-1]

该公式的示意图如下所示：

[图示：图片描述]（https://cspwiki.oss-cn-beijing.aliyuncs.com/S_06_01_3.svg）

也就是说 j 等价于子串 s[pi[i]-1] 的前缀函数值，对应于上图下半部分，即 j=pi[pi[i]-1] 。同理，次于 j 的第二长度等价于 s[j-1] 的前缀函数值， j^((2))=pi[j-1] .

显然我们可以得到一个关于 j 的状态转移方程： j^((n))=pi[j^((n-1))-1], (j^((n-1))>0)

最终算法

所以最终我们可以构建一个不需要进行任何字符串比较，并且只进行 O(n) 次操作的算法。
而且该算法的实现出人意料的短且直观：

```
vector<int> prefix_function(string s) {
int n = (int)s.length();
vector<int> pi(n);
for (int i = 1; i < n; i++) {
int j = pi[i - 1];
while (j > 0 && s[i] != s[j]) j = pi[j - 1];
if (s[i] == s[j]) j++;
pi[i] = j;
}
return pi;
}
```

这是一个 **在线** 算法，即其当数据到达时处理它——举例来说，你可以一个字符一个字符的读取字符串，立即处理它们以计算出每个字符的前缀函数值。该算法仍然需要存储字符串本身以及先前计算过的前缀函数值，但如果我们已经预先知道该字符串前缀函数的最大可能取值 M ，那么我们仅需要存储该字符串的前 M + 1 个字符以及对应的前缀函数值。

应用

在字符串中查找子串：Knuth - Morris - Pratt 算法

该算法由 Knuth、Pratt 和 Morris 在 1977 年共同发布，简称 KMP 算法。
该任务是前缀函数的一个典型应用。

过程

给定一个文本 t 和一个字符串 s ，我们尝试找到并展示 s 在 t 中的所有出现（occurrence）。
为了简便起见，我们用 n 表示字符串 s 的长度，用 m 表示文本 t 的长度。
我们构造一个字符串 s + # + t ，其中 # 为一个既不出现在 s 中也不出现在 t 中的分隔符。接下来计算该字符串的前缀函数。现在考虑该前缀函数除去最开始 n + 1 个值（即属于字符串 s 和分隔符的函数值）后其余函数值的意义。根据定义， pi[i] 为右端点在 i 且同时为一个前缀的最长真子串的长度，具体到我们的这种情况下，其值为与 s 的前缀相同且右端点位于 i 的最长子串的长度。由于分隔符的存在，该长度不可能超过 n 。而如果等式 pi[i] = n 成立，则意味着 s 完整出现在该位置（即其右端点位于位置 i ）。注意该位置的下标是对字符串 s + # + t 而言的。
因此如果在某一位置 i 有 pi[i] = n 成立，则字符串 s 在字符串 t 的 i - (n - 1) - (n + 1) = i - 2n 处出现。下图所示为索引的示意图。
[图示：图片描述]（https://cspwiki.oss-cn-beijing.aliyuncs.com/S_06_01_4.svg）
正如在前缀函数的计算中已经提到的那样，如果我们知道前缀函数的值永远不超过一特定值，那么我们不需要存储整个字符串以及整个前缀函数，而只需要二者开头的一部分。在我们这种情况下这意味着只需要存储字符串 s + # 以及相应的前缀函数值即可。我们可以一次读入字符串 t 的一个字符并计算当前位置的前缀函数值。
因此此算法用 O(n + m) 的时间以及 O(n) 的内存解决了该问题。

```
vector<int> find_occurrences(string text, string pattern) {
string cur = pattern + '#' + text;
int sz1 = text.size(), sz2 = pattern.size();
vector<int> v;
vector<int> lps = prefix_function(cur);
for (int i = sz2 + 1; i <= sz1 + sz2; i++) {
if (lps[i] == sz2) v.push_back(i - 2 * sz2);
}
return v;
}
```

字符串的周期

对字符串 s 和 0 < p <= |s| ，若 s[i] = s[i+p] 对所有 i 属于 [0, |s| - p - 1] 成立，则称 p 是 s 的周期。
对字符串 s 和 0 <= r < |s| ，若 s 长度为 r 的前缀和长度为 r 的后缀相等，就称 s 长度为 r 的前缀是 s 的 border。
由 s 有长度为 r 的 border 可以推导出 |s|-r 是 s 的周期。
根据前缀函数的定义，可以得到 s 所有的 border 长度，即 pi[n-1],pi[pi[n-1]-1], … 。[^ref1]
所以根据前缀函数可以在 O(n) 的时间内计算出 s 所有的周期。其中，由于 pi[n-1] 是 s 最长 border 的长度，所以 n - pi[n-1] 是 s 的最小周期。

统计每个前缀的出现次数

在该节我们将同时讨论两个问题。给定一个长度为 n 的字符串 s ，在问题的第一个变种中我们希望统计每个前缀 s[0 … i] 在同一个字符串的出现次数，在问题的第二个变种中我们希望统计每个前缀 s[0 … i] 在另一个给定字符串 t 中的出现次数。
首先让我们来解决第一个问题。考虑位置 i 的前缀函数值 pi[i] 。根据定义，其意味着字符串 s 一个长度为 pi[i] 的前缀在位置 i 出现并以 i 为右端点，同时不存在一个更长的前缀满足前述定义。与此同时，更短的前缀可能以该位置为右端点。容易看出，我们遇到了在计算前缀函数时已经回答过的问题：给定一个长度为 j 的前缀，同时其也是一个右端点位于 i 的后缀，下一个更小的前缀长度 k < j 是多少？该长度的前缀需同时也是一个右端点为 i 的后缀。因此以位置 i 为右端点，有长度为 pi[i] 的前缀，有长度为 pi[pi[i] - 1] 的前缀，有长度为 pi[pi[pi[i] - 1] - 1] 的前缀，等等，直到长度变为 0 。故而我们可以通过下述方式计算答案。

```
vector<int> ans(n + 1);
for (int i = 0; i < n; i++) ans[pi[i]]++;
for (int i = n - 1; i > 0; i--) ans[pi[i - 1]] += ans[i];
for (int i = 0; i <= n; i++) ans[i]++;
```

解释

在上述代码中我们首先统计每个前缀函数值在数组 pi 中出现了多少次，然后再计算最后答案：如果我们知道长度为 i 的前缀出现了恰好 ans[i] 次，那么该值必须被叠加至其最长的既是后缀也是前缀的子串的出现次数中。在最后，为了统计原始的前缀，我们对每个结果加 1 。
现在考虑第二个问题。我们应用来自 Knuth－Morris－Pratt 的技巧：构造一个字符串 s + # + t 并计算其前缀函数。与第一个问题唯一的不同之处在于，我们只关心与字符串 t 相关的前缀函数值，即 i >= n + 1 的 pi[i] 。有了这些值之后，我们可以同样应用在第一个问题中的算法来解决该问题。

一个字符串中本质不同子串的数目

给定一个长度为 n 的字符串 s ，我们希望计算其本质不同子串的数目。
我们将迭代的解决该问题。换句话说，在知道了当前的本质不同子串的数目的情况下，我们要找出一种在 s 末尾添加一个字符后重新计算该数目的方法。
令 k 为当前 s 的本质不同子串数量。我们添加一个新的字符 c 至 s 。显然，会有一些新的子串以字符 c 结尾。我们希望对这些以该字符结尾且我们之前未曾遇到的子串计数。
构造字符串 t = s + c 并将其反转得到字符串 t^(~) 。现在我们的任务变为计算有多少 t^(~) 的前缀未在 t^(~) 的其余任何地方出现。如果我们计算了 t^(~) 的前缀函数最大值 pi_(max) ，那么最长的出现在 s 中的前缀其长度为 pi_(max) 。自然的，所有更短的前缀也出现了。
因此，当添加了一个新字符后新出现的子串数目为 |s| + 1 - pi_(max) 。
所以对于每个添加的字符，我们可以在 O(n) 的时间内计算新子串的数目，故最终复杂度为 O(n^2) 。
值得注意的是，我们也可以重新计算在头部添加一个字符，或者从尾或者头移除一个字符时的本质不同子串数目。

例题：

Status
Problem

Tags
P3375
.
【模板】KMP
字符串
P1470
.
最长前缀
字符串
P4391
.
无线传输
字符串
P2375
.
动物园
字符串
P3435
.
OKR-Periods of Words
字符串
字符串基础知识及常用术语
字符串哈希

字符串 03 · 字符串哈希

定义

我们定义一个把字符串映射到整数的函数 f ，这个 f 称为是 Hash 函数。
我们希望这个函数 f 可以方便地帮我们判断两个字符串是否相等。

Hash 的思想

Hash 的核心思想在于，将输入映射到一个值域较小、可以方便比较的范围。
这里的「值域较小」在不同情况下意义不同。
在哈希表中，值域需要小到能够接受线性的空间与时间复杂度。
在字符串哈希中，值域需要小到能够快速比较（ 10^9 、 10^(18) 都是可以快速比较的）。
同时，为了降低哈希冲突率，值域也不能太小。

性质

具体来说，哈希函数最重要的性质可以概括为下面两条：

1. 在 Hash 函数值不一样的时候，两个字符串一定不一样；
2. 在 Hash 函数值一样的时候，两个字符串不一定一样（但有大概率一样，且我们当然希望它们总是一样的）。 我们将 Hash 函数值一样但原字符串不一样的现象称为哈希碰撞。

解释

我们需要关注的是什么？
时间复杂度和 Hash 的准确率。
通常我们采用的是多项式 Hash 的方法，对于一个长度为 l 的字符串 s 来说，我们可以这样定义多项式 Hash 函数： f(s) = sum (i=1 ~ l) s[i] * b^(l-i) mod M 。例如，对于字符串 xyz ，其哈希函数值为 xb^2+yb+z 。
特别要说明的是，也有很多人使用的是另一种 Hash 函数的定义，即 f(s) = sum (i=1 ~ l) s[i] * b^(i-1) mod M ，这种定义下，同样的字符串 xyz 的哈希值就变为了 x+yb+zb^2 了。

显然，上面这两种哈希函数的定义函数都是可行的，但二者在之后会讲到的计算子串哈希值时所用的计算式是不同的，因此千万注意 **不要弄混了这两种不同的 Hash 方式**。
由于前者的 Hash 定义计算更简便、使用人数更多、且可以类比为一个 b 进制数来帮助理解，所以本文下面所将要讨论的都是使用 f(s) = sum (i=1 ~ l) s[i] * b^(l-i) mod M 来定义的 Hash 函数。
还有，有时为了方便和扩大模数，我们在 C++ 中我们会使用 `unsigned long long` 来定义 Hash 函数的结果。由于 C++ 的特性，我们相当于把模数 M 定为 2^(64) ，也是一个不错的选择。
准确率会在后面讨论。

Hash 的错误率分析

Hash 冲突

Hash 冲突是指两个不同的字符串映射到相同的 Hash 值。
我们设 Hash 的取值空间（所有可能出现的字符串的数量）为 d ，计算次数（要计算的字符串数量）为 n 。
则 Hash 冲突的概率为：
p(n,d) = (d!)/(d^n(d-n)!) ~ 1 - exp(-(n(n-1))/(2d))

卡大模数 Hash

注意到这个公式：
p(n,d) ~ 1 - exp(-(n(n-1))/(2d))
为了卡掉 Hash，我们要满足一下条件：

1. d 要大于模数。
2. 1-p(d,n) 要尽量小。

举个例子：
若字符集为 **大小写字母和数字**，模数为 10^9+7 时：
log[62]10^9+7 ~ 6
p(10^6,62^6) ~ 0.9
所以对于这个范围，我们随机生成 10^6 个长度为 6 的字符串，它们 Hash 值相同的概率高达 90% 。

卡自然溢出 Hash

这种 Hash 由于模数太大，用上面的方法卡不了，所以我们需要另一种方法。
首先，这种 Hash 是形如 f(s) = sum (i=1 ~ l) s[i] * b^(l-i) ，我们根据 b 来分类讨论。

b 为偶数

此时 f(s) = s[1]• b^(l) + s[2]• b^(l-1) + ... + s_(l)• b mod M ，其中 M 为 2^(64) 。
容易发现若 l >= 64 ， s[i]• b^(l) == 0 mod M 。
所以我们只要构造形如：

```
aaa...a
baa...a
```

且长度大于 64 的字符串就能冲突。

b 为奇数

定义 !s[i] 为把 s[i] 中所有字符反转。
例：
s[i] = abaab
!s[i] = babba
即把 `a` 变成 `b` ，把 `b` 变成 `a` 。
再定义 hash[i] 为 s[i] 的 Hash 值， !hash[i] 为 !s[i] 的 Hash 值。
不断构造 s[i] = s[i-1] + !s[i-1] 。
s[12] 和 !s[12] 就是我们要的两个字符串。

Hash 的改进

多值 Hash

看了上面这么多的卡法，当然也有解决办法。
多值 Hash，就是有多个 Hash 函数，每个 Hash 函数的模数不一样，这样就能解决 Hash 冲突的问题。
判断时只要有其中一个的 Hash 值不同，就认为两个字符串不同，若 Hash 值都相同，则认为两个字符串相同。
一般来说，双值 Hash 就够用了。

多次询问子串哈希

单次计算一个字符串的哈希值复杂度是 O(n) ，其中 n 为串长，与暴力匹配没有区别，如果需要多次询问一个字符串的子串的哈希值，每次重新计算效率非常低下。
一般采取的方法是对整个字符串先预处理出每个前缀的哈希值，将哈希值看成一个 b 进制的数对 M 取模的结果，这样的话每次就能快速求出子串的哈希了：
令 fi 表示 f(s[1..i]) ，即原串长度为 i 的前缀的哈希值，那么按照定义有 fi=s[1]• b^(i-1)+s[2]• b^(i-2)+…+s[i-1]• b+s[i]
现在，我们想要用类似前缀和的方式快速求出 f(s[l..r]) ，按照定义有字符串 s[l..r] 的哈希值为 f(s[l..r])=s[l]• b^(r-l)+s[l+1]• b^(r-l-1)+…+s[r-1]• b+s[r]
对比观察上述两个式子，我们发现 f(s[l..r])=fr-f_(l-1)(s) * b^(r-l+1) 成立（可以手动代入验证一下），因此我们用这个式子就可以快速得到子串的哈希值。其中 b^(r-l+1) 可以 O(n) 的预处理出来然后 O(1) 的回答每次询问（当然也可以快速幂 O(log n) 的回答每次询问）。

实现

模数 Hash：

注：效率较低，实际使用中不推荐。

```
using std::string;

constexpr int M = 1e9 + 7;
constexpr int B = 233;

using ll = long long;

int get_hash(const string& s) {
int res = 0;
for (int i = 0; i < s.size(); ++i) {
res = ((ll)res * B + s[i]) % M;
}
return res;
}

bool cmp(const string& s, const string& t) {
return get_hash(s) == get_hash(t);
}
```

双值 Hash：

```
using ull = unsigned long long;
ull base = 131;
ull mod1 = 212370440130137957, mod2 = 1e9 + 7;

ull get_hash1(std::string s) {
int len = s.size();
ull ans = 0;
for (int i = 0; i < len; i++) ans = (ans * base + (ull)s[i]) % mod1;
return ans;
}

ull get_hash2(std::string s) {
int len = s.size();
ull ans = 0;
for (int i = 0; i < len; i++) ans = (ans * base + (ull)s[i]) % mod2;
return ans;
}

bool cmp(const std::string s, const std::string t) {
bool f1 = get_hash1(s) != get_hash1(t);
bool f2 = get_hash2(s) != get_hash2(t);
return f1 || f2;
}
```

Hash 的应用

字符串匹配

求出模式串的哈希值后，求出文本串每个长度为模式串长度的子串的哈希值，分别与模式串的哈希值比较即可。

允许 k 次失配的字符串匹配

问题：给定长为 n 的源串 s ，以及长度为 m 的模式串 p ，要求查找源串中有多少子串与模式串匹配。 s’ 与 s 匹配，当且仅当 s’ 与 s 长度相同，且最多有 k 个位置字符不同。其中 1 <= n,m <= 10^6 ， 0 <= k <= 5 。

这道题无法使用 KMP 解决，但是可以通过哈希 + 二分来解决。

枚举所有可能匹配的子串，假设现在枚举的子串为 s’ ，通过哈希 + 二分可以快速找到 s’ 与 p 第一个不同的位置。之后将 s’ 与 p 在这个失配位置及之前的部分删除掉，继续查找下一个失配位置。这样的过程最多发生 k 次。

总的时间复杂度为 O(m+knlog[2]m) 。

最长回文子串

二分答案，判断是否可行时枚举回文中心（对称轴），哈希判断两侧是否相等。需要分别预处理正着和倒着的哈希值。时间复杂度 O(nlog n) 。

这个问题可以使用 manacher 算法在 O(n) 的时间内解决。

通过哈希同样可以 O(n) 解决这个问题，具体方法就是记 R[i] 表示以 i 作为结尾的最长回文的长度，那么答案就是 max_(i=1)^nR[i] 。考虑到 R[i] <= R[i-1]+2 ，因此我们只需要暴力从 R[i-1]+2 开始递减，直到找到第一个回文即可。记变量 z 表示当前枚举的 R[i] ，初始时为 0 ，则 z 在每次 i 增大的时候都会增大 2 ，之后每次暴力循环都会减少 1 ，故暴力循环最多发生 2n 次，总的时间复杂度为 O(n) 。

最长公共子字符串

问题：给定 m 个总长不超过 n 的非空字符串，查找所有字符串的最长公共子字符串，如果有多个，任意输出其中一个。其中 1 <= m, n <= 10^6 。

很显然如果存在长度为 k 的最长公共子字符串，那么 k-1 的公共子字符串也必定存在。因此我们可以二分最长公共子字符串的长度。假设现在的长度为 k ，`check(k)` 的逻辑为我们将所有所有字符串的长度为 k 的子串分别进行哈希，将哈希值放入 n 个哈希表中存储。之后求交集即可。

时间复杂度为 O(m+nlog n) 。

确定字符串中不同子字符串的数量

问题：给定长为 n 的字符串，仅由小写英文字母组成，查找该字符串中不同子串的数量。
为了解决这个问题，我们遍历了所有长度为 l=1,... ,n 的子串。对于每个长度为 l ，我们将其 Hash 值乘以相同的 b 的幂次方，并存入一个数组中。数组中不同元素的数量等于字符串中长度不同的子串的数量，并此数字将添加到最终答案中。
为了方便起见，我们将使用 h [i] 作为 Hash 的前缀字符，并定义 h[0]=0 。

```
int count_unique_substrings(string const& s) {
int n = s.size();

constexpr static int b = 31;
constexpr static int m = 1e9 + 9;
vector<long long> b_pow(n);
b_pow[0] = 1;
for (int i = 1; i < n; i++) b_pow[i] = (b_pow[i - 1] * b) % m;

vector<long long> h(n + 1, 0);
for (int i = 0; i < n; i++)
h[i + 1] = (h[i] + (s[i] - 'a' + 1) * b_pow[i]) % m;

int cnt = 0;
for (int l = 1; l <= n; l++) {
set<long long> hs;
for (int i = 0; i <= n - l; i++) {
long long cur_h = (h[i + l] + m - h[i]) % m;
cur_h = (cur_h * b_pow[n - i - 1]) % m;
hs.insert(cur_h);
}
cnt += hs.size();
}
return cnt;
}
```

例题
题面
给你若干个字符串，答案串初始为空。第 i 步将第 i 个字符串加到答案串的后面，但是尽量地去掉重复部分（即去掉一个最长的、是原答案串的后缀、也是第 i 个串的前缀的字符串），求最后得到的字符串。
字符串个数不超过 10^5 ，总长不超过 10^6 。
题解
每次需要求最长的、是原答案串的后缀、也是第 i 个串的前缀的字符串。枚举这个串的长度，哈希比较即可。
当然，这道题也可以使用 KMP 算法解决。
参考代码

```
#include <cassert>
#include <cstring>
#include <iostream>
#include <vector>
using namespace std;

constexpr int L = 1e6 + 5;
constexpr int HASH_CNT = 2;

int hashBase[HASH_CNT] = {29, 31};
int hashMod[HASH_CNT] = {int(1e9 + 9), 998244353};

struct StringWithHash {
char s[L];
int ls;
int hsh[HASH_CNT][L];
int pwMod[HASH_CNT][L];

void init() {  // 初始化
ls = 0;
for (int i = 0; i < HASH_CNT; ++i) {
hsh[i][0] = 0;
pwMod[i][0] = 1;
}
}

StringWithHash() { init(); }

void extend(char c) {
s[++ls] = c;                          // 记录字符数和每一个字符
for (int i = 0; i < HASH_CNT; ++i) {  // 双哈希的预处理
pwMod[i][ls] =
1ll * pwMod[i][ls - 1] * hashBase[i] % hashMod[i];  // 得到 b^ls
hsh[i][ls] = (1ll * hsh[i][ls - 1] * hashBase[i] + c) % hashMod[i];
}
}

vector<int> getHash(int l, int r) {  // 得到哈希值
vector<int> res(HASH_CNT, 0);
for (int i = 0; i < HASH_CNT; ++i) {
int t =
(hsh[i][r] - 1ll * hsh[i][l - 1] * pwMod[i][r - l + 1]) % hashMod[i];
t = (t + hashMod[i]) % hashMod[i];
res[i] = t;
}
return res;
}
};

bool equal(const vector<int> &h1, const vector<int> &h2) {
assert(h1.size() == h2.size());
for (unsigned i = 0; i < h1.size(); i++)
if (h1[i] != h2[i]) return false;
return true;
}

int n;
StringWithHash s, t;
char str[L];

void work() {
int len = strlen(str);  // 取字符串长度
t.init();
for (int j = 0; j < len; ++j) t.extend(str[j]);
int d = 0;
```

```
for (int j = min(len, s.ls); j >= 1; --j) {
if (equal(t.getHash(1, j), s.getHash(s.ls - j + 1, s.ls))) {  // 比较哈希值
d = j;
break;
}
}
for (int j = d; j < len; ++j) s.extend(str[j]);  // 更新答案数组
}

int main() {
cin.tie(nullptr)->sync_with_stdio(false);
cin >> n;
for (int i = 1; i <= n; ++i) {
cin >> str;
work();
}
cout << s.s + 1 << '\n';
return 0;
}
```

例题：

Status

Problem

Tags

P3370

.

【模板】字符串哈希

字符串

P1381

.

单词背诵

字符串

P2318

.

虚拟内存

字符串

P3167

.

通配符匹配

字符串

前缀函数与字符串匹配

Trie 树

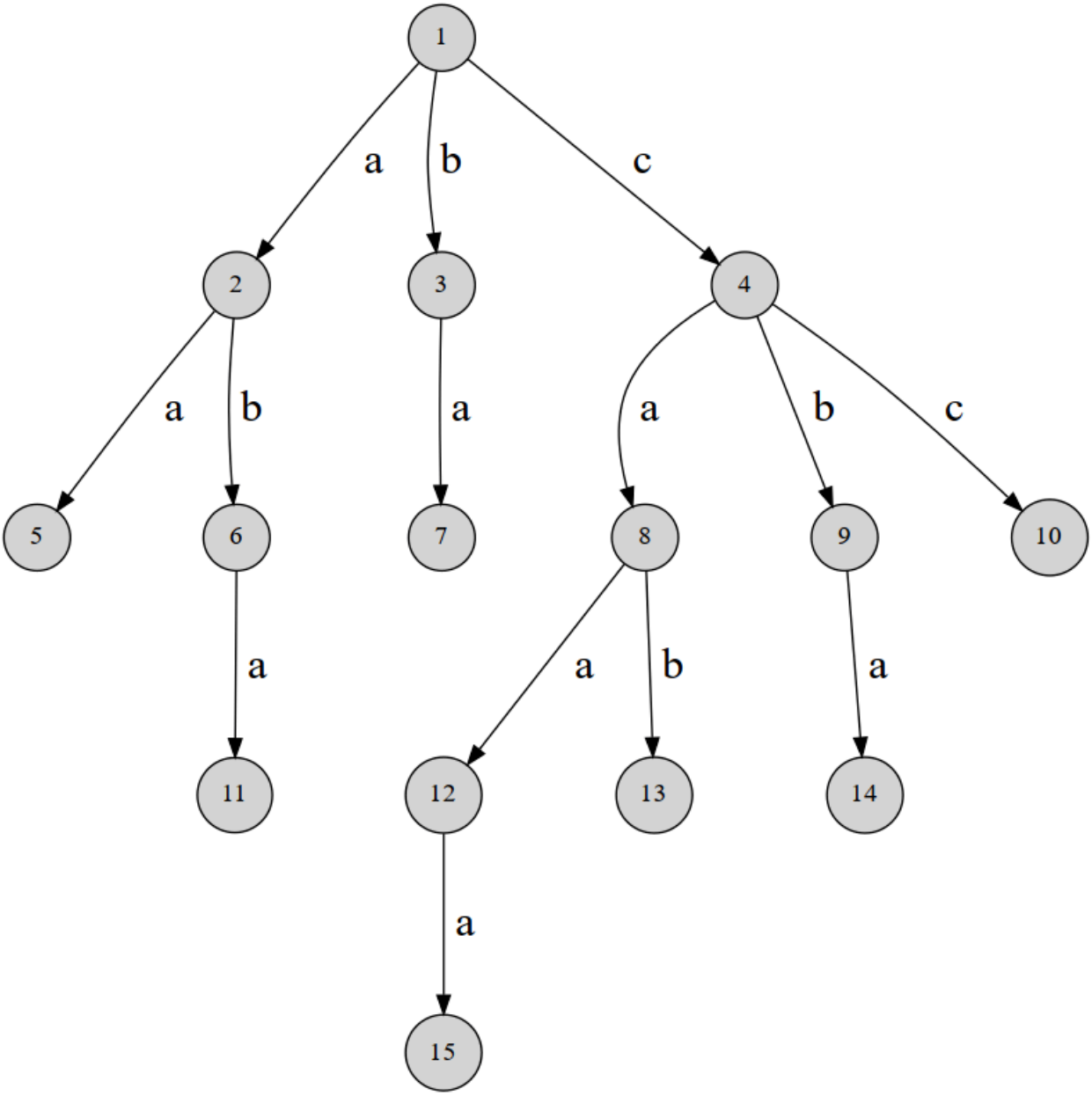
字符串 04 • Trie 树

定义

字典树，英文名 trie。顾名思义，就是一个像字典一样的树。

引入

如下图：

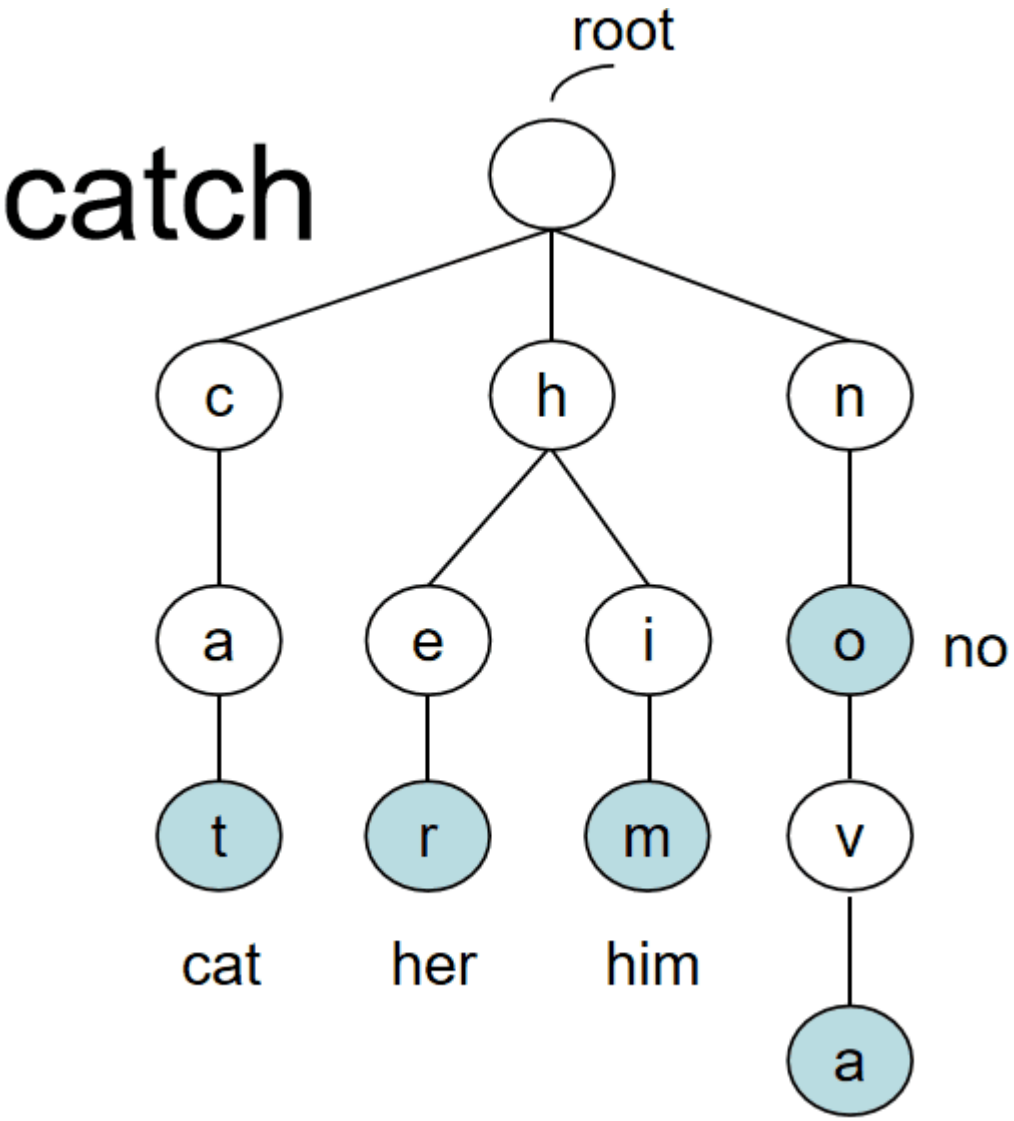


Trie

可以发现，这棵字典树用边来代表字母，而从根结点到树上某一结点的路径就代表了一个字符串。举个例子， 1 -> 4 -> 8 -> 12 表示的就是字符串 `caa` 。

trie 的结构非常好懂，我们用 delta(u,c) 表示结点 u 的 c 字符指向的下一个结点，或着说是结点 u 代表的字符串后面添加一个字符 c 形成的字符串的结点。（ c 的取值范围和字符集大小有关，不一定是 0 ~ 26 。）

有时需要标记插入进 trie 的是哪些字符串，每次插入完成时在这个字符串所代表的节点处打上标记即可。



实现

放一个结构体封装的模板：

```
struct trie {
int nex[100000][26], cnt;
bool exist[100000];  // 该结点结尾的字符串是否存在

void insert(char *s, int l) {  // 插入字符串
int p = 0;
for (int i = 0; i < l; i++) {
int c = s[i] - 'a';
if (!nex[p][c]) nex[p][c] = ++cnt;  // 如果没有，就添加结点
p = nex[p][c];
}
exist[p] = true;
}

bool find(char *s, int l) {  // 查找字符串
int p = 0;
for (int i = 0; i < l; i++) {
int c = s[i] - 'a';
if (!nex[p][c]) return 0;
p = nex[p][c];
}
return exist[p];
}
};
```

应用

检索字符串

字典树最基础的应用——查找一个字符串是否在「字典」中出现过。

题目于是他错误的点名开始了

题面

给你 n 个名字串，然后进行 m 次点名，每次你需要回答「名字不存在」、「第一次点到这个名字」、「已经点过这个名字」之一。

1 <= n <= 10^4 ， 1 <= m <= 10^5 ，所有字符串长度不超过 50 。

参考代码

```
#include <cstdio>
using namespace std;
constexpr int N = 500010;

char s[N];
int n, m, ch[N][26], tag[N], tot = 1;

int main() {
scanf("%d", &n);

for (int i = 1; i <= n; ++i) {
scanf("%s", s + 1);
int u = 1;
for (int j = 1; s[j]; ++j) {
int c = s[j] - 'a';
// 如果这个节点的子节点中没有这个字符，添加上并将该字符的节点号记录为++tot
if (!ch[u][c]) ch[u][c] = ++tot;
u = ch[u][c];  // 往更深一层搜索
}
tag[u] = 1;  // 最后一个字符为节点 u 的名字未被访问到记录为 1
}

scanf("%d", &m);

while (m--) {
scanf("%s", s + 1);
int u = 1;
for (int j = 1; s[j]; ++j) {
int c = s[j] - 'a';
u = ch[u][c];
if (!u) break;  // 不存在对应字符的出边说明名字不存在
}
if (tag[u] == 1) {
tag[u] = 2;  // 最后一个字符为节点 u 的名字已经被访问
puts("OK");
} else if (tag[u] == 2)  // 已经被访问，重复访问
puts("REPEAT");
else
puts("WRONG");
}

return 0;
}
```

例题：

Status

Problem

Tags

P8306

.

【模板】字典树

字符串

P2580

.

于是他错误的点名开始了

字符串

P4551

.

最长异或路径

字符串
P6088
.
字符串树
字符串
字符串哈希
Manacher 算法

字符串 05 • Manacher 算法

问题

给定一个长度为 n 的字符串 s ，请找到所有对 (i, j) 使得子串 s[i … j] 为一个回文串。当 t = t_(rev) 时，字符串 t 是一个回文串（ t_(rev) 是 t 的反转字符串）。

解释

显然在最坏情况下可能有 O(n^2) 个回文串，因此似乎一眼看过去该问题并没有线性算法。

但是关于回文串的信息可用 **一种更紧凑的方式** 表达：对于每个位置 i = 0 … n - 1 ，我们找出值 d[1][i] 和 d[2][i] 。二者分别表示以位置 i 为中心的长度为奇数和长度为偶数的回文串个数。换个角度，二者也表示了以位置 i 为中心的最长回文串的半径长度（半径长度 d[1][i] ， d[2][i] 均为从位置 i 到回文串最右端位置包含的字符个数）。

举例来说，字符串 s = abababc 以 s[3] = b 为中心有三个奇数长度的回文串，最长回文串半径为 3 ，也即 d[1][3] = 3 ：

a overbraceb a undersets[3]b a b^(d[1][3]=3) c

字符串 s = cbaabd 以 s[3] = a 为中心有两个偶数长度的回文串，最长回文串半径为 2 ，也即 d[2][3] = 2 ：

c overbraceb a undersets[3]a b^(d[2][3]=2) d

因此关键思路是，如果以某个位置 i 为中心，我们有一个长度为 l 的回文串，那么我们有以 i 为中心的长度为 l - 2 ， l - 4 ，等等的回文串。所以 d[1][i] 和 d[2][i] 两个数组已经足够表示字符串中所有子回文串的信息。

一个令人惊讶的事实是，存在一个复杂度为线性并且足够简单的算法计算上述两个「回文性质数组」d[1][] 和 d[2][] 。在这篇文章中我们将详细的描述该算法。

解法

总的来说，该问题具有多种解法：应用字符串哈希，该问题可在 O(n log n) 时间内解决，而使用后缀数组和快速 LCA 该问题可在 O(n) 时间内解决。

但是这里描述的算法 **压倒性** 的简单，并且在时间和空间复杂度上具有更小的常数。该算法由 Glenn K. Manacher 在 1975 年提出。

朴素算法

为了避免在之后的叙述中出现歧义，这里我们指出什么是「朴素算法」。

该算法通过下述方式工作：对每个中心位置 i ，在比较一对对应字符后，只要可能，该算法便尝试将答案加 1 。

该算法是比较慢的：它只能在 O(n^2) 的时间内计算答案。

该朴素算法的实现如下：

```
vector<int> d1(n), d2(n);
for (int i = 0; i < n; i++) {
d1[i] = 1;
while (0 <= i - d1[i] && i + d1[i] < n && s[i - d1[i]] == s[i + d1[i]]) {
d1[i]++;
}

d2[i] = 0;
while (0 <= i - d2[i] - 1 && i + d2[i] < n &&
s[i - d2[i] - 1] == s[i + d2[i]]) {
d2[i]++;
}
}
```

Manacher 算法

这里我们将只描述算法中寻找所有奇数长度子回文串的情况，即只计算 d[1][] ；寻找所有偶数长度子回文串的算法（即计算数组 d[2][] ）将只需对奇数情况下的算法进行一些小修改。

为了快速计算，我们维护已找到的最靠右的子回文串的 **边界 (l, r)**（即具有最大 r 值的回文串，其中 l 和 r 分别为该回文串左右边界的位置）。初始时，我们置 l = 0 和 r = -1 （-1需区别于倒序索引位置，这里可为任意负数，仅为了循环初始时方便）。

过程

现在假设我们要对下一个 i 计算 d[1][i] ，而之前所有 d[1][] 中的值已计算完毕。我们将通过下列方式计算：

- 如果 i 位于当前子回文串之外，即 i > r ，那么我们调用朴素算法。 因此我们将连续地增加 d[1][i] ，同时在每一步中检查当前的子串 [i - d[1][i] … i + d[1][i]] （ d[1][i] 表示半径长度，下同）是否为一个回文串。如果我们找到了第一处对应字符不同，又或者碰到了 s 的边界，则算法停止。在两种情况下我们均已计算完 d[1][i] 。此后，仍需记得更新 (l, r) 。
- 现在考虑 i <= r 的情况。我们将尝试从已计算过的 d[1][] 的值中获取一些信息。首先在子回文串 (l, r) 中反转位置 i ，即我们得到 j = l + (r - i) 。现在来考察值 d[1][j] 。因为位置 j 同位置 i 对称，我们 **几乎总是** 可以置 d[1][i] = d[1][j] 。该想法的图示如下（可认为以 j 为中心的回文串被「拷贝」至以 i 为中心的位置上）： …

overbrace
s_(l) …
underbrace
s_(j-d[1][j]+1) … s[j] … s_(j+d[1][j]-1)
[p]alindrome
…
underbrace
s_(i-d[1][j]+1) … s[i] … s_(i+d[1][j]-1)
[p]alindrome
… s[r]
^(p)alindrome
…

然而有一个 **棘手的情况** 需要被正确处理：当「内部」的回文串到达「外部」回文串的边界时，即 j - d[1][j] + 1 <= l （或者等价的说， i + d[1][j] - 1 >= r ）。因为在「外部」回文串范围以外的对称性没有保证，因此直接置 d[1][i] = d[1][j] 将是不正确的：我们没有足够的信息来断言在位置 i 的回文串具有同样的长度。

实际上，为了正确处理这种情况，我们应该「截断」回文串的长度，即置 d[1][i] = r - i 。之后我们将运行朴素算法以尝试尽可能增加 d[1][i] 的值。

该种情况的图示如下（以 j 为中心的回文串已经被截断以落在「外部」回文串内）：

…
overbrace
underbrace
s_(l) … s[j] … s_(j+(j-l))
[p]alindrome
…
underbrace
s_(i-(r-i)) … s[i] … s[r]
[p]alindrome
^(p)alindrome
underbrace
… … … … …
[t]ry moving here

该图示显示出，尽管以 j 为中心的回文串可能更长，以致于超出「外部」回文串，但在位置 i ，我们只能利用其完全落在「外部」回文串内的部分。然而位置 i 的答案可能比这个值更大，因此接下来我们将运行朴素算法来尝试将其扩展至「外部」回文串之外，也即标识为 "try moving here" 的区域。

最后，仍有必要提醒的是，我们应当记得在计算完每个 d[1][i] 后更新值 (l, r) 。

同时，再让我们重复一遍：计算偶数长度回文串数组 d[2][] 的算法同上述计算奇数长度回文串数组 d[1][] 的算法十分类似。

Manacher 算法的复杂度

因为在计算一个特定位置的答案时我们总会运行朴素算法，所以一眼看去该算法的时间复杂度为线性的事实并不显然。

然而更仔细的分析显示出该算法具有线性复杂度。此处我们需要指出，计算 Z 函数的算法和该算法较为类似，并同样具有线性时间复杂度。

实际上，注意到朴素算法的每次迭代均会使 r 增加 1 ，以及 r 在算法运行过程中从不减小。这两个观察告诉我们朴素算法总共会进行 O(n) 次迭代。

Manacher 算法的另一部分显然也是线性的，因此总复杂度为 O(n) 。

Manacher 算法的实现

分类讨论

为了计算 d[1][] ，我们有以下代码：

```
vector<int> d1(n);
for (int i = 0, l = 0, r = -1; i < n; i++) {
int k = (i > r) ? 1 : min(d1[l + r - i], r - i + 1);
while (0 <= i - k && i + k < n && s[i - k] == s[i + k]) {
k++;
}
d1[i] = k--;
if (i + k > r) {
l = i - k;
r = i + k;
}
}
```

计算 d[2][] 的代码十分类似，但是在算术表达式上有些许不同：

```
vector<int> d2(n);
for (int i = 0, l = 0, r = -1; i < n; i++) {
int k = (i > r) ? 0 : min(d2[l + r - i + 1], r - i + 1);
while (0 <= i - k - 1 && i + k < n && s[i - k - 1] == s[i + k]) {
k++;
}
d2[i] = k--;
if (i + k > r) {
l = i - k - 1;
r = i + k;
}
}
```

统一处理

虽然在讲解过程及上述实现中我们将 d[1][] 和 d[2][] 的计算分开考虑，但实际上可以通过一个技巧将二者的计算统一为 d[1][] 的计算。

给定一个长度为 n 的字符串 s ，我们在其 n + 1 个空中插入分隔符 # ，从而构造一个长度为 2n + 1 的字符串 s’ 。举例来说，对于字符串 s = abababc ，其对应的 s’ = #a#b#a#b#a#b#c# 。

对于字母间的 # ，其实际意义为 s 中对应的「空」。而两端的 # 则是为了实现的方便。

注意到，在对 s’ 计算 d[1][] 后，对于一个位置 i ， d[1][i] 所描述的最长的子回文串必定以 # 结尾（若以字母结尾，由于字母两侧必定各有一个 # ，因此可向外扩展一个得到一个更长的）。因此，对于 s 中一个以字母为中心的极大子回文串，设其长度为 m + 1 ，则其在 s’ 中对应一个以相应字母为中心，长度为 2m + 3 的极大子回文串；而对于 s 中一个以空为中心的极大子回文串，设其长度为 m ，则其在 s’ 中对应一个以相应表示空的 # 为中心，长度为 2m + 1 的极大子回文串（上述两种情况下的 m 均为偶数，但该性质成立与否并不影响结论）。综合以上观察及少许计算后易得，在 s’ 中， d[1][i] 表示在 s 中以对应位置为中心的极大子回文串的 **总长度加一**。

上述结论建立了 s’ 的 d[1][] 同 s 的 d[1][] 和 d[2][] 间的关系。

由于该统一处理本质上即求 s’ 的 d[1][] ，因此在得到 s’ 后，代码同上节计算 d[1][] 的一样。

例题：

Status

Problem

Tags

P3805

.

【模板】manacher

字符串

P3501

.

ANT-Antisymmetry

字符串

Trie 树

离散化

杂项 01 • 离散化

简介

离散化是一种数据处理的技巧，本质上可以看成是一种哈希，其保证数据在哈希以后仍然保持原来的顺序关系。

通俗地讲就是当有些数据因为本身很大或者类型不支持，自身无法作为数组的下标来方便地处理，而影响最终结果的只有元素之间的相对大小关系时，我们可以将原来的数据按照排名来处理问题，即离散化。

用来离散化的可以是大整数、浮点数、字符串等等。

实现

将一个数组离散化，并进行查询是比较常用的应用场景。

方法一

通常原数组中会有重复的元素，一般把相同的元素离散化为相同的数据。

方法如下：

1. 创建原数组的副本。
2. 将副本中的值从小到大排序。
3. 将排序好的副本去重。
4. 查找原数组的每一个元素在副本中的位置，位置即为排名，将其作为离散化后的值。

```
// arr[i] 为初始数组，下标范围为 [1, n]

for (int i = 1; i <= n; ++i)  // step 1
tmp[i] = arr[i];
std::sort(tmp + 1, tmp + n + 1);                              // step 2
int len = std::unique(tmp + 1, tmp + n + 1) - (tmp + 1);  // step 3
for (int i = 1; i <= n; ++i)                                  // step 4
arr[i] = std::lower_bound(tmp + 1, tmp + len + 1, arr[i]) - tmp;
```

同样地，我们也可以对 std::vector 进行离散化：

```
// std::vector<int> arr;
std::vector<int> tmp(arr);  // tmp 是 arr 的一个副本
std::sort(tmp.begin(), tmp.end());
tmp.erase(std::unique(tmp.begin(), tmp.end()), tmp.end());
for (int i = 0; i < n; ++i)
arr[i] = std::lower_bound(tmp.begin(), tmp.end(), arr[i]) - tmp.begin();
```

方法二

根据题目要求，有时候会把相同的元素根据输入顺序离散化为不同的数据。

此时再用 `std::lower_bound()` 函数实现就有些困难了，需要换一种思路：

1. 创建原数组的副本，同时记录每个元素出现的位置。
2. 将副本按值从小到大排序，当值相同时，按出现顺序从小到大排序。
3. 将离散化后的数字放回原数组。

```
struct Data {
int idx, val;

bool operator<(const Data& o) const {
if (val == o.val)
return idx < o.idx;  // 当值相同时，先出现的元素离散化后的值更小
return val < o.val;
}
} tmp[MAXN];  // 也可以使用 std::pair

for (int i = 1; i <= n; ++i) tmp[i] = Data{i, arr[i]};
std::sort(tmp + 1, tmp + n + 1);
for (int i = 1; i <= n; ++i) arr[tmp[i].idx] = i;
```

复杂度

对于方法一，去重复杂度为 O(n) ，排序复杂度为 O(n log n) ，最后的 n 次查找复杂度为 O(n log n) 。

对于方法二，排序复杂度为 O(n log n) 。

故两种方法的总时间复杂度都为 O(n log n) 。

空间复杂度为 O(n) 。

例题：
Status
Problem
Tags
P1496
.
火烧赤壁
离散化
P1884
.
Overplanting S
离散化
Manacher 算法
双指针

杂项 02 • 双指针

引入

双指针是一种简单而又灵活的技巧和思想，单独使用可以轻松解决一些特定问题，和其他算法结合也能发挥多样的用处。

双指针顾名思义，就是同时使用两个指针，在序列、链表结构上指向的是位置，在树、图结构中指向的是节点，通过或同向移动，或相向移动来维护、统计信息。

接下来我们来看双指针的几个具体使用方法。

维护区间信息

如果不和其他数据结构结合使用，双指针维护区间信息的最简单模式就是维护具有一定单调性，新增和删去一个元素都很方便处理的信息，就比如正数的和、正整数的积等等。

例题 1

给定一个长度为 n 的正整数数组 nums 和整数 k ，找出该数组内乘积小于 k 的连续子数组的个数。其中， 1 <= n <= 3 * 10^4, 1 <= nums[i] <= 1000, 0 <= k <= 10^6 。

过程

设两个指针分别为 l,r ，另外设置一个变量 tmp 记录 [l,r] 内所有数的乘积。最开始 l,r 都在最左面，先向右移动 r ，直到第一次发现 tmp >= k ，这时就固定 r ，右移 l ，直到 tmplt k 。那么对于每个 r ， l 是它能延展到的左边界，由于正整数乘积的单调性，此时以 r 为右端点的满足题目条件的区间个数为 r-l+1 个。

实现

```
int numSubarrayProductLessThanK(vector<int>& nums, int k) {
long long ji = 1ll, ans = 0;
int l = 0;
for (int i = 0; i < nums.size(); ++i) {
ji *= nums[i];
while (l <= i && ji >= k) ji /= nums[l++];
ans += i - l + 1;
}
return ans;
}
```

使用双指针维护区间信息也可以与其他数据结构比如差分、单调队列、线段树、主席树等等结合使用。另外将双指针技巧融入算法的还有莫队，莫队中将询问离线排序后，一般也都是用两个指针记录当前要处理的区间，随着指针一步步移动逐渐更新区间信息。

例题 2

接下来看一道在树上使用双指针并结合树上差分的例题：

给定一颗 n 个点的有根树，边有边权，节点从 1 至 n 编号，1 号节点是这棵树的根。再给出一个参数 t ，对于树上的每个节点 u ，请求出 u 的子树中有多少节点满足该节点到 u 的距离不大于 t 。
数据范围： 1 <= n <= 2 * 10^5,1 <= t <= 10^(18),1 <= p[i] lt i,1 <= w[i] <= 10^(12)

过程

从根开始用 dfs 遍历整棵树，使用一个栈来记录根到当前节点的树链，设一个指针 u 指向当前节点，另一个指针 p 指向与 u 距离不大于 t 的节点中深度最小的节点。记录到根的距离，每次二分查找确定 p 。此时 u 对 p 到 u 路径上的所有节点都有一个贡献，可以用树上差分来记录。
注意不能直接暴力移动 p ，否则时间复杂度可能会退化至 O(n^2) 。

子序列匹配

给定一个字符串 s 和一个字符串数组 dictionary 作为字典，找出并返回字典中最长的字符串，该字符串可以通过删除 s 中的某些字符得到。

过程

此类问题需要将字符串 s 与 t 进行匹配，判断 t 是否为 s 的子序列。解决这种问题只需先将两个指针一个 i 放在 s 开始位置，一个 j 放在 t 开始位置，如果 s[i]=t[j] 说明 t 的第 j 位已经在 s 中找到了第一个对应，可以进而检测后面的部分了，那么 i 和 j 同时加一。如果上述等式不成立，则 t 的第 j 位仍然没有被匹配上，所以只给 i 加一，在 s 的后面部分再继续寻找。最后，如果 j 已经移到了超尾位置，说明整个字符串都可以被匹配上，也就是 t 是 s 的一个子序列，否则不是。

实现

```
string findLongestWord(string s, vector<string>& dictionary) {
sort(dictionary.begin(), dictionary.end());
int mx = 0, r = 0;
string ans = "";
for (int i = dictionary.size() - 1; i >= 0; i--) {
r = 0;
for (int j = 0; j < s.length(); ++j) {
if (s[j] == dictionary[i][r]) r++;
}
if (r == dictionary[i].length()) {
if (r >= mx) {
mx = r;
ans = dictionary[i];
}
}
}
return ans;
}
```

这种两个指针指向不同对象然后逐步进行比对的方法还可以用在一些 dp 中。

利用序列有序性

很多时候在序列上使用双指针之所以能够正确地达到目的，是因为序列的某些性质，最常见的就是利用序列的有序性。
给定一个已按照 **升序排列** 的整数数组 `numbers` ，请你从数组中找出两个数满足相加之和等于目标数 `target` 。

过程

这种问题也是双指针的经典应用了，虽然二分也很方便，但时间复杂度上多一个 logn ，而且代码不够简洁。
接下来介绍双指针做法：既然要找到两个数，且这两个数不能在同一位置，那其位置一定是一左一右。由于两数之和固定，那么两数之中的小数越大，大数越小。考虑到这些性质，那我们不妨从两边接近它们。
首先假定答案就是 1 和 n，如果发现 num[1]+num[n]gt target ，说明我们需要将其中的一个元素变小，而 num[1] 已经不能再变小了，所以我们把指向 n 的指针减一，让大数变小。

同理如果发现 num[l]+num[n]lt target ，说明我们要将其中的一个元素变大，但 num[n] 已经不能再变大了，所以将指向 l 的指针加一，让小数变大。
推广到一般情形，如果此时我们两个指针分别指在 l,r 上，且 llt r ，如果 num[l]+num[r]gt target ，就将 r 减一，如果 num[l]+num[r]lt target ，就将 l 加一。这样 l 不断右移， r 不断左移，最后两者各逼近到一个答案。
实现

```
vector<int> twoSum(vector<int>& numbers, int target) {
int r = numbers.size() - 1, l = 0;
vector<int> ans;
ans.clear();
while (l < r) {
if (numbers[l] + numbers[r] > target)
r--;
else if (numbers[l] + numbers[r] == target) {
ans.push_back(l + 1), ans.push_back(r + 1);
return ans;
} else
l++;
}
return ans;
}
```

在归并排序中，在 O(n+m) 时间内合并两个有序数组，也是保证数组的有序性条件下使用的双指针法。
例题：
Status
Problem
Tags
B3773
.
完美字符串
双指针
P4805
.
合并饭团
双指针
P6465
.
课程安排
双指针
离散化
悬线法

杂项 03 • 悬线法

引入
悬线法的适用范围是单调栈的子集。具体来说，悬线法可以应用于满足以下条件的题目：

- 需要在扫描序列时维护单调的信息；
- 可以使用单调栈解决；
- 不需要在单调栈上二分。

看起来悬线法可以被替代，用处不大，但是悬线法概念比单调栈简单，更适合初学 OI 的选手理解并解决最大子矩阵等问题。

例题

大意：在一条水平线上有 n 个宽为 1 的矩形，求包含于这些矩形的最大子矩形面积。

悬线，就是一条竖线，这条竖线有初始位置和高度两个性质，可以在其上端点不超过当前位置的矩形高度的情况下左右移动。

对于一条悬线，我们在这条上端点不超过当前位置的矩形高度且不移出边界的前提下，将这条悬线左右移动，求出其最多能向左和向右扩展到何处，此时这条悬线扫过的面积就是包含这条悬线的尽可能大的矩形。容易发现，最大子矩形必定是包含一条初始位置为 i ，高度为 h[i] 的悬线。枚举实现这个过程的时间复杂度为 O(n ^2) ，但是我们可以用悬线法将其优化到 O(n) 。

我们考虑如何快速找到悬线可以到达的最左边的位置。

过程

定义 l[i] 为当前找到的 i 位置的悬线能扩展到的最左边的位置，容易得到 l[i] 初始为 i ，我们需要进一步判断还能不能进一步往左扩展。

- 如果当前 l[i] = 1 ，则已经扩展到了边界，不可以。
- 如果当前 a[i] > a_(l[i] - 1) ，则从当前悬线扩展到的位置不能再往左扩展了。
- 如果当前 a[i] <= a_(l[i] - 1) ，则从当前悬线还可以往左扩展，并且 l[i] - 1 位置的悬线能向左扩展到的位置， i 位置的悬线一定也可以扩展到，于是我们将 l[i] 更新为 l_(l[i] - 1) ，并继续执行判断。

通过摊还分析，可以证明每个 l[i] 最多会被其他的 l[j] 遍历到一次，因此时间复杂度为 O(n) 。

实现

```
#include <algorithm>
#include <iostream>
using std::max;
constexpr int N = 100010;
int n, a[N];
int l[N], r[N];
long long ans;
using std::cin;
using std::cout;

int main() {
cin.tie(nullptr)->sync_with_stdio(false);
while (cin >> n, n) {
ans = 0;
for (int i = 1; i <= n; i++) cin >> a[i], l[i] = r[i] = i;
for (int i = 1; i <= n; i++)
while (l[i] > 1 && a[i] <= a[l[i] - 1]) l[i] = l[l[i] - 1];
for (int i = n; i >= 1; i--)
while (r[i] < n && a[i] <= a[r[i] + 1]) r[i] = r[r[i] + 1];
for (int i = 1; i <= n; i++)
ans = max(ans, (long long)(r[i] - l[i] + 1) * a[i]);
cout << ans << '\n';
}
return 0;
}
```

对于一个长度为 n 的数列，找出一个子区间，使子区间内的最小值与子区间长度的乘积最大，要求在满足舒适值最大的情况下最小化长度，最小化长度的情况下最小化左端点序号。

本题中我们可以考虑枚举最小值，将每个位置的数 a[i] 当作最小值，并考虑从 i 向左右扩展，找到满足 min _ j = l ^(r) a[j] = a[i] 的尽可能向左右扩展的区间 [l, r] 。这样本题就被转化成了悬线法模型。

```
#include <cstring>
#include <iostream>
constexpr int N = 100010;
int n, a[N], l[N], r[N];
long long sum[N];
long long ans;
int ansl, ansr;
bool fir = true;
using std::cin;
using std::cout;

int main() {
cin.tie(nullptr)->sync_with_stdio(false);
while (cin >> n) {
memset(a, -1, sizeof(a));
if (!fir)
cout << '\n';
else
fir = false;
ans = 0;
ansl = ansr = 1;
for (int i = 1; i <= n; i++) {
cin >> a[i];
sum[i] = sum[i - 1] + a[i];
l[i] = r[i] = i;
}
for (int i = 1; i <= n; i++)
while (a[l[i] - 1] >= a[i]) l[i] = l[l[i] - 1];
for (int i = n; i >= 1; i--)
while (a[r[i] + 1] >= a[i]) r[i] = r[r[i] + 1];
for (int i = 1; i <= n; i++) {
long long x = a[i] * (sum[r[i]] - sum[l[i] - 1]);
if (ans < x || (ans == x && ansr - ansl > r[i] - l[i]))
ans = x, ansl = l[i], ansr = r[i];
}
cout << ans << '\n' << ansl << ' ' << ansr << '\n';
}
return 0;
}
```

最大子矩形

给定一个 n * m 的包含 `'F'` 和 `'R'` 的矩阵，求其面积最大的子矩阵的面积 * 3 ，使得这个子矩阵中的每一位的值都为 `'F'` 。

我们会发现本题的模型和第一题的模型很像。仔细分析，发现如果我们每次只考虑某一行的所有元素，将位置 (x, y) 的元素尽可能向上扩展的距离作为该位置的悬线长度，那最大子矩阵一定是这些悬线向左右扩展得到的尽可能大的矩形中的一个。

```
#include <algorithm>
#include <iostream>
int n, m, a[1010], l[1010], r[1010], ans;
using std::cin;
using std::cout;

int main() {
cin.tie(nullptr)->sync_with_stdio(false);
cin >> n >> m;
for (int i = 1; i <= n; i++) {
for (int j = 1; j <= m; j++) {
l[j] = r[j] = j;
}
char s[3];
for (int j = 1; j <= m; j++) {
cin >> s;
if (s[0] == 'F')
a[j]++;
else if (s[0] == 'R')
a[j] = 0;
}
for (int j = 1; j <= m; j++)
while (l[j] != 1 && a[l[j] - 1] >= a[j]) l[j] = l[l[j] - 1];
for (int j = m; j >= 1; j--)
while (r[j] != m && a[r[j] + 1] >= a[j]) r[j] = r[r[j] + 1];
for (int j = 1; j <= m; j++) ans = std::max(ans, (r[j] - l[j] + 1) * a[j]);
}
cout << ans * 3;
return 0;
}
```

例题：

Status

Problem

Tags

P1169

.

棋盘制作

悬线法

P1578

.

奶牛浴场

悬线法

双指针

扫描线

杂项 04 • 扫描线

引入

扫描线一般运用在图形上面，它和它的字面意思十分相似，就是一条线在整个图上扫来扫去，它一般被用来解决图形面积，周长，以及二维数点等问题。

二维矩形面积并问题

在二维坐标系上，给出多个矩形的左下以及右上坐标，求出所有矩形构成的图形的面积。

过程

根据图片可知总面积可以直接暴力即可求出面积，如果数据大了怎么办？这时就需要讲到 **扫描线** 算法。

现在假设我们有一根线，从下往上开始扫描：
[图示：图片描述]（https://cspwiki.oss-cn-beijing.aliyuncs.com/S_07_04_1.svg）
如图所示，把整个矩形分成如图各个颜色不同的小矩形，小矩形的高是扫过的距离，然而矩形的水平宽一直在变化。
给每一个矩形的上下边进行标记，下面的边标记为 1，上面的边标记为 -1。每遇到一个水平边时，让这条边（在横轴投影区间）的权值加上这条边的标记。
这个操作类似遍历括号序列：开括号加 1，闭括号减 1，「权值」对应当前位置的深度，「权值」是否大于 0，对应当前在不在括号里，也就是这段区间是否记入小矩形的宽度。
小矩形（不一定只有一个）的宽度就是整个数轴上权值大于 0 的区间总长度。
实现
用线段树维护矩形的长，也就是整个数轴上覆盖次数大于 0 的点。需求列举如下：

- 一段区间权值加 1、减 1。
- 统计整个数轴上，区间权值大于 0 的「区间长度和」。

如果你尝试直接用普通线段树模板来实现的话，也许会遇到些挫折。具体地，由于在区间加时，即使修改区间和节点管理区间重合，我们还是不能常数时间知道覆盖次数如何变化。这是因为我们不能直接知道：管理范围里有多长的区间会从 1 变成 0（从 0 变成 1）。
这道题只需要朴素的分治就能实现：维护每个节点管理区间中「**整体** 修改的权值和 `w[]` 」（类似不用下放的懒惰标记）和「覆盖长度 `v[]` 」两个信息。
需要离散化。
例题
模板
【模板】扫描线 & 矩形面积并
参考代码

```
#include <algorithm>
#include <iostream>

using ll = long long;
constexpr int N = 1e5 + 1;

int n, a[N * 2], tot;  // a[] 和 tot 用于把 x 离散化
ll v[N * 8], w[N * 8];  // 完全覆盖区间的次数、已覆盖的长度

struct St {
ll x1, x2, y, o;
} b[N * 2];  // 矩形上下边缘

int f(int y) {  // 离散化，把坐标映射到 a 中的下标
return std::lower_bound(a, a + tot, y) - a;
}

void up(int u, int ul, int ur) {  // pushup
if (v[u]) w[u] = a[ur] - a[ul];
// 如果对叶子节点调用 w[u*2+1]，那么线段树需要开 8 倍空间
// 乘上矩形上下两边就是 16 倍
else if (ul + 1 == ur)
w[u] = 0;
else
w[u] = w[u * 2 + 1] + w[u * 2 + 2];
}

void add(int lf, int rg, ll o, int u = 0, int ul = 0, int ur = tot - 1) {
// 区间加
if (lf == ul && rg == ur) return v[u] += o, up(u, ul, ur), void();
int um = (ul + ur) / 2;
if (lf < um) add(lf, std::min(rg, um), o, u * 2 + 1, ul, um);
if (um < rg) add(std::max(lf, um), rg, o, u * 2 + 2, um, ur);
up(u, ul, ur);
}

int main() {
std::cin >> n;
for (int i = 0, x1, x2, y1, y2; i < n; i++) {
// y1 是局部变量不会重名
std::cin >> x1 >> y1 >> x2 >> y2;
b[i] = {x1, x2, y1, 1};
b[i + n] = {x1, x2, y2, -1};
a[i] = x1, a[i + n] = x2;
}

std::sort(a, a + n * 2), tot = 1;
for (int i = 1; i < n * 2; i++)
if (a[i] != a[tot - 1]) a[tot++] = a[i];  // 离散化

std::sort(b, b + n * 2,
[](St &i, St &j) -> bool { return i.y < j.y; });  // 操作排序

ll sum = 0;
add(f(b[0].x1), f(b[0].x2), 1);
for (int i = 1; i < n * 2; i++) {
int x1 = f(b[i].x1), x2 = f(b[i].x2);
sum += (b[i].y - b[i - 1].y) * w[0];  // 对每个小矩形面积求和
add(x1, x2, b[i].o);
}
std::cout << sum << '\n';
}
```

例题：

Status
Problem
Tags
P5490
.
【模板】扫描线 & 矩形面积并
扫描线
P1856
.
矩形周长Picture
扫描线
P7530
.
United Cows of Farmer John P
扫描线
悬线法